
Introduction to Reinforcement Learning

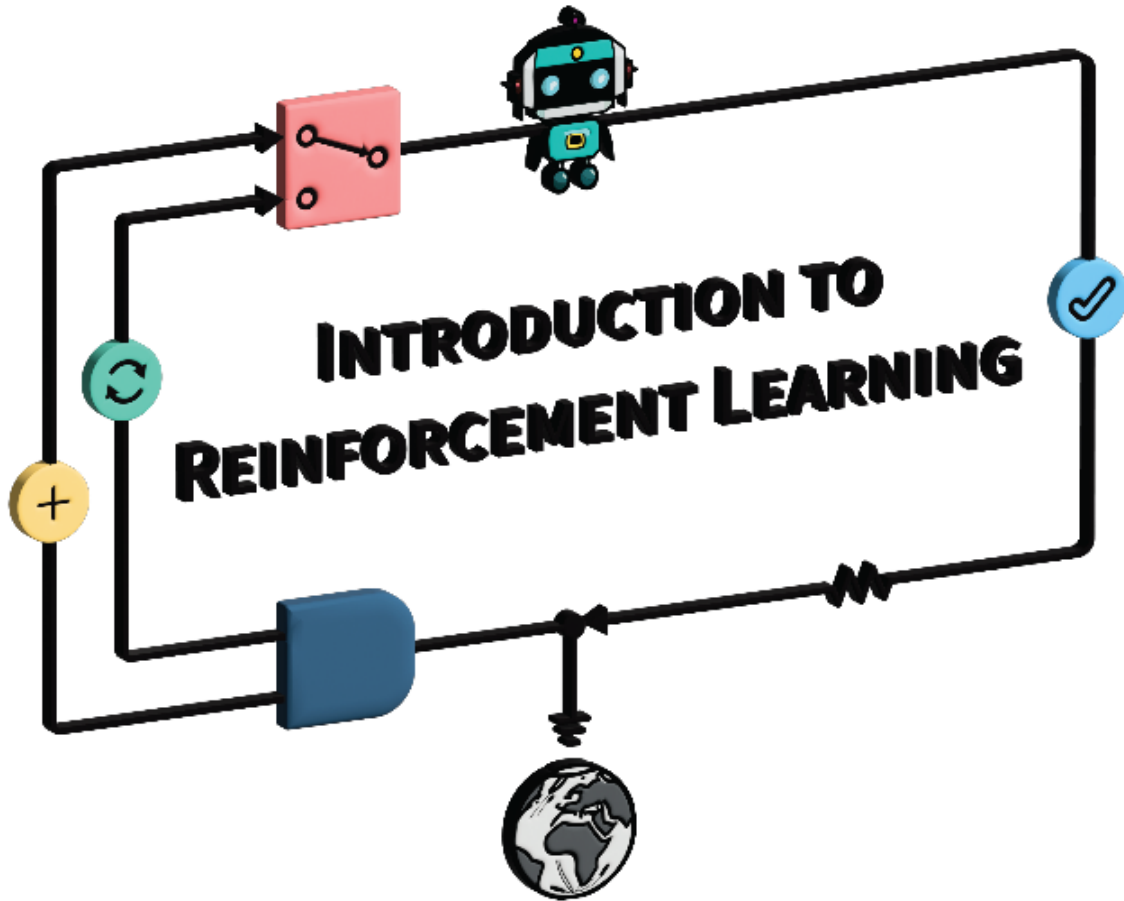
Tim Miller, The University of Queensland

Jan 05, 2024

CONTENTS

I	Single-agent Reinforcement Learning	5
1	Markov Decision Processes	7
1.1	Chapter Overview	7
1.2	Markov Decision Processes	8
1.3	Policies	14
1.4	Optimal Solutions for MDPs	16
1.5	Policy extraction	17
1.6	Partially Observable MDPs	19
1.7	Takeaways	19
2	Value-based methods	21
2.1	Value Iteration	21
2.2	Multi-armed bandits	40
2.3	Temporal difference reinforcement learning	55
2.4	n-step reinforcement learning	79
2.5	Monte-Carlo Tree Search (MCTS)	88
2.6	Q-function approximation	105
2.7	Reward shaping	123
3	Policy-based methods	135
3.1	Policy iteration	135
3.2	Policy gradients	141
3.3	REINFORCE	142
3.4	Advantages and disadvantages of policy gradients (compared to value-based techniques)	150
3.5	Actor critic methods	151
4	Modelling and abstraction for MDPs	157
4.1	Overview	157
4.2	Abstraction	157
4.3	State abstraction	158
4.4	Action abstraction	158
4.5	Rewarding sub-goals	159
4.6	Partially-observable state variables	160
4.7	Heuristics	160
4.8	Takeaways	161
II	Multi-agent Reinforcement Learning	163
5	Normal form games	165

5.1	Chapter overview	165
5.2	Solutions for normal form games: strategies	167
5.3	Best response and Nash equilibria	168
5.4	Calculating best response and Nash equilibria	168
5.5	Mixed strategies	170
5.6	Takeaways	173
6	Extensive form games	175
6.1	Overview	175
6.2	Perfect information extensive form games	176
6.3	Solutions for extensive form games	179
6.4	Backward induction	180
6.5	Multi-agent reinforcement learning	185
7	Modelling and abstraction for multi-agent games	191
7.1	Overview	191
7.2	Multi-agent to single-agent	191
7.3	Limited reasoning of opponent actions	192
7.4	Takeaways	192
III	Appendix	193
8	Introduction to basic probability theory	195
8.1	Overview	195
8.2	Some prerequisite set theory	195
8.3	Sample spaces and outcomes	196
8.4	Probability	197
	Proof Index	199



Document	Modified	Method	Run Time (s)	Status
appendix/intro-to-probability-theory	2024-01-04 21:40	cache	3.54	✓
multi-agent/backward-induction	2024-01-04 21:41	cache	20.46	✓
multi-agent/extensive-form	2024-01-04 21:41	cache	2.02	✓
multi-agent/multi-agent-rl	2024-01-04 21:41	cache	2.89	✓
multi-agent/normal-form	2024-01-04 21:41	cache	2.89	✓
single-agent/MDPs	2024-01-04 21:41	cache	3.56	✓
single-agent/function-approximation	2024-01-05 00:18	cache	29.51	✓
single-agent/mcts	2024-01-04 21:41	cache	2.5	✓
single-agent/multi-armed-bandits	2024-01-04 21:42	cache	14.75	✓
single-agent/n-step	2024-01-04 21:42	cache	2.4	✓
single-agent/policy-gradients	2024-01-04 21:43	cache	94.76	✓
single-agent/policy-iteration	2024-01-04 21:43	cache	5.18	✓
single-agent/reward-shaping	2024-01-04 21:44	cache	20.96	✓
single-agent/temporal-difference-learning	2024-01-05 00:03	cache	27.2	✓
single-agent/value-iteration	2024-01-04 21:44	cache	17.15	✓

This book provides a foundational introduction to the problem of **reinforcement learning**. It combines narrative, maths, and code, to help the reader gain an introduction to the area, why it exists, how to solve reinforcement learning problems, and the strengths and weaknesses of different approaches.

The **aim** of the book is to provide the reader with sufficient foundation that they can design and implement agents using reinforcement learning, and can understand more advanced topics covered in research papers.

We start with dynamic programming approaches, such as *value iteration*. While the current hype is around deep reinforcement learning, understanding value iteration helps learners to appreciate what techniques like *deep policy gradients* and *actor critic* methods are optimising, and how they work. Oh and, for many MDP problems, value iteration is a much better solution than deep reinforcement learning.

In Part I of these notes, we introduce *Markov Decision Processes* (MDPs). MDPs allow us to model problems in which the outcomes of actions are probabilistic; that is, we do not know the outcome beforehand, but we know there is some probability distribution over a set of possible outcomes. We look at *model-based* techniques, where these probabilistic outcomes are given to use, and *model-free* techniques, which are flexible enough the probabilities are unknown, but we can sample enough times that we can still learn good behaviour.

In Part II of these notes, we look at *multi-agent MDPs* (sometimes called *games*), in which there are multiple (possibly adversarial) actors in a problem, and we need to plan our actions while also considering what the other actors in the environment will do. Again, we look at both model-based and model-free techniques.

The book

This book is written using Markdown, and compiled into HTML using *Jupyter Book* — an extension of Jupyter notebooks.

Each individual HTML page can be downloaded individually as a *Jupyter notebook*, but note that you need to install the code and dependencies below.

Code

All code in this book is executable. You can download the code from [here](#).

Note: The code in this book is written for understandability rather than efficiency. It is not intended to be production-level code, such as the *Keras RL* reinforcement learning package.

If you understand the code in these notes, you will have little problem using production-level packages such as Keras RL.

The code in this book is written with the attempt to use very little Python-specific syntax, to enable those less familiar with Python to understand code snippets.

The code in this book is written using as few external libraries as possible, to make this easy to download and run yourself.

Once you have downloaded, unzip the code and add the folder to your PYTHONPATH variable if you want to download the Jupyter notebooks.

Most files in the code have a `main` function that can be run using just `python <filename>.py`. For most of these, no external libraries are required. However, if you want to plot the graphs or draw the trees, you will need to install:

1. The *Matplotlib library* for plotting graphs. You can download from the website or install with `pip install matplotlib`.
2. The *Scipy library* for helping with the graph plotting. You can download from the website or install with `pip install scipy`.
3. The *Graphviz Python library* for drawing trees. You can download from the website or use `pip install graphviz`. To render the generated graphs, you will also need to install *Graphviz the tool*, which is called by the Python package.

The Author

These notes are written and maintained by [Tim Miller](#), Professor of Artificial Intelligence at [The University of Queensland](#), Brisbane/Meaanjin, Australia.

If you find any errors or would like to provide other feedback, feel free to [email me](#).

If you use this as part of your teaching or learning in a course, please [let me know](#)! I'd love to hear from you.

Acknowledgements

Thanks to Alan Lewis for his excellent idea of demonstrating *policy gradients using a logistic regression policy*; and furthermore, for implementing the source for this and the *deep policy gradient agent*. Thanks also to Alan for setting up the library for play GIF files, which supports the interactive visualisations that are so useful in this book.

Thanks to Emma Baillie for the idea and implementation of the Contested Crossing examples.

Part I

Single-agent Reinforcement Learning

MARKOV DECISION PROCESSES

Learning outcomes

The learning outcomes of this chapter are:

1. Define **Markov Decision Process**.
 2. Identify situations in which Markov Decisions Processes (MDPs) are a suitable model of a problem.
 3. Compare MDPs to models of search, such as classical planning and heuristic search.
 4. Explain how Bellman equations are solutions to MDP problems
-

1.1 Chapter Overview

A **Markov Decision Process** (MDPs) is a framework for describing sequential decision making problems. In machine learning, problems such as classification and regression are one-time tasks. That is, a classification/regression model is given an input and returns an output. Then, the next input it is given is entirely independent from the first. In sequential decision making problems, we need to make a series of decisions over time, in which each decision influences the possible future. For example, navigating from one place to another requires us to choose a direction and velocity, move in that direction at the velocity, and then make this decision again and again until we reach our destination. So, even at that first step, we have to consider how each move affects the future. As another example, a clinician making medical treatment decisions has to consider whether decisions taken today will impact future decisions about their patients.

In the language of reinforcement learning, we say that the decision maker is the **agent**, and their decisions are **actions** that they execute in an **environment** or **state**.

Techniques like heuristic search and classical planning algorithms assume that action are **deterministic** – that is, before an agent executes an action in a state, it knows that the outcome of the action will be. MDPs remove the assumption of deterministic events and instead assume that each action could have multiple outcomes, with each outcome associated with a probability. If there is only one outcome for each action (with probability 1), then the problem is deterministic. Otherwise, it is **non-deterministic**. MDPs consider **stochastic** non-determinism; that is, where there is a probability distribution over outcomes.

Here are some examples of stochastic actions:

- Flipping a coin has two outcomes: heads ($\frac{1}{2}$) and tails ($\frac{1}{2}$)
- Rolling two dices together has twelve outcomes: 2 ($\frac{1}{36}$), 3 ($\frac{1}{18}$), 4 ($\frac{3}{36}$), ..., 12 ($\frac{1}{36}$)
- When trying to pick up an object with a robot arm, there could be two outcomes: successful ($\frac{4}{5}$) and unsuccessful ($\frac{1}{5}$)

- When we connect to a web server, there is a 1% chance that the document we are requesting will not exist (404 error) and 99% it will exist.
- When we send a patient for a test, there is a 20% the test will come back negative, and an 80% chance it will come back positive.

MDPs have been successfully applied to planning in many domains: robot navigation, planning which areas of a mine to dig for minerals, treatment for patients, maintenance scheduling on vehicles, and many others.

1.2 Markov Decision Processes

Definition – Markov Decision Process

A **Markov Decision Process** (MDP) is a **fully observable, probabilistic** state model. The most common formulation of MDPs is a **Discounted-Reward Markov Decision Process**. A discount-reward MDP is a tuple $(S, s_0, A, P, r, \gamma)$ containing:

- a state space S
 - initial state $s_0 \in S$
 - actions $A(s) \subseteq A$ applicable in each state $s \in S$ that our agent can execute.
 - **transition probabilities** $P_a(s' | s)$ for $s \in S$ and $a \in A(s)$
 - **rewards** $r(s, a, s')$ positive or negative of transitioning from state s to state s' using action a
 - a **discount factor** $0 \leq \gamma < 1$
-

Let's break down the above points in model detail.

States are the possible situations in which the agent can be in. Each state captures the information required to make a decision. For example, in robot navigation, the state consists of the position of the robot, the current velocity of the robot, the direction it is heading, and the position of obstacles, doors, etc. In an application for scheduling maintenance on vehicles for a delivery company, the state would consist of vehicle IDs, vehicle properties such as make, maximum load, etc., location of vehicles, number of kilometres since their last maintenance check, etc.

The **state space** is simply the set of all possible states. That is, the combination of all vehicle IDs, vehicle properties, maximum loads, etc.

Actions allow agents to affect the environment/state — that is, actions transition the environment from one state to another. They are also the choices that are available to an agent in each state: *which action should I choose now?* For now, we assume that the agent is the only entity that can affect a state.

As noted above, an action can have multiple possible outcomes. Exactly one outcome will occur, but the agent does not know which one until after the action is executed.

Transition probabilities tell us the effect(s) of each action, including the probabilities of each outcome. For example, in the vehicle maintenance task, when our agent schedules a vehicle to be inspected, possible outcomes could be: (a) no further maintenance required (80% chance); (b) minor maintenance required (15% chance); or (c) major maintenance required (5% chance).

Rewards specifies the benefit or cost of executing a particular action in a particular state. For example, a robot navigating to its destination receives a positive reward (benefit) for reaching its destination, a small negative reward (cost) for running into objects on the way; and a large negative reward for running into people.

The **discount factor** determines how much a future reward should be discounted compared to a current reward.

For example, would you prefer \$100 today or \$100 in a year's time? We (humans) often *discount* the future and place a higher value on nearer-term rewards.

Assume our agent receives rewards $r_1, r_2, r_3, r_4, \dots$ in that order. If γ is the discount factor, then the discounted reward is:

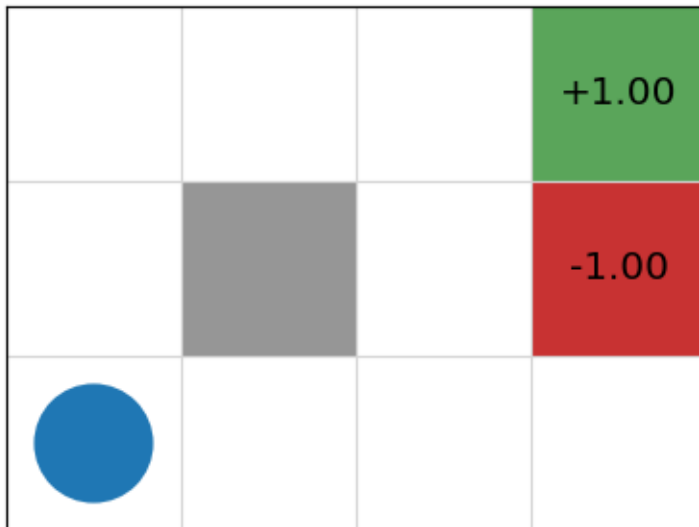
$$\begin{aligned} V &= r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots \\ &= r_1 + \gamma(r_2 + \gamma(r_3 + \gamma(r_4 + \dots))) \end{aligned}$$

If V_t is the value received at time-step t , then $V_t = r_t + \gamma V_{t+1}$. So, the further away a reward is from the start state s_0 , the less actual reward we will receive from it.

In an MDP, a discount reward must be strictly less than 1. Later, we will see why this is important.

Example MDP: Grid World

An agent is in the bottom left cell of a grid. The grey cell is a wall. The two coloured cells give a **reward**. There is a reward of 1 of being in the top-right (green) cell, but a negative value of -1 for the cell immediately below (red).



But! Things can go wrong — sometimes the effects of the actions are not what we want:

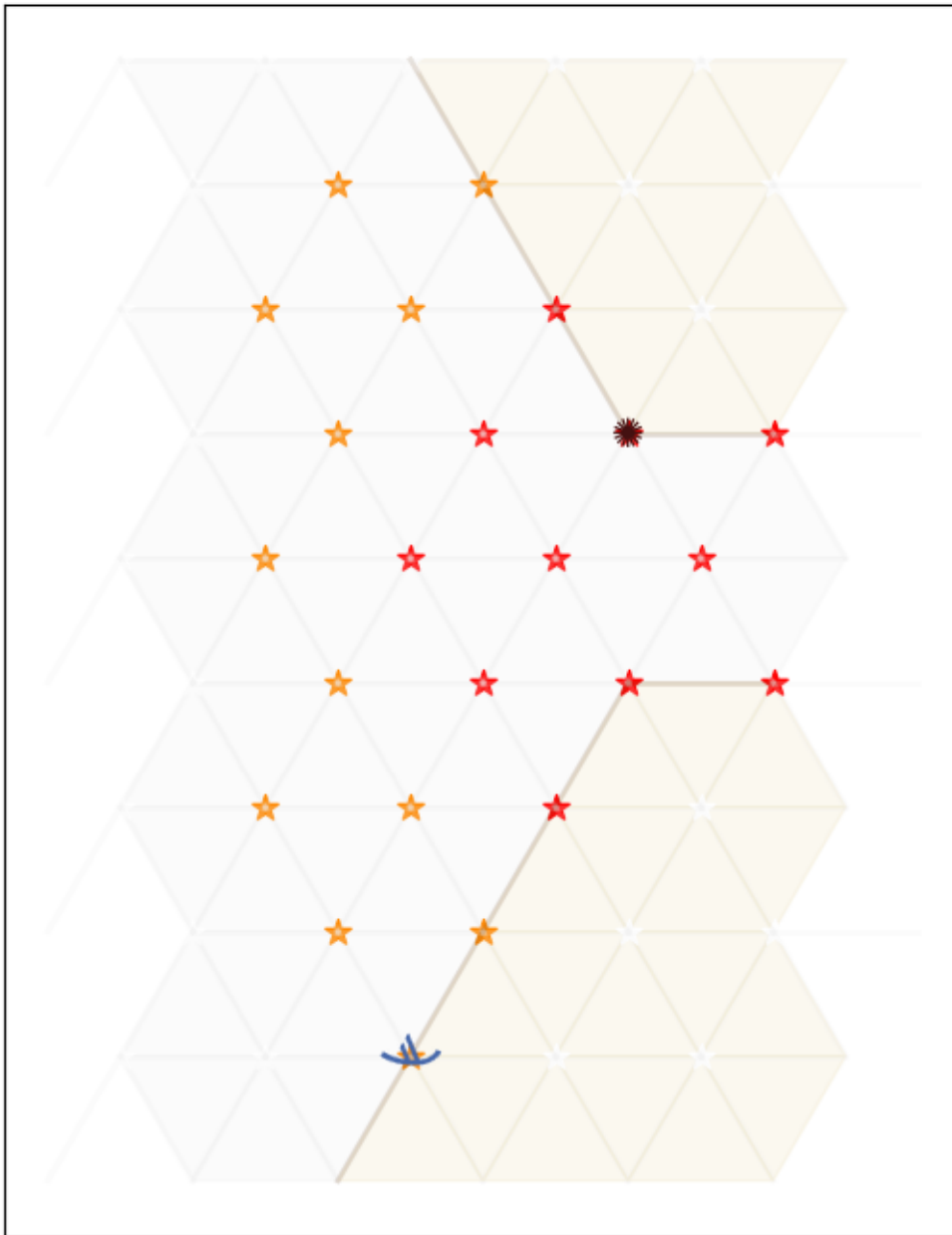
- If the agent tries to move north, 80% of the time, this works as planned (provided the wall is not in the way)
- 10% of the time, trying to move north takes the agent west (provided the wall is not in the way);
- 10% of the time, trying to move north takes the agent east (provided the wall is not in the way)
- If the wall is in the way of the cell that would have been taken, the agent stays in the current cell.

The task is to navigate from the start cell in the bottom left to maximise the expected reward. What would the best sequence of actions be for this problem?

Example MDP: Contested Crossing

An agent (a ship), denoted using s ; is at the south shore of a body of water. It may sail between points on the hexagonal grid where the terrain is water (pale grey), but not on land (pale yellow), choosing a different direction at each step (West, North-West, North-East, East, South-East or South-West). There is a reward of 10 for reaching the north shore, but a negative value of -10 for sinking on the way.

Initial Positions



At the closest point of the north shore is an enemy, denoted using the $\star$ character. The enemy will shoot at the ship when it is in areas of danger (yellow or red stars). It will do so once for each step. Therefore, the enemy's behaviour is completely determined and no choice needs to be made.

In locations with yellow or red stars, the ship may also shoot at the enemy, but it cannot do so and turn at the same time. If it chooses to shoot, it will continue sailing in the same direction.

- In areas of low danger (yellow), a shot will damage the target 10% of the time (either the ship firing at the enemy, or the enemy firing at the ship).
- In areas of high danger (red), a shot will damage the target 99% of the time.
- When the ship is damaged, it has a chance of failing to move in its step. At full health, the ship moves successfully 100% of the time, with damage level 1 it moves successfully 67% and at damage level 2, 33%; and at damage level

3, it sinks.

- When the enemy is at damage level 1, there is no change in its behaviour. When it is at damage level 2 it is destroyed. At this point the ship is in no further danger.

The ship can observe the entire state: its location, the enemy location, its own health, and the health of the enemy.

In this task, the agent again has the problem of navigating to a place where a reward can be gained, but there is extra complexity in deciding the best plan. There are multiple different high reward end states and low reward end states. There are paths to the reward which are slow, but guarantee achieving the high reward, and there are other paths which are faster, but more risky.

Example MDP model

Probabilistic PDDL is one way to represent an MDP. It extends PDDL with a few additional constructs. Of most relevance is that outcomes can be associated with probabilities. The following describes the “Bomb and Toilet” problem, in which one of two packages contains a bomb. The bomb can be diffused by dunking it into a toilet, but there is a 0.05 probability of the bomb clogging the toilet.

```
(define (domain gridworld)

  (:requirements :conditional-effects :probabilistic-effects)

  (:predicates (bomb-in-package ?pkg) (toilet-clogged) (bomb-defused))

  (:action dunk-package
   :parameters (?pkg)
   :effect (and (when (bomb-in-package ?pkg) (bomb-defused))
                (probabilistic 0.05 (toilet-clogged))))
```

MDPs can also be expressed as code, rather than just as a model. An algorithm for solving the MDP creates an instance of a class and obtains the information that it requires to solve it.

Example MDP model as Python code: Grid World

First, we have an interface that defines what an MDP is:

```
class MDP:
    """ Return all states of this MDP """
    def get_states(self):
        abstract

    """ Return all actions with non-zero probability from this state """
    def get_actions(self, state):
        abstract

    """ Return all non-zero probability transitions for this action
        from this state, as a list of (state, probability) pairs
    """
    def get_transitions(self, state, action):
        abstract

    """ Return the reward for transitioning from state to
        nextState via action
    """
```

(continues on next page)

(continued from previous page)

```

def get_reward(self, state, action, next_state):
    abstract

    """ Return true if and only if state is a terminal state of this MDP """
def is_terminal(self, state):
    abstract

    """ Return the discount factor for this MDP """
def get_discount_factor(self):
    abstract

    """ Return the initial state of this MDP """
def get_initial_state(self):
    abstract

    """ Return all goal states of this MDP """
def get_goal_states(self):
    abstract

```

Then, we need to implement this interface to create an MDP. Below is the implementation for `getTransitions` and `getReward` for `GridWorld`:

```

class GridWorld(MDP):

    ...

def get_transitions(self, state, action):
    transitions = []

    if state == self.TERMINAL:
        if action == self.TERMINATE:
            return [(self.TERMINAL, 1.0)]
        else:
            return []

    # Probability of not slipping left or right
    straight = 1 - (2 * self.noise)

    (x, y) = state
    if state in self.get_goal_states().keys():
        if action == self.TERMINATE:
            transitions += [(self.TERMINAL, 1.0)]

    elif action == self.UP:
        transitions += self.valid_add(state, (x, y + 1), straight)
        transitions += self.valid_add(state, (x - 1, y), self.noise)
        transitions += self.valid_add(state, (x + 1, y), self.noise)

    elif action == self.DOWN:
        transitions += self.valid_add(state, (x, y - 1), straight)
        transitions += self.valid_add(state, (x - 1, y), self.noise)
        transitions += self.valid_add(state, (x + 1, y), self.noise)

    elif action == self.RIGHT:
        transitions += self.valid_add(state, (x + 1, y), straight)
        transitions += self.valid_add(state, (x, y - 1), self.noise)

```

(continues on next page)

(continued from previous page)

```

        transitions += self.valid_add(state, (x, y + 1), self.noise)

    elif action == self.LEFT:
        transitions += self.valid_add(state, (x - 1, y), straight)
        transitions += self.valid_add(state, (x, y - 1), self.noise)
        transitions += self.valid_add(state, (x, y + 1), self.noise)

    # Merge any duplicate outcomes
    merged = defaultdict(lambda: 0.0)
    for (state, probability) in transitions:
        merged[state] = merged[state] + probability

    transitions = []
    for outcome in merged.keys():
        transitions += [(outcome, merged[outcome])]

    return transitions

def valid_add(self, state, new_state, probability):
    # If the next state is blocked, stay in the same state
    if probability == 0.0:
        return []

    if new_state in self.blocked_states:
        return [(state, probability)]

    # Move to the next space if it is not off the grid
    (x, y) = new_state
    if x >= 0 and x < self.width and y >= 0 and y < self.height:
        return [(x, y), probability]

    # If off the grid, state in the same state
    return [(state, probability)]

def get_reward(self, state, action, new_state):
    reward = 0.0
    if state in self.get_goal_states().keys() and new_state == self.TERMINAL:
        reward = self.get_goal_states().get(state)
    else:
        reward = self.action_cost
    step = len(self.episode_rewards)
    self.episode_rewards += [reward * (self.discount_factor ** step)]
    return reward

```

Let's break this down into its parts:

- The state space s here is the location of the robot in Cartesian coordinates: (x, y) , where x is the column and y the row. The state space is the set of all possible coordinates from $(0, 0)$ to $(3, 2)$, as well as a special state called *Terminal* from which we cannot escape and transitioning to it has no reward. We can expand the state space to be larger.
- The initial state s_0 is the location $(0, 0)$ in the bottom left.
- The actions are *Up*, *Down*, *Left*, and *Right*, as well as the special action *Terminate*, which transitions into the *Terminal* state from a goal state.
- `getTransitions` defines the transition probabilities. For example, we can see that if the action is *Left*, then

there are three possible transitions: to the left with 0.8 probability, and either up or down with 0.1 probability each. The method `validAdd` ensures that if we agent cannot go in that direction due to a wall, it remains in the current state for its outcome.

- The reward is modelled as receiving +1 or -1 for exiting a goal state and transitioning into the terminal state. We will see later that this models our desire that the goal states have the value of 1 and -1 respectively.
- Finally, the discount factor is simply a parameter to the MDP in this code, with default value 0.9.

In later chapters, we will see how to use these code-based models in several ways, including both model-based and model-free methods.

1.2.1 MDPs vs deterministic search

You may have studied search algorithms previously, such as classical planning or heuristic search algorithms like A*

What is different between an MDP and search-based solutions? There are four main differences:

- The transition function is not deterministic. Each action has a probability of $P_a(s' | s)$ of ending in state s' if a is executed in the state s , whereas in classical planning, the outcome of each action is known in advance.
- There are no goal states. Each action receives a reward when applied. The value of the reward is dependent on the state in which it is applied.
- There are no action costs. Actions costs are modelled as negative rewards.
- We have a discount factor. In classical planning, executing an action typically has a cost. A discount factor of less than 1 implicitly rewards shorter plans, all other things being equal.

1.3 Policies

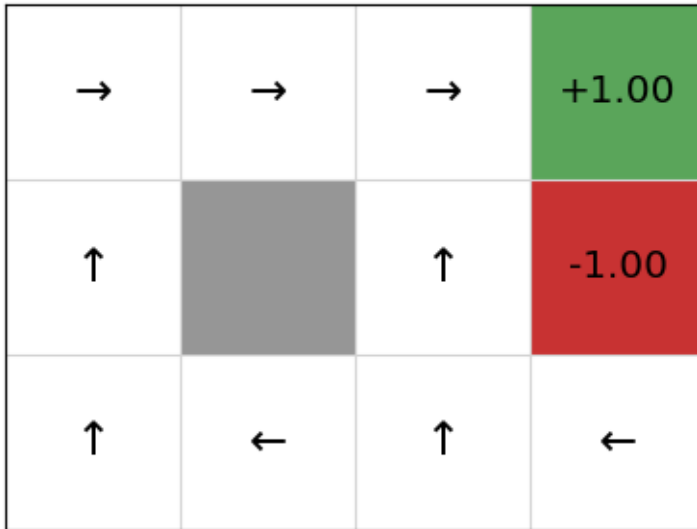
The planning problem for discounted-reward MDPs is different to that of classical planning or heuristic search because the actions are non-deterministic. Instead of a sequence of actions, an MDP produces a *policy*.

Definition – Policy

A **policy** π is a function that tells an agent which is the best action to choose in each state. A policy can be **deterministic** or **stochastic**.

1.3.1 Deterministic vs. stochastic policies

A **deterministic policy** $\pi : S \rightarrow A$ is a function that maps states to actions. It specifies which action to choose in every possible state. Thus, if we are in state s , our agent should choose the action defined by $\pi(s)$. A graphical representation of the policy for Grid World is:



So, in the initial state (bottom left cell), following this policy the agent should go up. If it accidentally slips right, it should go left again to return to the initial state.

Of course, agents do not work with graphical policies. The output from a planning algorithm would be a dictionary-like object or a function that takes a state and returns an action.

A **stochastic policy** $\pi : S \times A \rightarrow \mathbb{R}$ specifies the **probability distribution** from which an agent should select an action. Intuitively, $\pi(s, a)$ specifies the probability that action a should be executed in state s .

To execute a stochastic policy, we could just take the action with the maximum $\pi(s, a)$. However, in some domains, it is better to select an action based on the probability distribution; that is, choose the action probabilistically such that actions with higher probability are chosen proportionally to their relative probabilities.

We will focus mostly on deterministic policies, but stochastic policies have their place when we discuss *policy gradient methods*.

1.3.2 Representing policies

Policies can be represented in several ways, but all have the same basic interface: the ability to update the policy and the ability to get an action for a state (in a deterministic policy) or get the value or probability of playing an action (in a stochastic policy):

```
class Policy:
    def select_action(self, state, action):
        abstract

class DeterministicPolicy(Policy):
    def update(self, state, action):
        abstract

class StochasticPolicy(Policy):
    def update(self, states, actions, rewards):
        abstract
```

(continues on next page)

(continued from previous page)

```
def get_probability(self, state, action):  
    abstract
```

The simplest way to represent a policy is a tabular policy, which keeps a table that maps from each state to the action for that state. We implement this as a dictionary in Python:

```
import random  
from collections import defaultdict  
from policy import DeterministicPolicy  
  
class TabularPolicy(DeterministicPolicy):  
    def __init__(self, default_action=None):  
        self.policy_table = defaultdict(lambda: default_action)  
  
    def select_action(self, state, actions):  
        return self.policy_table[state]  
  
    def update(self, state, action):  
        self.policy_table[state] = action
```

As we see later in the section on *policy gradients*, policies can be represented using other means, such as machine learning models, which do not require us to keep an explicit answer for every state.

1.4 Optimal Solutions for MDPs

For discounted-reward MDPs, optimal solutions maximise the **expected discounted accumulated reward** from the initial state s_0 . But what is the expected discounted accumulated reward?

Definition – Expected discounted reward

The **expected discounted reward** from s for a policy π is:

$$V^\pi(s) = E_\pi \left[\sum_i \gamma^i r(s_i, a_i, s_{i+1}) \mid s_0 = s, a_i = \pi(s_i) \right]$$

So, $V^\pi(s)$ defines the expected value of following the policy π from state s .

For our Grid World example, assuming only the -1 and +1 states have rewards, the expected value is:

$$\begin{aligned} & \gamma^5 \times 1 \times (0.8^5) && \text{(optimal movement)} \\ + & \gamma^7 \times 1 \times (0.8^7) && \text{(first move only fails)} \\ + & \dots \text{(etc.)} \end{aligned}$$

Definition – Bellman equation

The **Bellman equation**, identified by Richard Bellman, describes the condition that must hold for a policy to be optimal. The Bellman equation is defined recursively as:

$$V(s) = \max_{a \in A(s)} \sum_{s' \in S} P_a(s' \mid s) [r(s, a, s') + \gamma V(s')]$$

Therefore, V is optimal **if** for all states s , $V(s)$ describes the total discounted reward for taking the action with the highest reward over an indefinite/infinite horizon.

Let's break this down:

$$V(s) = \overbrace{\max_{a \in A(s)}}^{\text{best action from } s} \overbrace{\sum_{\substack{s' \in S \\ \text{for every state}}} P_a(s' | s) [\underbrace{r(s, a, s')}_{\text{immediate reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{V(s')}_{\text{value of } s'}]}_{\text{expected reward of executing action } a \text{ in state } s}$$

First, we calculate the expected reward for each action. The reward of an action is: the sum of the immediate reward for all states possibly resulting from that action plus the discounted future reward of those states. The discounted future reward is the γ (discount reward) times the value of s' , where s' is the state that we end up in. However, because we can end up in multiple states, we must multiple the reward by the probability of it happening: $P_a(s' | s)$.

Second, the value $V(s)$ is the value of the action with the maximum the expected reward. This is because the Bellman equation assumes that once we know the best states, we will always take the action that leads to the best state.

Bellman equations can be described slightly differently, using what are known as Q -values.

If $V(s)$ is the expected value of being in state s and acting optimally according to our policy, then we can also describe the Q -value of being in a state s , choosing action a and then acting optimally according to our policy as.

Definition – Q-value

The **Q-value** for action a in state s is defined as:

$$Q(s, a) = \sum_{s' \in S} P_a(s' | s) [r(s, a, s') + \gamma V(s')]$$

This represents the value of choosing action a in state s and then following this same policy until termination.

This is just the expression inside the max expression in the Bellman equation. Using this, we can then Bellman equation then defined as:

$$V(s) = \max_{a \in A(s)} Q(s, a)$$

The two definitions are equivalent, and you may seem them defined in both ways. However, when we move onto Q-learning later, we will use Q -values more explicitly.

1.5 Policy extraction

Given a value function V , how should we then select the action to play in a given state? It is reasonably straightforward: select the action that maximises our expected utility!

So, if the value function V is optimal, we can select the action with the highest expected reward using:

$$\pi(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s' \in S} P_a(s' | s) [r(s, a, s') + \gamma V(s')]$$

This is known as **policy extraction**, because it extracts a policy for a value function (or Q-function). This can be calculated 'on the fly' at runtime, or we can extract a policy beforehand and use this.

Alternatively, given a Q-function instead of a value function, we can use:

$$\pi(s) = \operatorname{argmax}_{a \in A(s)} Q(s, a)$$

This is simpler than using the value functions because we do not need to sum over the set of possible output states, but we need to store $|A| \times |S|$ values in a Q-function, but just $|S|$ values in a value function.

1.5.1 Implementation

Policy extraction takes a value function and extracts a tabular policy. In this implementation, we extract a tabular policy using policy extraction from a value function:

```
from tabular_policy import TabularPolicy

class ValueFunction():

    def update(self, state, value):
        abstract

    def merge(self, value_table):
        abstract

    def get_value(self, state):
        abstract

    """ Return the Q-value of action in state """
    def get_q_value(self, mdp, state, action):
        q_value = 0.0
        for (new_state, probability) in mdp.get_transitions(state, action):
            reward = mdp.get_reward(state, action, new_state)
            q_value += probability * (
                reward
                + (mdp.get_discount_factor() * self.get_value(new_state))
            )

        return q_value

    """ Return a policy from this value function """

    def extract_policy(self, mdp):
        policy = TabularPolicy()
        for state in mdp.get_states():
            max_q = float("-inf")
            for action in mdp.get_actions(state):
                q_value = self.get_q_value(mdp, state, action)

                # If this is the maximum Q-value so far,
                # set the policy for this state
                if q_value > max_q:
                    policy.update(state, action)
                    max_q = q_value

        return policy
```

For each state, we find the best action in the state (the action that maximises the Q-value from that state); and for that state, we set that max action as the action to select in that state.

1.6 Partially Observable MDPs

MDPs assume that the agent always knows exactly what state it is in — the problem is fully-observable. However, this is not valid for many tasks; e.g. an unmanned aerial vehicle searching in a earthquake zone for survivors will by definition not know the location of survivors; a card-playing agent playing solitaire will not know the cards that are face-down;, etc.

Definition – Partially-observable MDP

Partially-observable MDPs (POMDPs) relax the assumption of full-observability. A POMDP is defined as:

- states $s \in S$
 - actions $A(s) \subseteq A$
 - transition probabilities $P_a(s' | s)$ for $s \in S$ and $a \in A(s)$
 - initial **belief state** b_0
 - reward function $r(s, a, s')$
 - a set of possible observations Obs
 - a **sensor model** given by probabilities $O_a(o | s)$, $o \in Obs$
-

The sensor model allows the agent to observe the environment. If an agent executes an action a , it has probability $O_a(o | s')$ of observing state s' .

Solving POMDPs is similar to solving MDPs. In fact, the same algorithms can be applied. The only difference is that we case the POMDP problem as a standard MDP problem with a new state space: each state is a **probability distribution** over the set S . Thus, each state of the POMDP is a **belief state**, which defined the probability of being in each state S . This leads to an exponentially-larger state space, so POMDPs are typically harder problems to solve.

Like MDPs, solutions are policies that map belief states into actions. Optimal policies maximise the expected reward.

We will not cover this in detail in these notes. However, POMDPs are a generalisation of MDPs, and they are more suited to practical solutions in planning for autonomy than standard MDPs because it is unusual to always know the true state of the world in which an agent is acting.

1.7 Takeaways

Takeaways

- **Markov Decision Processes** (MDPs) model sequential decision-making problems in which the outcome of an action is stochastic; although the agent can observe the state once the action is executed.
 - MDPs can be defined using code, simulators, or models.
 - The solution to an MDP is a **policy**.
 - A **deterministic policy** is a mapping from states to actions. For policy π , the term $\pi(s)$ returns the action to execute in state s .
 - A **stochastic policy** is a mapping from state actions pairs to probabilities. For policy π , the term $\pi(s, a)$ returns the probability of selecting action a in state s .
 - An **optimal policy** maximises the expected discounted reward.
 - The **Bellman equation** describes the condition that must hold for a policy to be optimal.
-

VALUE-BASED METHODS

Techniques for solving MDPs (and POMDPs) can be separated into three categories:

1. **Value-based** techniques aim to learn the value of states (or learn an estimate for value of states) and actions: that is, they learn value functions or Q functions. We then use policy extraction to get a policy for deciding actions.
2. **Policy-based** techniques learn a policy directly, which completely by-passes learning values of states or actions all together. This is important if for example, the state space or the action space are massive or infinite. If the action space is infinite, then using policy extraction as defined in Part I is not possible because we must iterate over all actions to find the optimal one. If we learn the policy directly, we do not need this.
3. **Hybrid** techniques that combine value- and policy-based techniques.

In this chapter, we investigate some of the foundational techniques for value-based reinforcement learning, and follow up with policy-based and hybrid techniques in the follow chapters.

2.1 Value Iteration

Learning outcomes

The learning outcomes of this chapter are:

1. Apply value iteration to solve small-scale MDP problems manually and program value iteration algorithms to solve medium-scale MDP problems automatically
 2. Construct a policy from a value function
 3. Discuss the strengths and weaknesses of value iteration
-

2.1.1 Overview

Value Iteration is a method for finding the optimal value function V^* by solving the Bellman equations iteratively. It uses the concept of dynamic programming to maintain a value function V that approximates the optimal value function V^* , iteratively improving V until it converges to V^* (or close to it).

2.1.2 Algorithm

Once we understand the Bellman equation, the value iteration algorithm is straightforward: we just repeatedly calculate V using the Bellman equation until we converge to the solution or we execute a pre-determined number of iterations.

Algorithm 1 (Value iteration)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Output : Value function V

Set V to arbitrary value function; e.g., $V(s) = 0$ for all s

repeat

$\Delta \leftarrow 0$

for each $s \in S$

$V'(s) \leftarrow \max_{a \in A(s)} \underbrace{\sum_{s' \in S} P_a(s' | s) [r(s, a, s') + \gamma V(s')]}_{\text{Bellman equation}}$

$\Delta \leftarrow \max(\Delta, |V'(s) - V(s)|)$

$V \leftarrow V'$

until $\Delta \leq \theta$

As we can see, this is just applying the Bellman equation iteratively until either the value function V doesn't change anymore, or until it changes in by a very small amount (θ).

We could also write the algorithm using the idea of Q-values, which is closer to a code-based implementation. For this, the loop is:

$\Delta \leftarrow 0$

for each $s \in S$

for each $a \in A(s)$

$Q(s, a) \leftarrow \sum_{s' \in S} P_a(s' | s) [r(s, a, s') + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |\max_{a \in A(s)} Q(s, a) - V(s)|)$

$V(s) \leftarrow \max_{a \in A(s)} Q(s, a)$

Value iteration converges to the optimal policy as iterations continue: $V \mapsto V^*$ as $i \mapsto \infty$, where i is the number of iterations. So, given an infinite amount of iterations, it will be optimal.

Value iteration **converges** to the optimal value function V^* asymptotically, but in practice, the algorithm terminates when the **residual** Δ reaches some pre-determined threshold θ – that is, when the largest change in the values between iterations is “small enough”.

A policy can now be easily defined: in a state s , given V , choose the action with the highest expected reward using *policy extraction*. The loss of the result greedy policy is bound by $\frac{2\gamma\Delta}{1-\gamma}$.

Note that we do not need an optimal value function V to obtain an optimal policy. A value function that is “close enough” can still give an optimal policy because the small values do not change the resulting policy. Of course, we would not *know* whether a policy is optimal unless we know the value function is optimal.

2.1.3 Implementation

Below is a Python implementation for value iteration. In this implementation, the parameter `iterations` is the number of iterations around the loop, which will terminate before convergence is the maximum number of iterations is reach. The parameter `theta` is θ in the value iteration algorithm above. Once the difference (Δ) is less than `theta`, the loop will terminate.

```
from tabular_value_function import TabularValueFunction
from qtable import QTable

class ValueIteration:
    def __init__(self, mdp, values):
        self.mdp = mdp
        self.values = values

    def value_iteration(self, max_iterations=100, theta=0.001):

        for i in range(max_iterations):
            delta = 0.0
            new_values = TabularValueFunction()
            for state in self.mdp.get_states():
                qtable = QTable()
                for action in self.mdp.get_actions(state):
                    # Calculate the value of Q(s,a)
                    new_value = 0.0
                    for (new_state, probability) in self.mdp.get_transitions(
                        state, action
                    ):
                        reward = self.mdp.get_reward(state, action, new_state)
                        new_value += probability * (
                            reward
                            + (
                                self.mdp.get_discount_factor()
                                * self.values.get_value(new_state)
                            )
                        )

                    qtable.update(state, action, new_value)

                # V(s) = max_a Q(sa)
                (_, max_q) = qtable.get_max_q(state, self.mdp.get_actions(state))
                delta = max(delta, abs(self.values.get_value(state) - max_q))
                new_values.update(state, max_q)

            self.values.merge(new_values)

        # Terminate if the value function has converged
        if delta < theta:
            return i
```

Example: Value iteration for GridWorld

Given this, we can create a GridWorld MDP, and solve using value iteration. The code below computes a value function using value iteration for 100 iterations:

```
from gridworld import GridWorld
from value_iteration import ValueIteration
from tabular_value_function import TabularValueFunction
from value_policy import ValuePolicy
from stochastic_value_policy import StochasticValuePolicy
```

```
gridworld = GridWorld()
values = TabularValueFunction()
```

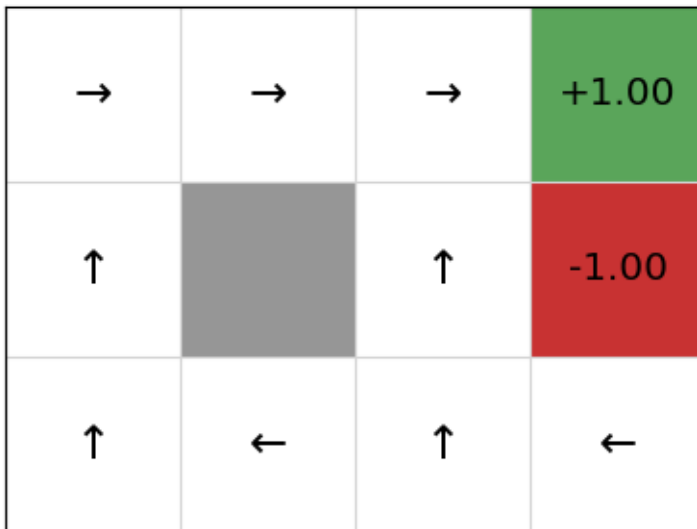
```
ValueIteration(gridworld, values).value_iteration(max_iterations=100)
gridworld.visualise_value_function(values, "Value function after iteration 100")
```



From the value function, we extract a policy:

```
policy = ValuePolicy(gridworld, values)
gridworld.visualise_policy(policy, "Policy after iteration 100")
```

Policy after iteration 100



Stepping through the 100 iterations, we can see that using value iteration, the values converge within about 10 iterations (to two decimal places), with each iteration giving us diminishing returns:

```
gridworld = GridWorld()
for iterations in [0, 1, 2, 3, 4, 5, 10, 100]:
    values = TabularValueFunction()
    ValueIteration(gridworld, values).value_iteration(max_iterations=iterations)
    gridworld.visualise_value_function(values,
                                       "After iteration %s" % (iterations),
                                       grid_size=0.75)
```

After iteration 0



After iteration 1

+0.00	+0.00	+0.00	+1.00
+0.00		+0.00	-1.00
+0.00	+0.00	+0.00	+0.00

After iteration 2

+0.00	+0.00	+0.72	+1.00
+0.00		+0.00	-1.00
+0.00	+0.00	+0.00	+0.00

After iteration 3

+0.00	+0.52	+0.78	+1.00
+0.00		+0.43	-1.00
+0.00	+0.00	+0.00	+0.00

After iteration 4

+0.37	+0.66	+0.83	+1.00
+0.00		+0.51	-1.00
+0.00	+0.00	+0.31	+0.00

After iteration 5

+0.51	+0.72	+0.84	+1.00
+0.27		+0.55	-1.00
+0.00	+0.22	+0.37	+0.13

After iteration 10

+0.64	+0.74	+0.85	+1.00
+0.56		+0.57	-1.00
+0.48	+0.41	+0.47	+0.27

After iteration 100

+0.64	+0.74	+0.85	+1.00
+0.57		+0.57	-1.00
+0.49	+0.43	+0.48	+0.28

2.1.4 Evaluating policies

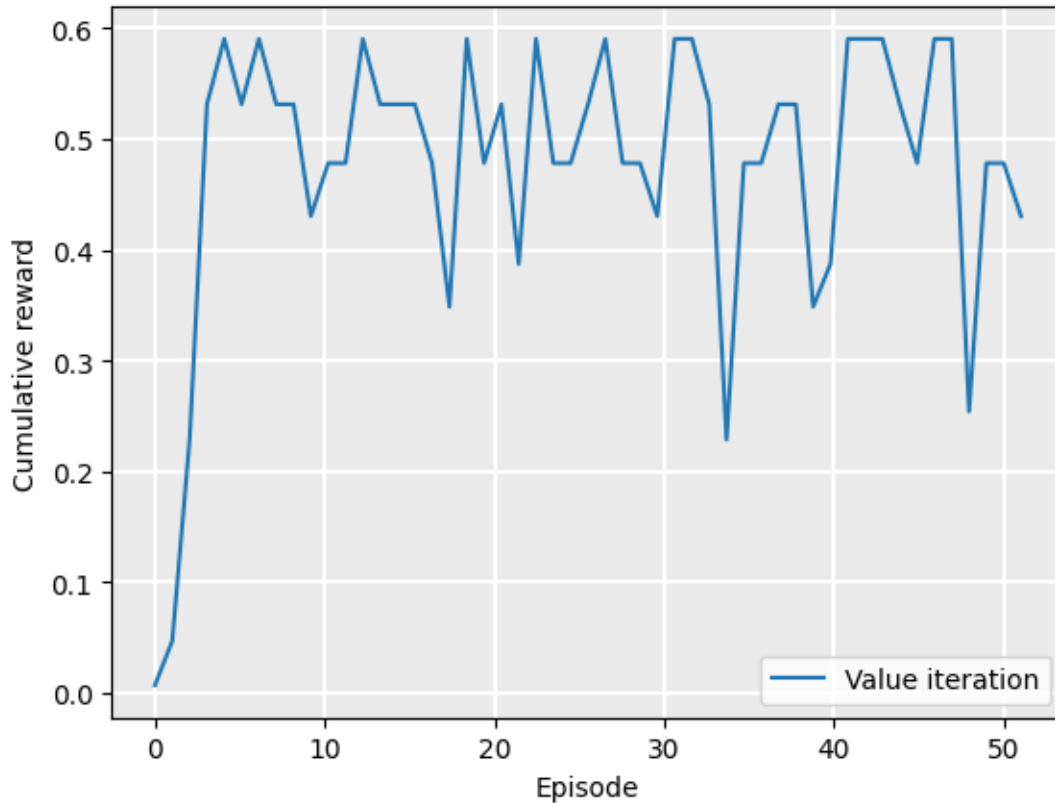
We can see the improvement that value iteration has on each iteration by extracting the policy after each iteration, running the policy on the GridWorld, and plotting the cumulative reward that is received. We run value iteration on GridWorld for 1 iteration, but 50 times, using the same value function each time, meaning the the value iteration algorithm will update the value function. After each iteration, we extract the policy and execute the policy for 1 episode, recording the cumulative reward that was received:

```
gridworld = GridWorld()
values = TabularValueFunction()
policy = ValuePolicy(gridworld, values)
rewards = gridworld.execute_policy(policy, episodes=1)
for _ in range(50):
    ValueIteration(gridworld, values).value_iteration(max_iterations=1)
    policy = ValuePolicy(gridworld, values)
    rewards += gridworld.execute_policy(policy, episodes=1)
```

The rewards variables contains the cumulative reward for each of the 50 episodes. We can plot this:

```
from tests.plot import Plot

Plot.plot_cumulative_rewards(["Value iteration"], [rewards], smoothing_factor=0.0)
```



As we can see, there is quite a bit of noise. This is because of the randomness in the GridWorld problem: each time an agent executes an action, it is successful 80% of the time, but fails 20%. Sometimes, the agent will fall into the hole in cell (2,1), and will receive reward -1.

However, we can smooth out the noise by plotting the exponential moving average (EMA), which averages the rewards received so far, but exponentially discounting rewards that are further in the past (earlier in the list). This is a standard way of smoothing time series data. Given a **smoothing factor** $\alpha \in [0, 1]$, we calculate the smoothed value s_t at step (or time) t , the EMA for a sequence $\vec{x}$ is calculated:

$$s_0 = \vec{x}_0$$

$$s_t = (1 - \alpha) \cdot \vec{x}_t + \alpha \cdot s_{t-1} \quad \text{when } t > 0$$

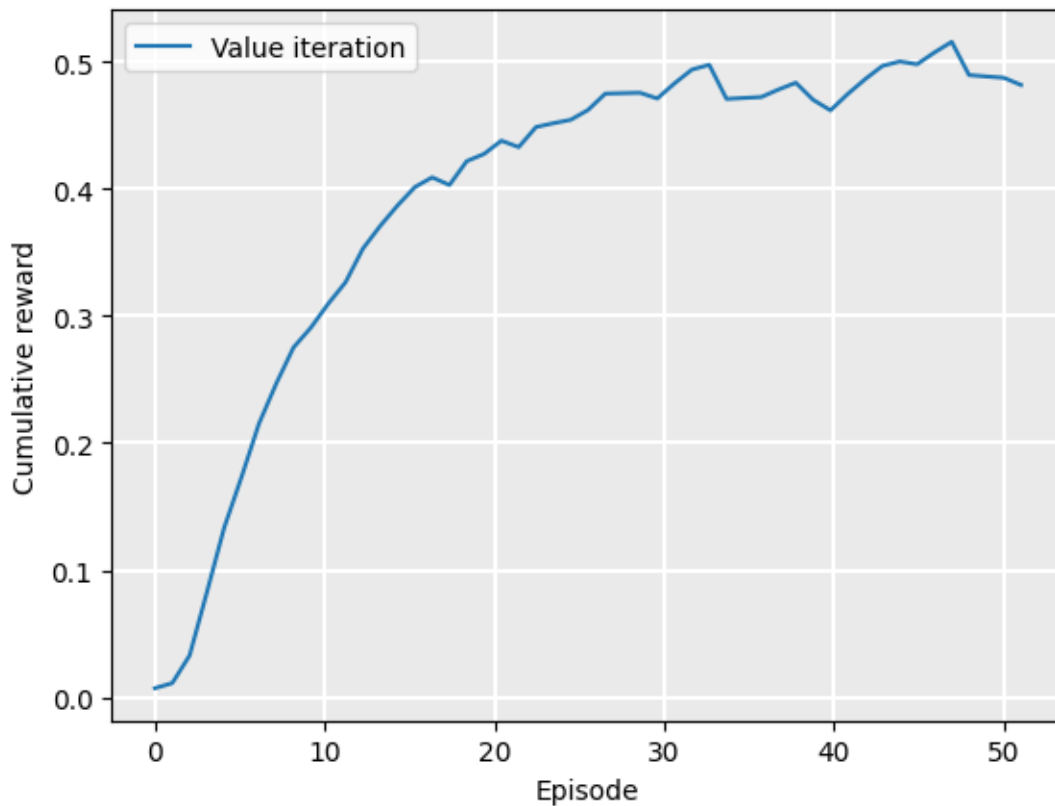
A higher value of α smooths the data more — that is, earlier values have more influence on s_t . A value of $\alpha = 0$ just means that $s_t = \vec{x}_t$ (see the example above where we plot the cumulative rewards with `smoothing_factor=0.0`); while a value of $\alpha = 1$ means that $s_t = \vec{x}_0$ for any t .

An implementation of the EMA in Python is shown below:

```
def get_ema(rewards, smoothing_factor=0.9):
    smoothed_rewards = []
    for reward in rewards:
        if smoothed_rewards == []:
            smoothed_rewards = [reward]
        else:
            smoothed_rewards += [
                smoothed_rewards[-1] * smoothing_factor
                + reward * (1 - smoothing_factor)
            ]
    return smoothed_rewards
```

Setting the smoothing factor $\alpha = 0.9$, we can better see the trend that the policy gets increasingly better values:

```
Plot.plot_cumulative_rewards(["Value iteration"], [rewards], smoothing_factor=0.9)
```



Even though the value function (and policy) are monotonically converging towards their optimal values, the curve still has some noise because of randomness in the GridWorld problem.

We can show this by creating an instance of a deterministic GridWorld class, using the parameter `noise=0.0`. The `noise` parameter controls action failures in GridWorld, so that when the agent selects an action, it will fail with probability `noise`. When `noise==0.0`, the action will always be successful:

```
values = TabularValueFunction()
gridworld = GridWorld(noise=0.0)
policy = StochasticValuePolicy(gridworld, values)
rewards = gridworld.execute_policy(policy, episodes=1)
for _ in range(50):
    ValueIteration(gridworld, values).value_iteration(max_iterations=1)
    policy = StochasticValuePolicy(gridworld, values)
    rewards += gridworld.execute_policy(policy, episodes=1)
```

Note: Compared to when we evaluated the policy earlier, there is a difference in the code above (other than `noise=0.0`): when use a `StochasticValuePolicy`, which uses a *multi-armed bandit* to sometimes (5% of the time) selects a random action instead of the action that leads to the best value.

Why do this?

Because a policy can contain loops! That is, in state s_1 , the policy tells us to execute action a_1 , taking us to state s_2 . Then from state s_2 , the policy tells us to execute action a_2 , which takes us back to state s_1 . Again, the policy will tell us to

execute action a_1 , return to state s_2 :

$$\dots s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots$$

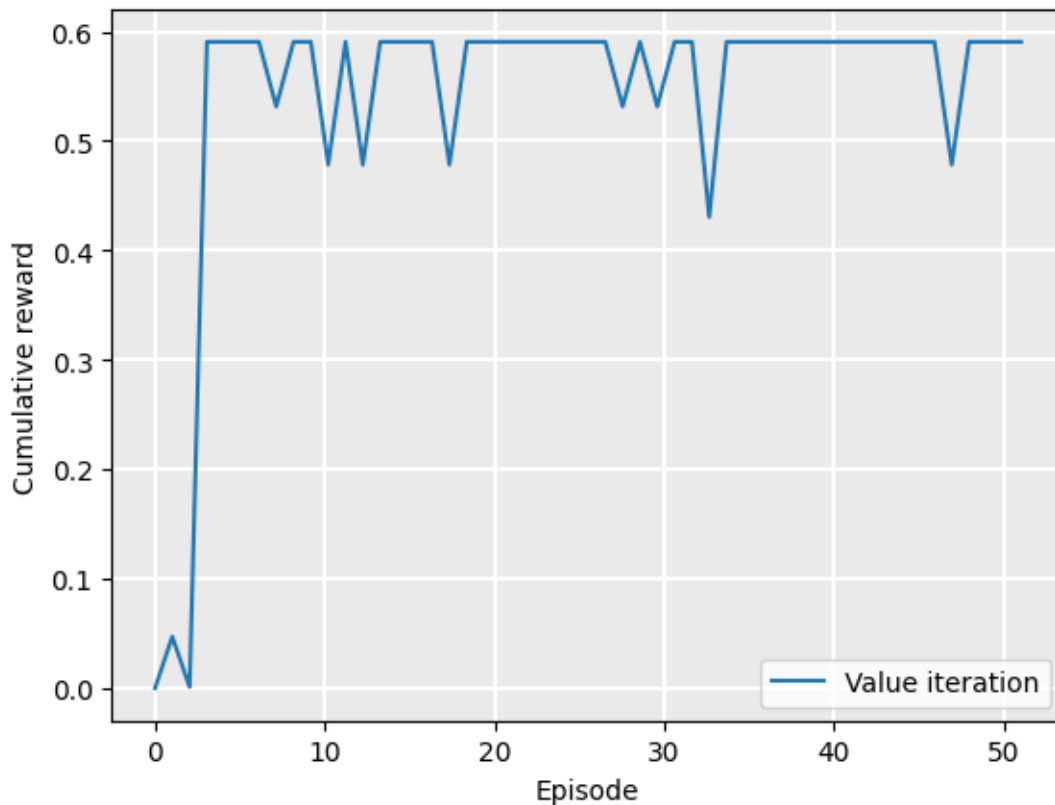
This will loop infinitely long when: (a) the policy is deterministic (not stochastic); and (b) the environment actions are deterministic (not stochastic).

When `noise>0.0` in GridWorld, a loop does not matter — eventually the randomness will break us out of this loop.

However, when `noise=0.0`, we can end up in an infinite loop. This is particularly the case in early iterations, when we have minimal information to select actions. By sometimes selecting a random action, we will eventually exit the loop.

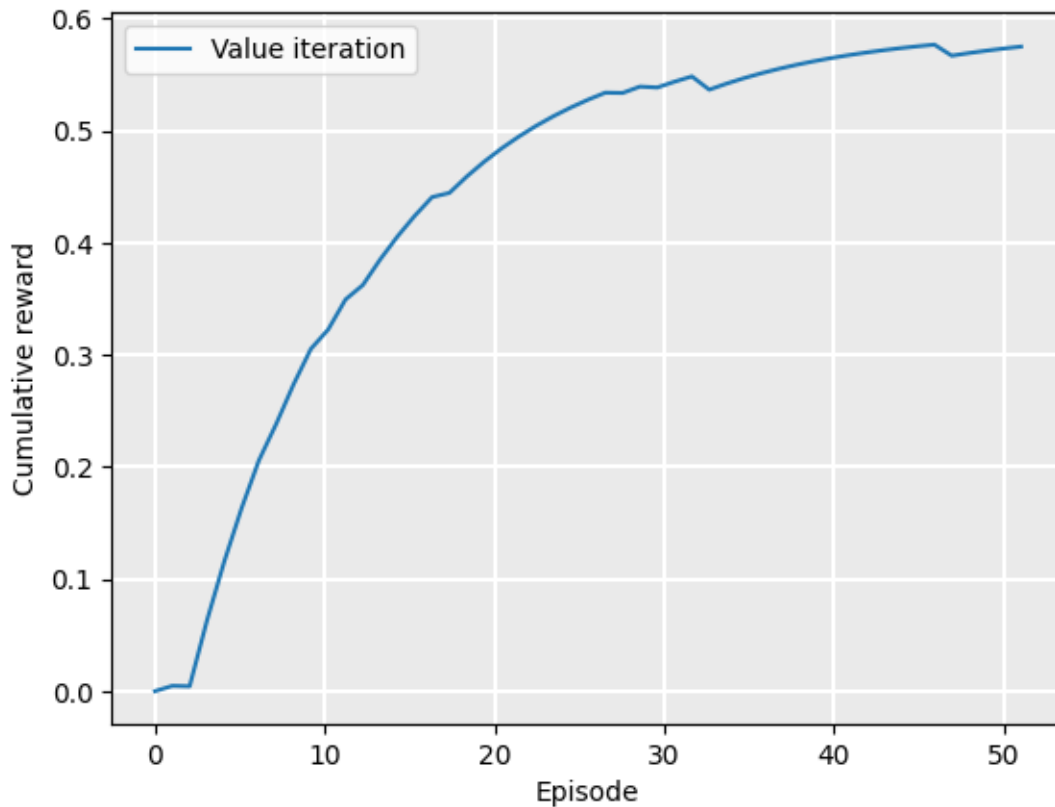
Plotting this with no smoothing shows that we reach the optimal policy in much fewer iterations than value iteration terminates:

```
Plot.plot_cumulative_rewards(["Value iteration"], [rewards], smoothing_factor=0.0)
```



Plotting this with smoothing at $\alpha = 0.9$ shows the gradual convergence nicely:

```
Plot.plot_cumulative_rewards(["Value iteration"], [rewards], smoothing_factor=0.9)
```



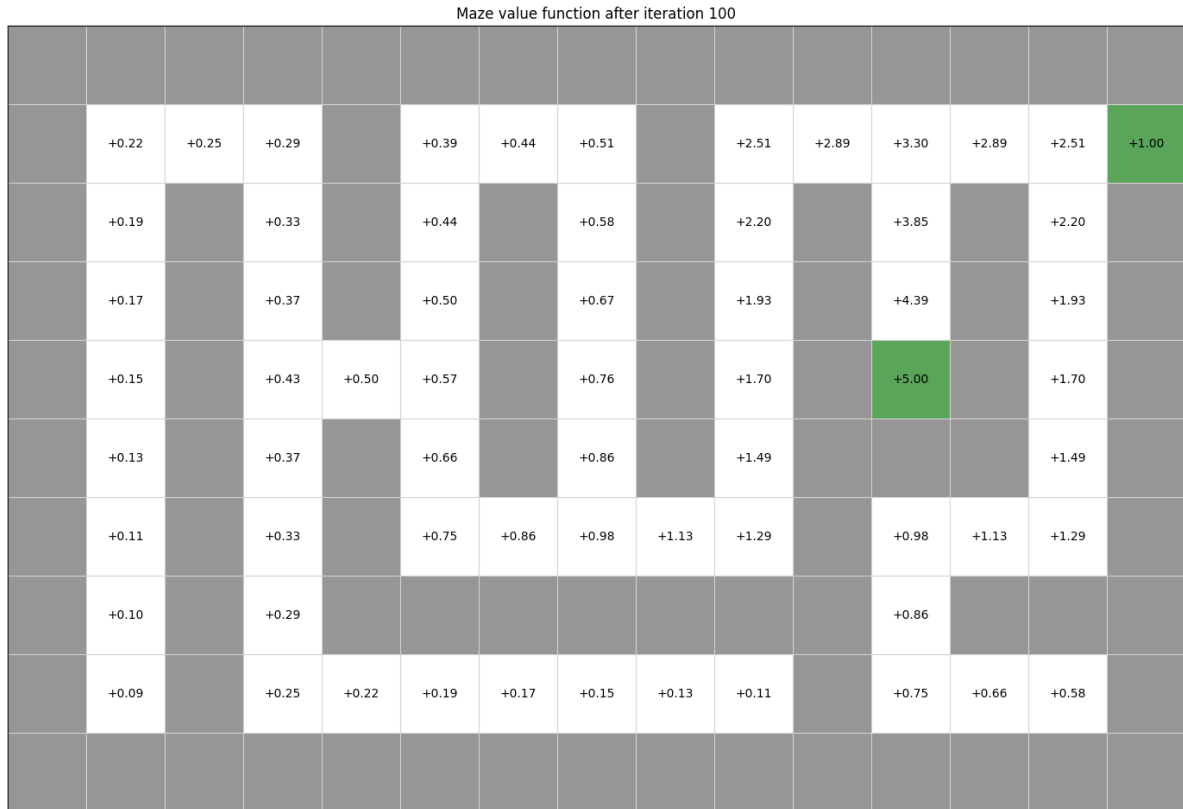
2.1.5 Further examples

Example: Value iteration for maze solving

We can use the same technique to solve a maze. Below is the value function, obtained using value iteration, of a small maze with two rewards: +1 for existing the maze, and +5 for picking up an object along the way. It is not possible to visualise the value function in a single static image like this, because one part of the MDP state is a Boolean indicating whether the +5 reward has been collected already. However, we can see here the value function would produce a policy that ensure the +5 reward was collected before going to the exit:

```
from gridworld import GridWorld
from value_iteration import ValueIteration
from tabular_value_function import TabularValueFunction

maze = GridWorld.open('../python_code/layouts/maze.txt')
values = TabularValueFunction()
ValueIteration(maze, values).value_iteration(max_iterations=100)
maze.visualise_value_function(values, "Maze value function after iteration 100")
```



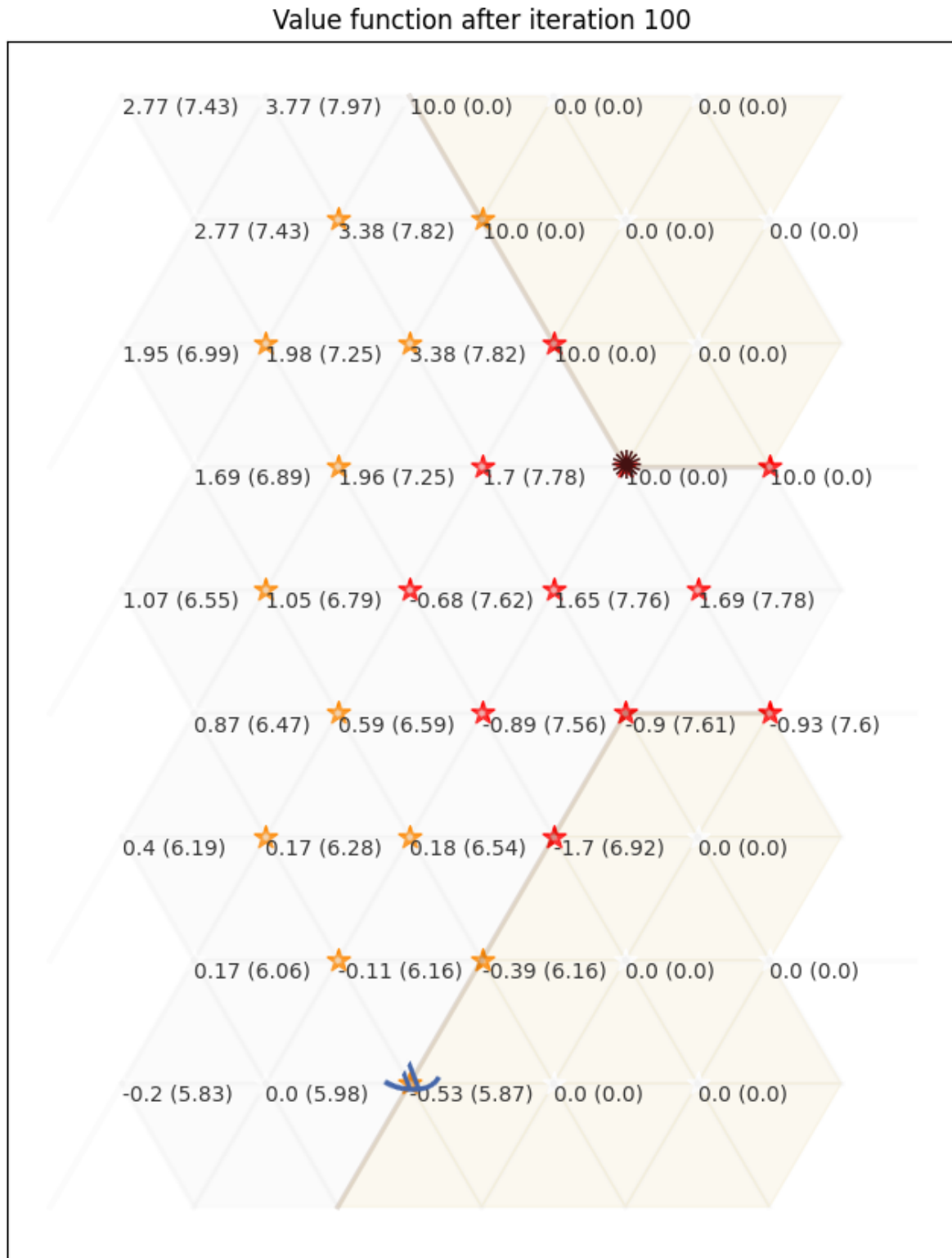
Example: Value iteration in Contested Crossing

The process works in exactly the same way for the Contested Crossing problem. However, in this case, the state space is no longer represented only by the coordinates of the agent. We also have to take into account non-locational information - damage to the ship, damage to the enemy, and the previous direction of travel - since these features will also affect the likelihood of the ship reaching its goal.

This can be difficult to visualise graphically. We can no longer simply assign one value to each physical location and so map the progress of the agent from each value to the largest adjacent one. We can, however, use other ways to get a general sense of how the state affects the behaviour of an agent at any one point. For instance, we can show the mean and standard deviation values for all states that are at a single location, as follows

```
from contested_crossing import ContestedCrossing
from value_iteration import ValueIteration
from tabular_value_function import TabularValueFunction

ccross = ContestedCrossing()
values = TabularValueFunction()
ValueIteration(ccross, values).value_iteration(max_iterations=100)
ccross.visualise_value_function(
    values, "Value function after iteration 100", mode=3, cell_size=1.25
)
```



In the above, the value shown is NOT the value of the state at that location, because there are multiple states in the MDP that contain that location. Other factors, such as the ship and enemy health, are average into the figure.

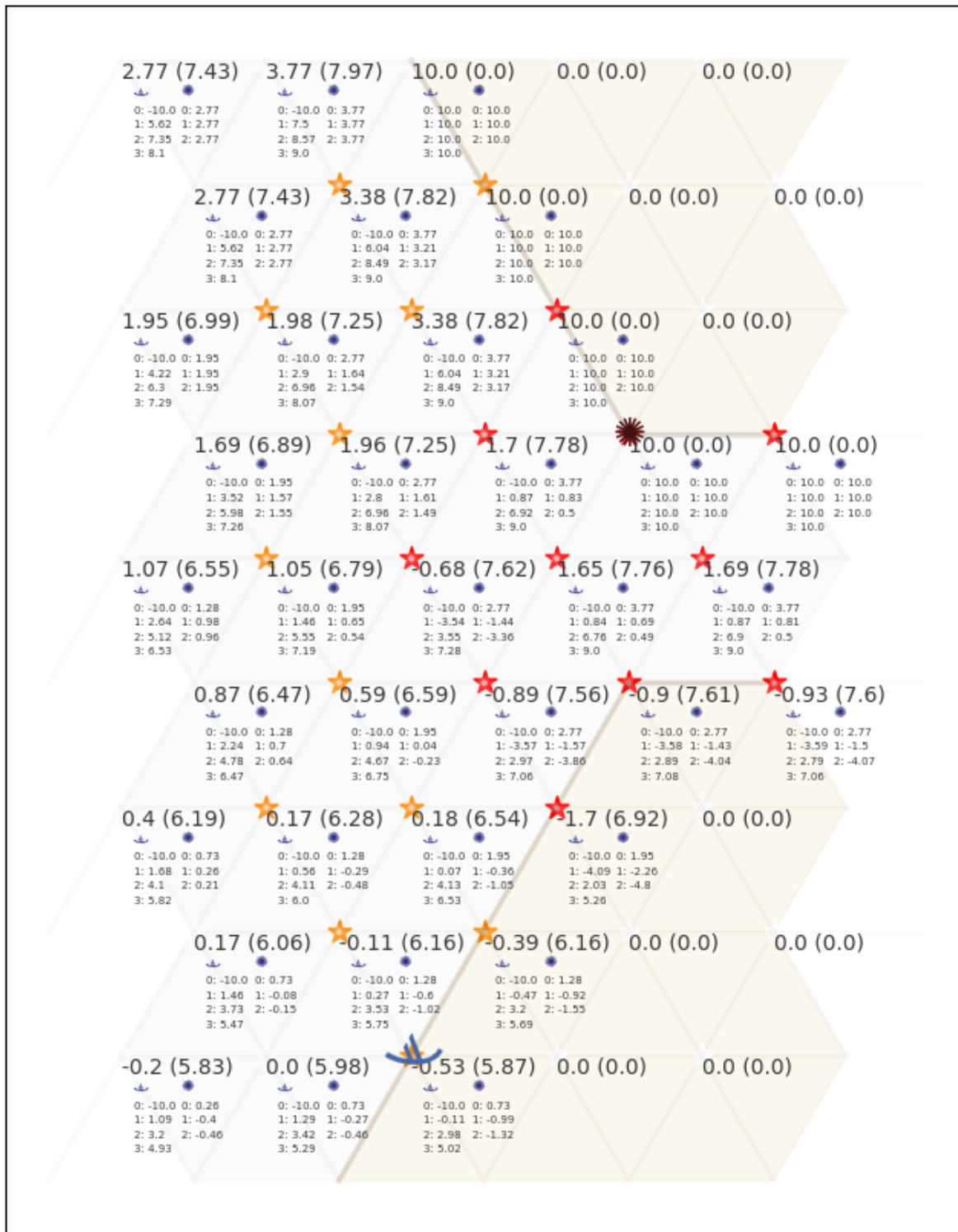
An agent simply traversing the map to each successive location with the highest mean value of all states would choose the

safe path - heading west to the nearest no-danger location, traversing around the safe zones and only briefly cutting back in through the low-danger zones at the end.

Below, we include a visualisation that includes means for different values of ship health and enemy health (the two smaller columns). In this, there are two columns associated with each location: one for the ship and one for the enemy. Each column shows the mean value of $V(s)$ if the ship (enemy) has the corresponding health value. So, in the top left location, if the ship has health value of 0, the average value of the state is -10.0.

```
ccross.visualise_value_function(  
    values, "Value function after iteration 100, with sub-tables", mode=0, cell_  
    size=1.25  
)
```

Value function after iteration 100, with sub-tables



The behaviour we could extract from this shows a more complicated picture than if we just consider the mean value of the states at the location. In states where the ship has full health the highest value first move is north-west and subsequent highest-value moves are all north-east, straight to the opposite shore. However, where the ship has sustained damage (health values 1 and 2) it is more likely that high-value states are found in the low-danger and no-danger areas, meaning

that the ship will choose a safer path.

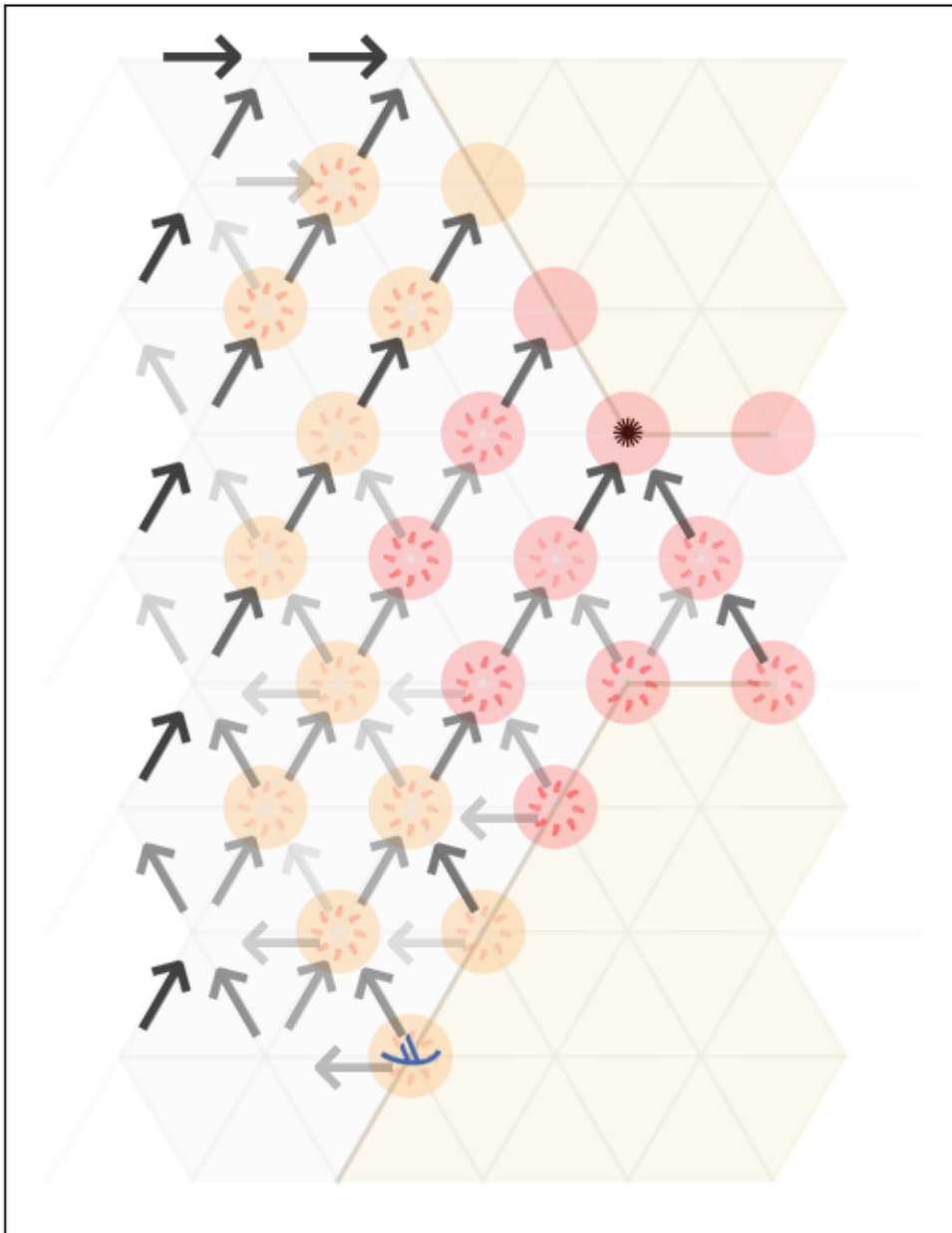
Another way to visualise this is by aggregate policy plot or path plot of actual uses of the policy. An aggregate policy plot (aggregating at each location over all states which include that location) shows which policies may be preferred at which locations, for different full state values. The opacity of each arrow (or of the starburst which represents the 'shoot' action) shows how many different states have the action as policy. This is different from a stochastic policy, because the choice of policy is not based on probabilities. This is simply a representation of the fact that multiple states are visualised at one location because the problem is complex enough that visualising individual states is difficult.

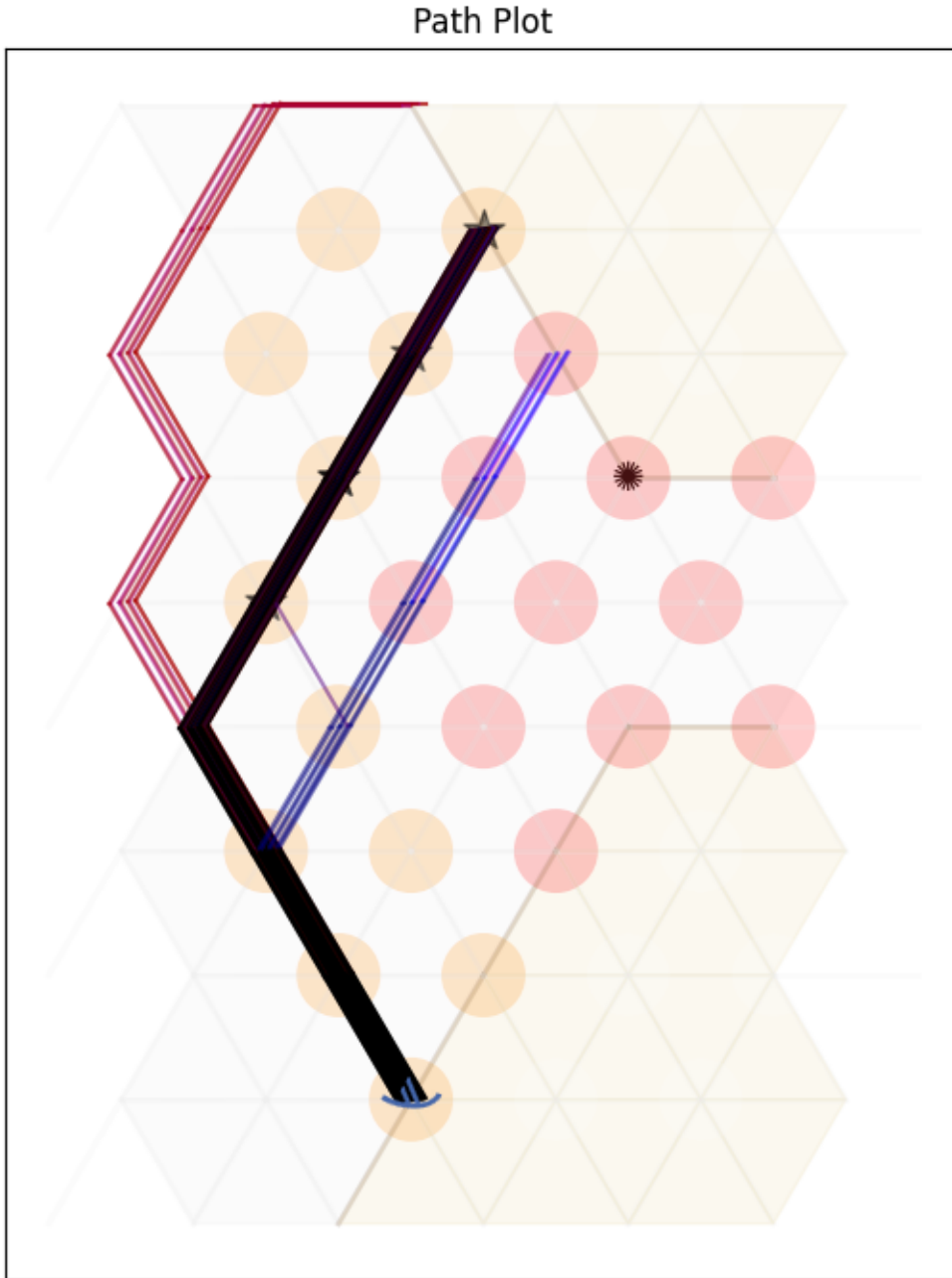
The path plot gives a number of episodes of the actual movement of an agent following the policy, over multiple iterations. Since the outcome of 'shoot' actions is random (as is the outcome of being shot at by the enemy), different paths may be taken by agents following the same policy. In this plot, the path colour becomes more red for higher values of ship damage (lower values of ship health) and becomes more blue for higher values of enemy damage. In this way we can see how the results of following the policy change depending on random outcomes during the operation of the agent. Agents that have been damaged tend to follow the safer path round the outside of the map while agents that have damaged the enemy (to the extent of destroying it completely) head straight for the nearest shore. Occasionally an agent is sunk (represented by a black star) - although the policy is optimal on average, this does not guarantee success at every iteration.

```
policy = ValuePolicy(ccross, values)

ccross.visualise_as_image(policy=policy, title="Policy Plot", mode=0, plot=True)
ccross.visualise_as_image(policy=policy, title="Path Plot", mode=1, plot=True)
```

Policy Plot





2.1.6 Complexity

The complexity of each iteration is $O(|S|^2|A|)$. On each iteration, we iterate in an outer loop over all states in S , and in each outer loop iteration, we need to iterate over all states ($\sum_{s' \in S}$), meaning $|S|^2$ iterations. But also within each outer loop iteration, we need to calculate the value for every action to find the maximum.

It is clear to see that the value iteration can be easily parallelised by updating the value of many states at once: the values of states at step $t + 1$ are dependent only on the value of other states at step t .

2.1.7 Takeaways

Takeaways

- Value iteration is an algorithm for calculating a value function V , from which a policy can be extracted using policy extraction.
 - It produces an optimal policy an infinite amount of time.
 - For medium-scale problems, it can converge on the optimal policy in a “reasonable” amount of time, but does not scale as well as some other techniques.
-

2.2 Multi-armed bandits

Learning outcomes

The learning outcomes of this chapter are:

1. Select and apply multi-armed bandit algorithms for a given problem
 2. Compare and contrast the strengths and weaknesses of different multi-armed bandit algorithms
-

2.2.1 Overview

Multi-armed bandit techniques are not techniques for solving MDPs, but they are used throughout a lot of reinforcement learning techniques that do solve MDPs.

The problem of multi-armed bandits can be illustrated as follows:



Imagine that you have N number of slot machines (or poker machines in Australia), which are sometimes called **one-armed bandits**, due to the “arm” on the side that people pull to run again. Over time, each bandit pays a random reward from an unknown probability distribution. Some bandits are more likely to get a winning payoff than others – we just do not know which ones at the start. The goal is to maximize the total rewards of a sequence of lever pulls of the machine.

The question is: without knowing the probability distribution beforehand, how should we select the arms to increase our total payoff? Multi-armed bandit techniques aim to solve this problem.

2.2.2 The multi-armed bandit problem

Definition – Multi-armed bandit

A **multi-armed bandit** (also known as an **N -armed bandit**) is defined by a set of random variables $X_{i,k}$ where:

- $1 \leq i \leq N$, such that i is the **arm** of the bandit; and
- k the index of the **play** of arm i ;

Successive plays $X_{i,1}, X_{j,2}, X_{k,3} \dots$ are assumed to be independently distributed, but we do not know the probability distributions of the random variables.

The idea is that a gambler iteratively plays rounds, observing the reward from the arm after each round, and can adjust their strategy each time. The aim is to maximise the sum of the rewards collected over all rounds.

Multi-arm bandit strategies aim to learn a **policy** $\pi(k)$, where k is the play.

Given that we do not know the probability distributions, a simple strategy is simply to select the arm given a uniform distribution; that is, select each arm with the same probability. This is just uniform sampling.

Then, the Q-value for an action a can be estimated using the following formula:

$$Q(a) = \frac{1}{N(a)} \sum_{i=1}^t X_{a,i}$$

where t is the number of rounds so far, $N(a)$ is the number of times a selected in previous rounds, and $X_{a,i}$ is the **reward** obtained in the i -th round for playing arm a .

The idea here is that for a multi-armed bandit problem, we explore the options uniformly for some time, and then once we are confident we have enough samples (when the changes to the values of $Q(a)$ start to stabilise), we start selecting $\operatorname{argmax}_a Q(a)$. This is known as the ϵ -first strategy, where the parameter ϵ (epsilon), determines how many rounds to select random actions before moving to the greedy action.

But what is the issue? Time is wasted equally in all actions using the uniform distribution. Instead, we can focus on the *most promising actions* given the rewards we have received so far.

2.2.3 Exploration vs. Exploitation

What we want is to play only the good actions; so just keep playing the actions that have given us the best reward so far. However, at first, we do not have information to tell us what the best actions are. Thus, we want strategies that **exploit** what we think are the best actions so far, but still **explore** other actions.

But how much should we exploit and how much should we explore? This is known as the **exploration vs. exploitation dilemma**. It is driven by the *The Fear of Missing Out* (FOMO). FOMO drives us to search for strategies that **minimise regret**.

Definition — Regret

Given a policy π and t number of arm pulls, **regret** is defined formally as:

$$\mathcal{R}(\pi, t) = t \cdot \max_a Q^*(a) - \mathbb{E}\left[\sum_{k=1}^t X_{\pi(k),k}\right]$$

where $Q^*(a)$ is actual average return of playing arm a . We do not know $Q^*(a)$ of course – otherwise we could simply play $\operatorname{argmax}_a Q^*(a)$ each round.

Informally: If we follow policy π by playing arm $\pi(k)$ in round each round k , our regret over the t pulls is the *best possible cumulated reward* minus the *expected reward of playing using policy π* . So, regret is the **expected loss** from not taking the best action. If I take always action $\arg\max_a Q^*(a)$ (the best action), my regret is 0.

The aim of a multi-armed bandit strategy to learn a policy that minimises the total regret.

A **zero-regret** strategy is a strategy whose average regret each round approaches zero as the number of rounds approached infinity. So, this means that a zero-regret strategy will converge to an optimal strategy given enough rounds.

Implementation

The implementation for each strategy we discuss inherits from the class `MultiArmedBandit`:

```
class MultiArmedBandit():

    """ Select an action for this state given from a list given a Q-function """

    def select(self, state, actions, qfunction):
        abstract

    """ Reset a multi-armed bandit to its initial configuration """

    def reset(self):
        self.__init__()
```

Each strategy must implement the `select` method, which takes the list of available actions and their Q-values. The `reset` method resets to the bandit to its initial configuration, and is used for demonstration purposes later.

Simulation

To demonstrate the effect of different multi-armed bandit strategies and their parameters, we use the following simple simulation. The simulation is an implementation of a simple multi-armed bandit problem with five actions, `actions = [0, 1, 2, 3, 4]`. Each action has a probability associated with it: `probabilities = [0.1, 0.3, 0.7, 0.2, 0.1]`. The simulation runs 2000 episodes of a bandit problem, with each episode being 1000 steps long. At each step, the agent must chose an action. For the action a it chooses, it receives a reward of 5 with a probability `probability[a]`. With a probability of $1 - \text{probability}[a]$ it receives a reward of 0. At the beginning of each episode, the bandit strategies are reset.

The simulation returns a list of lists, representing the reward received at each step of each episode. The aim for the bandit is to maximise the expected rewards over each episode.

```
from collections import defaultdict
import random
from qtable import QTable

""" Run a bandit algorithm for a number of episodes, with each episode
being a set length.
"""

def run_bandit(bandit, episodes=200, episode_length=500, drift=True):

    # The actions available
    actions = [0, 1, 2, 3, 4]
```

(continues on next page)

(continued from previous page)

```

# A dummy state
state = 1

rewards = []
for _ in range(0, episodes):
    bandit.reset()

    # The probability of receiving a payoff of 1 for each action
    probabilities = [0.1, 0.3, 0.7, 0.2, 0.1]

    # The number of times each arm has been selected
    times_selected = defaultdict(lambda: 0)
    qtable = QTable()

    episode_rewards = []
    for step in range(0, episode_length):

        # Halfway through the episode, change the probabilities
        if drift and step == episode_length / 2:
            probabilities = [0.5, 0.2, 0.0, 0.3, 0.3]

        # Select an action using the bandit
        action = bandit.select(state, actions, qtable)

        # Get the reward for that action
        reward = 0
        if random.random() < probabilities[action]:
            reward = 5

        episode_rewards += [reward]

        times_selected[action] = times_selected[action] + 1
        qtable.update(
            state,
            action,
            (reward / times_selected[action])
            - (qtable.get_q_value(state, action) / times_selected[action]),
        )

    rewards += [episode_rewards]

return rewards

```

2.2.4 Solutions for minimising regret

There are several basic strategies for minimising regret. In each of these techniques, we record the average return of each arm over time. For consistency with the rest of these notes, we will call these Q-values, and will use $Q(a)$ to represent the Q value of arm a .

The multi-armed bandit solutions we will look at in these notes all follow the same basic format below, where T is the number of rounds or pulls that we will play, which may be infinite, and A is the set of arms available.

Algorithm 2 (Abstract multi-armed bandit)

Input : Multi-armed bandit problem $M = \langle \{X_{i,k}\}, A, T \rangle$

Output : Q-function Q

$Q(a) \leftarrow 0$ for all arms $a \in A$

$N(a) \leftarrow 0$ for all arms $a \in A$

$k \leftarrow 1$

while $k \leq T$ **do**

$a \leftarrow \text{select}(k)$

 Execute arm a for round k and observe reward $X_{a,k}$

$N(a) \leftarrow N(a) + 1$

$Q(a) \leftarrow Q(a) + \frac{1}{N(a)}[X_{a,k} - Q(a)]$

$k \leftarrow k + 1$

So, these algorithms choose an arm, and then add the reward from that to a cumulative average of playing that arm so far.

The key difference between the solutions we will look at is how the select function is implemented. This is what determines the policy $\pi(k)$ for the multi-armed bandit problem.

ϵ -greedy strategy

The ϵ -greedy (pronounced “epsilon-greedy”) strategy is a simple and effective way of balancing exploration and exploitation. In this algorithm, the parameter $\epsilon \in [0, 1]$ (pronounced “epsilon”) controls how much we explore and how much we exploit.

Each time we need to choose an action, we do the following:

- With probability $1 - \epsilon$ we choose the arm with the maximum Q value: $\text{argmax}_a Q(a)$. If there is a tie between multiple actions with the largest Q-value, break the tie randomly.
- With probability ϵ we choose a random arm with uniform probability.

The best value for ϵ depends on the particular problem, but typically, values around 0.05-0.1 work well as they exploit what they have learnt, while still exploring.

Implementation

The implementation for epsilon greedy uses `random()` to select a random number between 0 and 1. If that number is less than epsilon, an action is randomly selected. If r is greater than or equal to epsilon, it finds the actions with the maximum Q value, breaking ties randomly:

```
import random
from multi_armed_bandit.multi_armed_bandit import MultiArmedBandit

class EpsilonGreedy(MultiArmedBandit):
    def __init__(self, epsilon=0.1):
        self.epsilon = epsilon

    def reset(self):
        pass

    def select(self, state, actions, qfunction):
        # Select a random action with epsilon probability
        if random.random() < self.epsilon:
            return random.choice(actions)
```

(continues on next page)

(continued from previous page)

```
(arg_max_q, _) = qfunction.get_max_q(state, actions)
return arg_max_q
```

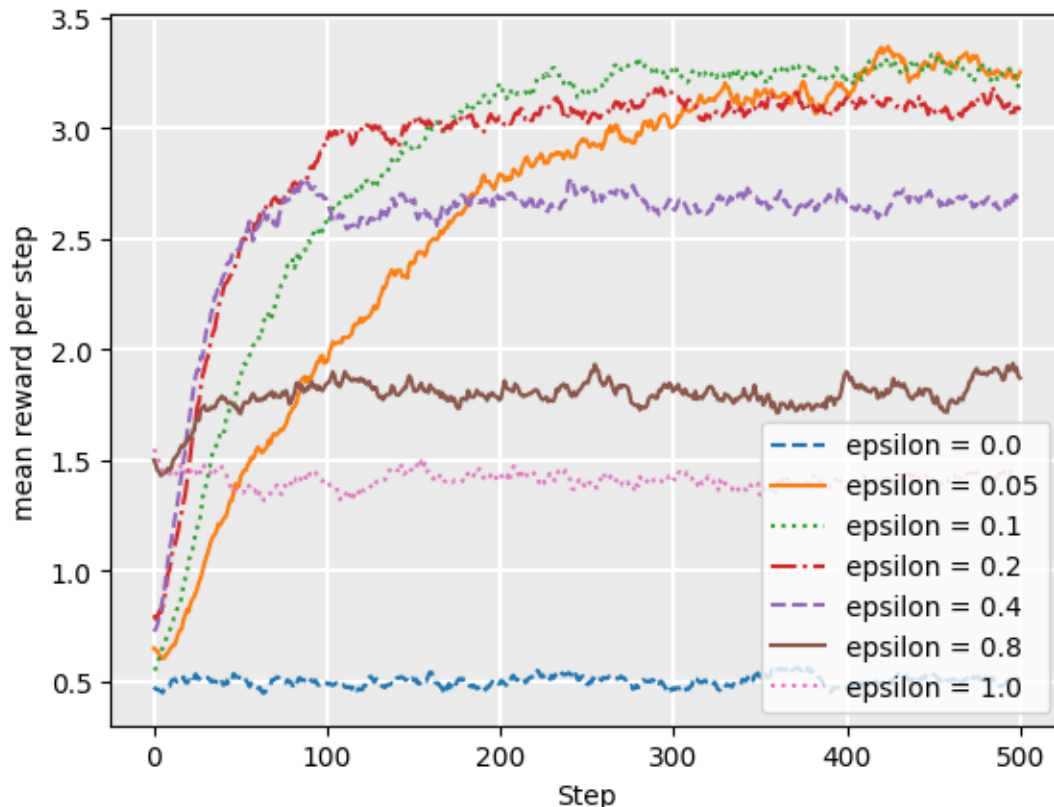
The following plot shows the reward for each step, averaged over 2000 episodes, over the simulation described in the *Simulation* section. Each episode is 1000 steps long. We evaluate different values of epsilon:

```
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy
from tests.multi_armed_bandit_tests.run_bandit import run_bandit
from tests.plot import Plot

def plot_epsilon_greedy(drift=False):
    epsilon000 = run_bandit(EpsilonGreedy(epsilon=0.00), drift=drift)
    epsilon005 = run_bandit(EpsilonGreedy(epsilon=0.05), drift=drift)
    epsilon01 = run_bandit(EpsilonGreedy(epsilon=0.1), drift=drift)
    epsilon02 = run_bandit(EpsilonGreedy(epsilon=0.2), drift=drift)
    epsilon04 = run_bandit(EpsilonGreedy(epsilon=0.4), drift=drift)
    epsilon08 = run_bandit(EpsilonGreedy(epsilon=0.8), drift=drift)
    epsilon10 = run_bandit(EpsilonGreedy(epsilon=1.0), drift=drift)

    Plot.plot_rewards(
        [
            "epsilon = 0.0",
            "epsilon = 0.05",
            "epsilon = 0.1",
            "epsilon = 0.2",
            "epsilon = 0.4",
            "epsilon = 0.8",
            "epsilon = 1.0",
        ],
        [epsilon000, epsilon005, epsilon01, epsilon02, epsilon04, epsilon08,
epsilon10],
    )

plot_epsilon_greedy()
```



As we can see, higher values of epsilon tend to have a lower reward over time, except that we need some non-zero value of epsilon. Higher values mean more exploration, so the bandit spends more time exploring less valuable actions, even after it has a good estimate of the value of actions. The actual choice of the epsilon parameter is entirely dependent on the particular application: there is no magic number. However, as in this particular case, an epsilon between 0.05-0.1 is usually a reasonable choice.

But we can also see that while epsilon = 0.05 ends up with a higher return after about 350 steps, initially it does not do as well as epsilon = 0.1, because it does not explore enough. Can we do better? Yes, we can!

ϵ -decreasing strategy

ϵ -decreasing (pronounced “epsilon-decreasing”) follows a similar idea to epsilon greedy, however, it recognises that initially, we have very little feedback so exploiting is not a good strategy to begin with: we need to explore first. Then, as we gather more data, we should exploit more.

The ϵ -decreasing strategy does this by taking the basic epsilon greedy strategy and introducing another parameter $\alpha \in [0, 1]$ (pronounced “alpha”), which is used to decrease ϵ over time. For this reason, α is called the **decay**.

The selection mechanism is the same as epsilon greedy, but then after each selection, we set $\epsilon := \epsilon \times \alpha$. We start initially with a higher value of ϵ to explore, and it will slowly decay to a low number such that we explore less and less as we gather more feedback.

Implementation

The following implementation for the ϵ -decreasing strategy uses the ϵ -greedy strategy, just decreasing the epsilon value each step:

```
from multi_armed_bandit.multi_armed_bandit import MultiArmedBandit
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy

class EpsilonDecreasing(MultiArmedBandit):
    def __init__(self, epsilon=0.2, alpha=0.999):
        self.epsilon_greedy_bandit = EpsilonGreedy(epsilon)
        self.initial_epsilon = epsilon
        self.alpha = alpha

    def reset(self):
        self.epsilon_greedy_bandit = EpsilonGreedy(self.initial_epsilon)

    def select(self, state, actions, qfunction):
        result = self.epsilon_greedy_bandit.select(state, actions, qfunction)
        self.epsilon_greedy_bandit.epsilon *= self.alpha
        return result
```

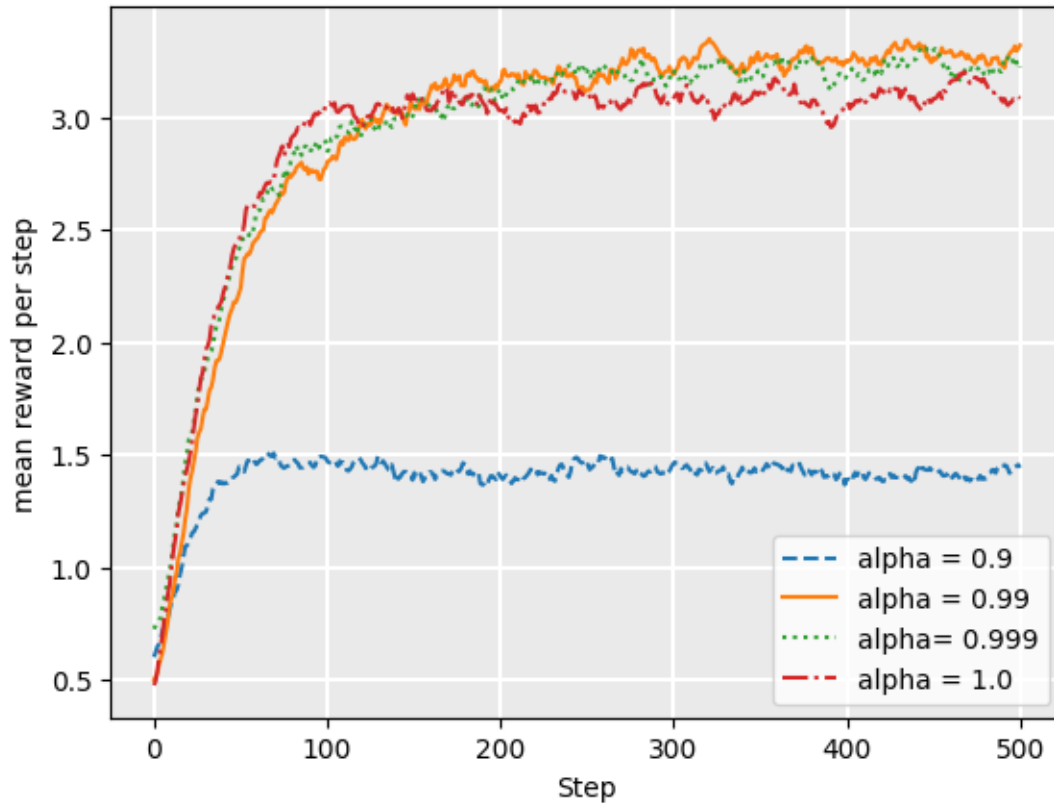
The following plot shows the average reward over our simulation, varying the value of α :

```
from multi_armed_bandit.epsilon_decreasing import EpsilonDecreasing
from tests.multi_armed_bandit_tests.run_bandit import run_bandit
from tests.plot import Plot

def plot_epsilon_decreasing(drift=False):
    alpha09 = run_bandit(EpsilonDecreasing(alpha=0.9), drift=drift)
    alpha099 = run_bandit(EpsilonDecreasing(alpha=0.99), drift=drift)
    alpha0999 = run_bandit(EpsilonDecreasing(alpha=0.999), drift=drift)
    alpha1 = run_bandit(EpsilonDecreasing(alpha=1.0), drift=drift)

    Plot.plot_rewards(
        ["alpha = 0.9", "alpha = 0.99", "alpha = 0.999", "alpha = 1.0"],
        [alpha09, alpha099, alpha0999, alpha1],
    )

plot_epsilon_decreasing()
```



This indicates that for this particular problem, a value of 0.99 for alpha has a better average return than lower values. This is because a lower value, such as 0.9, will result in epsilon approaching zero before we have explored enough. However, the choice of alpha depends both on the particular problem, and also the expected length of each episode: for longer episodes, decreasing slower would be more beneficial so we do not stop exploring too early.

Softmax strategy

Softmax is a probability matching strategy, which means that the probability of each action being chosen is dependent on its Q-value so far. Formally, softmax chooses an action because on the Boltzman distribution for that action:

$$\frac{e^{Q(a)/\tau}}{\sum_{b=1}^N e^{Q(b)/\tau}}$$

where N is the number of arms, and $\tau > 0$ (pronounced “tau”) is the **temperature**, which dictates how much of an influence the past data has on the decision. A higher value of τ would mean that the probability of selecting each action is close to each other (as τ approaches infinity, softmax approaches a uniform strategy), while a lower value of τ would imply that the probabilities are closer to their Q values. When $\tau = 1$, the probabilities are just $e^{Q(a)}$, and as τ approaches 0, softmax approaches a greedy strategy.

As with the ϵ -decreasing strategy, we can add a decay parameter α that allows the value of τ to decay until it reaches 1. This encourages exploration in earlier phases, and exploration less as we gather more feedback.

Implementation

The following implementation of the softmax strategy uses `random()` to generate a random number between 0 and 1, and divides this space 0-1 among the set of actions based on the value of $e^{Q(a)/\tau}$:

```
import math
import random
from multi_armed_bandit.multi_armed_bandit import MultiArmedBandit

class Softmax(MultiArmedBandit):
    def __init__(self, tau=1.0):
        self.tau = tau

    def reset(self):
        pass

    def select(self, state, actions, qfunction):
        # calculate the denominator for the softmax strategy
        total = 0.0
        for action in actions:
            total += math.exp(qfunction.get_q_value(state, action) / self.tau)

        rand = random.random()
        cumulative_probability = 0.0
        result = None
        for action in actions:
            probability = (
                math.exp(qfunction.get_q_value(state, action) / self.tau) / total
            )
            if cumulative_probability <= rand <= cumulative_probability + probability:
                result = action
            cumulative_probability += probability

        return result
```

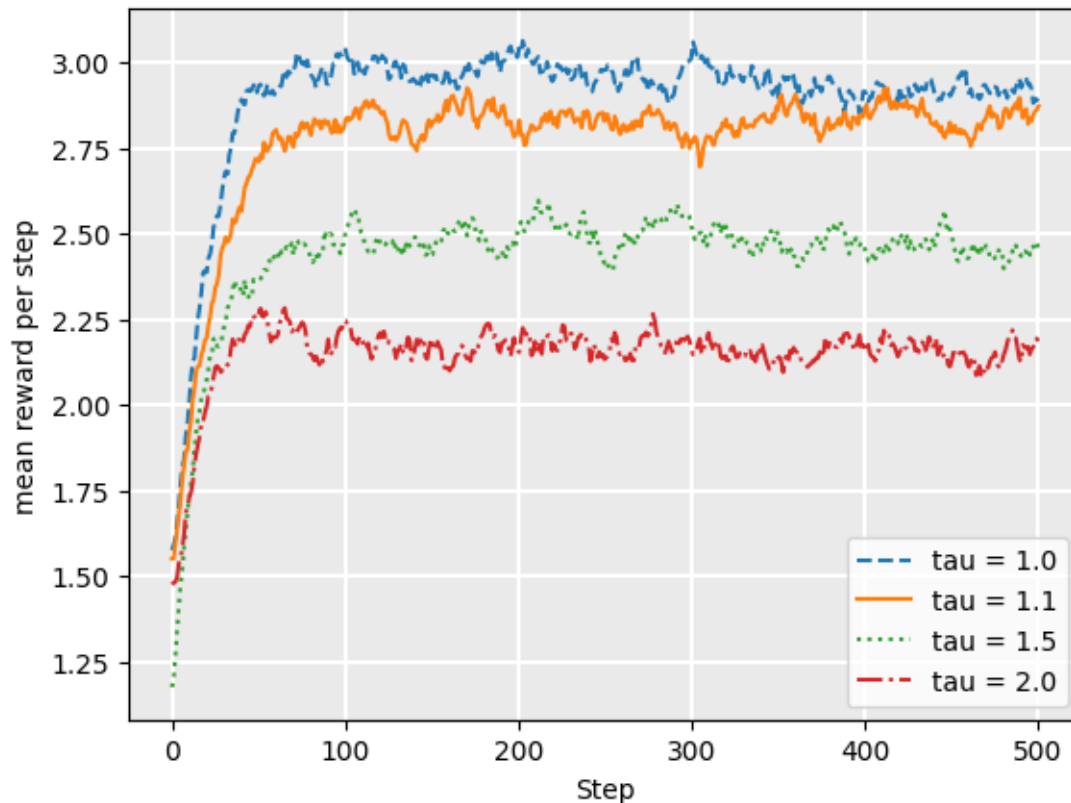
As before, we plot the average reward at each step of our simulation, this time varying values of tau:

```
from multi_armed_bandit.softmax import Softmax
from tests.multi_armed_bandit_tests.run_bandit import run_bandit
from tests.plot import Plot

def plot_softmax(drift=False):
    tau10 = run_bandit(Softmax(tau=1.0), drift=drift)
    tau11 = run_bandit(Softmax(tau=1.1), drift=drift)
    tau15 = run_bandit(Softmax(tau=1.5), drift=drift)
    tau20 = run_bandit(Softmax(tau=2.0), drift=drift)

    Plot.plot_rewards(
        ["tau = 1.0", "tau = 1.1", "tau = 1.5", "tau = 2.0"],
        [tau10, tau11, tau15, tau20],
    )

plot_softmax(drift=False)
```



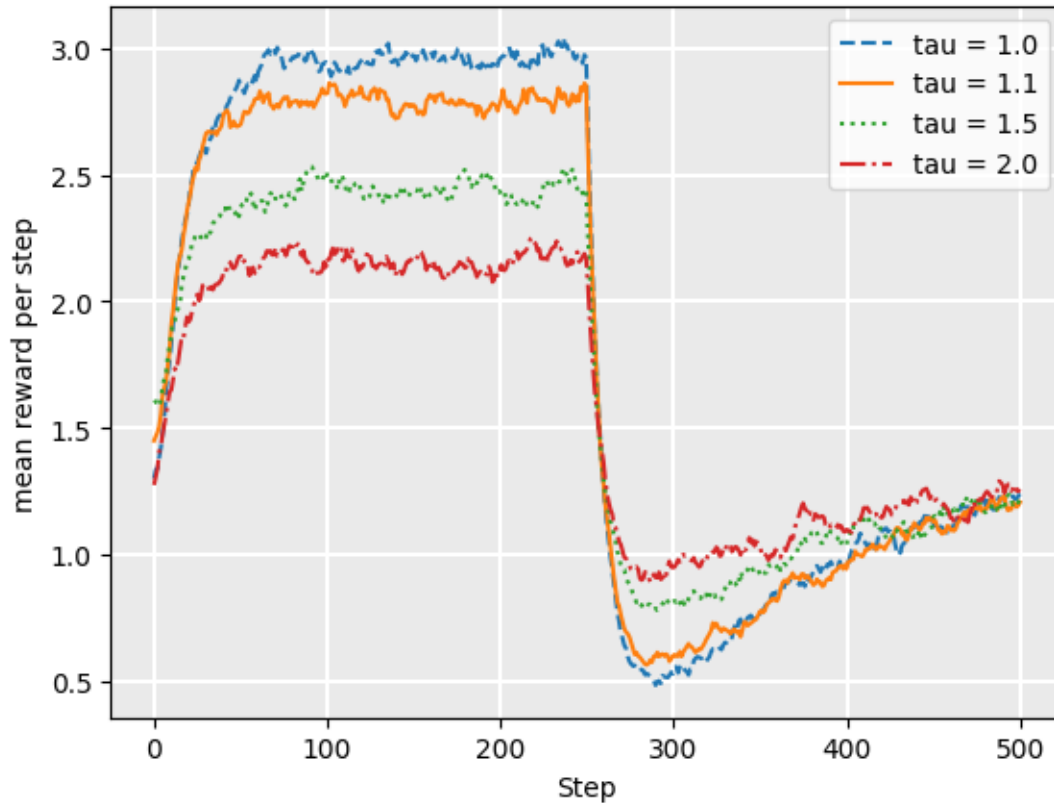
In this particular case, we see that $\tau = 1.0$ is a good choice, which means that the probability of selecting an action is directly proportional to $e^{Q(a)}$. So, why should we use τ at all?

The softmax strategy is designed to work well under **drift**; that is, when the underlying probability distributions (in this case, the probability of receiving a reward) change over time. In many real problems, the underlying probability distributions are not static. For example, if we are using a multi-armed bandit to determine which products to show to people visiting our website, where the reward is whether they click on the product link, the preferences of our visitors will change over time, depending on e.g. the news cycle, the weather, fashion, etc. Sometimes, preferences can change very quickly.

So, what happens if we change the value of our underlying probabilities? In the following evaluation, we change the probabilities of the underlying actions at the halfway point of each episode, from `probabilities = [0.1, 0.3, 0.7, 0.2, 0.1]` to `probabilities = [0.5, 0.2, 0.0, 0.3, 0.3]`. This is a sudden change in which the Q-values are almost worthless.

If we plot the performance of the softmax algorithm with this, the results are as follows:

```
plot_softmax(drift=True)
```



As one can see, the strategies that are less “committed” to their Q-values are less affected by the sudden change. Of course, if the drift is more gradual, values closer to 1.0 may be more suitable.

Note that higher values of tau also work when the drift is more gradual, rather than a sudden change as we simulate here.

Upper Confidence Bounds (UCB1)

A highly effective multi-armed bandit strategy is the **Upper Confidence Bounds (UCB1)** strategy.

Using the UCB1 strategy, we select the next action using the following:

$$\operatorname{argmax}_a \left(Q(a) + \sqrt{\frac{2 \ln t}{N(a)}} \right)$$

where t is the number of rounds so far, and $N(a)$ is the number of times times a has been chosen in all previous rounds. The term inside the square root is undefined if $N(a) = 0$. To avoid this, the typical strategy is to spend the first N rounds to select each of the N bandits once.

The left-hand side encourages exploitation: the Q-value is high for actions that have had a high reward.

The right-hand side encourages exploration: it is high for actions that have been explored less – that is, when $N(a)$ relative to other actions.

Interesting, the UCB formula is not a weighted formula – that is, there is no parameter giving weight to the $Q(a)$ or the square root expressions to balance exploration vs. exploitation. So how does it work? We will not get into all the details, but instead just give some intuition.

We want to learn the Q-function, which gives us the average return on each action a , such that it approximates the real (unknown) Q-function, which we will call Q^* . At each round, we select the action a that maximises the expression inside

the brackets. If arm a is optimal, then we want the following to hold for all actions $b \neq a$:

$$Q(b) + \sqrt{\frac{2 \ln t}{N(b)}} \leq Q^*(a)$$

If this holds, we have some confidence that $Q(a)$ is optimal. If $N(b)$ is low for some actions, we do not have this confidence.

If by chance the above expression does NOT hold for the optimal action a , then a is not chosen, but it should be. We want this to occur only with probability $\frac{1}{N}$ to minimise pseudo-regret. This leads us to $\ln t$ in the expression. We will not go into the technicalities of why $\ln t$ in these notes.

Implementation

```
import math
import random
from multi_armed_bandit.multi_armed_bandit import MultiArmedBandit

class UpperConfidenceBounds(MultiArmedBandit):
    def __init__(self):
        self.total = 0
        # number of times each action has been chosen
        self.times_selected = {}

    def select(self, state, actions, qfunction):
        # First execute each action one time
        for action in actions:
            if action not in self.times_selected.keys():
                self.times_selected[action] = 1
                self.total += 1
                return action

        max_actions = []
        max_value = float("-inf")
        for action in actions:
            value = qfunction.get_q_value(state, action) + math.sqrt(
                (2 * math.log(self.total)) / self.times_selected[action]
            )
            if value > max_value:
                max_actions = [action]
                max_value = value
            elif value == max_value:
                max_actions += [action]

        # if there are multiple actions with the highest value
        # choose one randomly
        result = random.choice(max_actions)
        self.times_selected[result] = self.times_selected[result] + 1
        self.total += 1
        return result
```

Because UCB does not have parameters, there is no exploration to be done, however, in the next section we compare UCB with the other three strategies.

Comparison

```

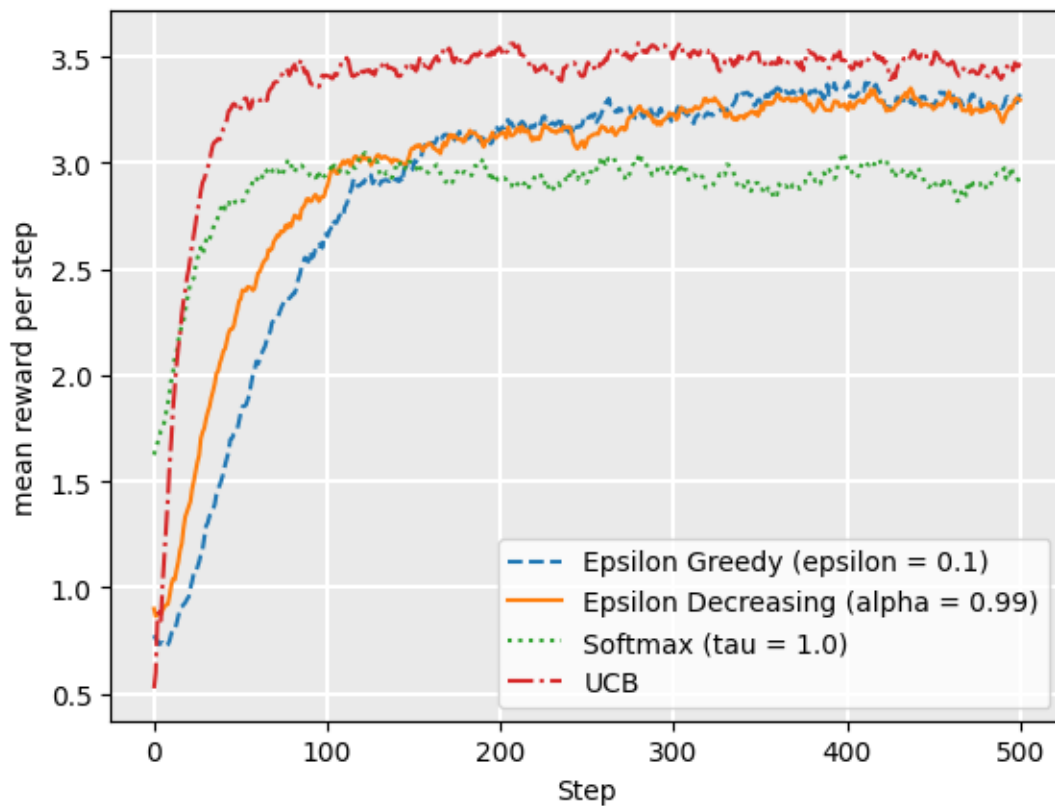
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy
from multi_armed_bandit.epsilon_decreasing import EpsilonDecreasing
from multi_armed_bandit.softmax import Softmax
from multi_armed_bandit.ucb import UpperConfidenceBounds
from tests.multi_armed_bandit_tests.run_bandit import run_bandit
from tests.plot import Plot

def plot_comparison(drift=False):
    epsilon_greedy = run_bandit(EpsilonGreedy(epsilon=0.1), drift=drift)
    epsilon_decreasing = run_bandit(EpsilonDecreasing(alpha=0.99), drift=drift)
    softmax = run_bandit(Softmax(tau=1.0), drift=drift)
    ucb = run_bandit(UpperConfidenceBounds(), drift=drift)

    Plot.plot_rewards(
        [
            "Epsilon Greedy (epsilon = 0.1)",
            "Epsilon Decreasing (alpha = 0.99)",
            "Softmax (tau = 1.0)",
            "UCB",
        ],
        [epsilon_greedy, epsilon_decreasing, softmax, ucb],
    )

plot_comparison(drift=False)

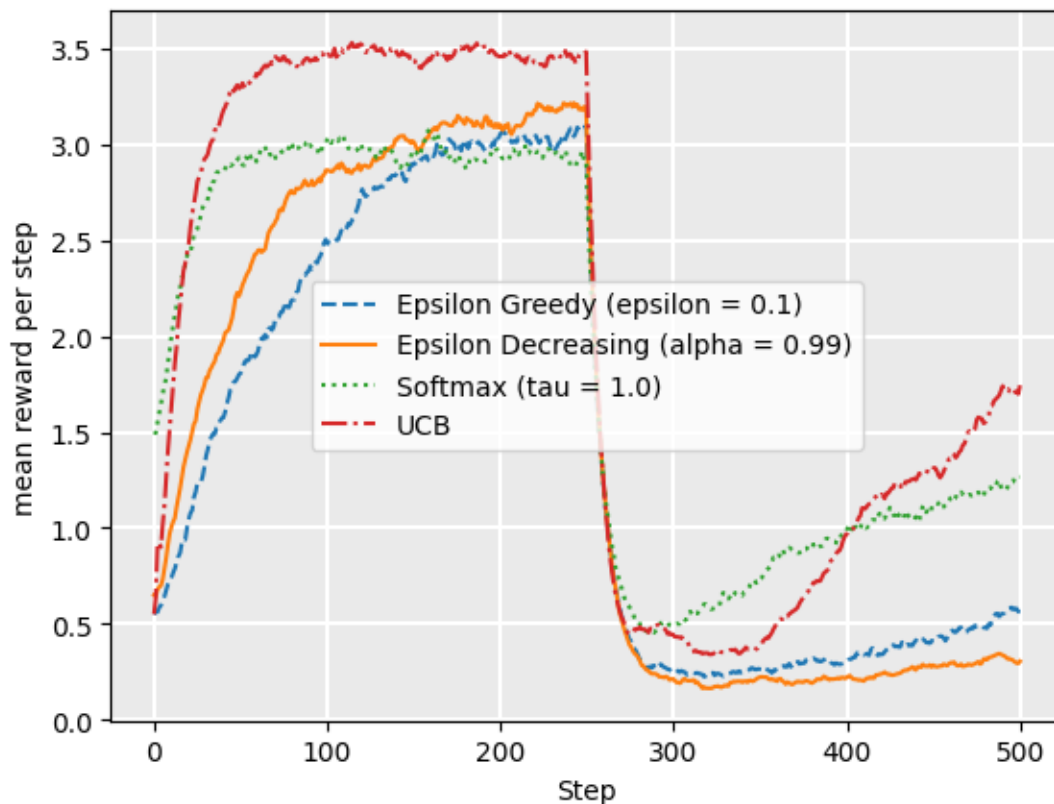
```



We can see from this that UCB1, on average, obtains the highest reward for the simulation. If we extend the simulation episodes to be longer, we would see that eventually epsilon decreasing would start to achieve similar rewards to UCB1, but it takes longer to converge to this.

From this, we may answer the question why softmax is considered a good strategy at all: it clearly does not perform well compared to the others. However, as noted already, softmax is able to adjust to drift more quickly, as is demonstrated when we suddenly change the probabilities as we did earlier:

```
plot_comparison(drift=True)
```



From this comparison, we can see that softmax, even with $\tau = 1.0$, adapts more quickly than other strategies. UCB1 recovers quite quickly too, soon out-performing softmax. For UCB1, the Q-values for the actions that were previously good are no longer good. This encourages exploration to other actions. But also, the actions that had poor Q-values previously but are now good actions would not have been visited as much previously, which also encourages exploration.

Epsilon decreasing never recovers because by the time the probabilities change, epsilon is low and it is committed to those values. For that reason, the ϵ -decreasing strategy is good only for static problems.

While in this particular case, UCB1 has a higher average reward over the entire episode, this may not be the case if the underlying probability distributions change or drift regularly. In those cases, softmax may be a better choice.

2.2.5 Takeaways

Takeaways

- Multi-armed bandits are problems in where we must make a selection from a set of options, but we do not know the probability of success (or the expected return) of each options
 - Several techniques can be used to solve these. This chapter looks at just a few: ϵ -greedy, ϵ -decreasing, softmax, and UCB1 strategy.
 - In a simple experiment, we found that UCB1 was the fastest learner and recovered from shift quickly. However, this is just one experiment – other domains will have different properties.
-

2.2.6 Further reading

Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems. Sébastien Bubeck and Nicolo Cesa-Bianchi, *Machine Learning* 5(1): 1-122, 2012

- A great analysis of multi-armed bandits.

2.3 Temporal difference reinforcement learning

Learning outcomes

The learning outcomes of this chapter are:

1. Identify situations in which model-free reinforcement learning is a suitable solution for an MDP
 2. Explain how model-free planning differs from model-based planning
 3. Apply temporal difference methods Q-learning and SARSA to solve small-scale MDP problems manually and program Q-learning and SARSA algorithms to solve medium-scale MDP problems automatically
 4. Compare and contrast off-policy reinforcement learning with on-policy reinforcement learning
-

2.3.1 Model-based vs model-free

Value iteration is part of a class of solutions known as **model-based** techniques. This means that we need to know the model; in particular, we have access to $P_a(s' | s)$ and $r(s, a, s')$.

In this section, we look at Q-learning and SARSA, which are **model-free** techniques. This means that we do NOT know $P_a(s' | s)$ and $r(s, a, s')$ of our model

How can we calculate a policy if we don't know the transitions and the rewards?! We **learn through experience** by trying actions and seeing what the results is, making this machine learning problem.

Importantly, in model-free reinforcement learning, we do NOT try to learn $P_a(s' | s)$ or $r(s, a, s')$ — we learn a value function or a policy directly.

There is something in between model-based and model-free: simulation-based techniques. In these cases, we have a model as a **simulator**, so we can **simulate** $P_a(s' | s)$ and $r(s, a, s')$ and learn a policy with a model-free technique, but we cannot “see” $P_a(s' | s)$ and $r(s, a, s')$, so model-based techniques like value iteration are not possible.

Exercise – The Mystery game

Consider the following game, implemented by Github user guiferviz, and available for download at https://github.com/guiferviz/rl_udacity/.

The aim of the game is to win the game. You have six actions available, which can be executed by pressing keys 1, 2, 3, 4, 5, and 6. You need to learn what the actions do and what the rewards are. No other instructions are given, but you will know when you receive any rewards/points. Try to play the game and see if you can win. Have fun!

Once you have played this, ask yourself the following questions:

- What was the process you took?
- What did you learn?
- What assumptions did you use?

I would guess that you experimented by pressing the keys 1 to 6, from the outcome of the keys, you learnt the effect of each of the keys, including the state transitions and the rewards. From this, you could easily construct a winning strategy. I also would guess that you perhaps used the colours as a way to guess what you should do.

This game is of interest because it is a **model-free** (at least initially) Markov decision process: you didn't know the transition function or the reward function; instead you had to learn it. Similarly, model-free reinforcement learning techniques don't know the transition function or the reward function of an MDP, so they just learn by trying different behaviours and observing what rewards they get. Over time, they learn which behaviours lead to positive rewards, so they reinforce the policy to try that behaviour more, and which behaviours lead to negative rewards, and they reinforce the policy to avoid that behaviour.

Imagine how hard it is for a computer that doesn't have any assumptions or intuition for this game though! It will not match the colours, nor will it really have any prior knowledge about similar games, unless it is explicitly told about it. Model-free reinforcement learning techniques start with no or minimal initial knowledge, and will learn a policy (e.g. a value function, a Q-function, or a policy directly) just by trying behaviours and seeing what happens. Most techniques (at least the ones covered in these notes) do not learn a model in the same way that you did — they just construct the policy directly.

Intuition of model-free reinforcement learning

There are many different techniques for model-free reinforcement learning, all with the same basis:

- We execute many different **episodes** of the problem we want to solve, and from that we learn a **policy**.
- During learning, we try to learn the value of applying particular actions in particular states.
- During each episode, we need to execute some actions. After each action, we get a reward (which may be 0) and we can see the new state.
- From this, we **reinforce** our estimates of applying the previous action in the previous state.
- We terminate when: (1) we run out of training time; (2) we think our policy has converged to the optimal policy (for each new episode we see no improvement); or (3) our policy is 'good enough' (for each new episode we see minimal improvement).

2.3.2 Monte-Carlo reinforcement learning

Monte-Carlo reinforcement learning is perhaps the simplest of reinforcement learning methods, and is based on how animals learn from their environment. The intuition is quite straightforward. Maintain a *Q-function* that records the value $Q(s, a)$ for every state-action pair. At each step: (1) choose an action using a multi-armed bandit algorithm; (2) apply that action and receive the reward; and (3) update $Q(s, a)$ based on that reward. Repeat over a number of episodes until ...when?

It is called **Monte-Carlo reinforcement learning** after the area within within Monaco (small principality on the French riviera) called Monte Carlo, which is best known for its extravagant casinos. As gambling and casinos are largely associated with chance, simulations that use some randomness to explore actions are often called Monte Carlo methods.

Algorithm 3 (Monte-Carlo reinforcement learning)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Output : Q-function Q

Initialise Q arbitrarily; e.g., $Q(s, a) \leftarrow 0$ for all s and a

$N(s, a) \leftarrow 0$ for all s and a

repeat

 Generate an episode $(s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T)$;

 e.g. using Q and a multi-armed bandit algorithm such as ϵ -greedy

$G \leftarrow 0$

$t \leftarrow T - 1$

while $t \geq 0$ **do**

$G \leftarrow r_{t+1} + \gamma \cdot G$

if s_t, a_t does not appear in $s_0, a_0, \dots, s_{t-1}, a_{t-1}$ **then**

$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{N(s_t, a_t)}[G - Q(s_t, a_t)]$

$N(s_t, a_t) \leftarrow N(s_t, a_t) + 1$

$t \leftarrow t - 1$

until Q converges

This algorithm generates an entire episode following some policy, such ϵ -greedy, observing the reward r_{t+1} at each step t . It then calculates the discounted future reward G at each step. If s_t, a_t occurs earlier in the episode, then we do not update $Q(s_t, a_t)$ as we will update it later in the loop. If it does not occur, we update $Q(s_t, a_t)$ as the cumulative average over all executions of s_t, a_t over all episodes. In this algorithm $N(s_t, a_t)$ represents the number of times that s_t, a_t have been evaluated over all episodes.

2.3.3 Q-Tables

Q-tables are the simplest way to maintain a Q-function. They are a table with an entry for every $Q(s, a)$. Thus, like value functions in value iteration, they do not scale to large state-spaces. (More on scaling in the next lecture).

Initially, we would have an arbitrary Q-table, which may look something like this if initialised with all zeros, taking the GridWorld example:

State	Up	Down	Right	Left
(0, 0)	0.00	0.00	0.00	0.00
(0, 1)	0.00	0.00	0.00	0.00
(0, 2)	0.00	0.00	0.00	0.00
(1, 0)	0.00	0.00	0.00	0.00
(1, 1)	0.00	0.00	0.00	0.00
(1, 2)	0.00	0.00	0.00	0.00
(2, 0)	0.00	0.00	0.00	0.00
(2, 1)	0.00	0.00	0.00	0.00
(2, 2)	0.00	0.00	0.00	0.00
(3, 0)	0.00	0.00	0.00	0.00
(3, 1)	0.00	0.00	0.00	0.00
(3, 2)	0.00	0.00	0.00	0.00

After some training, we may end up with a Q-function that looks something like this:

State	Up	Down	Right	Left
(0, 0)	0.50	0.42	0.39	0.42
(0, 1)	0.56	0.44	0.51	0.51
(0, 2)	0.58	0.51	0.63	0.57
(1, 0)	0.09	0.18	0.06	0.43
(1, 1)	0.00	0.00	0.00	0.00
(1, 2)	0.64	0.65	0.74	0.59
(2, 0)	0.41	0.00	0.00	0.00
(2, 1)	0.69	0.09	-0.24	0.24
(2, 2)	0.79	0.61	0.90	0.65
(3, 0)	-0.02	0.00	0.00	0.00
(3, 1)	0.00	0.00	0.00	0.00
(3, 2)	0.00	0.00	0.00	0.00

The following is an implementation of a Q-table using a Python dictionary:

```
import json
from collections import defaultdict
from qfunction import QFunction

class QTable(QFunction):
    def __init__(self, default=0.0):
        self.qtable = defaultdict(lambda: default)

    def update(self, state, action, delta):
        self.qtable[(state, action)] = self.qtable[(state, action)] + delta

    def get_q_value(self, state, action):
        return self.qtable[(state, action)]

    def save(self, filename):
        with open(filename, "w") as file:
            serialised = {str(key): value for key, value in self.qtable.items()}
            json.dump(serialised, file)

    def load(self, filename, default=0.0):
        with open(filename, "r") as file:
            serialised = json.load(file)
            self.qtable = defaultdict(
                lambda: default,
                {tuple(eval(key)): value for key, value in serialised.items()}
            )
```

2.3.4 Temporal difference (TD) reinforcement learning

Monte Carlo reinforcement learning is simple, but it has a number of problems. The most important is that it has high **variance**. Recall that we calculate the future discounted reward for an episode and use that to calculate the average reward for each state-action pair. However, the term γG is often *not* a good estimate of the average future reward that we would receive. If we execute action a in state s many times throughout different episodes, we might find that the future trajectories we execute after that vary significantly because we are using Monte Carlo simulation. This means that it will take a long term to learn a good estimate of the true average reward is for that state-action pair.

Temporal difference (TD) methods alleviate this problem using **bootstrapping**. Much the same way that value iteration bootstraps by using the last iteration's value function, in TD methods, instead of updating based on G – the actual future discounted reward received in the episode – we update based on the actual immediate reward received plus an estimate of our future discounted reward.

In TD methods, our update rules always follow a pattern:

$$Q(s, a) \leftarrow \underbrace{Q(s, a)}_{\text{old value}} + \underbrace{\hat{\alpha}}_{\text{learning rate}} \cdot \left[\underbrace{\hat{r}}_{\text{reward}} + \underbrace{\hat{\gamma} \cdot V(s')}_{\text{TD target}} - \underbrace{Q(s, a)}_{\text{do not count extra } Q(s, a)} \right]$$

$V(s')$ is our TD estimate of the average future reward, and is the bootstrapped value of our future discounted reward. The new information is weighted by a parameter $\alpha \in [0, 1]$ (pronounced “alpha”), which is the **learning rate**. A higher learning rate α will weight more recent information higher than older information, so will learn more quickly, but will make it more difficult to stabilise because it is strongly influenced by outliers.

The idea is that over time, the TD estimate will become be more stable than the actual rewards we receive in an episode, converging to the optimal value function $V(s')$ defined by the Bellman equation, which leads to $Q(s, a)$ converging more quickly.

Why is there the expression $-Q(s, a)$ inside the square brackets? This is because the old value is weighted $(1-\alpha) \cdot Q(s, a)$. We can expand this to $Q(s, a) - \alpha \cdot Q(s, a)$, and can then move the latter $-Q(s, a)$ inside the square brackets where the learning rate α is applied.

In this chapter, we will look at two TD methods that differ in the way that they estimate the future reward $V(s')$:

1. **Q-learning**, which is what we call an **off-policy** approach; and
2. **SARSA**, which is what we call an **on-policy** approach.

2.3.5 Q-Learning: Off-policy temporal-difference learning

Q-learning is a foundational method for reinforcement learning. It is TD method that estimates the future reward $V(s')$ using the Q-function itself, assuming that from state s' , the best action (according to Q) will be executed at each state.

Below is the Q-learning algorithm. It uses $\max_{a'} Q(s', a')$ as the estimate of $V(s')$; that is, it estimates this by using Q to estimate the value of the best action from s' . This helps to stabilise the learning.

Note that, unlike Monte Carlo reinforcement learning, because Q-learning uses the bootstrapped value, it can interleave execution and update, meaning that Q is improved at each step of an episode, rather than having to wait until the end of the episode to calculate G . This has benefit early episodes benefit from learning more than in Monte Carlo reinforcement learning.

Algorithm 4 (Q-learning)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Output : Q-function Q

Initialise Q arbitrarily; e.g., $Q(s, a) \leftarrow 0$ for all s and a

repeat

$s \leftarrow$ the first state in episode e

repeat (for each step in episode e)

Select action a to apply in s ;

e.g. using Q and a multi-armed bandit algorithm such as ϵ – greedy

Execute action a in state s

Observe reward r and new state s'

$\delta \leftarrow r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)$

$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \delta$

$s \leftarrow s'$

until s is the last state of episode e (a terminal state)

until Q converges

Updating the Q-function

Updating the Q-function is where the learning happens:

$$\delta \leftarrow \underbrace{\left[\underbrace{\hat{r}}_{\text{reward}} + \underbrace{\hat{\gamma}}_{\text{discount factor}} \cdot \underbrace{\max_{a'} Q(s', a')}_{\substack{V(s') \text{ estimate} \\ \text{do not count extra } Q(s, a)}} \right]}_{\text{TD target}} - \underbrace{Q(s, a)}_{\text{old value}}$$

$$Q(s, a) \leftarrow \underbrace{Q(s, a)}_{\text{old value}} + \underbrace{\hat{\alpha}}_{\text{learning rate}} \cdot \underbrace{\delta}_{\text{delta value}}$$

We can see that at each step, $Q(s, a)$ is update by taking the old value of $Q(s, a)$ and adding this to the new information.

The estimate from the new observations is given by $\delta \leftarrow r + \gamma \cdot \max_{a'} Q(s', a')$, where δ (pronounced “delta”) is the difference between the previous estimate and the most recent observation, r is the reward that was received by executing action a in state s , and $r + \gamma \cdot \max_{a'} Q(s', a')$ is the **temporal difference** target. What this says is that the estimate of $Q(s, a)$ based on the new information is the reward r , plus the estimated discounted future reward from being in state s' . The definition of δ is the update similar to that of the Bellman equation. We do not know $P_a(s' | s)$, so we cannot calculate the Bellman update directly, but we can estimate the value using r and the temporal difference target.

Note: Note that we estimate the future value using $\max_{a'} Q(s', a')$, which means it *ignores* the actual next action that will be executed, and instead updates based on the **estimated best action** for the update. This is known as **off policy** learning — more on this later.

Example – Q-learning update

Using the table above, we can illustrate the inner loop of the Q-learning algorithm. Assume that we are in state $s = (2, 2)$, and the action $a = Up$ is chosen and executed successfully, which would return to state $s' = (2, 2)$ as there is no cell above $(2, 2)$. Using the Q-table above, we would update the Q-value as follows:

$$\begin{aligned} Q((2, 2), Up) &\leftarrow Q((2, 2), Up) + \alpha[r + \gamma \max_{a'} Q((2, 2), a') - Q((2, 2), Up)] \\ &\leftarrow 0.79 + 0.1[0 + 0.9 \cdot Q((2, 2), Right) - Q((2, 2), Up)] \\ &\leftarrow 0.79 + 0.1[0 + 0.9 \cdot 0.90 - 0.79] \\ &\leftarrow 0.792 \end{aligned}$$

Theoretical guarantee

Using Q-tables to represent Q-functions, Q-learning will converge to the optimal policy under the assumption that all state-action pairs are sampled infinitely often.

Policy extraction using Q-functions

Using model-free learning, we iterate over as many episodes as possible, or until each episode hardly improves our Q-values. This gives us a (close to) optimal Q-function.

Once we have such a Q-function, we stop exploring and just exploit. We use *policy extraction*, exactly as we do for value iteration, except we extract from the Q-function instead of the value function:

$$\pi(s) = \operatorname{argmax}_{a \in A(s)} Q(s, a)$$

This selects the action with the maximum Q-value. Given an optimal Q-function (for the MDP), this results in optimal behaviour.

Implementation

To implement Q-learning, we first implement an abstract superclass `ModelFreeLearner` that defines the interface for any model-free learning algorithm:

```
class ModelFreeLearner:
    def execute(self, episodes=2000):
        abstract
```

Next, we implement a second abstract superclass `TemporalDifferenceLearner`, which contains most of the code we need:

```
from model_free_learner import ModelFreeLearner

class TemporalDifferenceLearner(ModelFreeLearner):
    def __init__(self, mdp, bandit, qfunction, alpha=0.1):
        self.mdp = mdp
        self.bandit = bandit
        self.alpha = alpha
        self.qfunction = qfunction

    def execute(self, episodes=2000):

        rewards = []
        for _ in range(episodes):
            state = self.mdp.get_initial_state()
            actions = self.mdp.get_actions(state)
            action = self.bandit.select(state, actions, self.qfunction)

            episode_reward = 0.0
            step = 0
            while not self.mdp.is_terminal(state):
                (next_state, reward) = self.mdp.execute(state, action)
```

(continues on next page)

(continued from previous page)

```

        actions = self.mdp.get_actions(next_state)
        next_action = self.bandit.select(next_state, actions, self.qfunction)
        q_value = self.qfunction.get_q_value(state, action)
        delta = self.get_delta(reward, q_value, state, next_state, next_
↪action)

        self.qfunction.update(state, action, delta)

        state = next_state
        action = next_action
        episode_reward += reward * (self.mdp.discount_factor ** step)
        step += 1

    rewards.append(episode_reward)

    return rewards

""" Calculate the delta for the update """

    def get_delta(self, reward, q_value, state, next_state, next_action):
        next_state_value = self.state_value(next_state, next_action)
        delta = reward + self.mdp.discount_factor * next_state_value - q_value
        return self.alpha * delta

""" Get the value of a state """

    def state_value(self, state, action):
        abstract

```

We will see later that we inherit from TemporalDifferenceLearner for other algorithms that are very similar to Q-learning.

We inherit from this class to implement the Q-learning algorithm:

```

from temporal_difference_learner import TemporalDifferenceLearner

class QLearning(TemporalDifferenceLearner):
    def state_value(self, state, action):
        (_, max_q_value) = self.qfunction.get_max_q(state, self.mdp.get_
↪actions(state))
        return max_q_value

```

We can see that the TemporalDifferenceLearner does most of the work. All the QLearning class has to do is define what the value of $V(s')$ for the new state s' , which is the state and the next action that will be executed. Why does we model it like this instead of just implementing all of this in a single algorithm? In the next section on [SARSA](#), we will see why.

Using this implementation, we execute 100 episodes on the GridWorld example, resulting in the following Q-function, where each cell represents a cell from the GridWorld example and the four entries correspond to the Q-values for the respective direction:

```

from gridworld import GridWorld
from qtable import QTable
from qlearning import QLearning
from q_policy import QPolicy

```

(continues on next page)

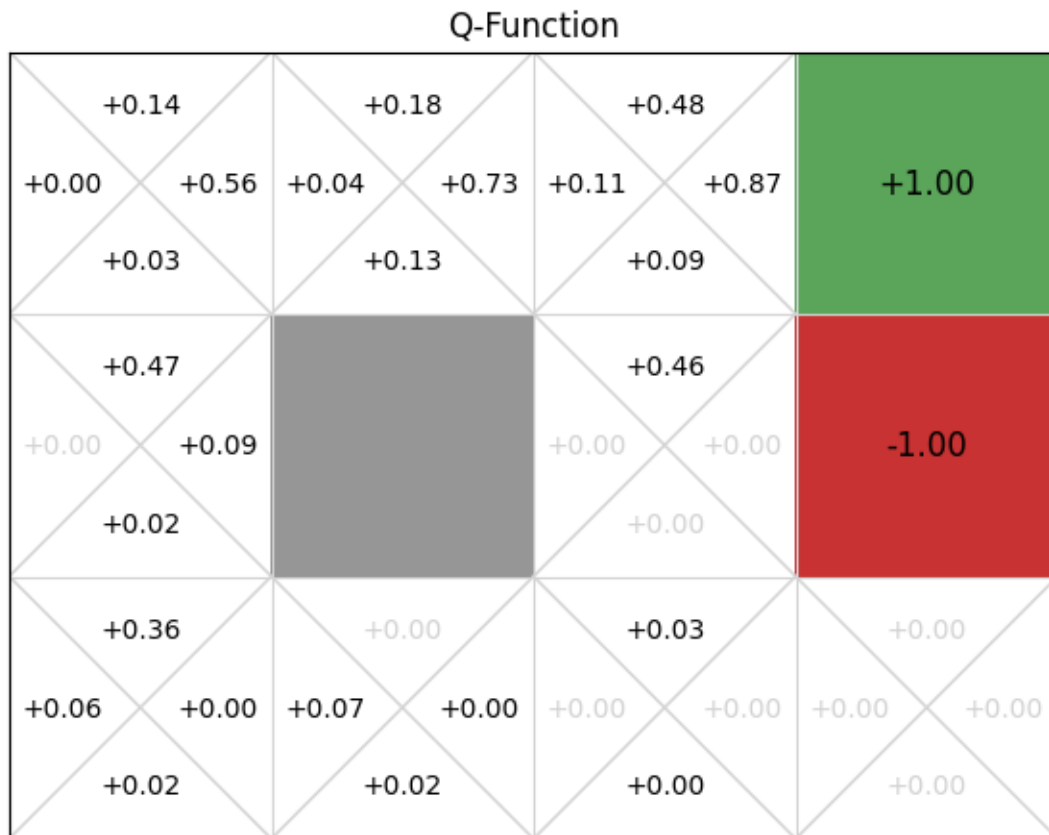
(continued from previous page)

```

from stochastic_q_policy import StochasticQPolicy
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy

gridworld = GridWorld()
qfunction = QTable()
QLearning(gridworld, EpsilonGreedy(), qfunction).execute(epochs=100)
gridworld.visualise_q_function(qfunction, "Q-Function", grid_size=1.5)

```



If we compare this to the value function for the *value iteration implementation*, we can see that the values learnt are not very accurate. Training for more episodes would result in more accurate values, but the α parameter means that recent information is weighted 0.1 in this case, and any unusual samples (from exploration or noise in the simulation), can affect the values.

Despite this, if we extract a policy from this, we still see that the policy corresponds to the optimal policy, although this is by no means guaranteed:

```

policy = QPolicy(qfunction)
gridworld.visualise_policy(policy)

```

→	→	→	+1.00
↑		↑	-1.00
↑	←	↑	↑

Below, we can explore how the Q-values for each state-action pair are learnt. If we play or step through the following visualisation, we can see that in early episodes, the Q-values that are learnt are not close to optimal because the initial episodes are quite long, so the discount factor means the rewards received are low. However, after just a few episodes, even these inaccurate Q-values give the multi-armed bandit some signal, and episodes start to become shorter, while the Q-values being learnt are updated to become more accurate:

2.3.6 SARSA: On-policy temporal difference learning

SARSA (State-action-reward-state-action) is an on-policy reinforcement learning algorithm. It is very similar to Q-learning, except that in its update rule, instead of estimate the future discount reward using $\max_{a \in A(s)} Q(s', a)$, it actually selects the next action that it will execute, and updates using that instead. Taking this approach is known as **on-policy reinforcement learning**. Later in this section, we'll discuss why this matters, but for now, let's look at the SARSA algorithm and on-policy learning a bit more.

Definition – On-policy and off-policy reinforcement learning

Instead of estimating $Q(s', a')$ for the best estimated future state during update, **on-policy reinforcement learning** uses the actual next action to update.

On-policy learning estimates $Q^\pi(s, a)$ state action pairs, for the current behaviour policy π , whereas **off-policy learning** estimates the policy independent of the current behaviour

To illustrate how this differs, let's take a look at the SARSA algorithm.

Algorithm 5 (SARSA)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Output : Q-function Q

Initialise Q arbitrarily; e.g., $Q(s, a) \leftarrow 0$ for all s and a

repeat

$s \leftarrow$ the first state in episode e

 Select action a to apply in s ;

 e.g. using Q and a multi-armed bandit algorithm such as ϵ – greedy

repeat (for each step in episode e)

 Execute action a in state s

 Observe reward r and new state s'

 Select action a to apply in s' ;

 e.g. using Q and a multi-armed bandit algorithm such as ϵ – greedy

$\delta \leftarrow r + \gamma \cdot Q(s', a') - Q(s, a)$

$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \delta$

$s \leftarrow s'$

$a \leftarrow a'$

until s is the last state of episode e (a terminal state)

until Q converges

The difference between the Q-learning and SARSA algorithms is what happens in the update in the loop body.

Q-learning: (1) selects an action a ; (2) takes that actions and observes the reward & next state s' ; and (3) updates **optimistically** by assuming the future reward is $\max_{a'} Q(s', a')$ – that is, it assumes that future behaviour will be optimal (according to its policy).

SARSA: (1) selects action a' for the *next* loop iteration; (2) in the next iteration, takes that action and observes the reward & next state s' ; (3) only then chooses a' for the next iteration; and (4) updates using the estimate for the actual next action chosen – which may not be the greediest one (e.g. it could be selected so that it can explore).

So what difference does this really make? There are two main differences:

- Q-learning will converge to the optimal policy irrelevant of the policy followed, because it is **off-policy**: it uses the greedy reward estimate in its update rather than following the policy such as ϵ -greedy). Using a random policy, Q-learning will still converge to the optimal policy, but SARSA will not (necessarily).
- Q-learning learns an optimal policy, but this can be ‘unsafe’ or risky *during training*.

Example – SARSA update

For this example, we will use the same Q-table as the earlier Q-learning example.

Assume that in state (2,2), the action ‘Up’ is chosen and executed successfully, which would return to state (2,2) there is no cell above (2,2). The next selected action is ‘Left’. Note that this is not the maximum action according to the Q-table – the selection function has explored instead of exploited. Using the Q-table above, we would update the Q-value using SARSA as follows:

$$\begin{aligned} Q((2,2), N) &\leftarrow Q((2,2), N) + \alpha[r + \gamma Q((2,2), W) - Q((2,2), N)] \\ &\leftarrow 0.79 + 0.1[0 + 0.9 \cdot Q((2,2), W) - Q((2,2), N)] \\ &\leftarrow 0.79 + 0.1[0 + 0.9 \cdot 0.72 - 0.79] \\ &\leftarrow 0.7758 \end{aligned}$$

Implementation

As with the Q-learning agent, we inherit from the `TemporalDifferenceLearner` class to implement SARSA. But the value of the next state $V(s')$ is calculated differently in the SARSA class:

```
from temporal_difference_learner import TemporalDifferenceLearner

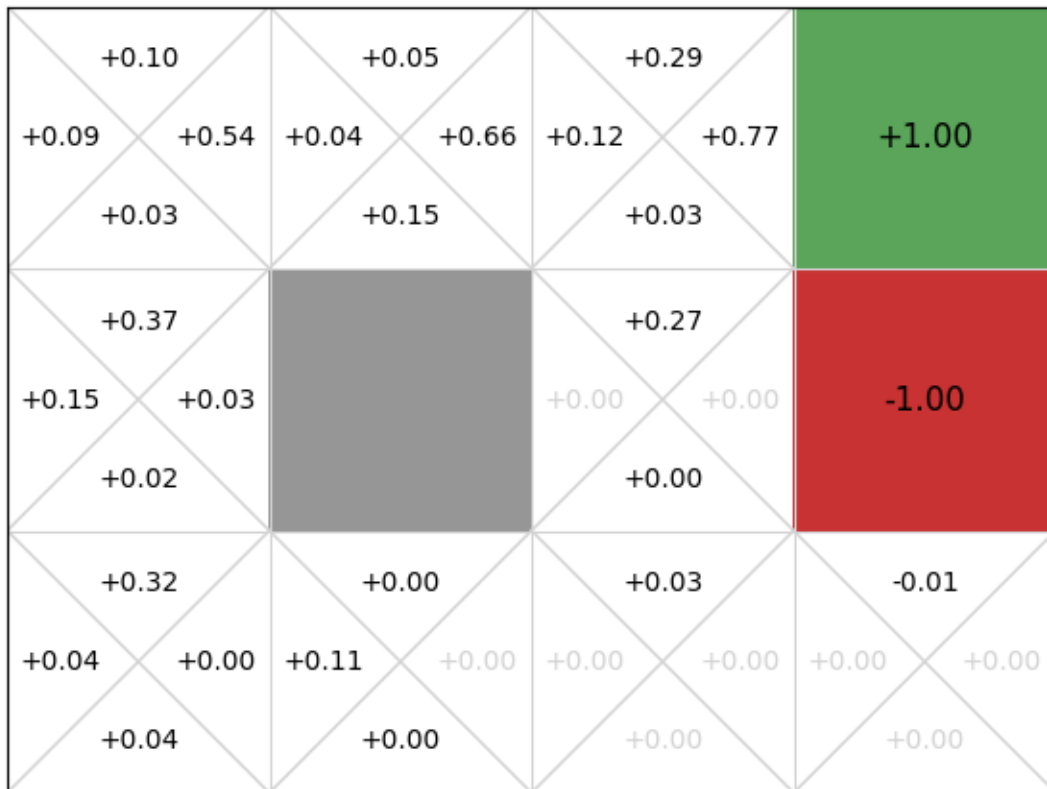
class SARSA(TemporalDifferenceLearner):
    def state_value(self, state, action):
        return self.qfunction.get_q_value(state, action)
```

So, as we can see, the value of state is instead $Q(s', a')$ instead of $\max_{a \in A} Q(s, a)$.

As with Q-learning, we execute SARSA for 100 episodes:

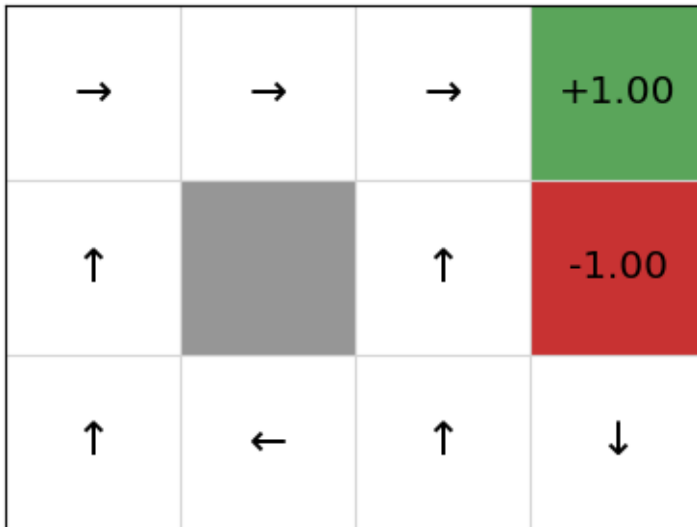
```
from gridworld import GridWorld
from qtable import QTable
from sarsa import SARSA
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy

gridworld = GridWorld()
qfunction = QTable()
SARSA(gridworld, EpsilonGreedy(), qfunction).execute(episodes=100)
gridworld.visualise_q_function(qfunction)
```



Again, we get an approximate Q-function. In this particular run, the policy is not optimal, because the action the policy selects from state (3,0) is down, not left, and the action from (2,0) is left, not up.

```
policy = QPolicy(qfunction)
gridworld.visualise_policy(policy)
```



This is (probably!) not because the SARSA implementation, but is because of the randomness in exploration combined with the value of alpha being quite high. A high value of alpha will learn more quickly, but this will also weight later updates more, so any unlikely events occurring late in the training will result in inaccurate Q-values. By selecting a lower value of alpha and training for more episodes, we can increase the likelihood of resulting in an optimal policy. This will require more time and resources to compute. In an example like GridWorld, this is not an issue, but for larger systems, it could be.

SARSA vs. Q-learning example: Cliff world

Consider the example below called “Cliff World”, which is taken from Chapter 6 of Sutton and Barto’s *Introduction to Reinforcement Learning* book (see further reading below). The bottom-left cell is the starting state and the bottom-right is the goal state, which receives a reward of 0. The four middle cells represent a cliff. Falling off the cliff receives a reward of -5. All other actions cost -0.05. Unlike the earlier GridWorld example, all actions are deterministic, which means that if the agent chooses to go to another cell, it will arrive at that cell with 100% probability. However, $P_a(s' | s)$ is unknown to the learning agent.

```
from gridworld import CliffWorld

cliffworld = CliffWorld()
cliffworld_image = cliffworld.visualise()
```



Let's try training this with Q-learning for 2000 episodes, using an epsilon greedy strategy with $\epsilon = 0.2$. The resulting Q-table is:

```
qfunction = QTable()
rewards = QLearning(cliffworld, EpsilonGreedy(epsilon=0.2), qfunction).
    ↪ execute(episodes=2000)
```

From this, we extract the following policy:

```
policy = QPolicy(qfunction)
cliffworld.visualise_policy(policy)
```

←	←	→	→	↓	↓
↓	→	→	→	→	↓
→	→	→	→	→	↓
↑	-5.00	-5.00	-5.00	-5.00	+0.00

We can see that the policy will initially move the agent up, and then along the cliff, going down to the terminal state at the end, receiving the reward of 5. We can see that the policy (and Q-table) for the upper cells are somewhat inaccurate: because they are low value states, they have not been explored as much as the states along the cliff.

Now, let's try training the same problem with SARSA:

```
cliffworld = CliffWorld()
qfunction = QTable()
rewards = SARSA(cliffworld, EpsilonGreedy(epsilon=0.2), qfunction).
->execute(epochs=2000)
```

Extracting the policy, we get:

```
policy = QPolicy(qfunction)
cliffworld.visualise_policy(policy)
```

→	→	→	→	→	↓
→	→	→	→	→	↓
↑	↑	→	→	→	↓
↑	-5.00	-5.00	-5.00	-5.00	+0.00

We can see that SARSA will instead not go along the cliff, but will take a sub-optimal path that avoids the cliff. *Why is this so?*

The answer is because SARSA uses a reward from the *actual* next action that is executed. Even with a mature Q-function, with $\epsilon = 0.2$ in the bandit used, the actual next action will be an exploratory action with probability 0.2, which means some of the time, the next action chosen will be “down”, so the agent falls off the cliff.

Consider each agent moving from state (1, 1) to state (2, 1). The Q-learning agent will update its Q-value for the preceding action by assuming that the agent continues along the cliff path, including $\max_{a \in A} Q(s, a)$ as the temporal difference reward. However, 10% of the time, the next action is NOT the optimal action because the agent will explore. Some of exploration actions will make the agent fall off the cliff, but this negative reward is not learnt by the Q-learning agent. The SARSA agent, on the other hand, selects its next action *before* the update, so in the cases where it chooses an action from state (2, 1) that falls off the cliff, the value $Q(s', a')$ will include this negative reward. As a result, the SARSA agent learns that staying close to the cliff is a “risky” behaviour, so will learn to instead take the safe path away from the cliff: exploring from the safe path does not result in a strong negative reward. As such, the SARSA agent will fall off the cliff less than the Q-learning agent during training.

However, *during* learning, the agent will still fall off the cliff sometimes when the agent is exploring actions. If trained using SARSA, the result will be a sub-optimal policy that learns the safe path. The SARSA learning agent will still fall off the cliff sometimes when exploring actions, however, it will fall off *less* than the Q-learning agent because it takes actions on the safe path more often during learning?

Consider the following in which we run both Q-learning and SARSA for 2000 episodes using epsilon greedy with $\epsilon = 0.2$. Then, we take the resulting Q-function and run another 2000 episodes following the policy (which is equivalent to using an epsilon greedy strategy with $\epsilon = 0.0$, initialising with the trained Q-function). If we plot the rewards for each episode for both SARSA and Q-Learning, we can see that SARSA receives more rewards the more we train, but at 2000 episodes when we start using the policy, Q-learning receives a higher reward per episode:

```
from gridworld import CliffWorld
from tests.compare_convergence_curves import qlearning_vs_sarsa
from gridworld import CliffWorld
```

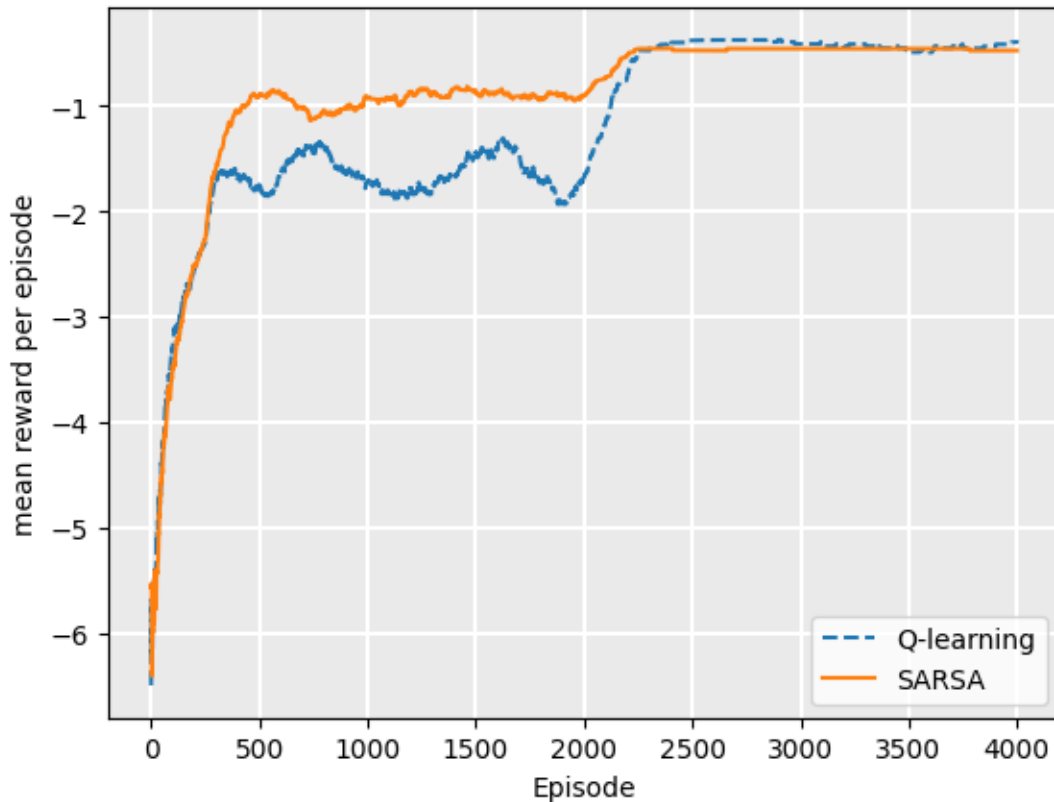
(continues on next page)

(continued from previous page)

```
from tests.plot import Plot

mdp_q = CliffWorld()
mdp_s = CliffWorld()

qlearning_vs_sarsa(mdp_q, mdp_s, episodes=2000)
```



During training, SARSA receives a higher average reward *per episode* than Q-Learning, because it falls off the cliff less as its policy improves. The Q-learning agent will follow the path along the cliff, but fall off when it explores, meaning that the average reward is lower. However, Q-learning learns the optimal policy, meaning that once we extract the policy, its rewards will be higher on average.

How is it possible that on-policy learning has a sub-optimal policy but higher rewards during training?

Consider a case of two agents training: one with Q-learning and one with SARSA, both using ϵ -greedy with $\epsilon=0.2$. Then, consider training episode 100 for each agent. From the plot above, we can see that both policies are close to converged.

However, once training is complete, we extract a policy. Because the actions are deterministic, the Q-learning policy is optimal: it will follow the path next to the cliff, but will not fall off. The SARSA agent will follow the safe path, but this safety is no longer required because no exploration is done.

The end result for the SARSA (on policy) method is a sub-optimal policy, but one that achieves stronger rewards during training.

SARSA vs. Q-learning example: Contested crossing

What happens when we train on a more complicated example?

In a game such as the Contested Crossing example, the addition of even a small number of extra actions and outcomes has a significant effect on the ability of both Q-learning and SARSA to find a good policy.

In the example of CliffWorld, multiple runs of the algorithm above (train for n episodes then test for n episodes) produce a stable optimal policy for when n is about 500. If we go through the procedure of training and testing for both Q-learning and SARSA multiple times, the final policy gives close to the same output each time.

In the case of Contested Crossing, there are seven actions available (compared to four for CliffWorld), the number of steps taken in executing the policy is somewhere between 6 and around 20 depending on the amount of damage the ship takes (compared to 7 for CliffWorld using Q-learning and 9 using SARSA) and the amount of chance involved is significantly higher, because a ship can shoot or be shot at any time within the danger zones, with medium or high probability.

The result of this is to considerably increase the amount of time taken to converge towards a good policy, for either Q-learning or SARSA. For $n = 2000$, the policy found by Q-learning is usually – but not always – superior to that found by SARSA. This is insufficient training for either algorithm to converge on an optimal policy.

At $n = 20,000$, the optimal policy for Q-learning appears to have been found. There is still variation in the reward generated from implementing it, because of the high amount of chance in the Contested Crossing task. The SARSA policies at $n = 20,000$ does not converge on the optimal policy because of its on-policy nature.

The resulting final policies, when executed after 20,000 episodes, also show that the Q-learning policy receives a higher reward per episode than SARSA.

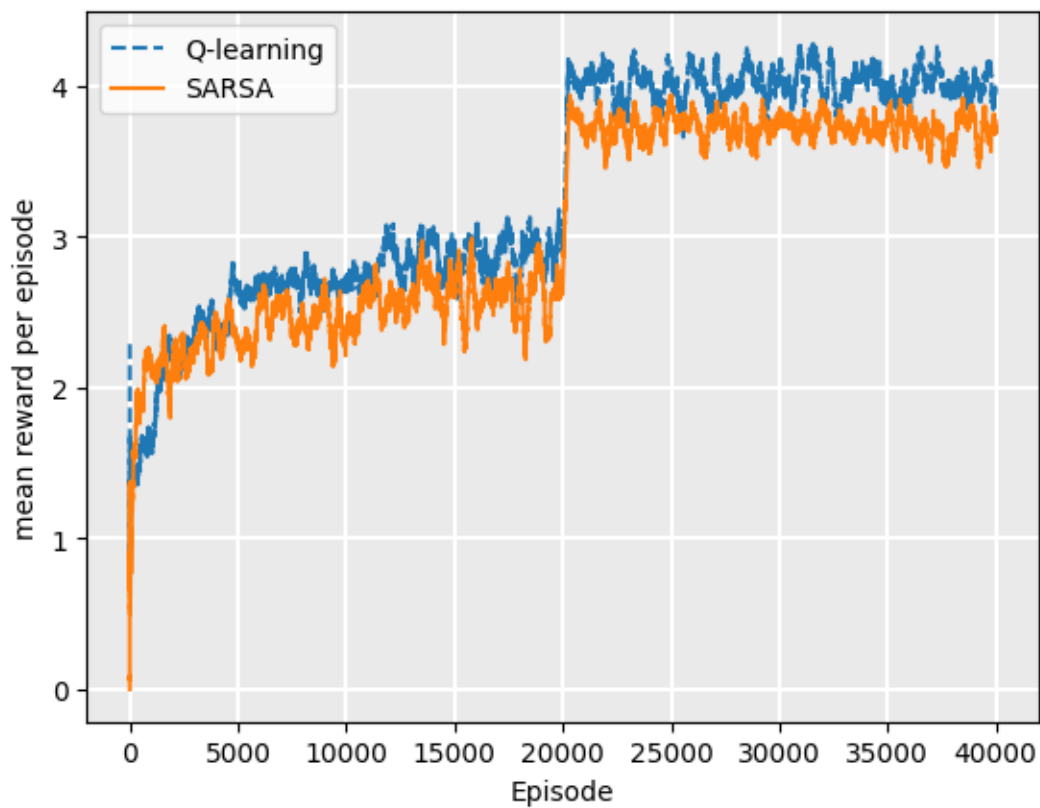
```
from contested_crossing import ContestedCrossing
from tests.compare_convergence_curves import qlearning_vs_sarsa
from tests.plot import Plot

mdp_q = ContestedCrossing()
mdp_s = ContestedCrossing()

qlearning_vs_sarsa(mdp_q, mdp_s, episodes=2000)

mdp_q = ContestedCrossing()
mdp_s = ContestedCrossing()

qlearning_vs_sarsa(mdp_q, mdp_s, episodes=20000)
```



2.3.7 On-policy vs. off-policy: Why do we have both?

There are a few reasons why we have both on-policy and off-policy learning.

Learning from prior experience

The main advantage of off-policy approaches is that they can use samples from sources other than their own policy. For example, off-policy agents can be given a set of episodes of behaviour from another agent, such as a human expert, and can learn a policy by **demonstration**. In Q-learning, this would mean instead of selecting action a to apply in state s using a multi-armed bandit algorithm on $Q(s, a)$, we can simply take the next action of a trajectory and then update Q as before. The policy that we are trying to learn is independent of the samples in the episodes. However, with SARSA, while we could in theory sample the same way, the update rule explicitly uses $Q(s', a')$, so the policy used to generate the trajectories in episodes is the same as the policy being learnt.

Learning on the job

The main advantage of on-policy approaches is that they are useful for ‘learning on the job’, meaning that it is better for cases in which we want an agent to learn optimal behaviour while operating in its environment.

For example, imagine a reinforcement learning agent that manages resources for a cloud-based platform and we have no prior data to inform a policy. We could program a simulator for this environment, but it is highly unlikely that the simulator would be an accurate reflection of the real world, as estimating the number of jobs, their lengths, their timing, etc., would be difficult without prior data. So, the only way to learn a policy is to get data from our cloud platform while it operates.

As such, if the average reward *per episode* is better using on-policy, this would give us better overall outcomes than off-policy learning, because the episodes are not practice – they actually influence real rewards, such as throughput, downtime, etc., and ultimately profit.

If we could run our reinforcement learning algorithm in a simulated environment before deploying (and we had reason to believe that simulated environment was accurate), off-policy learning may be better because its optimal policy could be followed.

Combining off-policy and on-policy learning

We can combine the two approaches for particular applications. For example, if we use take our cloud platform from above, it would be silly to start with a random policy. We would quickly lose customers as the scheduling etc., would be terrible.

A better approach would be to hand-craft an algorithm that gave good results initially, and use this with an off-policy approach to train a good initial policy. We could use the hand-crafted algorithm to select actions for training initially to gain an initial policy.

However, this new policy will merely mimic the hand-crafted algorithm. Presumably we are using reinforcement learning because the problem is so complex that a hand-crafted approach is not good enough. So, we can then take the policy trained using off-policy and optimise it further using on-policy learning.

Another common place for combining off-policy and on-policy learning is when we have an existing approach and we can use data from this with an off-policy approach to come up with an initial policy, which can be refined using on-policy.

2.3.8 Evaluation and termination

How do we know how many episodes we should train a model-free learning algorithm for?

With *value iteration*, we terminate the algorithm once the improvement in value function reaches some threshold. However, in the model-free environment, we are not aiming to learn a complete policy – only enough to get us from the initial state to an absorbing state; or, in the case of an infinite MDP, to maximise rewards. Due to the randomness of the exploration and the fact that an episode may visit a state it has rarely visited, it is likely that for each episode, a Q-value of at least one state-action pair changes.

With model-free learning, we can instead evaluate the policy directly by **executing it and recording the reward we receive**. Then, we terminate when the policy has reached **convergence**. By convergence, we mean that the average cumulative reward of the policy is no longer increasing during learning.

There are a few ways we can measure this:

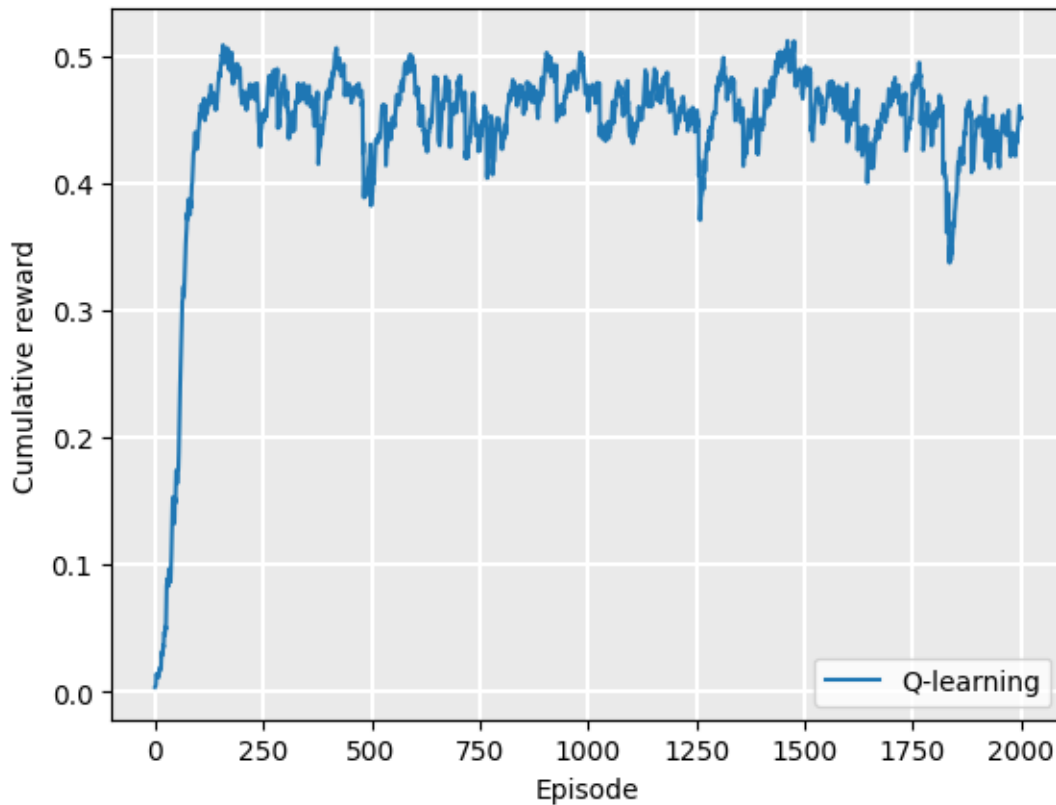
1. We can simply record the reward received during each episode of learning, and monitor how much this is increasing. The weakness with this is that a single episode can be noisy due to the exploration parameter used to select actions and the stochastic nature of the MDP. For example, if we use a *multi-armed bandit* to control exploration vs. exploitation during learning, then at each step, we choose a random action with probability ϵ . This means that we are not evaluating our real policy — we are evaluating our really policy plus some random actions.
2. We can pause our learning every now and then, and run our actual policy on the MDP. The strength of this is that we avoid randomness in action selection. However, we still have randomness caused by the stochastic nature of the MDP. The weakness is that it requires us to run more episodes; or more accurately, we are executing episodes and not learning from them.
3. To avoid stochasticity in both the environment and the exploration strategy, we can average the reward over a number of simulations. The weakness is that this is more costly – it requires us to run additional episodes that do not explore and learn.

In these notes, we use the strategy in item 2 above. Every 20 episodes, we run our policy and record the cumulative reward. As we don't know what the maximum reward could be, we monitor the learning and terminate when the *exponential moving average* of our reward is no longer increasing, similar to what we did in *evaluating the value iteration policy*. Here, we plot the **learning curve** for Q-learning on the GridWorld example:

```
qfunction = QTable()
mdp = GridWorld()
rewards = QLearning(mdp, EpsilonGreedy(), qfunction).execute(episodes=2000)
```

Once we have the rewards of the episodes, we can plot the rewards:

```
Plot.plot_cumulative_rewards(["Q-learning"], [rewards])
```

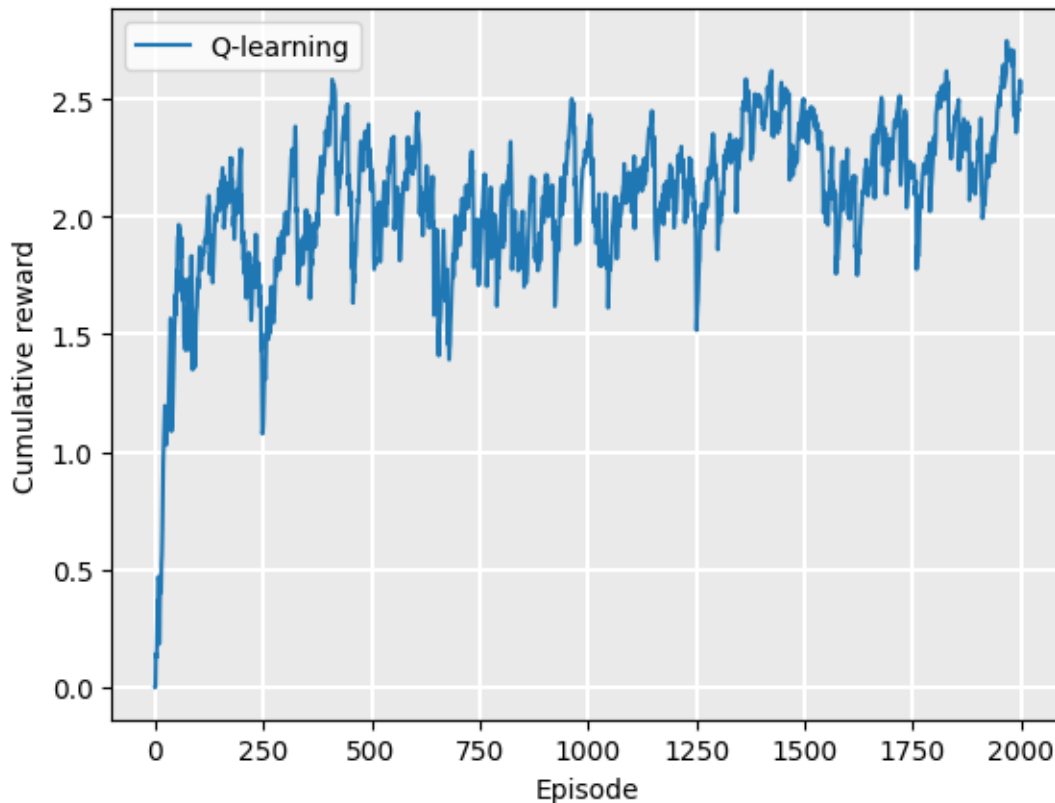


We can see that the policy converges at around 1000 episodes. However, recalling that this is smoothed using the exponential moving average, it probably converges in fewer episodes, but the early episodes weight on the average still.

Next, we do the same for the contested crossing example:

```
qfunction = QTable()
mdp = ContestedCrossing()
rewards = QLearning(mdp, EpsilonGreedy(), qfunction).execute(episodes=2000)

Plot.plot_cumulative_rewards(["Q-learning"], [rewards])
```



We can see a convergence at about 1250-1500 episodes.

2.3.9 Limitations of Q-learning and SARSA

The standard versions that we see in this section have two major limitations:

1. Because we need to select the best action a in Q-learning, we iterate over all actions. This limits Q-learning to discrete action spaces.
2. If we use a Q-table to represent our Q-function, both state spaces and action spaces must be discrete, and further, they must be modest in size or the Q-table will become too large to fit into memory or at least so large that it will take many episodes to sample all state-action pairs.

2.3.10 Applications of Reinforcement Learning

- Checkers (Samuel, 1959): first use of RL in an interesting real game
- (Inverted) Helicopter Flight (Ng et al. 2004), better than any human
- Computer Go (AlphaGo 2016): AlphaGo beats Go world champion Lee Sedol 4:1
- Atari 2600 Games (DQN & Blob-PROST 2015): human-level performance on half of 50+ games
- Robocup Soccer Teams (Stone & Veloso, Reidmiller et al.): World's best player of simulated soccer, 1999; Runner-up 2000
- Inventory Management (Van Roy, Bertsekas, Lee & Tsitsiklis): 10-15% improvement over industry standard methods

- Dynamic Channel Assignment (Singh & Bertsekas, Nie & Haykin): World's best assigner of radio channels to mobile telephone calls
- TD-Gammon and Jellyfish (Tesauro, Dahl): World's best backgammon player. Grandmaster level

2.3.11 Takeaways

Takeaways

- **On-policy** reinforcement learning uses the action chosen by the policy for the update.
 - **Off-policy** reinforcement learning assumes that the next action chosen is the action that has the maximum Q-value, but this may not be the case because with some probability the algorithm will explore instead of exploit.
 - **Q-Learning** (off-policy) does it relative to the greedy policy.
 - **SARSA** (on-policy) learns action values relative to the policy it follows.
 - If we know the MDP, we can use model-based techniques:
 - **Offline**: Value Iteration
 - **Online**: *Monte Carlo Tree Search* and friends.
 - We can also use model-free techniques if we know the MDP model: we just sample transitions and observe rewards from the model.
 - If we do **not** know MDP, we need to use model-free techniques:
 - **Offline**: Q-learning, SARSA, and friends.
-

2.3.12 Further Reading

- Chapter 6 of *Introduction to Reinforcement Learning*, Sutton and Barto

2.4 n-step reinforcement learning

Learning outcomes

The learning outcomes of this chapter are:

1. Manually apply n-step reinforcement learning approximation to solve small-scale MDP problems
 2. Design and implement n-step reinforcement learning to solve medium-scale MDP problems automatically
 3. Argue the strengths and weaknesses of n-step reinforcement learning
-

2.4.1 Overview

In the previous sections on of this chapter, we looked at two fundamental temporal difference (TD) methods for reinforcement learning: Q-learning and SARSA.

These two methods have some weaknesses in this basic format:

1. Unlike Monte-Carlo methods, which reach a reward and then backpropagate this reward, TD methods use bootstrapping (they estimate the future discounted reward using $Q(s, a)$), which means that for problems with sparse rewards, it can take a long time for rewards to propagate throughout a Q-function.
2. Rewards can be sparse, meaning that there are few state/actions that lead to non-zero rewards. This is problematic because initially, reinforcement learning algorithms behave entirely randomly and will struggle to find good rewards.
3. Both methods estimate a Q-function $Q(s, a)$, and the simplest way to model this is via a Q-table. However, this requires us to maintain a table of size $|A| \times |S|$, which is prohibitively large for any non-trivial problem.
4. Using a Q-table requires that we visit every reachable state many times and apply every action many times to get a good estimate of $Q(s, a)$. Thus, if we never visit a state s , we have no estimate of $Q(s, a)$, even if we have visited states that are very similar to s .

To get around limitations 1 and 2, we are going to look at **n-step temporal difference learning**: ‘Monte Carlo’ techniques execute entire episodes and then backpropagate the reward, while basic TD methods only look at the reward in the next step, estimating the future rewards. n-step methods instead look n steps ahead for the reward before updating the reward, and then estimate the remainder. In future parts of these notes, we’ll look at techniques for mitigating limitations 3 and 4.

n-step TD learning comes from the idea used in the image below, from Sutton and Barto (2020). Monte Carlo methods uses ‘deep backups’, where entire episodes are executed and the reward backpropagated. Methods such as Q-learning and SARSA use ‘shallow backups’, only using the reward from the 1-step ahead. n-step learning finds the middle ground: only update the Q-function after having explored ahead n steps.

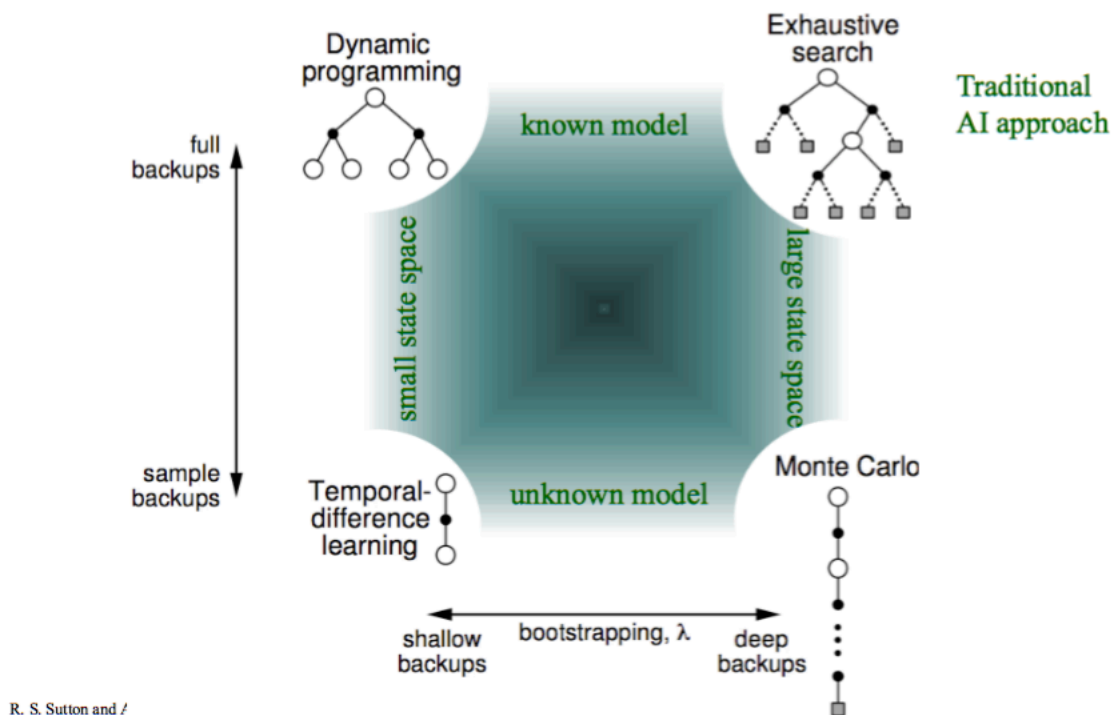


Fig. 2.1: Reinforcement learning approaches (from Sutton and Barto (2020))

2.4.2 n-step TD learning

We will look at n -step reinforcement learning, in which n is the parameter that determines the number of steps that we want to look ahead before updating the Q-function. So for $n = 1$, this is just “normal” TD learning such as Q-learning or SARSA. When $n = 2$, the algorithm looks one step beyond the immediate reward, $n = 3$ it looks two steps beyond, etc.

Both Q-learning and SARSA have an n -step version. We will look at n -step learning more generally, and then show an algorithm for n -step SARSA. The version for Q-learning is similar.

Intuition

The details and algorithm for n -step reinforcement learning making it seem more complicated than it really is. At an intuitive level, it is quite straightforward: at each step, instead of updating our Q-function or policy based on the reward received from the previous action, plus the discounted future rewards, we update it based on the last n rewards received, plus the discounted future rewards from n states ahead.

Consider the following interactive gif, which shows the update over an episode of five actions. The bracket represents a window size of $n = 3$:

At time $t = 0$, no update can be made because there is no action.

At time $t = 1$, after action a_0 , no update is done yet. In standard TD learning, we would update $Q(s_0, s_0)$ here. However, because we have only one reward instead of three ($n = 3$), we delay the update.

At time $t = 2$, after action a_1 , again we do not update because we have just two rewards, instead of three.

At time $t = 3$, after action a_2 , we have three rewards, so we do our first update. But note: we update $Q(s_0, a_0)$ – the first state-action pair in the sequence, rather than the most recent state action pair (s_2, a_2) . Notice that we consider the rewards r_1, r_2 , and r_3 (appropriately discounted), and use the discounted future reward $V(s_3)$. Effectively, the update of $Q(s_0, a_0)$ “looks forward” three actions into the future instead one, because $n = 3$.

At time $t = 4$, after action a_3 , the update is similar: we now update $Q(s_1, a_1)$.

At time $t = 5$, it is similar again, except that state s_5 is a terminal state, so we do not include the discounted future reward – there is no state s_6 such that we can estimate $V(s_6)$, so it is omitted.

At time $t = 6$, we can see that the window slides beyond the length of the episode. However, even though we have reached the terminal state, we continue updating — this time updating $Q(s_3, s_3)$. We need to do this because we have still not updated the Q-values for all state-action pairs that we have executed.

At time $t = 7$, we do the final update — this time for $Q(s_4, a_4)$; and the episode is complete.

So, we can see that, intuitively, n -step reinforcement learning is quite straightforward. However, to implement this, we need data structures to keep track of the last n states, actions, and rewards, and modifications to the standard TD learning algorithm to both delay Q-value updates in the first n steps of an episode, and to continue updating beyond the end of the episode for ensure the last n state-action pairs are updated. This “book-keeping” code can be confusing at first, unless we already have an intuitive understanding of what it achieves.

Discounted Future Rewards (again)

When calculating a discounted reward over an episode, we simply sum up the rewards over the episode:

$$G_t = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

We can re-write this as:

$$G_t = r_1 + \gamma(r_2 + \gamma(r_3 + \gamma(r_4 + \dots)))$$

If G_t is the value received at time-step t , then

$$G_t = r_t + \gamma G_{t+1}$$

In TD(0) methods such as Q-learning and SARSA, we do not know G_{t+1} when updating $Q(s, a)$, so we estimate using bootstrapping:

$$G_t = r_t + \gamma \cdot V(s_{t+1})$$

That is, the reward of the entire future from step t is estimated as the reward at t plus the estimated (discounted) future reward from $t + 1$. $V(s_{t+1})$ is estimated using the maximum expected return (Q-learning) or the estimated value of the next action (SARSA).

This is a **one-step return**.

Truncated Discounted Rewards

However, we can estimate a two-step return:

$$G_t^2 = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$$

a three-step return:

$$G_t^3 = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})$$

or **n-step returns**:

$$G_t^n = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^n V(s_{t+n})$$

In this above expression G_t^n is the full reward, **truncated** at n steps, at time t .

The basic idea of n-step reinforcement learning is that we do not update the Q-value immediately after executing an action: we wait n steps and update it based on the n-step return.

If T is the termination step and $t + n \geq T$, then we just use the full reward.

In Monte-Carlo methods, we go all the way to the end of an episode. Monte-Carlo Tree Search is one such Monte-Carlo method, but there are others that we do not cover.

Updating the Q-function

The update rule is then different. First, we need to calculate the truncated reward for n steps, in which τ is the time step that we are updating for (that is, τ is the action taken n steps ago):

$$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} r_i$$

This just sums the discounted rewards from time step $\tau + 1$ until either n steps ($\tau + n$) or termination of the episode (T), whichever comes first.

Then calculate the n -step expected reward:

$$\text{If } \tau + n < T \text{ then } G \leftarrow G + \gamma^n Q(s_{\tau+n}, a_{\tau+n}).$$

This adds the future expect reward if we are not at the end of the episode (if $\tau + n < T$).

Finally, we update the Q -value:

$$Q(s_\tau, a_\tau) \leftarrow Q(s_\tau, a_\tau) + \alpha[G - Q(s_\tau, a_\tau)]$$

In the update rule above, we are using a SARSA update, but a Q -learning update is similar.

n-step SARSA

While conceptually this is not so difficult, an algorithm for doing n -step learning needs to store the rewards and observed states for n steps, as well as keep track of which step to update. An algorithm for n -step SARSA is shown below.

Algorithm 6 (n-step SARSA)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$ number of steps n

Output : Q -function Q

Initialise Q arbitrarily; e.g., $Q(s, a) = 0$ for all s and a

repeat

 Select action a to apply in s using Q -values in Q and a multi-armed bandit algorithm such as ϵ - greedy

$ss = \langle s \rangle$

$as = \langle a \rangle$

$rs = \langle \rangle$

while ss is not empty **do**

if s is not a terminal state **then**

 Execute action a in state s

 Observe reward r and new state s'

$rs \leftarrow rs + \langle r \rangle$

if s' is not a terminal state **then**

 Select action a' to apply in s' using Q and a multi-armed bandit algorithm

$ss \leftarrow ss + \langle s' \rangle$

$as \leftarrow as + \langle a' \rangle$

if $|rs| = n$ **or** s is a terminal state **if**

$G \leftarrow \sum_{i=0}^{|rs|-1} \gamma^i rs_i$

if s is not a terminal state **then**

$G \leftarrow G + \gamma^n Q(s', a')$

$Q(ss_0, as_0) \leftarrow Q(ss_0, as_0) + \alpha[G - Q(ss_0, as_0)]$

$rs \leftarrow rs_{[1:n+1]}$

$ss \leftarrow ss_{[1:n+1]}$

$as \leftarrow as_{[1:n+1]}$

$s \leftarrow s'$

$a \leftarrow a'$

until Q converges

This is similar to standard SARSA, except that we are storing the last n states, actions, and rewards; and also calculating the rewards on the last five rewards rather than just one. The variables ss , as , and rs as the list of the last n states, actions,

and rewards respectively. We use the syntax ss_i to get the i^{th} element of the list, and the Python-like syntax $ss_{[1:n+1]}$ to get the elements between indices 1 and $n + 1$ (remove the first element).

As with SARSA and Q-learning, we iterate over each step in the episode. The first branch simply executes the selected action, selects a new action to apply, and stores the state, action, and reward.

It is the second branch where the actual learning happens. Instead of just updating with the 1-step reward r , we use the n -step reward G . This requires a bit of “book-keeping”. The first thing we do is calculate G . This simply sums up the elements in the reward sequence rs , but remembering that they must be discounted based on their position in rs . The next line adds the TD-estimate $y^n Q(s', a')$ to G , but only if the most recent state is not a terminal state. If we have already reached the end of the episode, then we must exclude the TD-estimate of the future reward, because there will be no such future reward. Of importance, also note that we multiply this by γ^n instead of γ . Why? This because the future estimated reward is n steps from state ss_0 . The $n - \text{step}$ reward in G comes first. Then we do the actual update, which updates the state-action pair (ss_0, as_0) that is n -steps back.

The final part of this branch removes the first element from the list of states, actions, and rewards, and moves on to the next state.

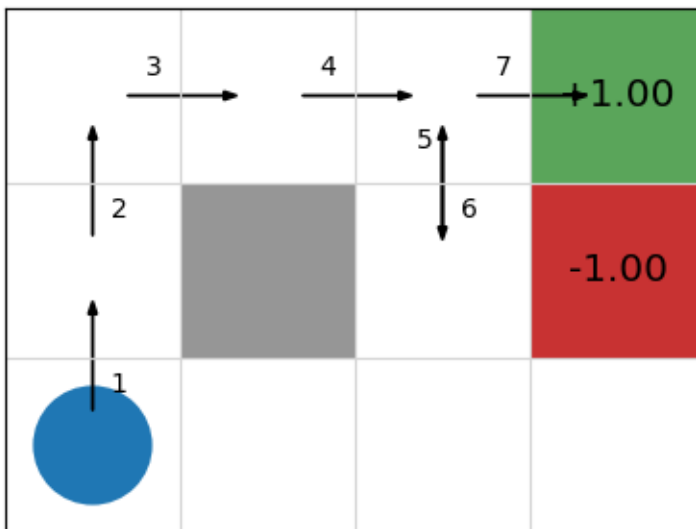
What is the effect of all of the computation? This algorithm differs from standard SARSA as follows: it only updates a state-action pair after it has seen the next n rewards that are returned, rather than just the single next reward. This means that there are no updates until the n^{th} steps of the episode; and no TD-estimate of the future reward in the last n steps of the episode.

Computationally, this is not much worse than 1-step learning. We need to store the last n states, but the per-step computation is small and uniform for n -step, just as for 1-step.

Example – n -step SARSA update

Consider our simple 2D navigation task, in which we do not know the probability transitions nor the rewards. Initially, the reinforcement learning algorithm will be required to search randomly until it finds a reward. Propagating this reward back n -steps will be helpful.

Imagine the first episode consisting of the following (very lucky!) episode:



Assuming $Q(s, a) = 0$ for all s and a , if we traverse the episode the labelled episode, what will our Q-function look like for a 5-step update with $\alpha = 0.5$ and $\gamma = 0.9$?

For the first $n - 1$ steps of the episode, no update is made to the Q-values, but rewards and states are stored for future processing.

On step 5, we reach the end of our n -step window, and we start to update values because $|rs| = n$. We calculate G from rs and update $Q(ss_0, as_0)$, and then similarly for the 6th step:

$$\begin{aligned} \text{Step 5} \quad & rs = \langle 0, 0, 0, 0, 0 \rangle \\ & G = \gamma^0 rs_0 + \dots + \gamma^4 rs_4 + \gamma^5 Q((2, 1), Up) = 0 \\ & Q((0, 0), Up) = 0 \\ \text{Step 6} \quad & rs = \langle 0, 0, 0, 0, 0 \rangle \\ & G = \gamma^0 rs_0 + \dots + \gamma^4 rs_4 + \gamma^5 Q((2, 2), Right) = 0 \\ & Q((0, 1), Up) = 0 + 0.5[0 - 0] = 0 \end{aligned}$$

We can see that there are no rewards in the first five steps, so the Q-values first two state-action pairs in the sequence $Q((0, 0), Up)$ and $Q((0, 1), Up)$ both remain 0, their original values.

When we reach step 7, however, we receive a reward. We can then update the Q-value $Q((0, 2), Right)$ of the state-action pair at step 3:

$$\begin{aligned} \text{Step 7} \quad & rs = \langle 0, 0, 0, 0, 1 \rangle \\ & G = \gamma^0 rs_0 + \dots + \gamma^4 rs_4 + \gamma^5 Q((3, 2), Terminate) = 0.9^4 \cdot 1 = 0.6561 \\ & Q((0, 2), Right) = 0 + 0.5[0.6561 - 0] = 0.32805 \end{aligned}$$

From this point, s is a terminal state, so we no longer select and execute actions, nor store the rewards, actions, and states. However, we continue to update the steps in the episode, leaving off $Q(ss_5, as_5)$ because there is no future discount reward beyond the end of the episode:

$$\begin{aligned} \text{Step 8} \quad & rs = \langle 0, 0, 0, 1 \rangle \\ & G = \gamma^0 rs_0 + \dots + \gamma^4 rs_3 = 0.9^3 \cdot 1 = 0.729 \\ & Q((1, 2), Right) = 0 + 0.5[0.729 - 0] = 0.3645 \\ \text{Step 9} \quad & rs = \langle 0, 0, 1 \rangle \\ & G = \gamma^0 rs_0 + \dots + \gamma^3 rs_2 = 0.9^2 \cdot 1 = 0.81 \\ & Q((2, 2), Down) = 0 + 0.5[0.81 - 0] = 0.405 \\ \text{Step 10} \quad & rs = \langle 0, 1 \rangle \\ & G = \gamma^0 rs_0 + \dots + \gamma^2 rs_1 = 0.9^1 \cdot 1 = 0.9 = 0.9 \\ & Q((2, 1), Up) = 0 + 0.5[0.9 - 0] = 0.45 \\ \text{Step 11} \quad & rs = \langle 1 \rangle \\ & G = \gamma^0 rs_0 = 0.9^0 \cdot 1 = 1 \\ & Q((2, 2), Right) = 0 + 0.5[1 \cdot 1 - 0] = 0.5 \end{aligned}$$

At this point, there are no further states left to update, so the inner loop terminates, and we start a new episode.

2.4.3 Implementation

Below is a Python implementation of n -step temporal difference learning. We first implement an abstract superclass `NStepReinforcementLearner`, which contains most of the code we need, except the part that determines the value of state s' in the Q-function update, which is left to the subclass so we can support both n -step Q-learning and n -step SARSA:

```
class NStepReinforcementLearner:
    def __init__(self, mdp, bandit, qfunction, n, alpha=0.1):
        self.mdp = mdp
        self.bandit = bandit
        self.alpha = alpha
        self.qfunction = qfunction
        self.n = n

    def execute(self, episodes=100):
        for _ in range(episodes):
```

(continues on next page)

(continued from previous page)

```

state = self.mdp.get_initial_state()
actions = self.mdp.get_actions(state)
action = self.bandit.select(state, actions, self.qfunction)

rewards = []
states = [state]
actions = [action]

while len(states) > 0:
    if not self.mdp.is_terminal(state):
        (next_state, reward) = self.mdp.execute(state, action)
        rewards += [reward]
        next_actions = self.mdp.get_actions(next_state)

        if not self.mdp.is_terminal(next_state):
            next_action = self.bandit.select(
                next_state, next_actions, self.qfunction
            )
            states += [next_state]
            actions += [next_action]

    if len(rewards) == self.n or self.mdp.is_terminal(state):
        n_step_rewards = sum(
            [
                self.mdp.discount_factor ** i * rewards[i]
                for i in range(len(rewards))
            ]
        )

        if not self.mdp.is_terminal(state):
            next_state_value = self.state_value(next_state, next_action)
            n_step_rewards = (
                n_step_rewards
                + self.mdp.discount_factor ** self.n * next_state_value
            )

        q_value = self.qfunction.get_q_value(
            states[0], actions[0]
        )

        self.qfunction.update(
            states[0],
            actions[0],
            self.alpha * (n_step_rewards - q_value),
        )

        rewards = rewards[1 : self.n + 1]
        states = states[1 : self.n + 1]
        actions = actions[1 : self.n + 1]

    state = next_state
    action = next_action

""" Get the value of a state """

def state_value(self, state, action):
    abstract

```

We inherit from this class to implement the n -step Q-learning algorithm:

```
from n_step_reinforcement_learner import NStepReinforcementLearner

class NStepQLearning(NStepReinforcementLearner):
    def state_value(self, state, action):
        (_, max_q_value) = self.qfunction.get_max_q(state, self.mdp.get_
        actions(state))
        return max_q_value
```

Example – 1-step Q-learning vs 5-step Q-learning

Using the interactive graphic below, we compare 1-step vs. 5-step Q-learning over the first 20 episodes of learning in the GridWorld task. Using 1-step Q-learning, reaching the reward only informs the state from which it is reached in the first episode; whereas for 5-step Q-learning, it informs the previous five steps. Then, in the 2nd episode, if any action reaches a state that has been visited, it can access the TD-estimate for that state. There are five such states in 5-step Q-learning; and just one in 1-step Q-learning. On all subsequent iterations, there is more chance of encountering a state with a TD estimate and those estimates are better informed. The end result is that the estimates ‘spread’ throughout the Q-table more quickly:

2.4.4 Values of n

Can we just increase n to be infinity so that we get the reward for the entire episode? Doing this is the same as *Monte-Carlo reinforcement learning*, as we would no longer use TD estimates in the update rule. As we have seen, this leads to more variance in the learning.

What is the best value for n then? Unfortunately, there is no theoretically best value for n . It depends on the particular application and reward function that is being trained. In practice, it seems that values of n around 4-8 give good updates because we can easily assign credit to each of the 4-8 actions; that is, we can tell whether the 4-8 actions in the lookahead contributed to the score, because we use the TD estimates.

2.4.5 Takeaways

Takeaways

- n -step reinforcement learning **propagates** rewards back n steps to help with learning.
- It is conceptually quite simple, but the implementation requires a lot of ‘book-keeping’.
- Choosing a value of n for a domain requires experimentation and intuition.

2.4.6 Further Reading

- Chapter 7 of *Introduction to Reinforcement Learning*, Sutton and Barto

2.5 Monte-Carlo Tree Search (MCTS)

Learning outcomes

The learning outcomes of this chapter are:

1. Explain the difference between offline and online planning for MDPs.
 2. Apply MCTS solve small-scale MDP problems manually and program MCTS algorithms to solve medium-scale MDP problems automatically
 3. Construct a policy from Q-functions resulting from MCTS algorithms
 4. Integrate multi-armed bandit algorithms (including UCB) to MCTS algorithms
 5. Compare and contrast MCTS to value iteration
 6. Discuss the strengths and weaknesses of the MCTS family of algorithms.
-

2.5.1 Offline Planning & Online Planning for MDPs

We saw value iteration in the previous section. This is an **offline** planning method because we solve the problem offline for all possible states, and then use the solution (a policy) online to act.

Yet the state space S is usually *far* too big to determine $V(s)$ or π exactly. Even games like Go, which are famously difficult for reinforcement learning, requiring computational power available to only a handful of organisations, are small compared to many real-world problems.

There are methods to approximate the MDP by reducing the dimensionality of S , but we will not discuss these until later.

In **online** planning, planning is undertaken immediately before executing an action. Once an action (or perhaps a sequence of actions) is executed, we start planning again from the new state. As such, planning and execution are interleaved such that:

- For each state s visited, the set of all available actions $A(s)$ partially evaluated
- The quality of each action a is approximated by averaging the expected reward of trajectories over S obtained by repeated simulations, giving as an approximation for $Q(s, a)$.
- The chosen action is $\operatorname{argmax}_a Q(s, a)$

In online planning, we need access to a **simulator** that approximates the transitions function $P_a(s'|s)$ and reward function r of our MDP. A model can be used, however, often it is easier to write a simulator that can choose outcomes with probability $P_a(s'|s)$ than it is to analytically calculate the probabilities for any state. For example, consider games like StarCraft. Calculating the probability of ending in a state for a given action is more difficult than simulating possible states.

The simulator allows us to run repeated simulations of possible futures to gain an idea of what moves are likely to be good moves compared to others.

The question is: how to we do the repeated simulations? **Monte Carlo** methods are by far the most widely-used approach.

2.5.2 Overview

Monte Carlo Tree Search (MCTS) is a name for a *set* of algorithms all based around the same idea. Here, we will focus on using an algorithm for solving single-agent MDPs in a model-based manner. Later, we look at solving single-agent MDPs in a model-free manner and multi-agent MDPs using MCTS.

Foundation: MDPs as ExpectiMax Trees

To get the idea of MCTS, we note that MDPs can be represented as trees (or graphs), called **ExpectiMax** trees:

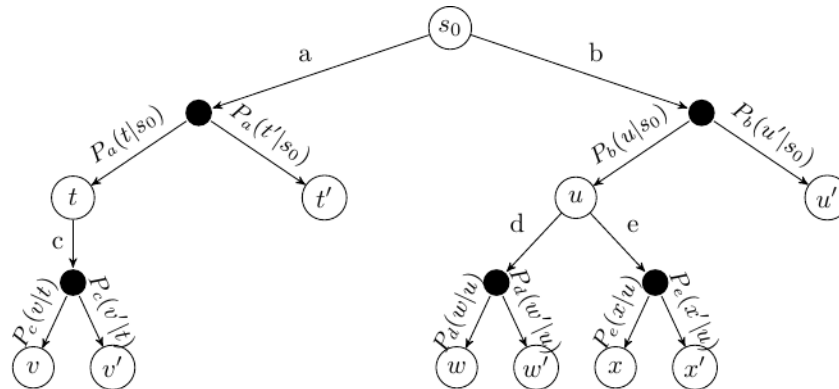


Fig. 2.2: Abstract example of an ExpectiMax Tree

The letters $a-e$ represent actions, and letters $s-x$ represent states. White nodes are state nodes, and the small black nodes represent the probabilistic uncertainty: the ‘environment’ choosing which outcome from an action happens, based on the transition function.

Monte Carlo Tree Search – Overview

The algorithm is online, which means the action selection is interleaved with action execution. Thus, MCTS is invoked every time an agent visits a new state.

Fundamental features:

1. The Q-value $Q(s, a)$ for each is approximated using **random simulation**.
2. For a single-agent problem, an ExpectiMax **search tree** is built incrementally
3. The search terminates when some pre-defined computational budget is used up, such as a time limit or a number of expanded nodes. Therefore, it is an **anytime** algorithm, as it can be terminated at any time and still give an answer.
4. The best performing action is returned.
 - This is complete if there are no dead-ends.
 - This is optimal if an entire search can be performed (which is unusual – if the problem is that small we should just use a dynamic programming technique such as value iteration).

2.5.3 The Framework: Monte Carlo Tree Search (MCTS)

The basic framework is to build up a tree using simulation. The states that have been evaluated are stored in a search tree. The set of evaluated states is **incrementally** built by iterating over the following four steps:

- **Select:** Select a single node in the tree that is **not fully expanded**. By this, we mean at least one of its children is not yet explored.
- **Expand:** Expand this node by applying one available action (as defined by the MDP) from the node.
- **Simulate:** From one of the outcomes of the expanded, perform a complete random simulation of the MDP to a terminating state. This therefore assumes that the simulation is finite, but versions of MCTS exist in which we just execute for some time and then estimate the outcome.
- **Backpropagate:** Finally, the value of the node is backpropagated to the root node, updating the value of each ancestor node on the way using expected value.

Selection

Start at the root node, and successively **select a child node** until we reach a node that is not fully expanded.

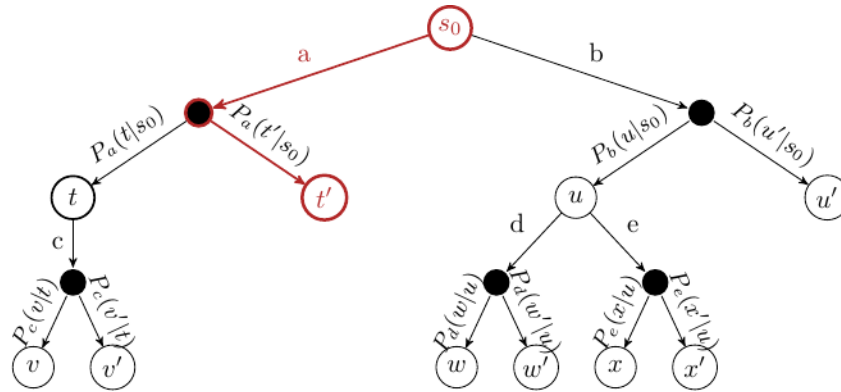


Fig. 2.3: MCTS Selection step. The red arcs and nodes are those that have been selected

Expansion

Unless the node we end up at is a terminating state, **expand the children** of the selected node by choosing an action and creating new nodes using the action outcomes.

Simulation

Choose one of the new nodes and perform a **random simulation** of the MDP to the terminating state:

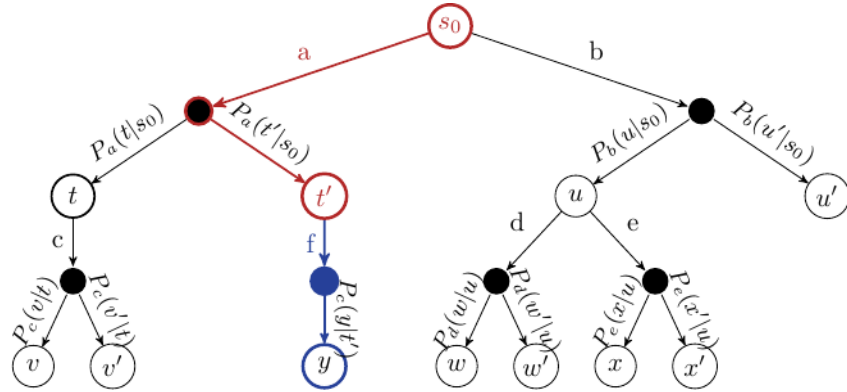


Fig. 2.4: MCTS Expansion step. The blue arcs and nodes are those that have been selected.

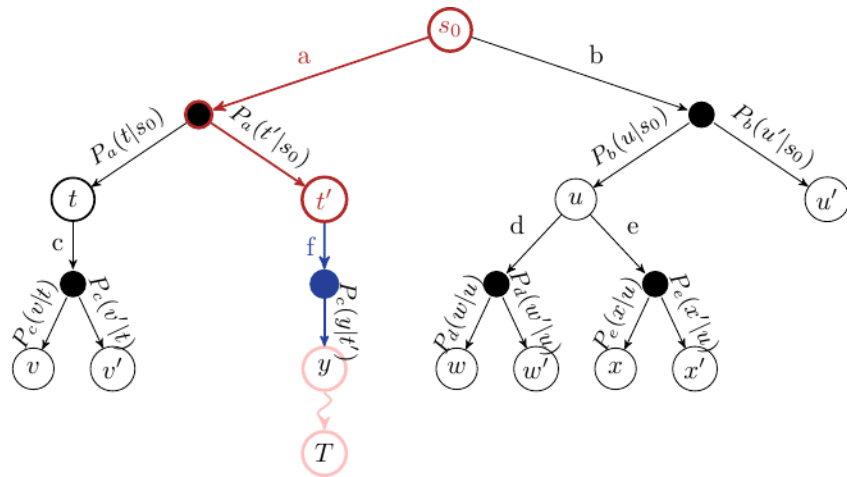


Fig. 2.5: MCTS Simulation step. The pink nodes and arcs represent the simulation.

Backpropagation

Given the reward r at the terminating state, **backpropagate** the reward to calculate the value $V(s)$ at each state along the path.

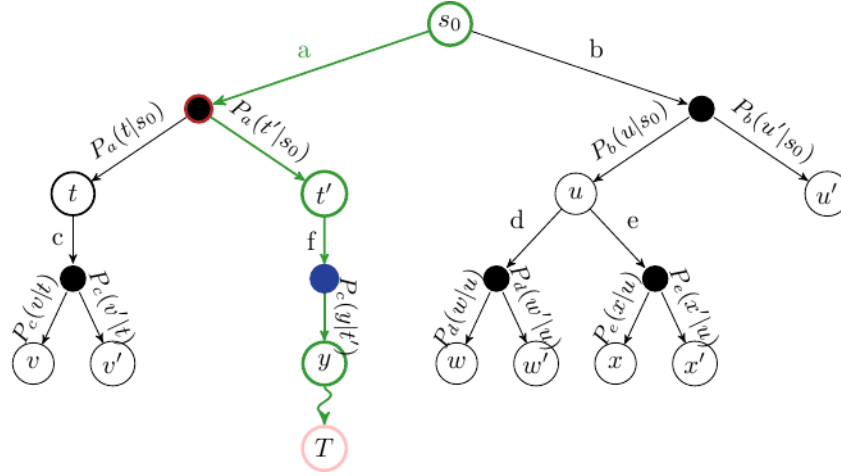


Fig. 2.6: MCTS Backpropagation step. The green arcs and nodes represent the states and actions whose Q-values will be updated.

2.5.4 Algorithm

In a basic MCTS algorithm we incrementally build of the search tree. Each node in the tree stores:

1. a set of children nodes;
2. pointers to its parent node and parent action; and
3. the number of times it has been visited.

We use this tree to explore different Monte-Carlo simulations to learn a Q-function Q .

Algorithm 7 (Monte-Carlo Tree Search)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$, base Q-function Q , time limit T

Output : updated Q-function Q

```

while  $currentTime < T$  do
     $selected\_node \leftarrow \text{Select}(s_0)$ 
     $child \leftarrow \text{Expand}(selected\_node)$  -- expand and choose a child to simulate
     $G \leftarrow \text{Simulate}(child)$  -- simulate from  $child$ 
     $\text{Backpropagate}(selected\_node, child, Q, G)$ 
return  $Q$ 
    
```

Given this, there are four main parts to the algorithm above:

1. **Selection:** The first loop progressively selects a branch in the tree using a multi-armed bandit algorithm using $Q(s, a)$. The outcome that occurs from an action is chosen according to $P(s' | s)$ defined in the MDP.

Algorithm 8 (Function – $\text{Select}(s : S)$)

Input : state s

Output : unexpanded states

while s is fully expanded **do**

 Select action a to apply in s using a multi-armed bandit algorithm

 Choose one outcome s' according to $P_a(s' | s)$

$s \leftarrow s'$

return s

2. **Expansion**: Select an action a to apply in state s , either randomly or using an heuristic. Get an outcome state s' from applying action a in state s according to the probability distribution $P(s' | s)$. Expand a new environment node and a new state node for that outcome.

Algorithm 9 (Function – Expand($s : S$))

Input : state s

Output : expanded states s'

if s is fully expanded **then**

 Randomly select action a to apply in s

 Expand one outcome s' according to $P_a(s' | s)$ and observe reward r

return s'

3. **Simulation**: Perform a randomised simulation of the MDP until we reach a terminating state. That is, at each choice point, randomly select an possible action from the MDP, and use transition probabilities $P_a(s' | s)$ to choose an outcome for each action. Heuristics can be used to improve the random simulation by guiding it towards more promising states. G is the cumulative discounted reward received from the simulation starting at s' until the simulation terminates.

To avoid memory explosion, we discard all nodes generated from the simulation. In any non-trivial search, we are unlikely to ever need them again.

4. **Backpropagation**: The reward from the simulation is backpropagated from the selected node to its ancestors recursively. We must not forget the discount factor! For each state s and action a selected in the Select step, update the cumulative reward of that state.

Algorithm 10 (Function – Backpropagation($s : S; a : A; Q : S \times A \rightarrow \mathbb{R}; G : \mathbb{R}$))

Input : state-action pair (s, a) , Q-function Q , rewards G

Output : none

do

$N(s, a) \leftarrow N(s, a) + 1$

$G \leftarrow r + \gamma G$

$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)}[G - Q(s, a)]$

$s \leftarrow \text{parent of } s$

$a \leftarrow \text{parent action of } s$

while $s \neq s_0$

Because action outcomes are selected according to $P_a(s' | s)$, this will converge to the average expected reward. This is why the tree is called an **ExpectiMax** tree: we maximise the expected return.

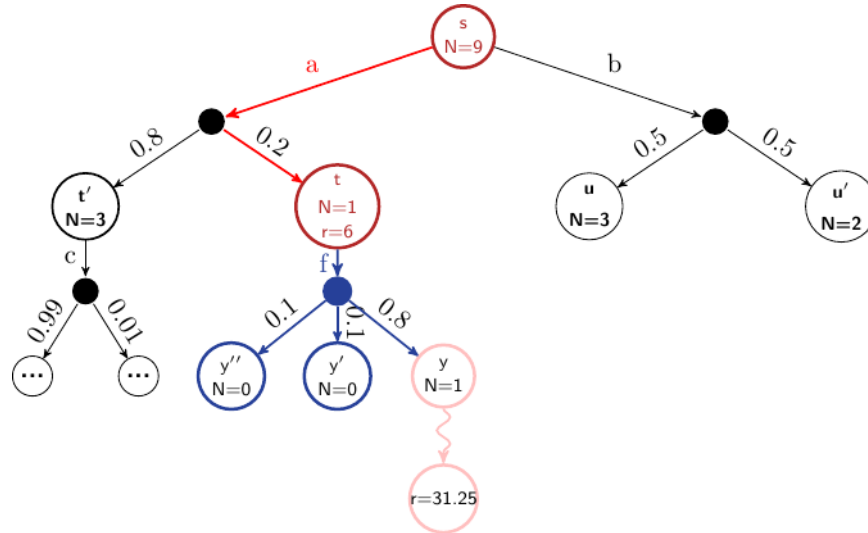
But: what if we do not know $P_a(s' | s)$?

Provided that we can **simulate** the outcomes; e.g. using a code-based simulator, then this does not matter. Over many simulations, the Select (and Expand/Execute steps) will sample $P_a(s' | s)$ sufficiently close that $Q(s, a)$ will converge

to the average expected reward. Note that this is **not a model-free** approach: we still need a model in the form of a simulator, but we do not need to have explicit transition and reward functions.

Example: Backpropagation

Consider the following ExpectiMax tree that has been expanded several times. Assume $\gamma = 0.8$, $r = X$ represents reward X received at a state, V represents the value of the state (the value $\max_{a' \in \text{children}} Q(s, a')$) and the length of the simulation is 14. After the simulation step, but before backpropagation, our tree would look like this:



In the next iteration, the red actions are selected, and the blue node is expanded to its three children nodes. A simulation is run from y , which terminates after 3 steps. A reward of 31.25 is received in the terminal state. This would mean that the discounted reward returned at node y would be $\gamma \times 31.25$, which is 20.

Before backpropagation, we have the following:

$$\begin{array}{rcl} Q(s, a) & = & 18 \\ Q(t, f) & = & 0 \end{array}$$

We leave out the remainder of the Q-function as it is not relevant to the example.

The backpropagation step is then calculated for the nodes y , t , and s as follows:

$$\begin{aligned} Q(y, g) &= \gamma^2 \times 31.25 \text{ (simulation is 3 steps long and receives reward of 31.25)} \\ &= 20 \end{aligned}$$

$$\begin{aligned} N(t, f) &\leftarrow N(t, f) + 1 = N(y) + N(y') + N(y'') + 1 = 2 \\ Q(t, f) &= Q(t, f) + \frac{1}{N(t, f)}[r + \gamma G - Q(t, f)] \\ &= 0 + \frac{1}{2}[0 + 0.8 \cdot 20 - 0] \\ &= 8 \end{aligned}$$

$$\begin{aligned} N(s, a) &\leftarrow N(s, a) + 1 = N(t) + N(t') + 1 = 5 \\ Q(s, a) &= Q(s, a) + \frac{1}{N(s, a)}[r + \gamma G - Q(s, a)] \\ &= 18 + \frac{1}{5}[6 + 0.8 \cdot (0.8 \cdot 20) - 18] \\ &= 18 + \frac{1}{5}[6 + 12.8 - 18] \\ &= 18.16 \end{aligned}$$

2.5.5 Action selection

Once we have run out of computational time, we select the action that maximises are expected return, which is simply the one with the highest Q-value from our simulations:

$$\operatorname{argmax}_{a \in A(s)} Q(s_0, a)$$

We execute that action and wait to see which outcome occurs for the action.

Once we see the outcome state, which we will call s' , we start the process all over again, except with $s_0 \leftarrow s'$.

However, importantly, we can *keep* the sub-tree from state s' , as we already have done simulations from that state. We discard the rest of the tree (all child of s_0 other than the chosen action) and incrementally build from s' .

2.5.6 Upper Confidence Trees (UCT)

When we select nodes, we select using some *multi-armed bandit algorithm*. We can use any multi-armed bandit algorithm, but in practice, using a slight variation of the UCB1 algorithm has proved to be successful in MCTS.

The **Upper Confidence Trees** (UCT) algorithm is the combination of MCTS with the UCB1 strategy for selecting the next node to follow:

$$UCT = MCTS + UCB1$$

The UCT selection strategy is similar to the UCB1 strategy:

$$\operatorname{argmax}_{a \in A(s)} Q(s, a) + 2C_p \sqrt{\frac{2 \ln N(s)}{N(s, a)}}$$

$N(s)$ is the number of times a state node has been visited, and $N(s, a)$ is the number of times a has been selected from this node. $C_p > 0$ is the exploration constant, which determines can be increased to encourage more exploration, and decreased to encourage less exploration. Ties are broken randomly.

Note: If $Q(s, a) \in [0, 1]$ and $C_p = \frac{1}{\sqrt{2}}$ then in two-player zero-sum, UCT converges to the well-known Minimax algorithm.

2.5.7 Implementation

Below is an implementation of MCTS in Python. This is a simulation-based implementation as it simulates outcomes and uses a moving average to calculate a value. However, the implementation keeps track of probabilities for the purpose of visualisation.

First, we create a class `Node`, which forms the basis for the tree:

```
import math
import time
import random
from collections import defaultdict

class Node:
```

(continues on next page)

(continued from previous page)

```

# Record a unique node id to distinguish duplicated states
next_node_id = 0

# Records the number of times states have been visited
visits = defaultdict(lambda: 0)

def __init__(self, mdp, parent, state, qfunction, bandit, reward=0.0, action=None):
    self.mdp = mdp
    self.parent = parent
    self.state = state
    self.id = Node.next_node_id
    Node.next_node_id += 1

    # The Q function used to store state-action values
    self.qfunction = qfunction

    # A multi-armed bandit for this node
    self.bandit = bandit

    # The immediate reward received for reaching this state, used for backpropagation
    self.reward = reward

    # The action that generated this node
    self.action = action

    """ Select a node that is not fully expanded """

def select(self): abstract

    """ Expand a node if it is not a terminal node """

def expand(self): abstract

    """ Backpropagate the reward back to the parent node """

def back_propagate(self, reward, child): abstract

    """ Return the value of this node """

def get_value(self):
    (_, max_q_value) = self.qfunction.get_max_q(
        self.state, self.mdp.get_actions(self.state)
    )
    return max_q_value

    """ Get the number of visits to this state """

def get_visits(self):
    return Node.visits[self.state]

```

(continues on next page)

(continued from previous page)

```

class MCTS:
    def __init__(self, mdp, qfunction, bandit):
        self.mdp = mdp
        self.qfunction = qfunction
        self.bandit = bandit

    """
    Execute the MCTS algorithm from the initial state given, with timeout in seconds
    """

    def mcts(self, timeout=1, root_node=None):
        if root_node is None:
            root_node = self.create_root_node()

        start_time = time.time()
        current_time = time.time()
        while current_time < start_time + timeout:

            # Find a state node to expand
            selected_node = root_node.select()
            if not self.mdp.is_terminal(selected_node):

                child = selected_node.expand()
                reward = self.simulate(child)
                selected_node.back_propagate(reward, child)

            current_time = time.time()

        return root_node

    """ Create a root node representing an initial state """

    def create_root_node(self): abstract

    """ Choose a random action. Heustics can be used here to improve simulations. """

    def choose(self, state):
        return random.choice(self.mdp.get_actions(state))

    """ Simulate until a terminal state """

    def simulate(self, node):
        state = node.state
        cumulative_reward = 0.0
        depth = 0
        while not self.mdp.is_terminal(state):
            # Choose an action to execute
            action = self.choose(state)

            # Execute the action
            (next_state, reward) = self.mdp.execute(state, action)

            # Discount the reward
            cumulative_reward += pow(self.mdp.get_discount_factor(), depth) * reward
            depth += 1

```

(continues on next page)

(continued from previous page)

```

        state = next_state

    return cumulative_reward

```

In our single-agent MCTS problem, we have two nodes in an ExpectiMax tree: nodes representing states, and nodes representing ‘choice points’ for the environment (that is, the filled nodes that correspond to an action outcome).

Our implementation mirrors this, with two different classes for nodes: `StateNode` and `EnvironmentNode`. The children of a `StateNode` are all `EnvironmentNode` instances, and vice versa.

For simplicity, we implement the select, expand, and backpropagate methods in these two node classes:

```

import random

from mcts import Node
from mcts import MCTS

class SingleAgentNode(Node):
    def __init__(
        self,
        mdp,
        parent,
        state,
        qfunction,
        bandit,
        reward=0.0,
        action=None,
    ):
        super().__init__(mdp, parent, state, qfunction, bandit, reward, action)

        # A dictionary from actions to a set of node-probability pairs
        self.children = {}

    """ Return true if and only if all child actions have been expanded """

    def is_fully_expanded(self):
        valid_actions = self.mdp.get_actions(self.state)
        if len(valid_actions) == len(self.children):
            return True
        else:
            return False

    """ Select a node that is not fully expanded """

    def select(self):
        if not self.is_fully_expanded() or self.mdp.is_terminal(self.state):
            return self
        else:
            actions = list(self.children.keys())
            action = self.bandit.select(self.state, actions, self.qfunction)
            return self.get_outcome_child(action).select()

    """ Expand a node if it is not a terminal node """

    def expand(self):

```

(continues on next page)

(continued from previous page)

```

    if not self.mdp.is_terminal(self.state):
        # Randomly select an unexpanded action to expand
        actions = self.mdp.get_actions(self.state) - self.children.keys()
        action = random.choice(list(actions))

        self.children[action] = []
        return self.get_outcome_child(action)
    return self

    """ Backpropagate the reward back to the parent node """

    def back_propagate(self, reward, child):
        action = child.action

        Node.visits[self.state] = Node.visits[self.state] + 1
        Node.visits[(self.state, action)] = Node.visits[(self.state, action)] + 1

        q_value = self.qfunction.get_q_value(self.state, action)
        delta = (1 / (Node.visits[(self.state, action)])) * (
            reward - self.qfunction.get_q_value(self.state, action)
        )
        self.qfunction.update(self.state, action, delta)

        if self.parent != None:
            self.parent.back_propagate(self.reward + reward, self)

    """ Simulate the outcome of an action, and return the child node """

    def get_outcome_child(self, action):
        # Choose one outcome based on transition probabilities
        (next_state, reward) = self.mdp.execute(self.state, action)

        # Find the corresponding state and return if this already exists
        for (child, _) in self.children[action]:
            if next_state == child.state:
                return child

        # This outcome has not occurred from this state-action pair previously
        new_child = SingleAgentNode(
            self.mdp, self, next_state, self.qfunction, self.bandit, reward, action
        )

        # Find the probability of this outcome (only possible for model-based) for
        ↪visualising tree
        probability = 0.0
        for (outcome, probability) in self.mdp.get_transitions(self.state, action):
            if outcome == next_state:
                self.children[action] += [(new_child, probability)]
            return new_child

class SingleAgentMCTS(MCTS):
    def create_root_node(self):
        return SingleAgentNode(
            self.mdp, None, self.mdp.get_initial_state(), self.qfunction, self.bandit
        )

```

We can visualise one of our MCTS trees to demonstrate another issue with this vanilla MCTS algorithm. Here, we

execute for just 0.03 seconds (to avoid a tree that is too large to visualise), we show the tree, and we only visualise up to the 6th level (including both state and environment nodes). At each node, $V(s)$ is the value of that node (the child with the highest Q-value). Right-click and open the image in a new tab to see a larger version:

```
from gridworld import GridWorld
from graph_visualisation import GraphVisualisation
from qtable import QTable
from single_agent_mcts import SingleAgentMCTS
from q_policy import QPolicy
from multi_armed_bandit.ucb import UpperConfidenceBounds

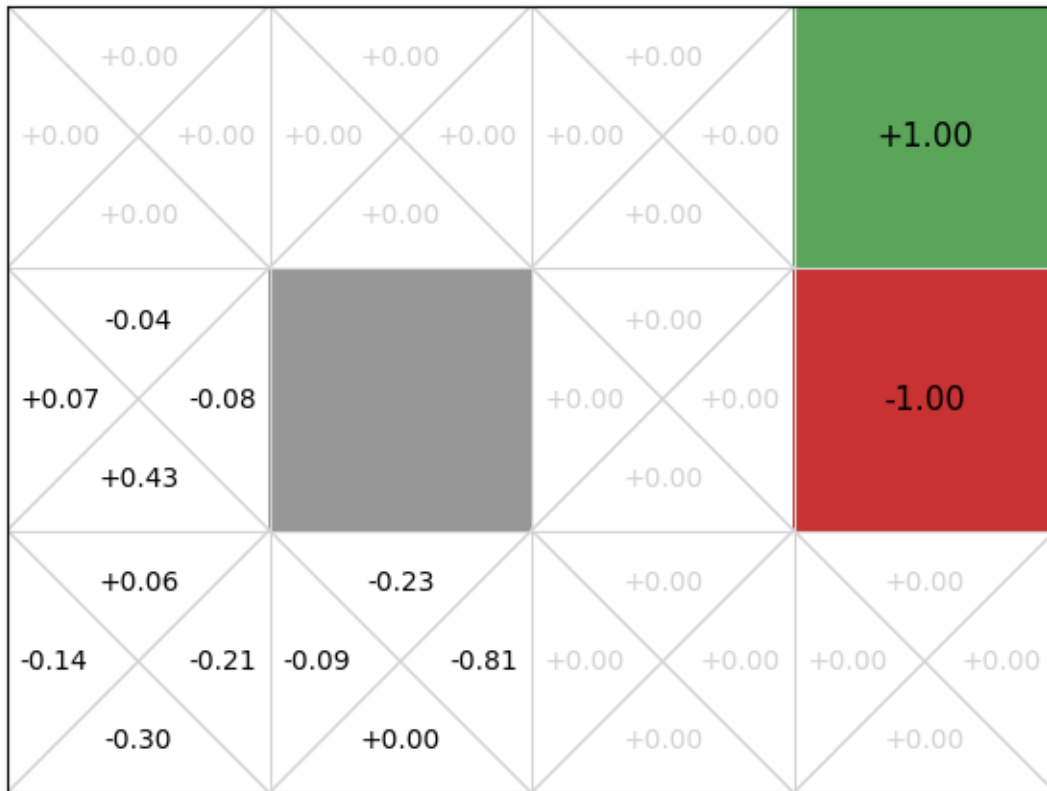
gridworld = GridWorld()
qfunction = QTable()
root_node = SingleAgentMCTS(gridworld, qfunction, UpperConfidenceBounds()).
    mcts(timeout=0.03)
gv = GraphVisualisation(max_level=6)
graph = gv.single_agent_mcts_to_graph(root_node, filename="mcts")
graph
```

```
<graphviz.graphs.Digraph at 0x7fb629b8f100>
```

The MCTS tree here demonstrates a weakness of this implementation: the same state expanded multiple times along a path, and will continue to be expanded. We can work around this by not expanding states that have already been visited, returning $V(s)$ as the reward for any expanded state, and not expanding it any more. For systems where repeated states are not an issue; e.g. some games, this problem does not arise.

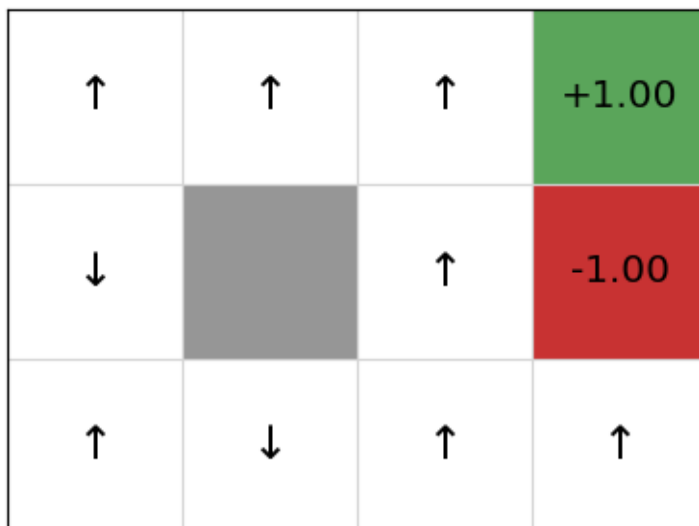
If we visualise the Q-function, we can see that only the actions that occur early in episodes have any informed Q-values:

```
gridworld.visualise_q_function(qfunction)
```



Therefore, the extracted policy is not yet very good:

```
policy = QPolicy(qfunction)
gridworld.visualise_policy(policy)
```



After 0.03 seconds, the rewards are improving but are still quite noisy. This makes sense. First, early random simulations are (a bit) more likely to terminate in the -1 state because it is four actions away from the initial state, while the +1 goal state is five actions away. Second, because the an agent can go back to previous states, the random simulations are often long sequences of actions and so they received a small discounted reward. Finally, there is actually little difference from

the start node between going left, up, and down: moving down or left from the initial state transitions back to the initial state with probability 0.9, so the difference between the three actions on average is just the discount factor.

Next, we execute this MCTS algorithm on the GridWorld problem for 1 second and visualise the corresponding Q-function every 0.01 second:

The final values after 1 second are much closer to what we expect.

Function approximation

As with standard Q-learning, we can use *Q-function approximation* to help generalise learning in MCTS.

In particular, we can use an off-line method such as Q-learning or SARSA with Q-function approximation to learn a general Q-function. Often this will work quite well, however, the issue with Q-function approximation is sometimes the approximation does not work well for certain states during execution.

To mitigate this, we can then use MCTS (online planning) to search from the actual state, but starting with the pre-trained Q-function. This has two benefits:

1. The MCTS supplements the pre-trained Q-function by running simulations from the actual initial state s_0 , which may reflect the real rewards more accurately than the pre-trained Q-function given that it is an approximation.
2. The pre-trained Q-function improves the MCTS search by guiding the selection step. In effect, the early simulations are not as random because there is some signal to use. This helps to mitigate the **cold start** problem, which is when we have no information to exploit at the start of learning.

Later in this chapter, we see an example of this with *AlphaZero*.

2.5.8 Why does it work so well (sometimes)?

It addresses exploitation vs. exploration comprehensively.

- UCT is **systematic**:
 - Policy evaluation is **exhaustive** up to a certain depth.
 - Exploration aims to **minimise regret**.

Below is a video of MCTS playing the game *Mario brothers*. The lines in front of Mario illustrate the exploration:

Here is UCT doing poorly playing a game of *Freeway*:

It fails here because the character does not receive a reward until it reaches the other side of the road, so UCT has no feedback to go on. This is the same problem that we see in GridWorld: random simulations spend a lot of time exploring for no reward. In this case, the length of a path to reach a reward is so long that it does not even get to the other side of the road. Heuristics are needed here.

Value iteration vs. MCTS

Often the set of states reachable from the initial state s_0 using an optimal policy is much smaller than the set of total states. In this regards, value iteration is exhaustive: it calculates behaviour from states that will never be encountered if we know the initial state of the problem.

MCTS (and other search methods) methods thus can be used by just taking samples starting at s_0 . However, the result is not as general as using value/policy iteration: the resulting solution will work only from the known initial state s_0 or any state reachable from s_0 using actions defined in the model. Whereas value iteration works from any state.

	Value iteration	MCTS
Cost	Higher cost (exhaustive)	Lower cost (does not solve for entire state space)
Coverage/ Robustness	Higher (works from any state)	Lower (works only from initial state or state reachable from initial state)

This is important: value iteration is then more expensive for many problems, however, for an agent operating in its environment, we only solve exhaustively once, and we can use the resulting policy many times no matter state we are in latter.

For MCTS, we need to solve **online** each time we encounter a state we have not considered before. As we see, even for a simple problem like GridWorld, doing many rollouts in a one second interval does not lead to a good policy; but with some careful crafting of the algorithm (avoid duplicate states, use a heuristic for rollouts), this can be improved.

2.5.9 Combining MCTS and TD learning: Alpha Zero

Alpha Zero (or more accurately its predecessor AlphaGo) made headlines when it beat Go world champion Lee Sedor in 2016. It uses a combination of MCTS and (deep) reinforcement learning to learn a policy.

A simple overview:

1. AlphaZero uses a deep neural network to estimate the Q-function. More accurately, it gives an estimate of the probability of selecting action a in state s ($P(a|s)$), and the value of the state ($V(s)$), which represents the probability of the player winning from s .
2. It is trained via **self-play***. Self-play is when the same policy is used to generate the moves of both the learning agent and any of its opponents. In AlphaZero, this means that initially, both players make random moves, but both also learn the same policy and use it to select subsequent moves.
3. At each move, AlphaZero:
 1. Executes an MCTS search using UCB-like selection: $Q(s, a) + P(s, a)/1 + N(s, a)$, which returns the probabilities of playing each move.
 2. The neural network is used to guide the MCTS by influencing $Q(s, a)$.
 3. The final result of a simulated game is used as the reward for each simulation.
 4. After a set number of MCTS simulations, the best move is chosen for self-play.
 5. Repeat steps 1-4 for each move until the self-play game ends.
 6. Then, feedback the result of the self-play game to update the Q function for each move.

AlphaZero is best summarised using the following figure from the Alpha Zero Nature paper (2017):

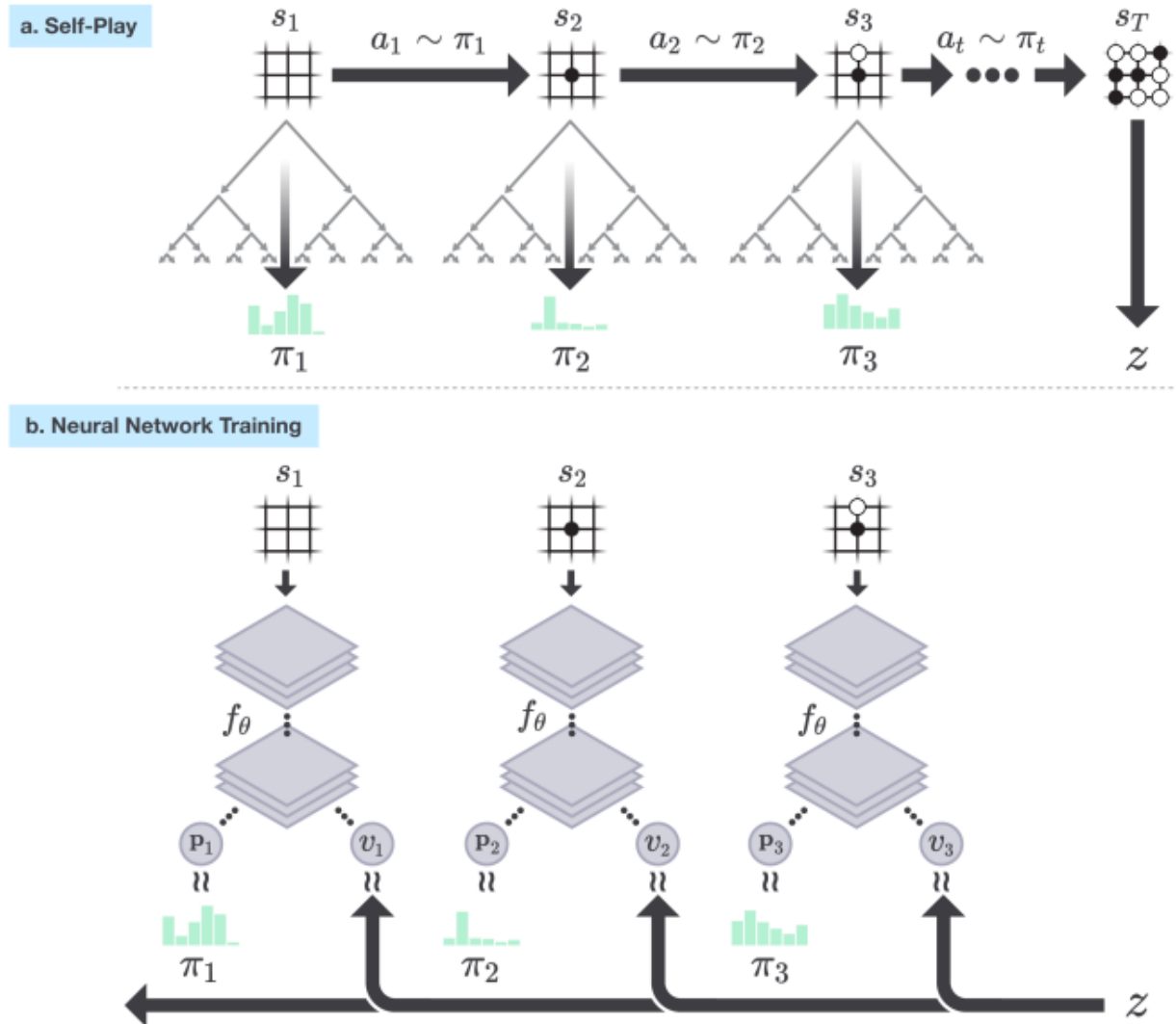


Fig. 2.7: The AlphaZero framework. *Mastering the Game of Go without Human Knowledge*. D. Silver, et al. *Nature* volume 550, pages 354–359 (2017)

2.5.10 Takeaways

Takeaways

- **Monte Carlo Tree Search** (MCTS) is an anytime search algorithm, especially good for stochastic domains, such as MDPs.
 - It can be used for model-based or simulation-based problems.
 - Smart selection strategies are *crucial* for good performance.
 - **UCT** is the combination of MCTS and UCB1, and is a successful algorithm on many problems.
-

2.5.11 Further Reading

- Chapters 2 and 5 of *Reinforcement Learning: An Introduction, second edition*. Freely downloadable at <http://incompleteideas.net/book/the-book.html>
- **Mastering the Game of Go without Human Knowledge**. D. Silver, et al. Nature volume 550, pages 354–359 (2017)
- **A Survey of Monte Carlo Tree Search Methods**. Cameron Browne, Edward Powley, Daniel Whitehouse, Simon Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis and Simon Colton. *IEEE Transactions on Computational Intelligence and AI in Games*, (4)1: 1-49, 2012
- **Monte-Carlo Tree Search: A New Framework for Game AI**. Guillaume Chaslot, Sander Bakkes, Istvan Szita, and Pieter Spronck. In *AIIDE*, 1-2, 2008.

2.6 Q-function approximation

Learning outcomes

The learning outcomes of this chapter are:

1. Manually apply linear Q-function approximation to solve small-scale MDP problems given some known features
 2. Select suitable features and design & implement Q-function approximation for model-free reinforcement learning techniques to solve medium-scale MDP problems automatically
 3. Argue the strengths and weaknesses of function approximation approaches
 4. Compare and contrast linear Q-learning with deep Q-learning
-

2.6.1 Overview

Using a Q-table has two main limitations:

1. It requires that we visit every reachable state many times and apply every action many times to get a good estimate of $Q(s, a)$. Thus, if we never visit a state s , we have no estimate of $Q(s, a)$, even if we have visited states that are very similar to s .
2. It requires us to maintain a table of size $|A| \times |S|$, which is prohibitively large for any non-trivial problem.

To get around these we will look at how to use machine learning to approximate Q-functions. In particular, we will look at **linear function approximation** and approximation using **deep learning** (deep Q-learning). Instead of calculating an exact Q-function, we approximate it using simple methods that both eliminate the need for a large Q-table (therefore the methods scale better), and also allowing use to provide reasonable estimates of $Q(s, a)$ *even if we have not applied action a in state s previously*.

Example — Freeway

Consider the game *Freeway*, in which a kangaroo needs to cross several lanes on a freeway without being run over by a car. A screenshot of the game is shown below:



Let us assume that there are 12 rows and about 40 columns. This grossly underestimates the actual number of rows and columns because the cars move a few pixels at a time, not in columns. This means there are 480 different positions that a kangaroo can be in, and there are two kangaroos. We also need to record whether there is a car in each location.

This leads to:

$$480^2 \times 2^{480} \approx 7 \times 10^{149} \text{ states}$$

There are four actions: left, right, up, down.

A Q-table would need to store 28×10^{149} entries. This is a huge Q-table for what is a trivial example compared to many other problems.

2.6.2 Linear Q-learning (Linear Function Approximation)

The key idea is to **approximate** the Q-function using a linear combination of **features** and their **weights**. Instead of recording everything in detail, we think about what is most important to know, and model that.

What are some features that are relevant to the Freeway example?

The overall process is:

1. For the states, consider what are the features that determine its representation.
2. During learning, perform updates based on the **weights of features** instead of states.
3. Estimate $Q(s, a)$ by summing the features and their weights.

Example — Features for Freeway

Instead of recording the position of both kangaroos and whether there is a car in every position, we just record the following features:

- the number of rows each kangaroo is away from the other side of the road in (two features – one for each kangaroo); and
- how far away the *closest* car is in the row above and below each kangaroo (four features — two for each kangaroo).

This requires just six features.

Linear Q-function Representation

In linear Q-learning, we store features and weights, not states. What we need to learn is how important each feature is (its **weight**) for each action.

To represent this, we have two vectors:

1. A **feature vector**, $f(s, a)$, which is a vector of $n \cdot |A|$ different functions, where n is the number of state features and $|A|$ the number of actions. Each function extracts the value of a feature for state-action pair (s, a) . We say $f_i(s, a)$ extracts the i th feature from the state-action pair (s, a) :

$$f(s, a) = \begin{pmatrix} f_1(s, a) \\ f_2(s, a) \\ \dots \\ f_{n \times |A|}(s, a) \end{pmatrix}$$

In the Freeway example, we have a vector with six state features times four actions. The function $f_1(s, Up)$ returns value of the feature that represents the distance kangaroo 1 is away from the goal. The function $f_3(s, Up)$ returns the distance to the nearest car in the row above the first kangaroo.

2. A **weight vector** w of size $n \times |A|$: one weight for each feature-action pair. w_i^a defines the weight of a feature i for action a .

Defining State-Action Features

Often it is easier to just define features for states, rather than state-action pairs. The features are just a vector of n functions of the form $f_i(s)$.

However, for most applications, the weight of a feature is related to the action. The weight of being one step away from the end in Freeway is different if we go Up to if we go Right.

It is straightforward to construct $n \times |A|$ state-pair features from just n state features:

$$f_{i,k}(s, a) = \begin{cases} f_i(s) & \text{if } a = a_k \\ 0 & \text{otherwise} \end{cases} \quad 1 \leq i \leq n, 1 \leq k \leq |A|$$

This effectively results in $|A|$ different weight vectors:

$$f(s, a_1) = \begin{pmatrix} f_{1,a_1}(s, a) \\ f_{2,a_1}(s, a) \\ 0 \\ 0 \\ 0 \\ \dots \end{pmatrix} \quad f(s, a_2) = \begin{pmatrix} 0 \\ 0 \\ f_{1,a_2}(s, a) \\ f_{2,a_2}(s, a) \\ 0 \\ \dots \end{pmatrix} \quad f(s, a_3) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ f_{1,a_3}(s, a) \\ f_{2,a_3}(s, a) \\ \dots \end{pmatrix} \quad \dots$$

Q-values from linear Q-functions

Give a feature vector f and a weight vector w , the Q-value of a state is a simple linear combination of features and weights:

$$\begin{aligned} Q(s, a) &= f_1(s, a) \cdot w_1^a + f_2(s, a) \cdot w_2^a + \dots + f_n(s, a) \cdot w_n^a \\ &= \sum_{i=0}^n f_i(s, a) w_i^a \end{aligned}$$

In practice, we also multiply the feature vector for weights w_n^b for all actions $b \neq a$, but as the feature values will be 0, we know that it does not influence the result.

Example — Approximate Q-function computation for Freeway

For the Freeway example, we would assume that moving up would give a better score than moving down, all else equal (that is, if the closest car in the next row up is the same distance away than the closest in the next row down). So, for state s where the kangaroo is in row 1:

$$Q(s, Up) = f_1(s, Up) \cdot 0.31 + \dots + f_6(s, Up) \cdot 0.04$$

Linear Q-function Update

To use approximate Q-functions in reinforcement learning, there are two steps we need to change from the standard algorithm: (1) initialisation; and (2) update.

For initialisation, initialise all weights to 0. Alternatively, you can try Q-function initialisation and assign weights that you think will be “good” weights.

For update, we now need to update the weights instead of the Q-table values. The update rule is now:

For each state-action feature i

$$w_i^a \leftarrow w_i^a + \alpha \cdot \delta \cdot f_i(s, a)$$

where δ depends on which algorithm we are using; e.g. Q-learning, SARSA.

As this is linear, it is therefore convex, so the weights will converge.

Note — Q-value propagation

Note that this has the effect of updating Q-values to states that have never been visited!

In Freeway, for example, if we receive our first reward by crossing the road (going Up from the final row), this will update the weight all features for Up, and now we have a Q-value for going Up from *any* position on the final row.

Example — Linear Q-function update for Freeway

Assume that all weights are 0, therefore, $Q(s, a) = 0$ for every state and action. Now, we receive the reward of 10 for getting to the other side of the road. If feature 6 has the value $\frac{r}{D}$, where r is the current row and D is the distance to the other side, then we have:

$$\begin{aligned} w_i^a &\leftarrow w_i^a + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]f_i(s, a) \\ w_6^{Up} &\leftarrow 0 + 0.5[10 + 0.9 \times 0]\frac{10}{10} \\ &= 5 \end{aligned}$$

From this, we now can get an estimate of $Q(s, Up)$ from any state because we have some weights in our linear function. Those that are closer to the other size of the road will get a higher Q-value than those further away (all other things being equal).

Implementation

To implement linear function approximation, we implement a new class that inherits from `QFunction` called `LinearQFunction`:

```
from qfunction import QFunction

class LinearQFunction(QFunction):
    def __init__(self, features, weights=None, default=0.0):
        self.features = features
        if weights == None:
            self.weights = [
                default
                for _ in range(0, features.num_actions())
                for _ in range(0, features.num_features())
            ]

    def update(self, state, action, delta):
        # update the weights
        feature_values = self.features.extract(state, action)
        for i in range(len(self.weights)):
            self.weights[i] = self.weights[i] + (delta * feature_values[i])

    def get_q_value(self, state, action):
        q_value = 0.0
        feature_values = self.features.extract(state, action)
        for i in range(len(feature_values)):
            q_value += feature_values[i] * self.weights[i]
        return q_value
```

A linear Q-function is initialised with either some given weights or with a default weight for all weights. The `update` method does as outlined [above]((sec:single-agent:q-function-approximation:linear-q-values)): updates each weight by adding $\delta \cdot f_i(s, a)$. Computing the Q-value implements the weighted sum outlined [above](#).

Example: Linear Q-function approximation on Gridworld

To use on an example, we need a feature extractor. The first thing we need to do is define some features for the task. As discussed above, feature engineering is not always straightforward. However, for the GridWorld task, it is reasonably clear that the distance from the goal cell is important. As such, we define three features here:

1. The distance from the goal on the X-axis.
2. The distance from the goal on the Y-axis.
3. The total distance from the goal as a Manhattan distance.

As noted above, normalising features is important to ensure that they are in the same magnitude. So, given the current position (x, y) as the state, we extract the values of the state features as follows:

1. $(x(s) + \epsilon) / (x(g) + \epsilon)$
2. $(y(s) + \epsilon) / (y(g) + \epsilon)$
3. $(x(g) - x(s) + y(g) - y(s) + \epsilon) / (x(g) + y(g) + \epsilon)$

where $x(s)$ and $y(s)$ return the x and y coordinates of the agent respectively, and g is the goal state. These expressions normalise the feature values to the range $[0, 1]$ by dividing by the goal state. The ϵ is some small value such as 0.01 to

ensure two things: (1) that we do not divide by 0 if the goal is at a coordinator where x or y are 0; and (2) that the state (0,0) has a non-zero value, otherwise there will be no Q-value for it.

Then, to extract state-action features, we need to define these as $f(s, a)$ is defined from $f(s)$ above.

We can implement these in a feature extractor class:

```
class FeatureExtractor:
    def extract_features(self, state, action):
        abstract

from feature_extractor import FeatureExtractor
from gridworld import GridWorld

class GridWorldFeatureExtractor(FeatureExtractor):

    def __init__(self, mdp):
        self.mdp = mdp

    def num_features(self):
        return 3

    def num_actions(self):
        return len(self.mdp.get_actions())

    def extract(self, state, action):
        goal = (self.mdp.width - 1, self.mdp.height - 1)
        x = 0
        y = 1
        e = 0.01
        feature_values = []
        for a in self.mdp.get_actions():
            if a == action and state != GridWorld.TERMINAL:
                feature_values += [(state[x] + e) / (goal[x] + e)]
                feature_values += [(state[y] + e) / (goal[y] + e)]
                feature_values += [
                    (goal[x] - state[x] + goal[y] - state[y] + e)
                    / (goal[x] + goal[y] + e)
                ]
            else:
                for _ in range(0, self.num_features()):
                    feature_values += [0.0]
        return feature_values
```

Now, we just simply pass this as a feature extractor to our implementation of `LinearQFunction`, and use this as the Q-function instead of a Q-table:

```
from gridworld import GridWorld
from qlearning import QLearning
from linear_qfunction import LinearQFunction
from gridworld_feature_extractor import GridWorldFeatureExtractor
from q_policy import QPolicy
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy

mdp = GridWorld()
features = GridWorldFeatureExtractor(mdp)
```

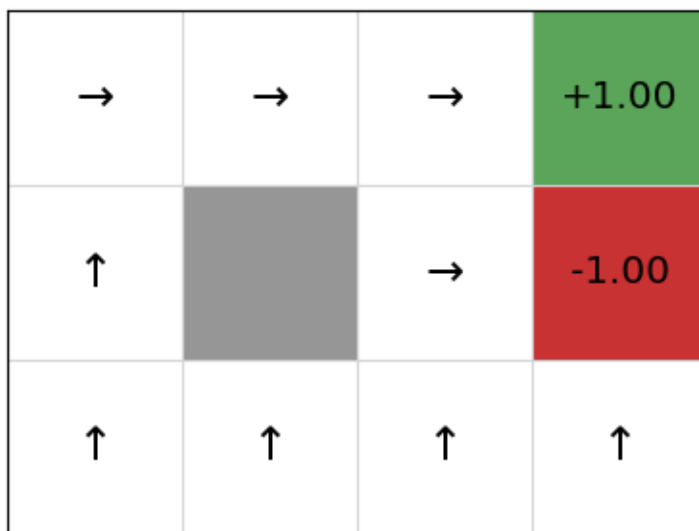
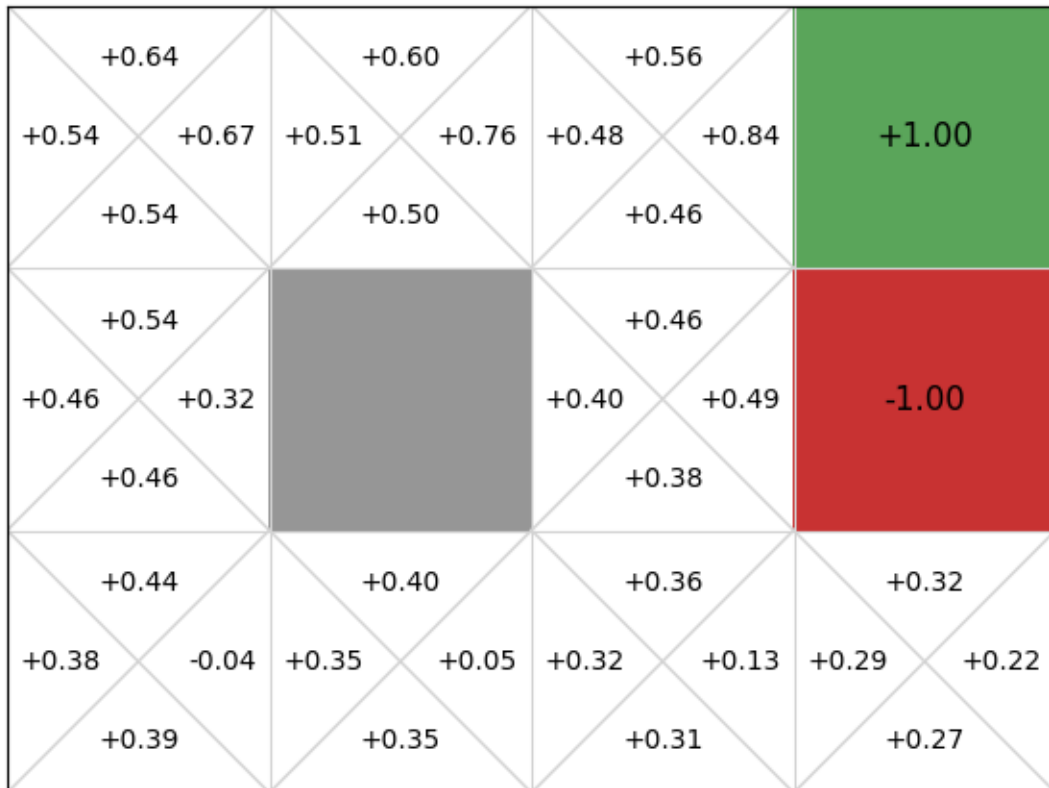
(continues on next page)

(continued from previous page)

```

qfunction = LinearQFunction(features)
QLearning(mdp, EpsilonGreedy(), qfunction).execute()
policy = QPolicy(qfunction)
mdp.visualise_q_function(qfunction)
mdp.visualise_policy(policy)

```



We can see that this gives ok Q-values and an ok policy, but there are issues. In particular, if we are in cell (2,1), the policy directs us to go right to the terminal state that gives us a -1 reward!

This is because our linear approximation learns one weight for going right, left, up, and down. Going right at the state (2, 2) is clearly good, so the weight will be learnt as positive, but every time an update is performed after the agent transitions from (2, 2) to the goal state (3, 2), the weight updates the value of $Q(s, \text{Right})$ for all states s , including the state (2, 1). As such, we learn that going right at (2,1) is good when it is not.

The choice of features is key to solving the problem. We have defined features that assume there is just the goal in the top-right corner. However, this does not help us avoid the negative reward.

We can solve this by improving our features using feature engineering. One way is to encode specific features that learn that we are e.g. in state (2, 1). Our learning algorithm will then learn that when this feature value is true, going right is a bad move.

However, more of these features we engineer, the more domain knowledge we are encoding into our solution.

A more general to engineering features about specific states is to add two new features that are true (1) if and only if we are in the same column (or row respectively) as the goal:

```
from feature_extractor import FeatureExtractor
from gridworld import GridWorld

class GridWorldBetterFeatureExtractor(FeatureExtractor):

    def __init__(self, mdp):
        self.mdp = mdp

    def num_features(self):
        return 5

    def num_actions(self):
        return len(self.mdp.get_actions())

    def extract(self, state, action):
        goal = (self.mdp.width - 1, self.mdp.height - 1)
        x = 0
        y = 1
        e = 0.01
        feature_values = []
        for a in self.mdp.get_actions():
            if a == action and state != GridWorld.TERMINAL:
                feature_values += [(state[x] + e) / (goal[x] + e)]
                feature_values += [(state[y] + e) / (goal[y] + e)]
                feature_values += [
                    (goal[x] - state[x] + goal[y] - state[y] + e)
                    / (goal[x] + goal[y] + e)
                ]
                # Features to determine if we are in goal row or column
                feature_values += [1 if goal[x] == state[x] else 0]
                feature_values += [1 if goal[y] == state[y] else 0]
            else:
                for _ in range(0, self.num_features()):
                    feature_values += [0.0]
        return feature_values
```

If we now run this on the GridWorld, we get better results:

```
from gridworld_better_feature_extractor import GridWorldBetterFeatureExtractor
```

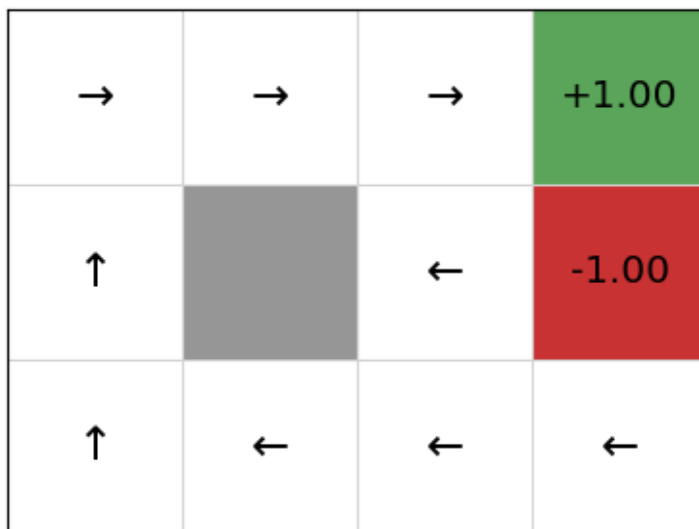
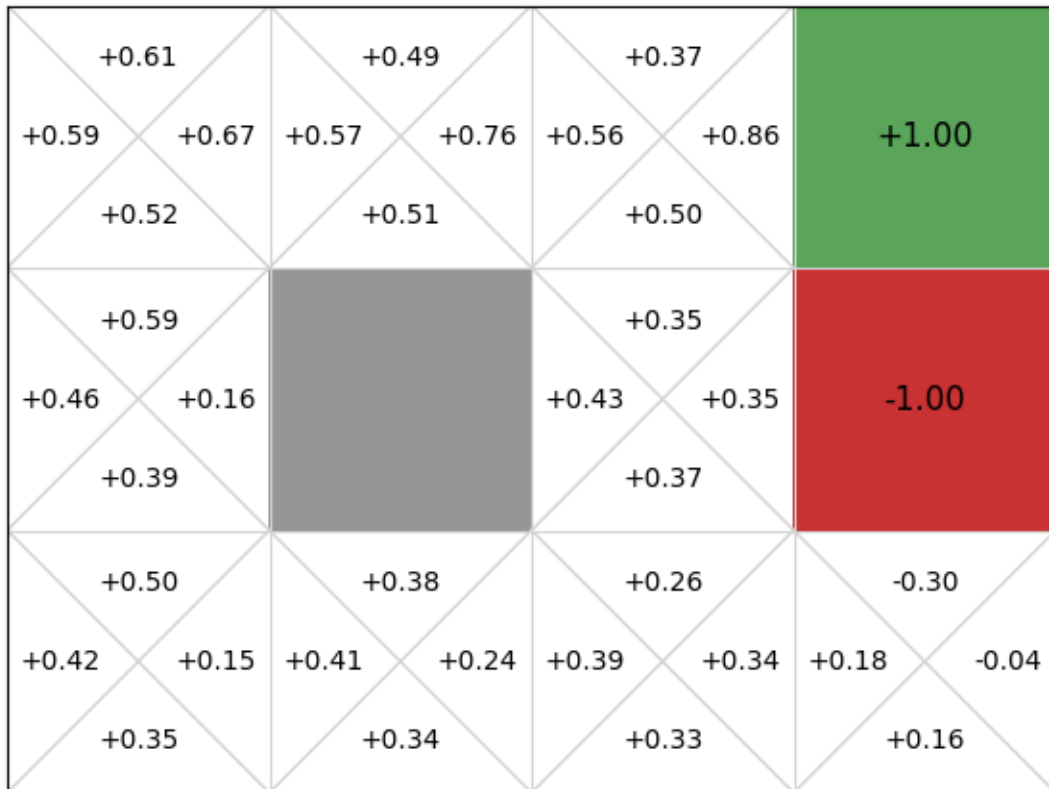
(continues on next page)

(continued from previous page)

```

mdp = GridWorld()
features = GridWorldBetterFeatureExtractor(mdp)
qfunction = LinearQFunction(features)
linear_qlearning_rewards = QLearning(mdp, EpsilonGreedy(), qfunction).
    ↪ execute(epochs=2000)
policy = QPolicy(qfunction)
mdp.visualise_q_function(qfunction)
mdp.visualise_policy(policy)

```



However, this is still not perfect. As we see, the policy recommends going up in state (1,0), which runs straight into the blocked cell. This is the downside of using linear function approximation. While it comes with convergence guarantees, it will not produce optimal policies if the underlying problem is non-linear.

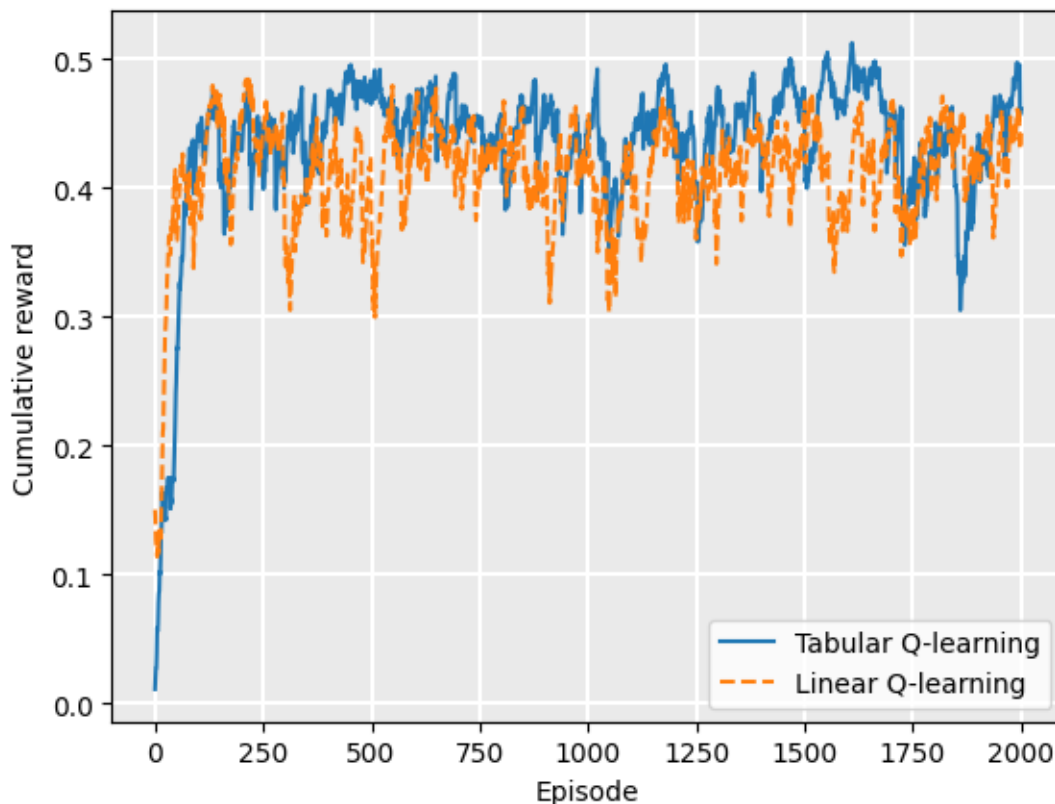
We could work around the above problem by adding two more features to avoid the blocked cells, but the more domain knowledge we require, the more effort we require in both engineering and maintenance. It is fine to encode domain knowledge, but eventually we end up encoding so much domain knowledge that we nearly encode the entire solution by hand. If it is feasible to encode the solution by hand, there is little point using reinforcement learning.

Is there much value using function approximation? Comparing the reward curves of tabular Q-learning and linear Q-function approximation, we see that the linear version converges to the optimal policy earlier, at about 350 episodes instead of about 600 for tabular Q-learning:

```
from qtable import QTable
mdp = GridWorld()
qfunction = QTable()
tabular_qlearning_rewards = QLearning(mdp, EpsilonGreedy(), qfunction).
    ↪execute(episodes=2000)
```

```
from plot import Plot

Plot.plot_cumulative_rewards(
    ["Tabular Q-learning", "Linear Q-learning"],
    [tabular_qlearning_rewards, linear_qlearning_rewards]
)
```



Example: Linear Q-function approximation on Contested Crossing

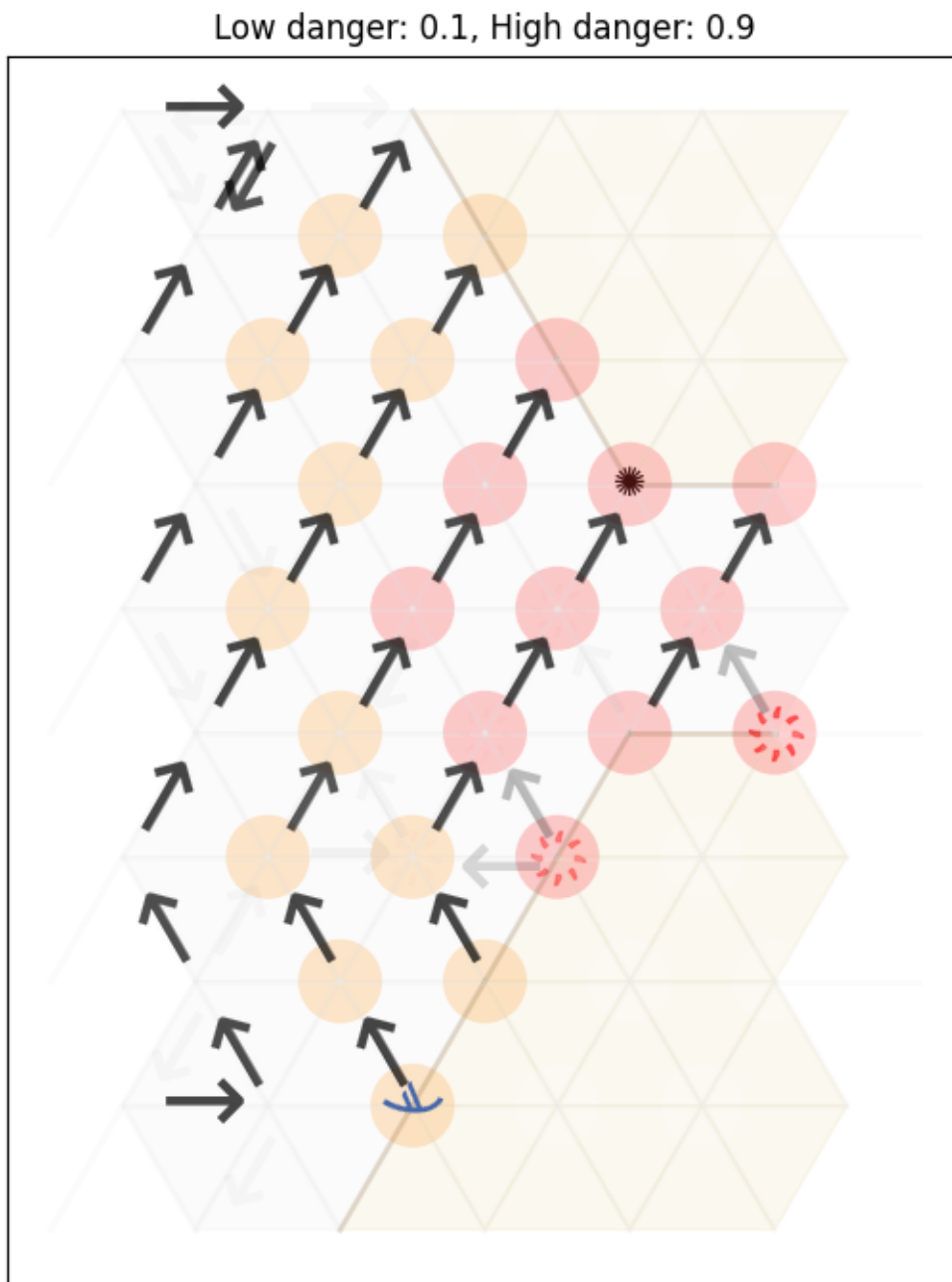
In the Contested Crossing example, a similar set of features to the Gridworld problem works: distance to the other side of the crossing. However, using only the distance ignores other important factors, such as the risk of being shot the amount of health remaining. For this example, we use four features:

1. the (normalised) x-distance to the other side of the crossing;
2. the (normalised) y-distance to the other side of the crossing;
3. the health level; and
4. the (normalised) difference between the Manhattan distance to the other side of the crossing minus the amount of damage done to the ship. Intuitively, the amount of moves required to successfully cross interacts with the health remaining, so have these as separate features in a linear model will not capture this interaction.

This results in the following policy visualisation:

```
from qlearning import QLearning
from linear_qfunction import LinearQFunction
from ccross_feature_extractor import CCrossFeatureExtractor
from stochastic_q_policy import StochasticQPolicy
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy
import contested_crossing

mdp = contested_crossing.ContestedCrossing()
features = CCrossFeatureExtractor(mdp)
qfunction = LinearQFunction(features)
QLearning(mdp, EpsilonGreedy(), qfunction).execute()
policy = StochasticQPolicy(qfunction)
mdp.visualise_as_image(
    policy=policy,
    mode=0,
    title="Low danger: {0}, High danger: {1}".format(mdp.low_danger, mdp.high_danger),
    plot=True
)
```



Example – Linear function approximation vs Q-tables

Below we see a comparison of how linear Q-functions update over time in Gridworld, compared to standard Q-tables. We can see that, even after a single update after episode 1, the values of the Up action are updated for all states.

Challenges and tips

The key challenge in linear function approximation for Q-learning is the feature engineering: selecting features that are meaningful and helpful in learning a good Q function. As well as estimating the Q-values of each action in a state, it also has to estimate the value of future states. As with any machine learning problem, feature engineering requires some experimentation and a careful combination of art and science.

Tip: Note that to make analysis and debugging easier, our feature values can be **normalised** using e.g. min-max normalisation or mean normalisation. This ensures that each feature weight is of the same magnitude, which makes it easier to understand the relative effect between features.

2.6.3 Deep Q-learning

The latest hype in reinforcement learning is all about the use of deep neural networks to approximate value and Q-functions.

Deep Q-function representation

In deep Q-learning, Q-functions are represented using deep neural networks. Instead of selecting features and training weights, we learn the parameters θ to a neural network. The Q-function is $Q(s, a; \theta)$, so takes the parameters as an argument.

This has the advantage (over linear Q-function approximation) that feature engineering is not required, the ‘features’ will be learnt as part of the hidden layers of the neural network.

A further advantage is that states can be non-structured (or less structured), rather than using a factored state representation. This means that states can be images, videos (sequences of images), or unstructured text.

Deep Q-function update

The update rule for deep Q-learning looks similar to that of updating a linear Q-function.

The deep reinforcement learning TD update is:

$$\theta \leftarrow \theta + \alpha \cdot \delta \cdot \nabla_{\theta} Q(s, a; \theta)$$

where $\nabla_{\theta} Q(s, a; \theta)$ is the **gradient** of the Q-function. In these notes, we will not cover how to calculate the gradient of the Q-function: there are many excellent text books that cover gradients, such as [Dive into Deep Learning](#).

Implementation

Deep Q-learning is identical to tabular or linear Q-learning, except that we use a deep neural network to represent the Q-function instead of a Q-table or a linear equation.

In this implementation, we use the [PyTorch deep learning framework](#). This is not a framework specifically for reinforcement learning — it is a general deep learning framework for production code, which means it is also suitable for reinforcement learning.

Using PyTorch, we create a sequential neural network with the following:

1. The first layer takes the state vector, so the input features are the features of the state. In the case of the GridWorld example, this would be just the x- and y-coordinates of the agent.
2. We have a hidden layer, with a default number of 64 hidden dimensions, but this is parameterised by the variable `hidden_dim` in the `__init__` constructor.
3. The third and final layer is the output layer, whose dimensionality is the same as the action space, so that each action has a Q-value associated with it.
4. We use a non-linear ReLU (rectified linear unit) between layers.

The input, hidden, and output layers are all `Linear` layers, which is the name for a dense (fully connected) layer in PyTorch. This just means that the layers each learn linear weights, and during inference, they feed these values forward to the next layer. The ReLU layers in between prevent values less than 0 from activating, creating non-linear effects between layers.

From this, we implement the `update` method, which uses the PyTorch implementation to update the network parameters θ , rather than calculating the gradient itself. We could also get the gradient and update θ , but using an off-the-shelf implementation allows us to take advantage of optimisations.

We also need to implement the `get_q_value` method to pass through the network to get our Q-values.

```
import torch
import torch.nn as nn
from qfunction import QFunction
from torch.optim import Adam
from deep_agent import DeepAgent

class DeepQFunction(QFunction, DeepAgent):
    """ A neural network to represent the Q-function.
        This class uses PyTorch for the neural network framework (https://pytorch.org/)
    """

    def __init__(
        self, mdp, state_space, action_space, hidden_dim=64, alpha=0.001
    ) -> None:

        # Create a sequential neural network to represent the Q function
        self.q_network = nn.Sequential(
            nn.Linear(in_features=state_space, out_features=hidden_dim),
            nn.ReLU(),
            nn.Linear(in_features=hidden_dim, out_features=hidden_dim),
            nn.ReLU(),
            nn.Linear(in_features=hidden_dim, out_features=action_space),
        )
        '''
        self.q_network = nn.Sequential(
            nn.Linear(state_space, 128),
```

(continues on next page)

(continued from previous page)

```

        nn.ReLU(),
        nn.Dropout(p=0.3),
        nn.Linear(128, 64), # First hidden layer
        nn.ReLU(),
        nn.Dropout(p=0.3),
        nn.Linear(64, 32), # Second hidden layer
        nn.ReLU(),
        nn.Dropout(p=0.3),
        nn.Linear(32, action_space)
    )
    '''
    self.optimiser = Adam(self.q_network.parameters(), lr=alpha)

    # A two-way mapping from actions to integer IDs for ordinal encoding
    actions = mdp.get_actions()
    self.action_to_id = {actions[i]: i for i in range(len(actions))}
    self.id_to_action = {
        action_id: action for action, action_id in self.action_to_id.items()
    }

    def update(self, state, action, delta):

        # Train the network based on the squared error.
        self.optimiser.zero_grad() # Reset gradients to zero
        (delta ** 2).backward() # Back-propagate the loss through the network
        self.optimiser.step() # Do a gradient descent step with the optimiser

        #self.multi_update([(state, action, delta)])

    def multi_update(self, experiences):
        (states, actions, deltas, done) = zip(*experiences)
        states_tensor = torch.tensor(states, dtype=torch.float32)
        actions_tensor = torch.tensor(actions, dtype=torch.long)
        deltas_tensor = torch.tensor(deltas, dtype=torch.float32)
        done_tensor = torch.tensor(done, dtype=torch.float32)

        q_values = self.q_network(states_tensor).gather(dim=1, index=actions_tensor.
→unsqueeze(1))

        loss = nn.functional.mse_loss(q_values, q_values + deltas_tensor.unsqueeze(1))
        self.optimiser.zero_grad()
        loss.backward()
        self.optimiser.step()

    def get_q_value(self, state, action):
        # Convert the state into a tensor
        state = self.encode_state(state)
        q_values = self.q_network(state)

        # Index q-values by action
        q_value = q_values[self.action_to_id[action]]

        return q_value

    def get_max_q(self, state, actions):
        # Convert the state into a tensor

```

(continues on next page)

(continued from previous page)

```

state = torch.as_tensor(self.encode_state(state), dtype=torch.float32)

# Since we have a multi-headed q-function, we only need to pass through the
↪network once
# call torch.no_grad() to avoid tracking the gradients for this network
↪forward pass
with torch.no_grad():
    q_values = self.q_network(state)
    arg_max_q = None
    max_q = float("-inf")
    for action in actions:
        value = q_values[self.action_to_id[action]].item()
        if max_q < value:
            arg_max_q = action
            max_q = value
    return (arg_max_q, max_q)

def save(self, filename):
    torch.save(self.q_network.state_dict(), filename)

def load(self, filename):
    self.q_network.load_state_dict(torch.load(filename))

```

Note in this implementation that PyTorch does not support strings as values, so we need to encode action names and states as numbers.

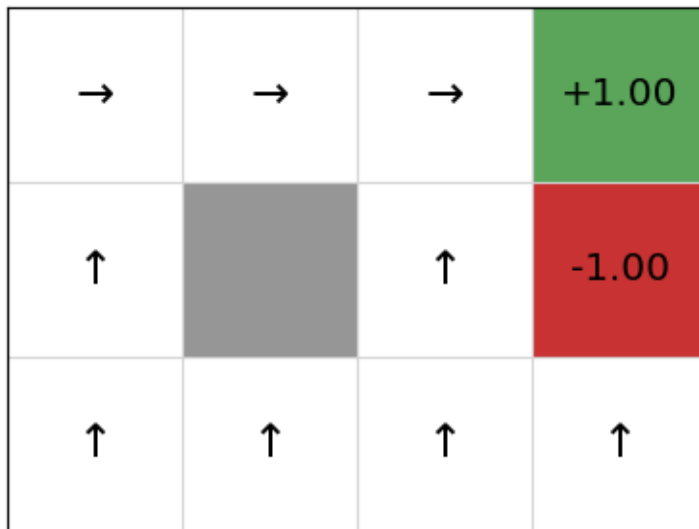
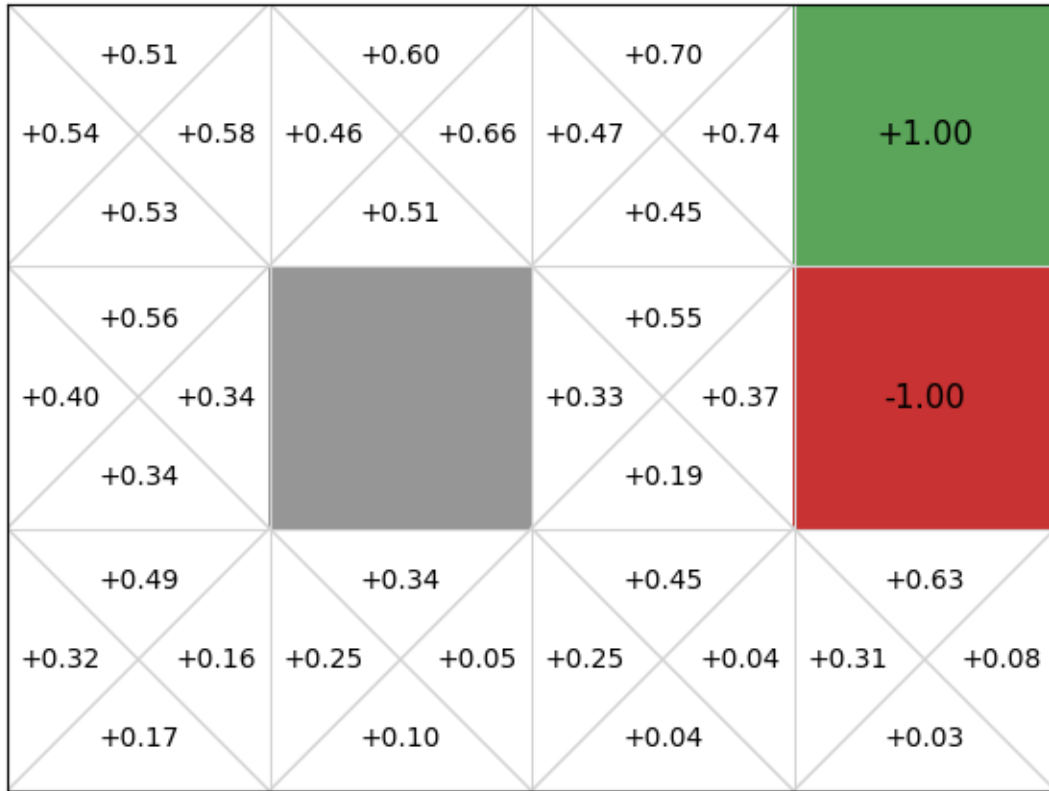
We can now use this implementation by creating a standard Q-learning agent with a deep Q network as the Q function:

```

from gridworld import GridWorld
from qlearning import QLearning
from deep_qfunction import DeepQFunction
from q_policy import QPolicy
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy

gridworld = GridWorld()
qfunction = DeepQFunction(gridworld, state_space=len(gridworld.get_initial_state()),
↪action_space=5, hidden_dim=16)
QLearning(gridworld, EpsilonGreedy(), qfunction, alpha=1.0).execute(episodes=100)
gridworld.visualise_q_function_as_image(qfunction)
policy = QPolicy(qfunction)
gridworld.visualise_policy_as_image(policy)

```



Note the value of the learning rate $\alpha = 1.0$. This is because the optimiser (called ADAM) that is used in the PyTorch implementation handles the learning rate in the `update` method of the `DeepQFunction` implementation. Therefore, we do not need to multiply the TD value by the learning rate α as the ADAM optimiser already does this. By setting $\alpha = 1.0$, this means that the learning rate is not used in the `update` method, except implicitly by the call to the optimiser.

Below we see a comparison between deep Q-functions and linear Q-functions.

We can see that because parameters in the deep Q-function are randomly initialised, the Q-values are random, whereas we initialise linear weights to 0, so Q-values are all 0.

There is minimal difference between the two policies. Note though that even though the deep Q-function does not assume

linearity, it still learns a poor policy in parts of the Q-function, such as in the bottom right state, in which it recommends going up instead of left. This is likely due to the neural network being too simple for this problem, and thus underfitting. We could improve this by adding an additional layer or adding more hidden parameters to the layers.

The main difference between the two is the linear Q-function approximation is guaranteed to converge to a global optima due to its convex loss function, whereas deep Q-function approximation has no such guarantees.

Advantages and disadvantages

Advantages of deep Q-function approximation (compared to linear Q-function approximation):

- **Feature selection:** We do not need to select features – the ‘features’ will be learnt as part of the hidden layers of the neural network.
- **Unstructured data:** The state s can be less structured, such as images or sequences of images (video).

Disadvantages:

- **Convergence:** There are no convergence guarantees.
- **Data hungry:** Deep neural networks are more data hungry because they need to learn features as well as “the Q-function”, so compared to a linear approximation with good features, learning good Q-functions can be difficult. Large amounts of computation are often required.

Despite this, deep Q-learning works remarkably well in some areas, especially for tasks that require vision (see the robotic arm grasping unknown objects).

2.6.4 Strengths and Limitations of Q-function Approximation

Approximating Q-functions using machine learning techniques such as linear functions or deep learning has advantages and disadvantages.

Advantages:

- **Memory:** More efficient representation compared with Q-tables because we only store weights/parameters for the Q-function, rather than the $|A| \times |S|$ entries for a Q-table.
- **Q-value propagation:** we do not need to apply action a in state s to get a value for $Q(s, a)$ because the Q-function generalises.

Disadvantages:

- The Q-function is now only an approximation of the real Q-function: states that share feature values will have the same Q-value according to the Q-function, but the actual Q-value according to the (unknown) optimal Q-function may be different.

2.6.5 Takeaways

Takeaways

- We can scale reinforcement learning by **approximating Q-functions**, rather than storing complete Q-tables.
 - Using simple **linear methods** in which we select features and learn weights are effective and guarantee convergence.
 - **Deep Q-learning** offers an alternative in which we can learn a state representation, but requires more training data (more episodes) and has no convergence guarantees.
-

2.6.6 Further Reading

- Chapter 9 (Approximate Solution Methods) of [Introduction to Reinforcement Learning, Sutton and Barto](#)
- Deep Q-learning for Atari. This uses Convolutional Neural Networks (NN) to estimate $Q(s, a)$. The input for the NN is the state, and the output is the estimated reward for each action. There are two papers worth reading on this:
 - [Human-level control through deep reinforcement learning](#). Mnih, V., et al. Nature 529 (2015).
 - [Playing Atari with Deep Reinforcement Learning](#). Mnih, V., et al. arXiv: preprint arXiv:1312.5602 (2013).
- Before AlphaGo there was TD-gammon, which was the first paper to combine reinforcement learning and neural networks: [TD-Gammon, A Self-Teaching Backgammon Program, Achieves Master-Level Play](#), : AAAI Technical Report FS-93-02 (1993).

2.7 Reward shaping

Learning outcomes

The learning outcomes of this chapter are:

1. Explain how reward shaping can be used to help model-free reinforcement learning methods to converge
 2. Manually apply reward shaping for a given potential function to solve small-scale MDP problems
 3. Design and implement potential functions to solve medium-scale MDP problems automatically
 4. Compare and contrast reward shaping with Q-value initialisation
-

2.7.1 Overview

In the previous chapters, we looked at fundamental temporal difference (TD) methods for reinforcement learning. As noted, these methods have some weaknesses, including that rewards are sometimes **sparse**. This means that there are few state/actions that lead to non-zero rewards. This is problematic because initially, reinforcement learning algorithms behave entirely randomly and will struggle to find good rewards. Remember the example of a *UCT algorithm playing Freeway*.

In this section, we look at two simple approaches that can improve temporal difference methods:

1. **Reward shaping:** If rewards are sparse, we can modify/augment our reward function to reward behaviour that we think moves us closer to the solution.
2. **Q-value Initialisation:** We can “guess” good Q-values at the start and initialise $Q(s, a)$ to be this at the start, which will guide our learning algorithm.

2.7.2 Reward shaping

Definition – Reward sharpening

Reward shaping is the use of small intermediate ‘fake’ rewards given to the learning agent that help it converge more quickly.

In many applications, you will have some idea of what a good solution should look like. For example, in our simple navigation task, it is clear that moving towards the reward of +1 and away from the reward of -1 are likely to be good solutions.

Can we then speed up learning and/or improve our final solution by nudging our reinforcement learner towards this behaviour?

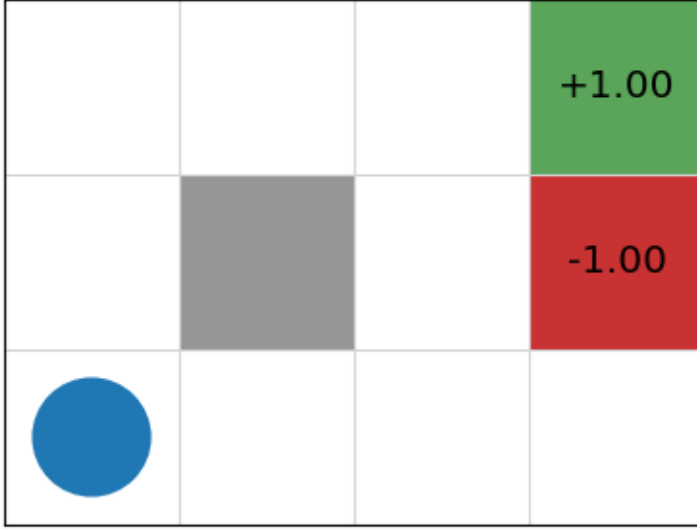
The answer is: Yes! We can modify our reinforcement learning algorithm slightly to give the algorithm some information to help, while also guaranteeing optimality.

This information is known as **domain knowledge** — that is, stuff about the domain that the human modeller knows about while constructing the model to be solved.

Exercise: Freeway What would be a good heuristic for the Freeway game to learn how to get the kangaroo across the freeway?



Exercise: GridWorld What would be a good heuristic for GridWorld?



Shaped Reward

In TD learning methods, we update a Q-function when a reward is received. E.g, for 1-step Q-learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

The approach to reward shaping is not to modify the reward function or the received reward r , but to just give some additional reward for some actions:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \underbrace{F(s, s')}_{\text{additional reward}} + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

The purpose of the function is to give an additional reward $F(s, s')$ when any action transitions from state s to state s' . The function $F : S \times S \rightarrow \mathbb{R}$ provides **heuristic domain knowledge** to the problem that is typically manually programmed.

We say that $r + F(s, s')$ is the **shaped reward** for an action.

Further, we say that $G^\Phi = \sum_{i=0}^{\infty} \gamma^i (r_i + F(s_i, s_{i+1}))$ is the shaped reward for the entire episode.

If we define $F(s, s') > 0$ for states s and s' , then this provides a small positive reward for transitioning from s to s' , thus encouraging actions that transition from s to s' in future exploitation. If we define $F(s, s') < 0$ for states s and s' , then this provides a small *negative* reward for transitioning from s to s' , thus discouraging actions that transition like this in future exploitation.

Potential-based Reward Shaping

Potential-based reward shaping is a particular type of reward shaping with nice theoretical guarantees. In potential-based reward shaping, F is of the form:

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

We call Φ the **potential function** and $\Phi(s)$ is the **potential** of state s .

So, instead of defining $F : S \times S \rightarrow \mathbb{R}$, we define $\Phi : S \rightarrow \mathbb{R}$, which is some heuristic measure of the value of each state $s \in S$.

Theoretical guarantee: this will still converge to the optimal policy under the assumption that all state-action pairs are sampled infinitely often.

This is quite straightforward to show as follows. Consider an episode with shaped reward G^Φ :

$$\begin{aligned} G^\Phi &= \sum_{i=0}^{\infty} \gamma^i (r_i + F(s_i, s_{i+1})) \\ &= \sum_{i=0}^{\infty} \gamma^i (r_i + \gamma \Phi(s_{i+1}) - \Phi(s_i)) \\ &= \sum_{i=0}^{\infty} \gamma^i r_i + \sum_{i=0}^{\infty} \gamma^{i+1} \Phi(s_{i+1}) - \sum_{i=0}^{\infty} \gamma^i \Phi(s_i) \\ &= G + \sum_{i=0}^{\infty} \gamma^i \Phi(s_i) - \Phi(s_0) - \sum_{i=0}^{\infty} \gamma^i \Phi(s_i) \\ &= G - \Phi(s_0) \end{aligned}$$

where G refers to the shaped reward for the episode, and s_0 is the starting state of the episode. What this says is that the shaped reward G^Φ is just the unshaped reward G minus the potential of the initial state s_0 . However, because F does not depend on the actions and G^Φ does not depend on shaped rewards beyond the initial state, the **shaped Q function**, which we refer to as Q^Φ , can be defined as just $Q^\Phi(s, a) = Q(s, a) + \Phi(s)$. Given this, any optimal policy extracted from Q^Φ will be equivalent to any optimal policy extracted from Q .

However! While it provides guarantees about the end result, potential-based reward shaping may either increase or decrease the time taken to learn. A well-designed potential function decrease the time to convergence.

Example – Potential Reward Shaping for GridWorld

For Grid World, we use the Manhattan distance to define the potential function, normalised by the size of the grid:

$$\Phi(s) = 1 - \frac{|x(g) - x(s)| + |y(g) - y(s)|}{width + height - 2}$$

in which $x(s)$ and $y(s)$ return the x and y coordinates of the agent respectively, g is the goal state. and $width$ and $height$ are the width and height of the grid respectively. Note that the coordinates are indexed from 0, so we subtract 2 from the denominator.

Even on the very first iteration, a greedy policy such as ϵ -greedy, will feedback those states closer to the +1 reward. From state (1,2) with $\gamma = 0.9$ if we go Right, we get:

$$\begin{aligned} F((1, 2), (2, 2)) &= \gamma \Phi(2, 2) - \Phi(1, 2) \\ &= 0.9 \cdot (1 - \frac{1}{5}) - (1 - \frac{2}{5}) \\ &= 0.12 \end{aligned}$$

We can compare the Q-values for these states for the four different possible moves that could have been taken from (1,2), using and $\alpha = 0.1$ and $\gamma = 0.9$:

Action	r	$F(s, s')$	$\gamma \max_{a'} Q(s', a')$	New $Q(s, a)$
Up	0	$0.9(1 - \frac{2}{5}) - (1 - \frac{2}{5}) = -0.06$	0	-0.006
Down	0	$0.9(1 - \frac{2}{5}) - (1 - \frac{2}{5}) = -0.06$	0	-0.006
Right	0	$0.9(1 - \frac{1}{5}) - (1 - \frac{2}{5}) = 0.12$	0	0.012
Left	0	$0.9(1 - \frac{3}{5}) - (1 - \frac{2}{5}) = -0.24$	0	-0.024

Thus, we can see that our potential reward function rewards actions that go towards the goal and penalises actions that go away from the goal. Recall that state (1,2) is in the top row, so action Up just leaves us in state (1,2) and Down similarly because we cannot go through the walls.

But! It will not always work. Compare states (0,0) and (0,1). Our potential function will reward (0,1) because it is closer to the goal, but we know from our value iteration example that (0,0) is a higher value state than (0,1). This is because our reward function does not consider the negative reward.

In practice, it is non-trivial to derive a perfect reward function – it is the same problem as deriving the perfect search heuristic. If we could do this, we would not need to even use reinforcement learning – we could just do a greedy search over the reward function.

Implementation

To implement potential-based reward shaping, we need to first implement a potential function. We implement potential functions as subclasses of `PotentialFunction`. For the `GridWorld` example, the potential function is 1 minus the normalised distance from the goal:

```
from potential_function import PotentialFunction
from gridworld import GridWorld

class GridWorldPotentialFunction(PotentialFunction):
    def __init__(self, mdp):
        self.mdp = mdp

    def get_potential(self, state):
        if state != GridWorld.TERMINAL:
            goal = (self.mdp.width, self.mdp.height)
            x = 0
            y = 1
            return 0.1 * (
                1 - ((goal[x] - state[x] + goal[y] - state[y]) / (goal[x] + goal[y]))
            )
        else:
            return 0.0
```

Reward shaping for Q-learning is then a simple extension of the `QLearning` class, overriding the `get_delta` method:

```
from qlearning import QLearning

class RewardShapedQLearning(QLearning):
    def __init__(self, mdp, bandit, potential, qfunction, alpha=0.1):
        super().__init__(mdp, bandit, qfunction=qfunction, alpha=alpha)
        self.potential = potential

    def get_delta(self, reward, q_value, state, next_state, next_action):
        next_state_value = self.state_value(next_state, next_action)
        state_potential = self.potential.get_potential(state)
        next_state_potential = self.potential.get_potential(next_state)
        potential = self.mdp.discount_factor * next_state_potential - state_potential
        delta = reward + potential + self.mdp.discount_factor * next_state_value - q_value
        return delta
```

We can run this on a `GridWorld` example with more states, to make the problem harder::

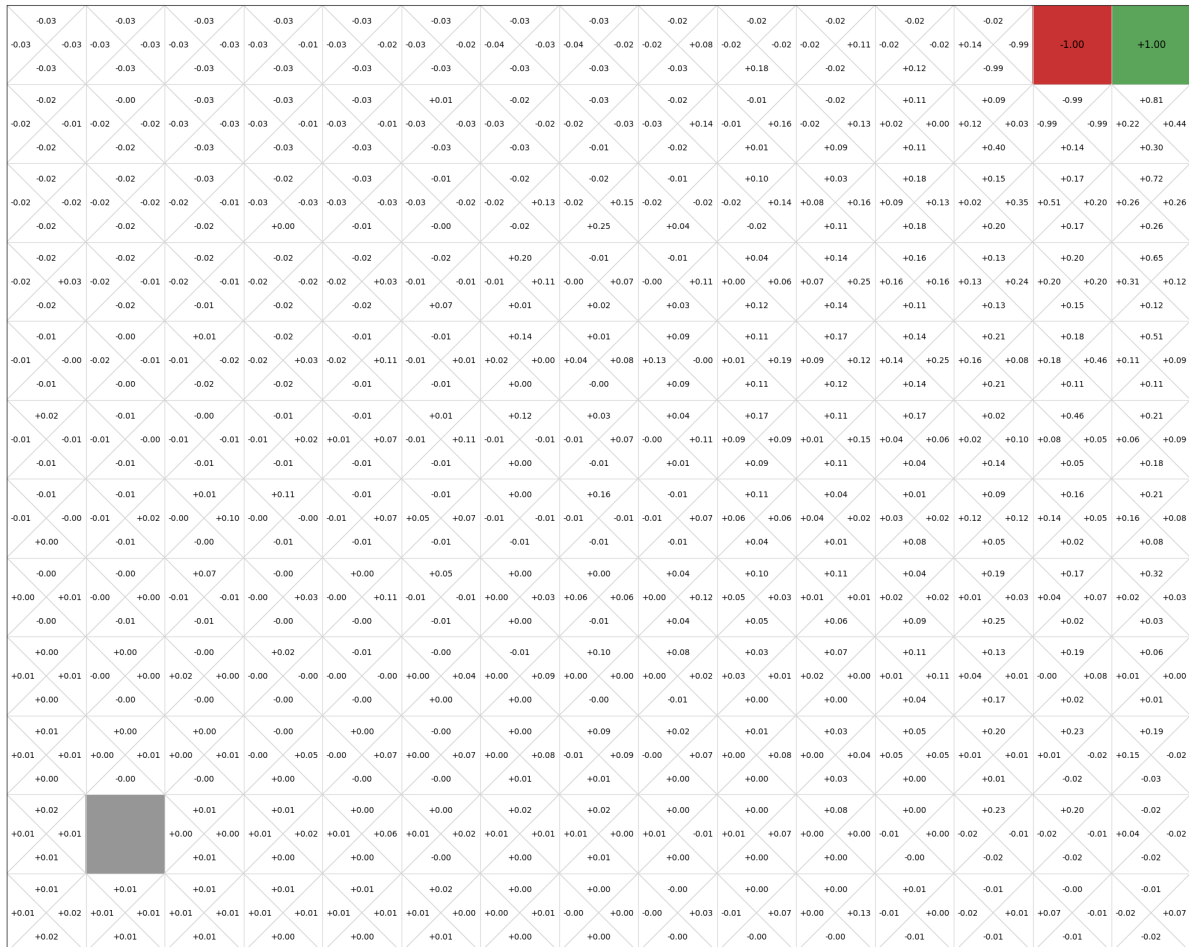
```
from qtable import QTable
from qlearning import QLearning
from reward_shaped_qlearning import RewardShapedQLearning
from gridworld_potential_function import GridWorldPotentialFunction
from q_policy import QPolicy
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy

mdp = GridWorld(width = 15, height = 12, goals = (((14,11), 1), ((13,11), -1)))
qfunction = QTable()
```

(continues on next page)

(continued from previous page)

```
potential = GridWorldPotentialFunction(mdp)
RewardShapedQLearning(mdp, EpsilonGreedy(), potential, qfunction).execute()
policy = QPolicy(qfunction)
mdp.visualise_q_function(qfunction)
mdp.visualise_policy(policy)
reward_shaped_rewards = mdp.get_rewards()
```



↑	←	→	→	→	→	↑	→	→	↓	→	↓	←	-1.00	+1.00
→	↑	↑	→	→	↑	→	↓	→	→	→	↑	↓	↓	↑
↑	↓	→	↓	↓	↓	→	↓	↓	→	→	↓	→	←	↑
→	→	↓	←	→	↓	↑	→	→	↓	→	↑	→	↑	↑
→	↑	↑	→	→	→	↑	→	←	→	↑	→	↓	→	↑
↑	→	↑	→	→	→	↑	→	→	↑	→	↑	↓	↑	↑
↓	→	→	↑	→	→	↑	↑	→	↑	↑	↓	→	↑	↑
→	→	↑	→	→	↑	→	→	→	↑	↑	↓	↓	↑	↑
→	→	←	↑	←	→	→	↑	↑	←	↑	↑	↓	↑	↑
→	→	→	→	→	→	→	→	→	→	→	↑	↑	↑	↑
↑		↓	→	→	→	↑	↑	←	→	↑	→	↑	↑	←
→	→	→	→	→	↑	→	↑	→	→	→	↑	→	←	→

Now, we compare this with Q-learning without reward shaping:

```

mdp = GridWorld(width = 15, height = 12, goals = [((14,11), 1), ((13,11), -1)])
qfunction = QTable()
QLearning(mdp, EpsilonGreedy(), qfunction).execute()
policy = QPolicy(qfunction)
mdp.visualise_q_function(qfunction)
mdp.visualise_policy(policy)
q_learning_rewards = mdp.get_rewards()

```

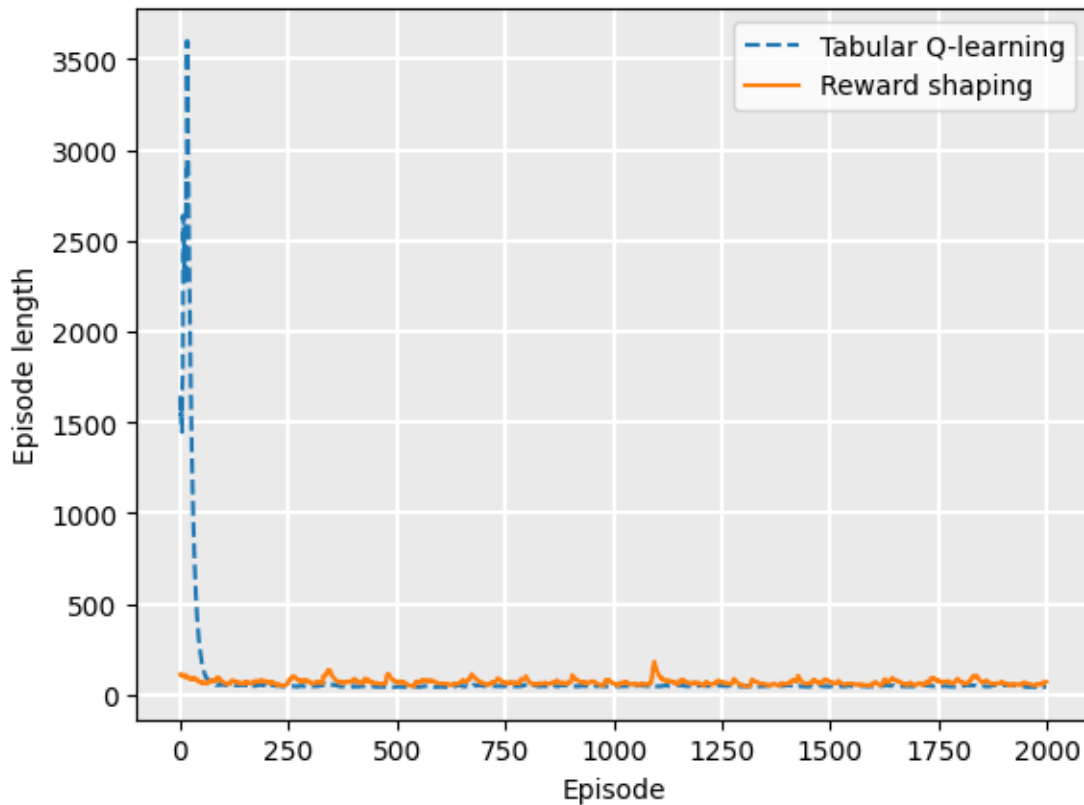
+0.03	+0.08	+0.09	+0.11	+0.12	+0.14	+0.16	+0.18	+0.21	+0.24	+0.28	+0.30	-0.03	-1.00	+1.00															
+0.02	+0.07	+0.07	+0.09	+0.08	+0.10	+0.09	+0.11	+0.11	+0.13	+0.12	+0.15	+0.14			+0.17	+0.16	+0.21	+0.19	+0.24	+0.21	+0.27	+0.25	+0.31	+0.30	+0.30	+0.33	-0.40		
+0.00	+0.07		+0.07		+0.09		+0.10		+0.12		+0.14				+0.16		+0.20		+0.23		+0.28		+0.35		+0.23				
+0.04	+0.08		+0.06		+0.10		+0.11		+0.13		+0.15		+0.15		+0.21		+0.24		+0.17		+0.34		+0.33		+0.31	-0.56	+0.88		
+0.03	+0.06	+0.06	+0.06	+0.07	+0.06	+0.02	+0.04	+0.05	+0.03	+0.08	+0.07	+0.08	+0.12	+0.07	+0.17	+0.10	+0.17	+0.10	+0.16	+0.13	+0.34	+0.32	+0.41	+0.36	+0.50	+0.28	+0.54	+0.55	+0.72
+0.03		+0.06		+0.05		+0.05		+0.05		+0.05		+0.04		+0.06		+0.11		+0.12		+0.16		+0.33		+0.39	+0.60		+0.64		
+0.05	+0.07		+0.05		+0.09		+0.09		+0.09		+0.10		+0.14		+0.16		+0.18		+0.04		+0.36		+0.43		+0.51		+0.75		
+0.05	+0.06	+0.05	+0.05	+0.06	+0.07	+0.01	+0.01	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.01	+0.00	+0.00	+0.02	+0.00	+0.03	+0.06	+0.28	+0.14	+0.23	+0.19	+0.38	+0.39	+0.64	+0.59	+0.61
+0.04		+0.05		+0.05		+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.07		+0.16		+0.18		+0.49		+0.57	
+0.05		+0.06		+0.06		+0.07		+0.00		+0.01		+0.00		+0.01		+0.00		+0.02		+0.13		+0.25		+0.37		+0.51		+0.64	
+0.04	+0.04	+0.04	+0.05	+0.04	+0.04	+0.01	+0.01	+0.04	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.01	+0.00	+0.02	+0.00	+0.01	+0.04	+0.05	+0.15	+0.27	+0.15	+0.15
+0.04		+0.04		+0.02		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.01	+0.00
+0.04		+0.05		+0.05		+0.01		+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.06		+0.06	+0.10
+0.03	+0.03	+0.04	+0.04	+0.01	+0.02	+0.03	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.03		+0.04		+0.02		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.04		+0.04		+0.03		+0.02		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.03	+0.03	+0.03	+0.03	+0.04	+0.01	+0.02	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.02		+0.03		+0.03		+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.04		+0.04		+0.03		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.02	+0.03	+0.03	+0.03	+0.02	+0.01	+0.03	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.02		+0.03		+0.02		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.03		+0.03		+0.03		+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.02	+0.02	+0.02	+0.02	+0.02	+0.02	+0.01	+0.02	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.01		+0.02		+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.02		+0.03		+0.02		+0.02		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.02	+0.02	+0.02	+0.02	+0.02	+0.01	+0.01	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.02		+0.02		+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.02	+0.02	+0.02	+0.02	+0.02	+0.02	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.02		+0.02		+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00	+0.00
+0.02	+0.02				+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.01					+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00
+0.02		+0.00			+0.01		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00
+0.01	+0.01	+0.01	+0.00		+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
+0.01		+0.00			+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00		+0.00

→	→	→	→	→	→	→	→	→	→	→	↓	←	-1.00	+1.00
→	↑	←	↑	↑	↑	↑	→	↑	↑	→	→	→	↓	↑
→	↑	→	↑	↑	↑	↑	↑	↑	↑	→	↑	↑	→	↑
↑	↑	↑	↑	←	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	←	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	←	↑	↑	←	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	←	↓	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	←	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
→	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	←	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑		↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	←	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑

If we plot the average episode length during training, we see that reward shaping reduces the length of the early episodes because it has knowledge nudging it towards the goal:

```
from tests.plot import Plot

Plot.plot_episode_length(
    ["Tabular Q-learning", "Reward shaping"],
    [q_learning_rewards, reward_shaped_rewards],
)
```



2.7.3 Q-value initialisation

An approach related to reward shaping is **Q-value initialisation**. Recall that TD learning methods can start at any arbitrary Q-function. The closer our Q-values are to the optimal Q-values, the quicker it will converge.

Imagine if we happened to initialise our Q-values to the optimal Q-value. It would converge in one step!

Q-value initialisation is similar to reward shaping: we use heuristics to assign higher values to 'better' states. If we just define $\Phi(s) = V_0(s)$, then they are equivalent. In fact, if our potential function is **static** (the definition does not change during learning), then Q-value initialisation and reward shaping are equivalent¹.

Example – Q-value Initialisation in GridWorld

Using the idea of Manhattan distance for a potential function, we can define an initial Q-function as follows for state (1,2) using our potential function:

$$\begin{aligned}
 Q((1,2), Up) &= 0.9(1 - \frac{2}{5}) - (1 - \frac{2}{5}) = -0.06 \\
 Q((1,2), Down) &= 0.9(1 - \frac{2}{5}) - (1 - \frac{2}{5}) = -0.06 \\
 Q((1,2), Right) &= 0.9(1 - \frac{1}{5}) - (1 - \frac{1}{5}) = 0.12 \\
 Q((1,2), Left) &= 0.9(1 - \frac{3}{5}) - (1 - \frac{3}{5}) = -0.24
 \end{aligned}$$

Once we start learning over episodes, we will select those actions with a higher heuristic value, and also we are already closer to the optimal Q-function, so will converge faster. As with reward shaping though, this entirely depends on having a good potential function! A poor potential function will give an inaccurate initial Q-values, which may take longer to converge.

¹ Wiewiora, Eric. "Potential-based shaping and Q-value initialization are equivalent." Journal of Artificial Intelligence Research 19 (2003): 205-208. <https://www.jair.org/index.php/jair/article/download/10338/24713/>

2.7.4 Takeaways

Takeaways

- A weakness of model-free methods is that they spend a lot of time exploring at the start of the learning. It is not until they find some rewards that the learning begins. This is particularly problematic when rewards are sparse.
 - **Reward shaping** takes in some domain knowledge that “nudges” the learning algorithm towards more positive actions.
 - **Q-value initialisation** is a “guess” of the initial Q-values to guide early exploration
 - Reward shaping and Q-value initialisation are equivalent if our potential function is static.
 - **Potential-based reward shaping** guarantees that the policy will converge to the same policy without reward shaping.
-

Related Reading

- Chapter 9 (Approximate Solution Methods) of [Introduction to Reinforcement Learning, Sutton and Barto](#)
-

POLICY-BASED METHODS

In this chapter, we cover **policy-based** methods for reinforcement learning. Policy-based methods learn a policy directly, rather than learning the value of states and actions.

As noted earlier, learning a policy directly has advantages, particularly for applications where the state space or the action space are massive or infinite. If the action space is infinite, then we cannot do *policy extraction* because that requires us to iterate over all actions and extract the one that maximises the reward. Learning the policy directly mitigates this.

In this chapter, we will look introduce just two policy-based methods:

1. [**Policy iteration*](sec:policy-iteration): Like value iteration, this is a dynamic programming-based method for model-based MDPs
2. *Policy gradients*: This is a model-free technique that performs gradient ascent, which is the same as the well-known gradient descent technique, but which maximises rewards instead of minimising error.

3.1 Policy iteration

Learning outcomes

The learning outcomes of this chapter are:

1. Apply policy iteration to solve small-scale MDP problems manually and program policy iteration algorithms to solve medium-scale MDP problems automatically
 2. Discuss the strengths and weaknesses of policy iteration
 3. Compare and contrast policy iteration to value iteration
-

3.1.1 Overview

The other common way that MDPs are solved is using **policy iteration** – an approach that is similar to value iteration. While value iteration iterates over value functions, policy iteration iterates over policies themselves, creating a strictly improved policy in each iteration (except if the iterated policy is already optimal).

Policy iteration first starts with some (non-optimal) policy, such as a random policy, and then calculates the value of each state of the MDP given that policy — this step is called **policy evaluation**. It then updates the policy itself for every state by calculating the expected reward of each action applicable from that state.

The basic idea here is that policy evaluation is easier to computer than value iteration because the set of actions to consider is fixed by the policy that we have so far.

3.1.2 Policy evaluation

An important concept in policy iteration is **policy evaluation**, which is an evaluation of the expected reward of a policy. The **expected reward** of policy π from s , $V^\pi(s)$, is the weighted average of reward of the possible state sequences defined by that policy times their probability given π .

Definition – Policy evaluation

Policy evaluation can be characterised as $V^\pi(s)$ as defined by the following equation:

$$V^\pi(s) = \sum_{s' \in S} P_{\pi(s)}(s' | s) [r(s, a, s') + \gamma V^\pi(s')]$$

where $V^\pi(s) = 0$ for terminal states.

Note that this is very similar to the *Bellman equation*, except $V^\pi(s)$ is not the value of the best action, but instead just as the value for $\pi(s)$, the action that would be chosen in s by the policy π . Note the expression $P_{\pi(s)}(s' | s)$ instead of $P_a(s' | s)$, which means we only evaluate the action that the policy defines.

Once we understand the definition of policy evaluation, the implementation is straightforward. It is the same as value iteration except that we use the policy evaluation equation instead of the Bellman equation.

Algorithm (Policy iteration)

Input : π the policy for evaluation, V^π value function, and
MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Output : Value function V^π

repeat

$\Delta \leftarrow 0$

for each $s \in S$

$$V'^\pi(s) \leftarrow \underbrace{\sum_{s' \in S} P_{\pi(s)}(s' | s) [r(s, a, s') + \gamma V^\pi(s')]}_{\text{Policy evaluation equation}}$$

$$\Delta \leftarrow \max(\Delta, |V'^\pi(s) - V^\pi(s)|)$$

$$V^\pi \leftarrow V'^\pi$$

until $\Delta \leq \theta$

The **optimal expected reward** $V^*(s)$ is $\max_\pi V^\pi(s)$ and the **optimal policy** is the $\arg \max$

3.1.3 Policy improvement

If we have a policy and we want to improve it, we can use **policy improvement** to change the policy (that is, change the actions recommended for states) by updating the actions it recommends based on $V(s)$ that we receive from the policy evaluation.

Let $Q^\pi(s, a)$ be the expected reward from s when doing a first and then following the policy π . Recall from the chapter on *Markov Decision Processes* that we define this as:

$$Q^\pi(s, a) = \sum_{s' \in S} P_a(s' | s) [r(s, a, s') + \gamma V^\pi(s')]$$

In this case, $V^\pi(s')$ is the value function from the policy evaluation.

If there is an action a such that $Q^\pi(s, a) > Q^\pi(s, \pi(s))$, then the policy π can be **strictly improved** by setting $\pi(s) \leftarrow a$. This will improve the overall policy.

3.1.4 Policy iteration

Pulling together policy evaluation and policy improvement, we can define **policy iteration**, which computes an optimal π by performing a sequence of interleaved policy evaluations and improvements:

Algorithm (Policy iteration)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Output : Policy π

Set V^π to arbitrary value function; e.g., $V^\pi(s) = 0$ for all s

Set π to arbitrary policy; e.g. $\pi(s) = a$ for all s , where $a \in A$ is an arbitrary action

repeat

 Compute $V^\pi(s)$ for all s using policy evaluation

for each $s \in S$

$\pi(s) \leftarrow \operatorname{argmax}_{a \in A(s)} Q^\pi(s, a)$

until π does not change

The policy iteration algorithm finishes with an optimal π after a finite number of iterations, because the number of policies is finite, bounded by $O(|A|^{|S|})$, unlike value iteration, which can theoretically require infinite iterations.

However, each iteration costs $O(|S|^2|A| + |S|^3)$. Empirical evidence suggests that the most efficient is dependent on the particular MDP model being solved, but that surprisingly few iterations are often required for policy iteration.

3.1.5 Implementation

Below is a Python implementation for policy iteration. In this implementation, the parameter `max_iterations` is the maximum number of iterations of the policy iteration, and the parameter `theta` the largest amount the value function corresponding to the current policy can change before the policy evaluation loop terminates.

```
from tabular_policy import TabularPolicy
from tabular_value_function import TabularValueFunction
from qtable import QTable

class PolicyIteration:
    def __init__(self, mdp, policy):
        self.mdp = mdp
        self.policy = policy

    def policy_evaluation(self, policy, values, theta=0.001):
        while True:
            delta = 0.0
            new_values = TabularValueFunction()
            for state in self.mdp.get_states():
                # Calculate the value of V(s)
                actions = self.mdp.get_actions(state)
                old_value = values.get_value(state)
                new_value = values.get_q_value(
                    self.mdp, state, policy.select_action(state, actions)
                )
                values.update(state, new_value)
            delta = max(delta, abs(old_value - new_value))
```

(continues on next page)

```

        # terminate if the value function has converged
        if delta < theta:
            break

    return values

    """ Implmentation of policy iteration iteration. Returns the number of iterations_
    expected """

    def policy_iteration(self, max_iterations=100, theta=0.001):

        # create a value function to hold details
        values = TabularValueFunction()

        for i in range(1, max_iterations + 1):
            policy_changed = False
            values = self.policy_evaluation(self.policy, values, theta)
            for state in self.mdp.get_states():
                actions = self.mdp.get_actions(state)
                old_action = self.policy.select_action(state, actions)

                q_values = QTable()
                for action in self.mdp.get_actions(state):
                    # Calculate the value of Q(s,a)
                    new_value = values.get_q_value(self.mdp, state, action)
                    q_values.update(state, action, new_value)

                    #  $V(s) = \operatorname{argmax}_a Q(s,a)$ 
                    (new_action, _) = q_values.get_max_q(state, self.mdp.get_
                    actions(state))
                    self.policy.update(state, new_action)

                policy_changed = (
                    True if new_action is not old_action else policy_changed
                )

            if not policy_changed:
                return i

        return max_iterations

```

From this, we can see that policy evaluation looks very similar to value iteration. The main difference is in the inner loop: instead of finding the action with the maximum Q-value, we simply find the value of the action that is given the policy: `policy.select_action(state)`.

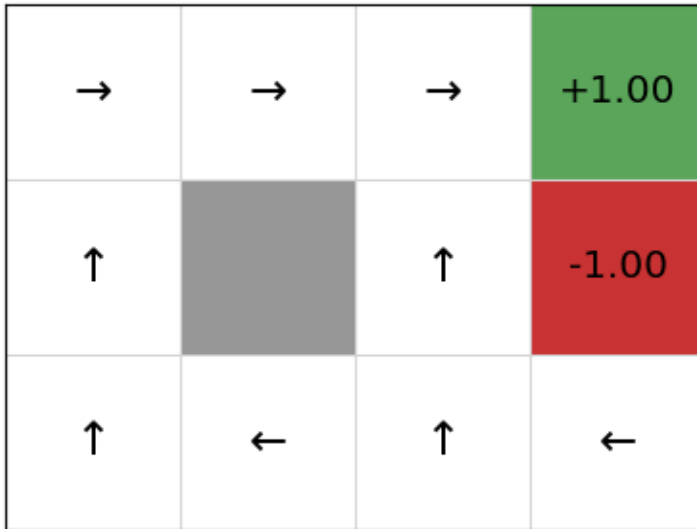
We can execute this to get the policy:

```

from gridworld import GridWorld
from policy_iteration import PolicyIteration
from tabular_policy import TabularPolicy

gridworld = GridWorld()
policy = TabularPolicy(default_action=gridworld.LEFT)
PolicyIteration(gridworld, policy).policy_iteration(max_iterations=100)
gridworld.visualise_policy(policy)

```



We can see that this matches the optimal policy according to *value iteration*.

Let's look at the policies that are generated after each iteration, noting that the initial policy is defined by taking a random action (left) and using that for every state:

We can see that this converges in just four iterations. Let's try on a larger state space of a 20 x 15 grid:

```
gridworld = GridWorld(width=20, height=15)
policy = TabularPolicy(default_action=gridworld.LEFT)
iterations = PolicyIteration(gridworld, policy).policy_iteration(max_iterations=100)
print("Number of iterations until convergence: %d" % (iterations))
gridworld.visualise_policy(policy)
```

```
Number of iterations until convergence: 19
```

→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	+1.00
→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	↑	↑	-1.00
→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	↑	↑	↑	↓
→	→	→	→	→	→	→	→	→	→	→	→	→	→	→	↑	↑	↑	←
↑	→	→	→	→	→	→	→	→	→	→	→	→	↑	↑	↑	↑	↑	←
↑	→	→	→	→	→	→	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑
↑	↑	→	→	→	→	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	→	→	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	↑	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	↑	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	↑	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	↑	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	↑	→	→	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑	↑
↑	↑	→	→	→	→	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑	↑
↑	█	→	→	→	→	→	→	→	→	→	→	↑	↑	↑	↑	↑	↑	↑
→	→	→	→	→	→	→	→	→	→	→	→	→	→	↑	↑	↑	↑	↑

This terminates in 19 iterations. We can see that the policy is optimal as it always directs the agent to terminating state at (3,2) with the positive reward. However, the number of iterations can change depending on the initial policy and the order in which actions are evaluated.

3.1.6 Takeaways

Takeaways

- **Policy iteration** is a dynamic programming technique for calculating a policy directly, rather than calculating an optimal $V(s)$ and extracting a policy; but one that uses the concept of values.
- It produces an optimal policy in a finite number of steps.
- Similar to value iteration, for medium-scale problems, it works well, but as the state-space grows, it does not scale well.

3.2 Policy gradients

Learning outcomes

The learning outcomes of this chapter are:

1. Apply policy gradients and actor critic methods to solve small-scale MDP problems manually and program policy gradients and actor critic algorithms to solve medium-scale MDP problems automatically
2. Compare and contrast policy-based reinforcement learning with value-based reinforcement learning

3.2.1 Overview

As noted earlier, policy-based methods search for a policy directly, rather than searching for a value function and extracting a policy. In this section, we look at a model-free method that optimises a policy directly. It is similar to Q-learning and SARSA, but instead of updating a Q-function, it updates the parameters θ of a policy directly using gradient ascent.

In policy gradient methods, we approximate the policy from the rewards and actions received in our episodes, similar to the way we do it with Q-learning. We can do this provided that the policy has two properties:

1. The policy is represented using some function that is **differentiable** with respect to its parameters. For a non-differentiable policy, we cannot calculate the gradient.
2. Typically, we want the policy to be **stochastic**. Recall from the section on *policies* that a stochastic policy specifies a **probability distribution** over actions, defining the probability with which each action should be chosen.

The goal of a policy gradient is to approximate the optimal policy $\pi_\theta(s, a)$ via gradient ascent on the expected return. Gradient ascent will find the best parameters θ for the particular MDP.

3.2.2 Policy improvement using gradient ascent

The goal of gradient ascent is to find weights of a policy function that maximises the expected return. This is done iteratively by calculating the gradient from some data and updating the weights of the policy.

The *expected value* of a policy π_θ with parameters θ is defined as:

$$J(\theta) = V^{\pi_\theta}(s_0)$$

where V^{π_θ} is the policy evaluation using the policy π_θ and s_0 is the initial state. This expression is computationally expensive to calculate, because we need to execute every possible episode from s_0 to every terminal state, which may be an infinite number of episodes. So, we use policy gradient algorithms to approximate this instead. These search for a local maximum in $J(\theta)$ by **ascending the gradient** of the policy with respect to the parameters θ , using episodic samples.

Definition – Policy gradient

Given a policy objective $J(\theta)$, the **policy gradient** of J with respect to θ , written $\nabla_\theta J(\theta)$ is defined as:

$$\nabla_\theta J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$

where $\frac{\partial J(\theta)}{\partial \theta_i}$ is the partial derivative of J with respect to θ_i .

If we want to follow the gradient towards the optimal $J(\theta)$ for our problem, we use the gradient and update the weights:

$$\theta \leftarrow \theta + \alpha \nabla J(\theta)$$

where α is a learning rate parameter that dictates how big the step in the direction of the gradient should be.

The question is: what is $\nabla J(\theta)$? The **policy gradient theorem** (see Sutton and Barto, Section 13.2) says that for any differentiable policy π_θ , state s , and action a , that $\nabla J(\theta)$ is:

$$\nabla J(\theta) = \mathbb{E}[\nabla \ln \pi_\theta(s, a) Q(s, a)]$$

The expression $\ln \pi_\theta(s, a)$ tells us how to change the weights θ : increase the log probability of selecting action a in state s . If the quality of selecting action a in s is positive, we would increase that probability; otherwise we decrease the probability. So, this is the expected return of taking action $\pi_\theta(s, a)$ multiplied by the gradient.

In these notes, we will not go into details about gradients or algorithms for solving them – this is itself a large topic that is relevant outside of reinforcement learning. Instead, we will just give the intuition and how to use this for model-free reinforcement learning.

3.3 REINFORCE

The REINFORCE algorithm is one algorithm for policy gradients. We cannot calculate the gradient optimally because this is too computationally expensive – we would need to solve for all possible trajectories in our model. In REINFORCE, we sample trajectories, similar to the sampling process in *Monte-Carlo reinforcement learning*.

Algorithm (REINFORCE)

Input : A differentiable policy $\pi_\theta(s, a)$, an MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Output : Policy $\pi_\theta(s, a)$

Initialise parameters θ arbitrarily

repeat

 Generate episode $(s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T)$ by following π_θ

for each (s_t, a_t) in the episode

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} r_k$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi_\theta(s, a)$$

until π_θ converges

REINFORCE generates an entire episode using Monte-Carlo simulation by following the policy so far; therefore, it generates better and better policies as π is improved. It then steps through each action in the episode, calculates G , the total future discounted reward of the trajectory. Using this reward, it calculates the gradient π and multiplies this in the direction of G .

Comparing to value-based techniques, we can see that REINFORCE (and other policy-gradient approaches) do not evaluate each action in the policy improvement process. In policy iteration, we update the policy $\pi(s) \leftarrow \operatorname{argmax}_{a \in A(s)} Q^\pi(s, a)$. In REINFORCE, the most recently sampled action and its reward are used to calculate the gradient and update.

This has the advantage that policy-gradient approaches can be applied when the action space or state space are continuous; e.g. there are one or more actions with a parameter that takes a continuous value. This is because it uses the gradient instead of doing the policy improvement explicitly. For the same reason, policy-gradient approaches are often more efficient than value-based approaches when there are a large number of actions.

Note: From the algorithm above, we can see that REINFORCE is an *on policy* approach. The sample trajectories from directly from π_θ and the update happens

3.3.1 Implementation

The implementation of REINFORCE takes a policy, but this policy must be differentiable. As such, we cannot use the tabular policy used in policy iteration. As we can see, unlike Q-learning and SARSA, the update happens only at the end of each episode.

```
class PolicyGradient:
    def __init__(self, mdp, policy, alpha) -> None:
        super().__init__()
        self.alpha = alpha # Learning rate (gradient update step-size)
        self.mdp = mdp
        self.policy = policy

    """ Generate and store an entire episode trajectory to use to update the policy """
    ↪

    def execute(self, episodes=100):
        for _ in range(episodes):
            actions = []
            states = []
            rewards = []

            state = self.mdp.get_initial_state()
            episode_reward = 0
            while not self.mdp.is_terminal(state):
                action = self.policy.select_action(state, self.mdp.get_actions(state))
                next_state, reward = self.mdp.execute(state, action)

                # Store the information from this step of the trajectory
                states.append(state)
                actions.append(action)
                rewards.append(reward)

                state = next_state

            deltas = self.calculate_deltas(rewards)
            self.policy.update(states=states, actions=actions, deltas=deltas)

    def calculate_deltas(self, rewards):
        """
        Generate a list of the discounted future rewards at each step of an episode
        Note that discounted_reward[T-2] = rewards[T-1] + discounted_reward[T-1] * ↪
        ↪gamma.
        We can use that pattern to populate the discounted_rewards array.
        """
        T = len(rewards)
        discounted_future_rewards = [0 for _ in range(T)]
        # The final discounted reward is the reward you get at that step
        discounted_future_rewards[T - 1] = rewards[T - 1]
        for t in reversed(range(0, T - 1)):
            discounted_future_rewards[t] = (
```

(continues on next page)

(continued from previous page)

```

        rewards[t]
        + discounted_future_rewards[t + 1] * self.mdp.get_discount_factor()
    )
    deltas = []
    for t in range(len(discounted_future_rewards)):
        deltas += [
            self.alpha
            * (self.mdp.get_discount_factor() ** t)
            * discounted_future_rewards[t]
        ]
    return deltas

```

We need a differentiable policy so that we can calculate the gradient and update. To illustrate this from first principles, let's look at an implementation of a policy that uses logistic regression. This basic implementation supports environments with only two actions, but it is sufficient to illustrate the basics of policy gradient methods from first principles.

The policy inherits from `StochasticPolicy`, which means that the policy is a *stochastic policy* $\pi_\theta(s, a)$ that returns the probability of action a being executed in state s . The `select_action` is stochastic, as can be seen below – it selects between the two actions using the policy's probability distribution.

```

import math
import random

from policy import StochasticPolicy

""" A two-action policy implemented using logistic regression from first principles """
↪

class LogisticRegressionPolicy(StochasticPolicy):

    """ Create a new policy, with given parameters theta (randomly if theta is None) """
    ↪

    def __init__(self, actions, num_params, theta=None):
        assert len(actions) == 2

        self.actions = actions

        if theta is None:
            theta = [0.0 for _ in range(num_params)]
            self.theta = theta

    """ Select one of the two actions using the logistic function for the given state """
    ↪

    def select_action(self, state, actions):
        # Get the probability of selecting the first action
        probability = self.get_probability(state, self.actions[0])

        # With a probability of 'probability' take the first action
        if random.random() < probability:
            return self.actions[0]
        return self.actions[1]

```

(continues on next page)

(continued from previous page)

```

""" Update our policy parameters according using the gradient descent formula:
    theta <- theta + alpha * G * nabla J(theta),
    where G is the future discounted reward
"""

def update(self, states, actions, deltas):
    for t in range(len(states)):
        gradient_log_pi = self.gradient_log_pi(states[t], actions[t])
        # Update each parameter
        for i in range(len(self.theta)):
            self.theta[i] += deltas[t] * gradient_log_pi[i]

""" Get the probability of applying an action in a state """

def get_probability(self, state, action):
    # Calculate y as the linearly weight product of the
    # policy parameters (theta) and the state
    y = self.dot_product(state, self.theta)

    # Pass y through the logistic regression function to convert it to a
    ↪probability
    probability = self.logistic_function(y)

    if action == self.actions[0]:
        return probability
    return 1 - probability

""" Computes the gradient of the log of the policy (pi),
which is needed to get the gradient of the objective (J).
Because the policy is a logistic regression, using the policy parameters (theta).
    pi(actions[0] | state) = 1 / (1 + e^(-theta * state))
    pi(actions[1] | state) = 1 / (1 + e^(theta * state))
When we apply a logarithmic transformation and take the gradient we end up with:
    grad_log_pi(left | state) = state - state * pi(left|state)
    grad_log_pi(right | state) = - state * pi(0|state)
"""

def gradient_log_pi(self, state, action):
    y = self.dot_product(state, self.theta)
    if action == self.actions[0]:
        return [s_i - s_i * self.logistic_function(y) for s_i in state]
    return [-s_i * self.logistic_function(y) for s_i in state]

""" Standard logistic function """

@staticmethod
def logistic_function(y):
    return 1 / (1 + math.exp(-y))

""" Compute the dot product between two vectors """

@staticmethod
def dot_product(vec1, vec2):
    return sum([v1 * v2 for v1, v2 in zip(vec1, vec2)])

```

The update method, as well as the `gradient_log_pi` method that it calls, are where the policy gradient theorem is applied. In update, for every state-action transition in the trajectory, we calculate the gradient and then update the

parameters θ using the corresponding partial derivative.

Because we use logistic regression to represent the policy in this simplified implementation that allows only two actions, the probability for each action (and therefore the policy) can be calculated as follows:

$$\pi_{\theta}(s, a_0) = \frac{1}{1 + e^{-\theta \cdot s}}$$

$$\pi_{\theta}(s, a_1) = \frac{1}{1 + e^{\theta \cdot s}}$$

This gives us the probability of choosing actions a_0 and a_1 in state s , which approximates the expression $\pi_{\theta}(s, a)Q(s, a)$ in the policy gradient theorem.

Since the policy follows a sigmoid function, we can use the following result from calculus to find the derivative. If $\sigma(x)$ is a sigmoid, then the derivative of $\sigma(x)$ with respect to x conforms to the following pattern:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1 - \sigma(x))$$

Using this result, we can then find $\nabla \ln \pi(a, s)$:

$$\nabla \ln \pi_{\theta}(a_0, s) = s - s \cdot \pi_{\theta}(s, a_0)$$

$$\nabla \ln \pi_{\theta}(a_1, s) = -s \cdot \pi_{\theta}(s, a_1)$$

This gives us a vector of the partial derivatives, which the `update` method then uses to update the corresponding θ_i value.

Let's try this implementation on an example using the REINFORCE algorithm and our logistic regression policy. Because our logistic regression policy supports only two actions, we cannot use the 4x3 GridWorld example, so we instead use an even simpler example (who thought that would be possible!) of a Gridworld that is 11x1 and has a -1 reward at one end and a +1 reward at the other:

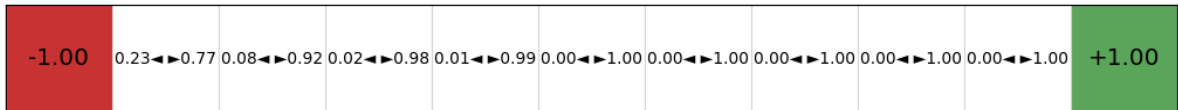
```
from gridworld import GridWorld
from gridworld import OneDimensionalGridWorld
from policy_gradient import PolicyGradient
from logistic_regression_policy import LogisticRegressionPolicy

gridworld = GridWorld(
    height=1, width=11, initial_state=(5, 0), goals=[((0, 0), -1), ((10, 0), 1)]
)
gridworld_image = gridworld.visualise()
```



Next, we create a `LogisticRegressionPolicy` with just two actions, `left` and `right`, and with two parameters corresponding to the x and y coordinates. This is used in a `PolicyGradient` instance to produce the following policy:

```
policy = LogisticRegressionPolicy(
    actions=[GridWorld.LEFT, GridWorld.RIGHT],
    num_params=len(gridworld.get_initial_state()),
)
PolicyGradient(gridworld, policy, alpha=0.1).execute(episodes=1000)
policy_image = gridworld.visualise_stochastic_policy(policy)
```



We can see here that this is a stochastic policy — instead of giving us an action (left or right) in each cell, we get the probability that the policy will select each of these actions. The policy learns to select right with a much higher probability because the positive reward is to the right.

To follow the policy, we can use this deterministically by simply selecting the action with the highest probability, but in some applications, the optimal behaviour is to select an action by sampling from the probability distribution.

The difference between value-based methods such as Q-learning and SARSA is demonstrated by the output: there is just a policy and no value function or Q-function because the policy is learnt directly.

If we step through the policy during training, we can see the gradient updates performing their role in policy improvement:

This policy only considers two actions, but it can be easily extended to support multiple actions using standard machine learning techniques like one-vs-rest classification.

3.3.2 Deep REINFORCE

Just like we can use deep neural networks to approximate Q-functions in *deep Q learning*, we can use deep neural networks to approximate policies. Neural networks are differentiable, so these can be used in policy gradient algorithms such as REINFORCE — the policy parameters θ are the parameters to the neural network.

As with deep Q learning, this has the advantage that features of the problem are learnt, features do not have to be independent, therefore supporting a larger set of problems compared to a logistic regression approach, and we can use unstructured data as input, such as images and videos.

To implement a deep policy gradient approach in REINFORCE, we just have to implement a new policy that uses a deep neural network.

In our implementation, we again use the PyTorch deep learning framework (<https://pytorch.org/>).

The policy uses a three layer network with the following:

1. The first layer takes the state vector, so the input features are the features of the state. In the case of the GridWorld example, this would be just the x- and y-coordinates of the agent.
2. We have a hidden layer, with a default number of 64 hidden dimensions, but this is parameterised by the variable `hidden_dim` in the `__init__` constructor.
3. The third and final layer is the output layer, which returns a categorical distribution with a dimensionality the same size as the action space, so that each action is associated with a probability of being selected.
4. We use a non-linear ReLU (rectified linear unit) between layers.

Recall from the section on *deep Q learning* that the `Linear` layers are dense layers in PyTorch.

It inherits from the `StochasticPolicy` and then implement the `update`, `select_action`, and `get_probability` methods. These take advantage of optimisations with the PyTorch framework, so the calculation of the gradient is 'hidden' by the PyTorch library. The line `self.optimiser.zero_grad()` then 'zeros' out the existing gradient so that the new gradient can be calculated. The gradient is calculated using `loss.backward()`. Then `self.optimiser.step()` adjusts the parameters θ in the direction of the gradient:

```
import torch
import torch.nn as nn
from torch.distributions.categorical import Categorical
from torch.optim import Adam
```

(continues on next page)

(continued from previous page)

```

import torch.nn.functional as F

from policy import StochasticPolicy

class DeepNeuralNetworkPolicy(StochasticPolicy):
    """
    An implementation of a policy that uses a PyTorch (https://pytorch.org/)
    deep neural network to represent the underlying policy.
    """

    def __init__(self, mdp, state_space, action_space, hidden_dim=64, alpha=0.001):
        self.mdp = mdp
        self.state_space = state_space
        self.action_space = action_space
        self.alpha = alpha

        # Define the policy structure as a sequential neural network.
        self.policy_network = nn.Sequential(
            nn.Linear(in_features=self.state_space, out_features=hidden_dim),
            nn.ReLU(),
            nn.Linear(in_features=hidden_dim, out_features=hidden_dim),
            nn.ReLU(),
            nn.Linear(in_features=hidden_dim, out_features=self.action_space),
        )

        # The optimiser for the policy network, used to update policy weights
        self.optimiser = Adam(self.policy_network.parameters(), lr=self.alpha)

        # A two-way mapping from actions to integer IDs for ordinal encoding
        actions = self.mdp.get_actions()
        self.action_to_id = {actions[i]: i for i in range(len(actions))}
        self.id_to_action = {
            action_id: action for action, action_id in self.action_to_id.items()
        }

    """ Select an action using a forward pass through the network """

    def select_action(self, state, actions):
        # Convert the state into a tensor so it can be passed into the network
        state = torch.as_tensor(state, dtype=torch.float32)
        action_logits = self.policy_network(state)
        action_distribution = Categorical(logits=action_logits)
        action = action_distribution.sample()
        return self.id_to_action[action.item()]

    """ Get the probability of an action being selected in a state """

    def get_probability(self, state, action):
        state = torch.as_tensor(state, dtype=torch.float32)
        with torch.no_grad():
            action_logits = self.policy_network(state)
        # A softmax layer turns action logits into relative probabilities
        probabilities = F.softmax(input=action_logits, dim=-1).tolist()

        # Convert from a tensor encoding back to the action space

```

(continues on next page)

(continued from previous page)

```

    return probabilities[self.action_to_id[action]]

def evaluate_actions(self, states, actions):
    action_logits = self.policy_network(states)
    action_distribution = Categorical(logits=action_logits)
    log_prob = action_distribution.log_prob(actions.squeeze(-1))
    return log_prob.view(1, -1)

def update(self, states, actions, deltas):
    # Convert to tensors to use in the network
    deltas = torch.as_tensor(deltas, dtype=torch.float32)
    states = torch.as_tensor(states, dtype=torch.float32)
    actions = torch.as_tensor([self.action_to_id[action] for action in actions])

    action_log_probs = self.evaluate_actions(states, actions)

    # Construct a loss function, using negative because we want to descend,
    # not ascend the gradient
    loss = -(action_log_probs * deltas).mean()
    self.optimiser.zero_grad()
    loss.backward()

    # Take a gradient descent step
    self.optimiser.step()

```

As with the *deep Q learning* implementation, PyTorch does not support strings as values, so we need to encode action names and states as integers.

We can now use this implementation by creating a REINFORCE agent with a `DeepNeuralNetworkPolicy` instance as the policy, and use it to learn a policy for the GridWorld example:

```

from gridworld import GridWorld
from policy_gradient import PolicyGradient
from deep_nn_policy import DeepNeuralNetworkPolicy

gridworld = GridWorld()
policy = DeepNeuralNetworkPolicy(
    gridworld, state_space=len(gridworld.get_initial_state()), action_space=4
)
PolicyGradient(gridworld, policy, alpha=0.1).execute(epochs=1000)
gridworld_image = gridworld.visualise_stochastic_policy(policy)

gridworld.visualise_policy(policy)

```

0.16 ▲ 0.00 ← 0.84 ▼ 0.00	0.06 ▲ 0.00 ← 0.93 ▼ 0.00	0.14 ▲ 0.00 ← 0.86 ▼ 0.00	+1.00
0.93 ▲ 0.01 ← 0.06 ▼ 0.00		0.88 ▲ 0.04 ← 0.08 ▼ 0.01	-1.00
0.98 ▲ 0.01 ← 0.00 ▼ 0.00	0.94 ▲ 0.05 ← 0.00 ▼ 0.01	0.93 ▲ 0.07 ← 0.00 ▼ 0.00	0.95 ▲ 0.04 ← 0.00 ▼ 0.00

→	→	→	+1.00
↑		↑	-1.00
↑	↑	↑	↑

Again, we can see that this policy is stochastic: each action has a probability of being executed in a state.

Simulating the process of training the policy, we can see that initially, all four actions have (approximately) the same probability of being executed, but deep REINFORCE learns a good Q-function and therefore policy:

3.4 Advantages and disadvantages of policy gradients (compared to value-based techniques)

3.4.1 Advantages

- **High-dimensional problems:** The major advantage of policy-gradient approaches compared to value-based techniques like Q-learning and SARSA is that they can handle high-dimensional action and state spaces, including actions and states that are continuous. This is because we do not have to iterate over all actions using $\operatorname{argmax}_{a \in A(s)}$ as we do in value-based approaches. For continuous problems, $\operatorname{argmax}_{a \in A(s)}$ is not possible to calculate, while for a high number of actions, the computational complexity is dependent on the number of actions.

3.4.2 Disadvantages

- **Sample inefficiency:** Since the policy gradients algorithm takes an entire episode to do the update, it is difficult to determine which of the state-action pairs are those that effect the value G (the episode reward), and therefore which to sample.
- **Loss of explainability:** Model-free reinforcement learning is a particularly challenging case to understand and explain why a policy is making a decision. This is largely due to the model-free property: there are no action definitions that can be used as these are unknown. However, policy gradients are particularly difficult because the values of states are unknown: we just have a resulting policy. With value-based approaches, knowing V or Q provides some insight into why actions are chosen by a policy; although explainability problems still remain.

3.5 Actor critic methods

The sample efficiency problem in REINFORCE leads to issues with policy convergence. As with Monte-Carlo simulation, the high variance in the cumulative rewards G over episodes leads to instability.

Actor critic methods aim to mitigate this problem. The idea is that instead of learning a value function or a policy, we learn both. The policy is called the **actor** and the value function is called the **critic**. The primary idea is that the actor produces actions, and as in *temporal difference learning*, the value function (the critic) provides feedback or “criticism” about these actions as a way of bootstrapping.

The **Q Actor Critic** algorithm uses a Q-function as the critic.

Algorithm (Q Actor Critic)

Input : MDP $M = \langle S, s_0, A, P_a(s' | s), r(s, a, s') \rangle$

Input : A differentiable actor policy $\pi_\theta(s, a)$

Input : A differentiable critic Q-function $Q(s, a)$

Output : Policy $\pi_\theta(s, a)$

Initialise actor π parameters θ and critic parameters w arbitrarily

repeat (for each episode e)

$s \leftarrow$ the first state in episode e

 Select action $a \sim \pi_\theta(s)$

repeat (for each step in episode e)

 Execute action a in state s

 Observe reward r and new state s'

 Select action $a' \sim \pi_\theta(s')$

$\delta \leftarrow r + \gamma \cdot Q_w(s', a') - Q_w(s, a)$

$w \leftarrow w + \alpha_w \cdot \delta \cdot \nabla Q_w(s, a)$

$\theta \leftarrow \theta + \alpha_\theta \cdot \delta \cdot \nabla \ln \pi_\theta(s, a)$

$s \leftarrow s'; a \leftarrow a'$

until s is the last state of episode e (a terminal state)

until π_θ converges

Note that we have two different learning rates α_w and α_θ for the Q-function and policy respectively.

Let's analyse the key parts in more detail. The line that updates δ is the same as the δ calculation in SARSA: it is temporal difference value for executing action a in state s , with the estimate of the future discount reward being $Q_w(s', a')$.

Once the δ value is calculated, we update both the actor and the critic. The weights of the critic Q_w are updated by following the gradient $\nabla Q_w(s, a)$ of the critic Q-function at s, a , and then the parameters of the actor θ are updated the

same way as in REINFORCE, except that the value of δ uses the temporal difference estimate based on $Q_w(s, a)$ instead of using G .

So, this simultaneously learns the policy (actor) π_θ and a critic (Q-function) Q_w , but the critic is learnt only to provide the temporal difference update, not to extract the policy.

But wait! Didn't we say early that the weakness of value-based methods was that they could not extend to continuous action spaces? Haven't we now gone backwards by including a Q-function? Why not just use the Q-function directly?

The reason the actor critic methods still work like this is because the actor policy π_θ selects actions for us, while the critic $Q_w(s, a)$ is only ever used to calculate the temporal difference estimate for an already selected action. We do not use the critic Q-function to select actions – we just use the policy. As such, this will still extend to continuous state spaces and be more efficient for large action space.

3.5.1 Implementation

To implement the Q Actor Critic framework, we first create a new base class called `ActorCritic`, which can be used as a base for other types of actor critic methods, such as **advantage actor critics**, which we will not discuss here.

The `ActorCritic` class is an abstract class that looks similar to that of `QLearning`, except that we update both the actor and the critic:

```
class ActorCritic:
    def __init__(self, mdp, actor, critic, alpha=0.1):
        self.alpha = alpha # Learning rate (gradient update step-size)
        self.mdp = mdp
        self.actor = actor # Actor (policy based) to select actions
        self.critic = critic # Critic (value based) to evaluate actions

    def execute(self, episodes=100):
        for _ in range(episodes):
            actions = []
            states = []
            rewards = []
            next_states = []

            state = self.mdp.get_initial_state()
            while not self.mdp.is_terminal(state):
                action = self.actor.select_action(state, self.mdp.get_actions(state))
                next_state, reward = self.mdp.execute(state, action)
                self.update_critic(reward, state, action, next_state)

                # Store the information from this step of the trajectory
                states.append(state)
                actions.append(action)
                rewards.append(reward)
                next_states.append(next_state)

                state = next_state

            self.update_actor(
                rewards=rewards, states=states, actions=actions, next_states=next_
↪states
            )

        """ Update the actor using a batch of rewards, states, actions, and next states """
↪"""
```

(continues on next page)

(continued from previous page)

```

def update_actor(self, rewards, states, actions, next_states):
    abstract

    """ Update the critic using a reward, state, action, and next state """

def update_critic(self, reward, state, action, next_state):
    abstract

```

Note from the code above that we use the actor (the policy) to choose an action, and then update both the critic and the actor. In this particular implementation, we batch update the actor policy at the end of the episode.

Next, we have to instantiate the ActorCritic class as a QActorCritic class to implement the update_actor and update_critic classes:

```

from actor_critic import ActorCritic
from deep_agent import DeepAgent

class QActorCritic(ActorCritic, DeepAgent):
    def __init__(self, mdp, actor, critic, alpha=0.1):
        super().__init__(mdp, actor, critic, alpha)

    def update_actor(self, rewards, states, actions, next_states):
        q_values = [
            self.critic.qfunction.get_q_value(state, action)
            for state, action in zip(states, actions)
        ]
        self.actor.update(states, actions, q_values)

    def update_critic(self, reward, state, action, next_state):
        next_state = self.encode_state(next_state)
        actions = self.mdp.get_actions(next_state)
        next_action = self.actor.select_action(next_state, actions)
        q_value = self.critic.qfunction.get_q_value(state, action)
        delta = self.critic.get_delta(reward, q_value, state, next_state, next_action)
        self.critic.qfunction.update(state=state, action=action, delta=delta)

```

Now, we can create a policy and Q-function using any differentiable policy and any Q-function implementation. We choose DeepNeuralNetworkPolicy and DeepQFunction with QLearning updates.

```

from deep_nn_policy import DeepNeuralNetworkPolicy
from q_actor_critic import QActorCritic
from deep_qfunction import DeepQFunction
from gridworld import GridWorld
from multi_armed_bandit.epsilon_greedy import EpsilonGreedy
from qlearning import QLearning

gridworld = GridWorld()

# Instantiate the critic
qfunction = DeepQFunction(
    gridworld,
    state_space=len(gridworld.get_initial_state()),
    action_space=5,
    hidden_dim=16,

```

(continues on next page)

(continued from previous page)

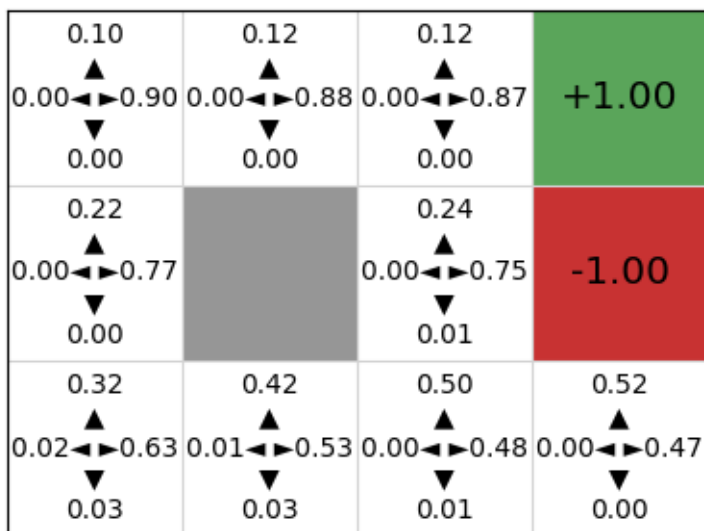
```

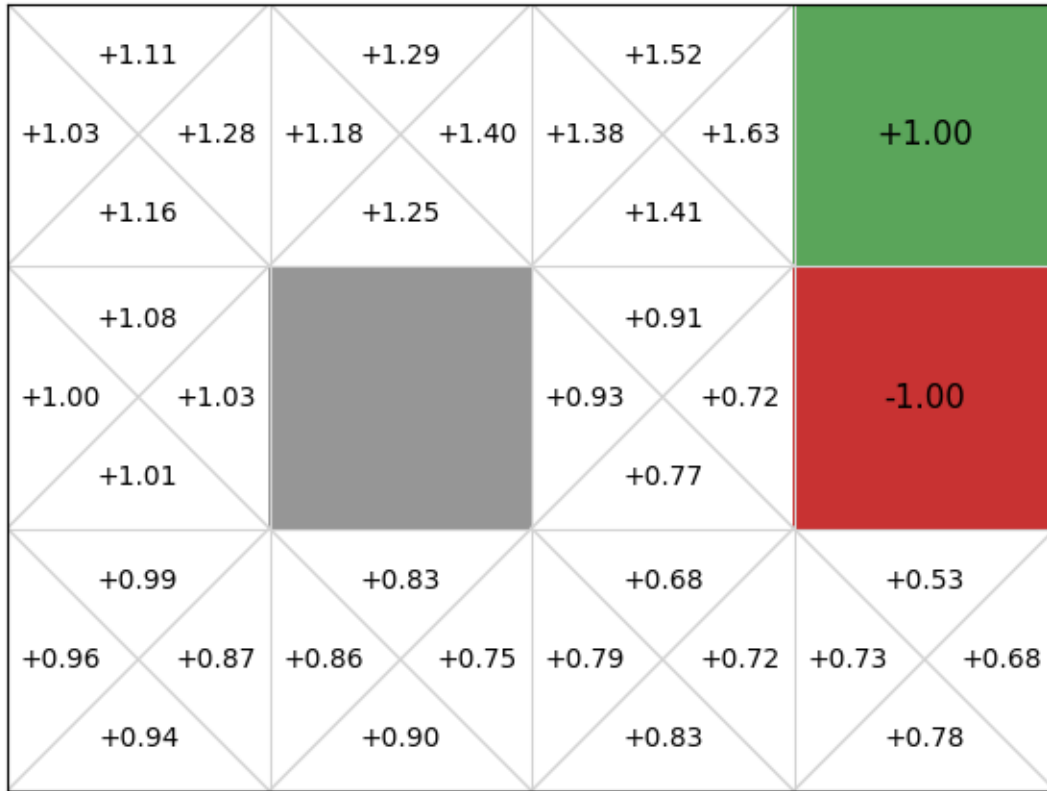
)
critic = QLearning(gridworld, EpsilonGreedy(), qfunction, alpha=1.0)

# Instantiate the actor
actor = DeepNeuralNetworkPolicy(
    gridworld, state_space=len(gridworld.get_initial_state()), action_space=4
)

# Instantiate the actor critic agent
q_actor_critic = QActorCritic(mdp=gridworld, actor=actor, critic=critic).execute(
    episodes=1000
)
gridworld.visualise_stochastic_policy(actor)
gridworld.visualise_q_function(critic.qfunction)

```





We can see that the actor critic agent has learnt both a policy that is very good, but also a Q-function critic that could be used as a policy (because our action space is finite and very small). In a continuous state space, we would be able to learn the critic, but not use it as a policy because we cannot iterate over the possible actions.

3.5.2 Takeaways

Takeaways

- **Policy gradient methods** such as REINFORCE and actor-critic approaches directly learn a policy instead of first learning a value function or Q-function.
- Using trajectories, the parameters for a policy are updated by **following the gradient upwards** – the same as gradient descent but in the opposite direction.
- Unlike policy iteration, policy gradient approaches are **model free**.
- **Actor critic** methods also learn a value function or Q-function to reduce the variance in the cumulative rewards.

MODELLING AND ABSTRACTION FOR MDPS

Learning outcomes

The learning outcomes of this chapter are:

1. Describe modelling and abstraction strategies to scale MDP algorithms to problems.
 2. Apply modelling and abstraction strategies to non-trivial MDP problems..
-

4.1 Overview

As discussed through Part I of this book, often our reinforcement learning algorithms struggle with **scale**. That is, as the state and action spaces of problems get larger, the time taken to arrive at good policies becomes increases exponentially.

Two strategies that are useful if you can apply them are to : (1) apply more computational power to the problem so that you can run more episodes or assess more states; or (2) increase the efficiency of a simulator (if you are using a simulation-based technique).

However, often we may already be using all the computational power we have access to, or maybe we are not working in a simulated environment at all. In these cases, one way is to simplify the problem and solve that.

In this section, I give some tips that I have found useful when applying reinforcement learning techniques. Implementing these tips requires us to understand the domain that we are working in. Such an implementation will only work on that domain or others that are very similar – they will not likely generalise to other domains. However, these tips/strategies themselves are general, and would work on many domains if we take the time to apply them.

4.2 Abstraction

One solution is to define an abstracted version of the MDP, solve that abstracted problem, and then translate the policy back to the original problem. Using abstraction, we take the original MDP model M that we have, and define a new MDP model M_A that is smaller than the original: it has fewer states and/or fewer actions than the original model.

The process is that the modeller should ask themselves the following questions:

1. What are the important decisions that I need to make in this problem to reach achieve some objective?
2. What information do I need to do this?
3. Can I translate the original MDP action and state spaces into smaller action and state spaces so that I can access this information and discard some information that is less important?

We then define an abstract model $M' = (S', s'_0, A', P', r', \gamma')$ that is an abstraction of M , such that there is some mapping $f_s : S \rightarrow S'$ such that $f_s(s)$ abstracts state s into another state s' ; and similarly mappings for A , P , r' , and even γ (the length of plans will be different if the action space is smaller).

We solve the new model M' , obtaining abstract policy $\pi' : S' \rightarrow A'$. We then need to define a mapping $g : A' \rightarrow A$ to map each abstract actions to actions in the original model. Given this, we can then define the policy for M as:

$$\pi(s) = g(\pi'(f_s(s)))$$

This means that $\pi(s)$ translates s to $s' \in S'$ using f_s , then applies abstract policy π' to get an abstraction actions $a' \in A'$, and then maps this to concrete action $a \in A$.

By “model”, I don’t just mean for model-based reinforcement learning: even in a model-free environment we can abstract the actions and states that we see, and modify the rewards, by putting a “layer” in between the environment and the reinforcement learning algorithm.

For example, in these notes we use the GridWorld problem to illustrate various techniques because it is simple and intuitive. This is already an abstraction of a real navigation problem. In a real navigation problem, a robot would have to move in a continuous environment, with actions such as rotating and accelerating. However, our objective in the problem is to find the best route, not to move the wheels. For the purposes of finding the best route, we model the problem differently by dividing the state space into grids and mapping the actions into directions, as this is all the information that we need to find a good, but perhaps not optimal, route. If we wanted to actually move a robot, we would need a layer in-between that: (1) translates the real states into our abstracted grid coordinators and identifies when we are in a reward state, which is function f_s ; and (2) translates our abstracted actions back into actions in the environment, which is function g . For example, the action `Right` may translate into rotating 90 degrees and then moving forward 2 metres.

Sometimes a problem may already be as abstract as it can be. For example, if we are learning a controller for a robotic arm in a manufacturing facility, the information set from the sensors and to the actuators may strongly influence the effect of higher-level actions, so cannot be abstracted away. However, in practice, there are usually abstractions we can use.

In reality, most of the techniques in this section are abstraction techniques of some sort, but some other tricks can help.

4.3 State abstraction

In some cases, the size of the state space can be abstracted by combining some states that are different, but are either equivalent from a semantic view, or are similar enough that they will lead to the same action most of the time.

AlphaGoZero, the famous Go-playing agent, used state abstraction by noting that the state is game is symmetric: it is a 19x19 grid and if the first move is to e.g. place a stone in one of the corners, then *which* corner is not relevant: placing a stone in all four corners corresponds to the same abstract state. This reduces the size of the state space to a quarter of its original size, and does not lose any information

Other abstraction techniques may lose information, but still be useful. For example, the state abstraction in GridWorld from precise coordinates into a simple grid.

4.4 Action abstraction

Similar to state abstraction, we can abstract actions. There are two main ways:

1. Abstract sequences of smaller actions into single higher-level actions, such as in the GridWorld domain where we abstract physical navigation actions to just `Left`, `Right`, `Up`, and `Down`.
2. We can also **group similar actions** into a single action. For example, in the GridWorld domain, if we have lower-level actions like `Rotate`, which rotates by a given number of degrees, instead of reasoning about 360 different version of `Rotate` (one for each degree), we could group actions into buckets: one abstract action for turning

0-15, one for 16-30, etc. We **lose information/precision**, but in many cases, the solution may be good enough, and the problem is much easier to solve.

4.5 Rewarding sub-goals

In some problems, there are final goals that need to be achieved, and we receive rewards for transitioning to states where those goals are achieved. One problem we saw in these notes, such as in the Freeway example, is when those rewards are a long way from the start state. As we often start with a random policy or value/Q-function, we spend a lot of time doing random simulation before we receive a reward by achieving the goal.

However, often these problems have **sub-goals** that we need to achieve on the way. If we do not achieve the sub-goals, we cannot achieve the primary goal. For example, in the Freeway game, we need to reach row 1, then row 2, etc., until we reach the other side.

If we can identify these sub-goals, we can give “partial” rewards to the sub-goals on the way to the goal. In this case, we identify key states (or more accurately, key transitions) that are like sub-goals of our problem. In Freeway, assuming the reward for reaching the other side is 100 points, we could give 1 point for reaching row 1, then 2 points for reaching row 2, etc., and then $100 - (1 + 2 + \dots + n)$ for reaching the other side, where n is the 2nd-last row.

For an MDP $(S, s_0, A, P, r, \gamma)$, the process is:

1. Identify sub-goals that need to be achieved to achieve the final goal.
2. Create a new reward function r' where the rewards are re-distributed to sub-goals, and where cumulative reward of using the original reward r of any episode e that reaches a real goal is roughly equal to the cumulative reward for e using r' .
3. Solve a new MDP $(S, s_0, A, P, r', \gamma)$ to obtain policy π (the MDP is the same but just with a different reward function).
4. Evaluate π on the original MDP.

For example, consider the single-agent card game *Solitaire*. The aim of the game is to win by having all cards placed in four piles, with various rules for when you can move a card to the pile. Each pile must be in order from ace to king, all of the same suit.

Imagine we are given a simulator for Solitaire and the only reward received for winning the game by arriving in one of the terminal states where the cards are in the correct piles. This reward would only be given in that terminal state, and requires perhaps a few hundred actions to arrive at.

Simulations of a game like this would have to be long because there are no rewards until the end of the game, and we would spend a LOT of random actions before we first won the game. By re-distributed sub-goals that give a reward whenever a correct card is placed on one of the piles, we feed rewards forward much earlier and give information for our reinforcement learning algorithm to exploit.

This idea can be implemented using *reward shaping*. In these notes, we focused on potential-based reward shaping, which gives small “fake” rewards. This effectively asks that you define a heuristic for states that guide the agent towards promising actions early during learning when there is not much information to exploit. By using sub-goals, we can give a shaped reward $\Phi(s)$ to any state that is a sub-goal, and assign $\Phi(s) = 0$ for any state that does not. Thinking about reward shaping as sub-goals can often be conceptually simpler to implement than considering it as an heuristic.

Alternatively, we can just “intercept” the real reward function with our new reward function and not use reward shaping, but reward shaping is a more principled way of doing this and offers convergence guarantees when using Q-tables and linear approximation.

4.6 Partially-observable state variables

Often we have situations where some variables are not observable for at least part of the game. For example, in Solitaire, the draw cards are not visible until we take each one. In this case, the model is a *partially-observable Markov Decision Process* (POMDP). We can translate this to an MDP where the states are not represented as states in the real-world, but are **beliefs** over states in the real world. However, the computational complexity of this is high.

One option is to simplify by **ignoring the partially observable variables**. In Solitaire, it is simple enough to just not track the cards in the deck and only consider actions for the cards that we can already see. In Solitaire, this will work well. If you have ever played the game, you will know that you spend very little time wondering about the cards in the deck.

Of course, this will not always be a good idea: sometimes it is important to keep track of the partially-observable variables; for example, by mapping the probability that certain cards could be played in the future.

4.7 Heuristics

Using heuristics is a good way to speed up learning. There are three places where we can often use heuristics.

State/action pruning. First, if we have a reasonable heuristic, we can choose to “prune” states that are not promising, and explore only those that appear to be promising. For example, consider using *MCTS* for Solitaire. If we are using a model or a simulator, we may be able to peek at the states resulting from actions. If one action results in us being able to move a block of cards to the goal pile, we may prioritise doing simulations using this action. Given we expand the first node fully before expanding any other nodes, we could look at the heuristic value of each node and prune it from the tree if it is too low. This would cut down our search space. Of course, a poor heuristic could do more damage than good! But it is not just MCTS that benefits. For example, using Q-learning or SARSA, we could terminate episodes when we reach unpromising states, either start new episodes or backtracking to an earlier promising node is using a simulator.

Exploiting. Second, heuristics can be good for helping to guide episodes towards good solutions. In any model-free or simulation-based technique, early episodes effectively choose each action with a uniform probability, as there is no good information to exploit. Like reward shaping, we can use heuristics to choose the most promising actions more often, meaning that simulations are still randomised, but informed. In MCTS, when we expand a node and then simulate, this is particularly valuable, as if we can find reasonable simulations, we get a better idea of the value of the expanded node. The trick here is to balance the heuristic against the information that we learn into our Q-function. Either we have a weighted measure, or we stop using the heuristic after we gain sufficient information in our Q-functions.

Shorter episodes and simulations. Finally, we can use heuristics to terminate episodes/simulations early. As we move towards terminal states, our heuristic values are likely to be more accurate, because it is “easier” to estimate how good a state is as we gain more information. As such, we can use heuristics to terminate episodes early. For example, in MCTS, instead of simulating until a terminal state, we can simulate for X number of actions and then return the heuristic value of the state. This is an estimate, but early in the learning process, it can often be more valuable to shorter simulations that return heuristic values rather than much longer ones that take more time and often involve a lot of actions that were not helpful. In fact, this is precisely what temporal different methods like *Q-learning* and *SARSA* do! They use $\max_a Q(s', a')$ and $Q(s', a')$ as heuristic estimates of the value of state s' so that we can assign ‘credit’ to the previously executed action. In those techniques, we are learning the heuristic. Hand-crafting a heuristic will be more helpful in earlier episodes.

4.8 Takeaways

Takeaways

- Techniques for solving MDPs face **scalability** issues.
 - Using modelling tricks, we can find problems that are easier to solve, and apply them back to the original problem.
 - Sometimes the smaller problem is enough to solve our problem; other times, it is not.
-

Part II

Multi-agent Reinforcement Learning

NORMAL FORM GAMES

In the following chapters, we will look at **games**. By “games”, we do not only mean games like Chess or digital games – the term “game” is a more general term to describe a problem that involves **multiple agents or players**.

The standard definition of an MDP is for a single agent, who controls all of the actions. In a multi-agent system, we face further challenges: the effects of actions and the rewards we receive are dependent also on the actions of the agents. In fact, the other agents can even be our adversaries, so they may be working to minimise our rewards.

First, we look at **normal form games**, which are single-shot (non-sequential) games where a group of agents each has to play a move (execute an action) at the same time as all others, but the reward (or **payoff** that they receive is dependent on the moves of the other agents.

Then, we look at **extensive form games**, which are sequential games, meaning that there are multiple actions played in sequence.

Learning outcomes

The learning outcomes of this chapter are:

1. Identify situations in which normal form games are a suitable model of a problem.
 2. Manually calculate the best responses and Nash equilibria for two-player normal form games.
 3. Compare and contrast pure strategies and mixed strategies.
-

5.1 Chapter overview

Normal form games capture many different applications in the field of multi-agent systems. We will just look at the foundation of these, focusing on both deterministic and stochastic strategies for playing the games. We will cover how to solve these analytically.

Exercise – Prisoner’s dilemma

The best known example of a normal form game is known as the *Prisoner’s dilemma*, which is as follows.

Consider the following (hopefully fictional) scenario. You and your partner in crime have been arrested for armed robbery. The police would like to avoid a lengthy trial because they have little evidence that you committed the crime, however, both of you were caught carrying illegal weapons, so the police can use this against you.

You and your partner in crime are placed in separate cells, unable to communicate with each other. The police offer you each the following deal:

- If neither of you admit your guilt, you will be charged with carrying illegal weapons, and receive 1 year each in prison.

- If one of you admits both are guilty, while the other does not, the prisoner that admitted will get off free in exchange for the confession, while the other prisoner will receive 4 years in prison.
- If both of you admit you are guilty, you will each receive 2 years in prison — the length is reduced from 4 years in exchange for your confession.

What do you do: admit or deny?

This problem can be represented by the following two-dimensional matrix:

		Prisoner B	
		admit	deny
Prisoner A	admit	-2, -2	0, -4
	deny	-4, 0	-1, -1

Prisoner A and *Prisoner B* are the **players**, *admit* and *deny* are the **actions**, and the values in the cells are the **utility** or **payoffs** given to the player. For example, if both players choose to admit, both will receive two years in prison, so the utility is -2 for each player. The left cell is Prisoner A and the right cell is Prisoner B.

So, what should each prisoner do? Let's first list some assumptions about the game, which are general assumptions that we are going to hold throughout this chapter:

1. We assume that all agents are **rational**, which means that they aim to maximise their total utility.
2. We assume that all agents are **self-interested**, which means that they do not care about the other agents' utility.
3. We assume that the game is a **perfect information** game, which means that rules of the game are **common knowledge** for all agents; that is, all agents know the rules, including the actions and utilities for all other agents, and all agents know that all agents know the rules, and all agents know that all agents know that all agents know the rules, and all agents know that.... *ad infinitum*. In short, there is no way to "trick" another agent by taking advantage of something that they don't know.

Given these assumptions, both agents should choose to admit, with the result being that both receive two years in prison. This may seem surprising: If both agents choose to deny, they would both receive just one year in prison. Why do these choose to spend more time in prison?

The answer all comes down to the self interest: they are not trying to minimise the amount of time that they both spend in prison. Instead, each of them is trying to minimise the amount of time they spend in prison themselves.

Let's look at this from the perspective of one agent. Your initial intuition may be to think like this: "if I choose to admit, then my opponent could either admit or deny. If they admit, then my utility is -2", etc. However, perhaps a better way to think is to instead reverse this: "if my opponent admits, what should I do?"

If we reason this way, it is clear to see why both players admit. The game is symmetric, so we only need to reason about one player:

1. If my opponent admits, then I should admit, which has utility -2, rather than deny, which has utility -4.
2. If my opponent denies, then I should admit, which has utility 0, rather than deny, which has utility -1.

So, whatever my opponent does, my best action is to admit. Similarly, my opponent can reason in the same way, and their best action is to admit as well. Because both of us reason like this and both of us are self interested, we both end up spending two years in prison instead of one. Interestingly, we cannot use this to out-reason our opponent. If we "know" they will admit, switching to deny is not a good response: we will end up with four years in prison instead of two! This is why it is known as the Prisoner's *dilemma*: both prisoners know there is a better outcome for them both, but neither of them has an incentive to deny.

Now that we have seen an example, let's look at a more formal definition of normal form game.

Definition – Normal form game

A **normal form game** is a tuple $G = (N, A, u)$

- N is a set of n number of players
- $A = A_1 \times \dots \times A_n$ is an **action profile**, where A_i is the set of actions for player i . Thus, an action profile $a = (a_1, \dots, a_n)$ describes the simultaneous moves by all players.
- $u : A \rightarrow \mathbb{R}^N$ is a reward function that returns an N -tuple specifying the payoff each player receives in state S . This is called the **utility** for an action.

Normal game games can be visualised as matrices, as shown above for the prisoner's dilemma, with each agent representing one dimension of the matrix, each row represents an action for a player, and each cell represents the utility received when the players each take the action.

5.2 Solutions for normal form games: strategies

In normal form games, the solution for a player in the game is known as a **strategy**. There are several types of strategy.

Definition – Pure strategy

A **pure strategy** for an agent is when the agent selects a single action and plays it. If the agent were to play the game multiple times, they would choose the same action every time.

Definition – Mixed strategy

A **mixed strategy** is when an agent selects the action to play based on some probability distribution. That is, we choose action a with probability 0.8 and action b with probability 0.2. If the agent were to play the game an infinite number of times, it would select a 80% of the time and b 20% of the time.

Note that a pure strategy is a special case of a mixed strategy where one of the actions has a probability of 1.

We call the set of strategies for an agent a **strategy profile**. The strategy profile for agent i is denoted S_i . Note that $S_i \neq A_i$ because S_i contains mixed strategies. The set of mixed-strategy profiles for all agents is simply $S = S_1 \times \dots \times S_n$.

We use the notation S_{-i} to denote the set of mixed-strategy profiles for all agents except for agent i , and $s_{-i} \in S_{-i}$ to denote an element of this.

It is not immediately obvious why an agent would want to use randomisation when select an action, but we will see examples where this is important.

Definition – Dominant strategy

Strategy s_i for player i **weakly dominates** strategy s'_i if the utility received by the agent for playing strategy s_i is greater than or equal to the utility received by that agent for playing s'_i . Formally, s_i weakly dominates s'_i iff and only if:

$$\text{for all } s_{-i} \in S_{-i}, \text{ we have that } u_i(s_i, s_{-i}) \geq u_i(s'_i, s_{-i})$$

Strategy s_i **strongly dominates** strategy s'_i if its utility is strictly greater. Formally:

$$\text{for all } s_{-i} \in S_{-i}, \text{ we have that } u_i(s_i, s_{-i}) > u_i(s'_i, s_{-i})$$

A strategy is a **weakly (resp. strictly) dominant strategy** if it weakly (resp. strictly) dominates all other strategies.

A dominant strategy is **strategy-proof**: we can even tell our opponents beforehand that this is the strategy that we are going to choose and this would not give them any advantage: it would still be a dominant strategy.

In the Prisoner's dilemma game, the strategy to admit is strictly dominant: is the best action to take for any of the strategies that an opponent can play.

5.3 Best response and Nash equilibria

First, we look at how to solve games from the perspective of one of the agents, known as the agent's **best response**.

Then, we look at solutions at the concept of **equilibria**, which captures solutions to the entire game.

Informally, the concept of a best response refers to the best strategy that an agent could select *if* it knew how all of the other agents in the game were going to play.

Definition – Best response

The **best response** for an agent i if its opponents play strategy profile $s_{-i} \in S_{-i}$ is a mixed strategy $s_i^* \in S_i$ such that $u(s_i^*, s_{-i}) \geq u(s'_i, s_{-i})$ for all strategies $s'_i \in S_i$

Note that for many problems, there can be multiple best responses.

Using this, we can define the Nash equilibrium of a game, which is named after the famous mathematician John Nash, who in his PhD thesis provide that all finite normal form games have a Nash equilibrium. Informally, as Nash equilibrium is a **stable** strategy profile for all agents in N such that no agent has an incentive to change strategy if all other agents kept their strategy the same.

Definition – Nash equilibrium

A strategy profile $s = (s_1, \dots, s_n)$ is a **Nash equilibrium** if for all agents i and for all strategies s_i is a best response to the strategy s_{-i} .

If the strategies in a Nash equilibrium are all pure strategies, then we call this a **pure-strategy Nash equilibrium**. Otherwise, it is a **mixed-strategy Nash equilibrium**.

5.4 Calculating best response and Nash equilibria

The set of best responses for a normal form game can be calculated by searching through all strategies to find those with the highest payoff.

Algorithm 15 (Best response)

Input : Normal form game $G = (N, A, u)$
Input : agent i
Input : and strategy profile s_{-i} for agents other than i
Output : Set of best responses

```

best_response =  $\emptyset$ 
best_response_value =  $-\infty$ 
for each  $s_i \in S_i$ 
    if  $u(s_i, s_{-i}) > \text{best\_response\_value}$  then
        best_response =  $\{s_i\}$ 
    elseif  $u(s_i, s_{-i}) = \text{best\_response\_value}$  then
        best_response = best_response  $\cup \{s_i\}$ 
return best_response

```

This has complexity $O(|S_i|)$: we check each strategy once.

As an example, consider the Prisoner's dilemma. Clearly, the best response for both agents is the unique strategy to admit.

Finding Nash equilibria involves searching through all strategy profiles and finding those in which all agents strategies are a best response in that profile. Here is an algorithm for finding the Nash equilibria for a two-player normal form game:

Algorithm 16 (Nash equilibria)

Input : Normal form game $G = (N, A, u)$
Output : Set of Nash equilibria

```

nash_equilibria =  $\emptyset$ 
for each  $s_1 \in A_1$ 
    for each  $s_2 \in A_2$ 
        if  $s_i \in \text{BestResponse}(i, s_j)$  and  $s_j \in \text{BestResponse}(j, s_i)$  then
            nash_equilibria = nash_equilibria  $\cup \{(s_i, s_j)\}$ 
return nash_equilibria

```

Informally, given a normal form game, we can look at each cell (s_1, s_2) of the game, and search along the rows and columns to determine whether any player can do better by changing their strategy (while all other agents' strategies state the same).

To search for pure strategy equilibria, we just set $S_i \leftarrow A_i$; that is, we search only over pure strategies.

Example – Nash equilibria for Prisoner's dilemma

Prisoner's dilemma has a unique pure-strategy Nash equilibrium: $(\text{admit}, \text{admit})$. In fact, for any game in which all agents have a dominant strategy, that will form a unique Nash equilibrium.

We can verify this by looking at each cell and reasoning as follows:

- At $(\text{admit}, \text{admit})$ the utility is $(-2, -2)$. If either agent switches to *deny*, then that agent will receive -4 , so they have no reason to deviate from *admit*. Therefore, this is a Nash equilibrium.
 - At $(\text{deny}, \text{deny})$ the utility is $(-1, -1)$. If either agent switches to *admit*, their utility will increase to 0 , so they have an incentive to deviate. Therefore, this is NOT a Nash equilibrium.
 - For $(\text{admin}, \text{deny})$ or $(\text{deny}, \text{admit})$, the agent that denies has an incentive to admit to increase its utility from -1 to 0 , therefore, neither is a Nash equilibrium.
-

Example – Nash equilibria for *Split or steal*

Split or steal is a game in which two agents need to decide whether to split a pot of prize money, or try to steal it from the other. If they share, both receive half of the prize money. If one steals and one shares, the stealer receives all of the prize and the other agent receives nothing. If they both steal, both receive nothing. The game matrix for this can be described as follows:

		Agent 2	
		split	steal
Agent 1	split	1, 1	[0, 2]
	steal	[2, 0]	[0, 0]

What are the Nash equilibria. There are in fact three Nash equilibria for this game, highlighted using the square brackets above! Let's reason about them:

- $(steal, steal)$ is a Nash equilibrium for both agents, because if either agent deviates by playing *split*, their utility remains at 0.
- $(steal, split)$ and $(split, steal)$ are both equilibria because if either agent deviates from *steal* their utility decreases from 2 to 1, while if either deviates from *split* their utility remains at 0
- $(split, split)$ is not a Nash equilibrium because both agents have incentive to deviate to *steal*, which would increase their utility from 1 to 2.

5.5 Mixed strategies

Recall from earlier in this chapter where we defined **mixed strategies**, which are strategies that use randomisation. To illustrate why these are necessary, consider the following simple game.

Example – Matching Pennies

In the *Matching Pennies* game, two players each have a penny, and simultaneously they need to show either a *head* or a *tail* of their penny to their opponent. The *Odd* player will win if there is just one head, and the *Even* player will win if there are two heads. We can model this as follows:

		Player Odd	
		heads	tails
Player Even	heads	1, -1	-1, 1
	tails	-1, 1	1, -1

It is clear that neither agent has a dominant strategy and there are no pure-strategy equilibria: in every call, there is an incentive for the agent receiving -1 to deviate from their strategy.

So, what strategy should we play? If we were to play this game a number of times, clearly picking either *heads* or *tails* every time would be a bad strategy: our opponent would learn this and would start picking their strategy to beat us. Instead, we need to **randomise** by choosing *heads* sometimes and *tails* sometimes. Intuitively, we would choose each with probability 0.5; but can we calculate this analytically?

To do this, we need to define the concepts of **expected utility** and **indifference**.

Definition – Expected utility of a pure strategy

Expected utility is the weighted average received by an playing a particular pure strategy. For an action $a_i \in A_i$, the expected utility of that action is:

$$U_i(a_i) = p_1 \times u_i(a_i, a_{-i}^1) + \dots + p_m \times u_i(a_i, a_{-i}^m)$$

where $a_{-i}^1, \dots, a_{-i}^m$ are the action profiles for all agents other than i , and $p_1, \dots, p_m$ are the probabilities of our opponents playing those action profiles.

In theory, we can maximise our overall utility by picking the pure strategy with the highest expected utility. However, in a game, we do not know the probabilities that our opponents will play those moves! This is where indifference comes in.

Definition – Indifference

An agent i is **indifferent** between a set of pure strategies $I \subseteq A_i$ if for all $a_i, a_j \in I$, we have that $U_i(a_i) = U_i(a_j)$.

Informally, this states that an agent is indifferent between a set of pure strategies if the expected return of all strategies is the same. The agent is therefore indifferent between these pure strategies because it does not matter which action they choose.

Definition – Mixed-strategy Nash equilibria

A **mixed-strategy Nash equilibria** is a mixed-strategy profile S such that the strategy for each agent $i \in N$ is a tuple of probabilities $P_i = (p_1, \dots, p_m)$, one for each pure strategy, such that $p_1 + \dots + p_m = 1$ and that all opponents j are indifferent to their pure strategies A_j .

Informally, this states that each agent should choose a mixed strategy such that it makes their opponents indifferent to their own actions. Intuitively, this does not really make much sense: each agents' strategy is to make its opponents indifferent to their own strategy. However, if we analyse it from the perspective of the opponent, it becomes clear: if we select the probabilities for a mixed strategy such that our opponent is *not* indifferent, then this means there is at least one strategy that has a higher expected utility than all others. In that case, the opponent would play that strategy.

Example – Mixed strategies for matching pennies

For the matching pennies game, as agent *Odd*, we want to set the probabilities of *heads* and *tails* such that $U_{Even}(heads) = U_{Even}(tails)$.

Let Y and $1 - Y$ be the probabilities that agent *Odd* plays *heads* and *tails* respectively. We need to solve to Y to make *Even* indifferent to playing *heads* or *tails*:

$$\begin{aligned} U_{Even}(heads) &= U_{Even}(tails) \\ Y \times 1 + -1(1 - Y) &= Y \times -1 + 1(1 - Y) \\ 2Y - 1 &= 1 - 2Y \\ 4Y &= 2 \\ Y &= \frac{1}{2} \end{aligned}$$

Therefore, *Odd* should play *heads* and *tails* with probability $\frac{1}{2}$ each. If we solve for player *Even*, we would find the same answer.

In this example, the probabilities of the game are reasonably clear without having to solve. Now, let's look at a more complicated game, which is an example of a **security game**.

Example – Mixed strategies for the security game

A security game is a model of how to place resources to improve safety and security of key assets. In this example, we look at the issue of where to place resources each hour to defend from an attack at an airport that has two terminals. It is infeasible to always patrol all terminals, but with a randomised strategy, we can make it difficult for an adversary to know which terminal to target.

The difficulty is that the two terminals have different “values” for both the defender and the adversary. Both value terminal 1 more highly (e.g. it is a terminal that has more people on a typical day), but both also value the terminals differently to each other. This can be modelled with the following normal form game:

		Adversary	
		Terminal 1	Terminal 2
Defender	Terminal 1	5, -3	-1, 1
	Terminal 2	-5, 5	2, -1

It is clear that having a uniform strategy is not the best strategy, but we can use indifference to determine what is.

If Y is the probability that the adversary will attack Terminal 1, then the expected utility of the defender's two pure strategies are:

$$U_D(T1) = 5Y + -1(1 - Y) = 6Y - 1$$

$$U_D(T2) = -5Y + 2(1 - Y) = 2 - 7Y$$

If the adversary wants to make the defender indifferent between its two pure strategies, then the utility for the defender's two pure strategies must be the same.

$$\begin{aligned} U_D(T1) &= U_D(T2) \\ 6Y - 1 &= 2 - 7Y \\ 3 &= 13Y \\ Y &= \frac{3}{13} \end{aligned}$$

So, the adversary should target Terminal 1 with probability $\frac{3}{13}$ and Terminal 2 with probability $\frac{10}{13}$.

If X is the probability that the defender will defend Terminal 1, then the expected utility of the adversary's two pure strategies are:

$$U_A(T1) = -3X + 5(1 - X) = 5 - 8X$$

$$U_A(T2) = 1X + -1(1 - X) = 2X - 1$$

If the defender wants to make the adversary indifferent between its two pure strategies, then the utility for the adversary's two pure strategies must be the same:

$$\begin{aligned} U_A(T1) &= U_A(T2) \\ 5 - 8X &= 2X - 1 \\ 6 &= 10X \\ X &= \frac{6}{10} \end{aligned}$$

So, the defender should choose to defend Terminal 1 with the probability $\frac{3}{5}$ and Terminal 2 with $\frac{2}{5}$.

5.6 Takeaways

Takeaways

- **Normal form games** model non-sequential games where agents take actions simultaneously.
 - **Pure strategies** and **mixed strategies** can be used by agents. We analyse which strategies are good and also how these relate to Nash equilibria.
-

EXTENSIVE FORM GAMES

Learning outcomes

The learning outcomes of this chapter are:

1. Define ‘extensive form game’
 2. Identify situations in which extensive form games are a suitable model of a problem.
 3. Define the types of strategy for an extensive form game
 4. Manually apply backward induction to small extensive form games.
 5. Design and implement backward induction to solve medium-scale extensive form games automatically.
-

6.1 Overview

In this section, we look at **extensive form games**. An extensive form game is a sequential game, which includes a set of players, rules around which players can move when, and what they observe and the rewards they receive when they move. In an extensive form game, there are multiple players who can take moves in the game, but not simultaneously. At the end of the game, each player receives a reward, known as a **payoff**, which can be positive or negative.

We look at three main ways to solve extensive form games:

1. In a model-based game, we can use **backward induction**, which is where we calculate an equilibrium for every subgame of the game, and these to decide our moves.
2. In a model-free game, we can use **model-free reinforcement learning**. These are very similar to techniques such as Q-learning or policy gradient methods, except that there are other players that can effect our rewards, rather than just ourselves and the environment.
3. If we have a simulation, we can use model-free techniques or **multi-agent Monte-Carlo tree search (MCTS)**, which is similar to single-agent MCTS, except again there are other players that can effect our rewards, rather than just ourselves and the environment.

In these notes, we will look only at **perfect information** extensive form games, which means that the game state is **fully observable to all players**.

6.2 Perfect information extensive form games

Definition – Perfect information extensive form game

A perfect information extensive form game is a tuple $G = (N, S, s_0, A, T, r)$

- N is a set of n **number of players**
- S is a set of **states** (or **node**)
- s_0 is the **initial state**
- $A : S \rightarrow 2^A$ is a function that specifies the allowed actions from each state $s \in S$
- $P : S \rightarrow N$ is a function that specifies which player chooses the action at a node (whose turn it is)
- $T : S \times A \rightarrow S$ is a **transition function** that specifies the successor state for choosing an action in a state
- $r : S \rightarrow \mathbb{R}^N$ is a **reward function** that returns an N -tuple specifying the payoff each player receives in state S . In some texts, this may be called u instead of r , to represent **utility**, but we will use r in these notes.

An extensive form game forms a tree, due to its sequential nature.

Example – The sharing game

Consider the following simple game called the *Sharing Game*, from *Essentials in Game Theory* by Leyton-Brown and Shoham:

“Imagine a brother and sister following the following protocol for sharing two indivisible and identical presents from their parents. First the brother suggests a split, which can be one of three—he keeps both, she keeps both, or they each keep one. Then the sister chooses whether to accept or reject the split. If she accepts they each get their allocated present(s), and otherwise neither gets any gift.”

We assume that the brother and sister both value the presents equally, we can represent this as a tree, so the rewards vectors are (2,0), (1,1), and (0,2) where the first element is the brother’s reward and the second element is the sisters reward.

We can implement such a game in Python. First, we have an interface that defined what an extensive form game is:

```
class ExtensiveFormGame:

    ''' Get the list of players for this game as a list [1, ..., N] '''
    def get_players(self): abstract

    ''' Get the valid actions at a state '''
    def get_actions(self, state): abstract

    ''' Return the state resulting from playing an action in a state '''
    def get_transition(self, state, action): abstract

    ''' Return the reward for a state, return as a dictionary mapping players to
    rewards '''
    def get_reward(self, state, action, next_state): abstract

    ''' Return true if and only if state is a terminal state of this game '''
    def is_terminal(self, state): abstract
```

(continues on next page)

(continued from previous page)

```

''' Return the player who selects the action at this state (whose turn it is) '''
def get_player_turn(self, state): abstract

''' Return the initial state of this game '''
def get_initial_state(self): abstract

''' Return a game tree for this game '''
def game_tree(self):
    return self.state_to_node(self.get_initial_state())

def state_to_node(self, state):
    if self.is_terminal(state):
        node = GameNode(state, None, self.get_reward(state))
        return node

    player = self.get_player_turn(state)
    children = dict()
    for action in self.get_actions(state):
        next_state = self.get_transition(state, action)
        child = self.state_to_node(next_state)
        children[action] = child
    node = GameNode(state, player, None, children = children)
    return node

class GameNode:

    # record a unique node id to distinguish duplicated states
    next_node_id = 0

    def __init__(self, state, player_turn, value, is_best_action = False, children = dict()):
        self.state = state
        self.player_turn = player_turn
        self.value = value
        self.is_best_action = is_best_action
        self.children = children

        self.id = GameNode.next_node_id
        GameNode.next_node_id += 1

```

Then, we need to implement this interface to create an extensive form game:

```

BROTHER = "B"
SISTER = "S"

from extensive_form_game import ExtensiveFormGame

class SharingGame(ExtensiveFormGame):

    ''' Get the list of players for this game as a list [1, ..., N] '''
    def get_players(self):
        return [BROTHER, SISTER]

    ''' Get the valid actions at a state '''
    def get_actions(self, state):
        actions = dict()

```

(continues on next page)

(continued from previous page)

```

actions[1] = ["2-0", "1-1", "0-2"]
actions[2] = ["yes", "no"]
actions[3] = ["yes", "no"]
actions[4] = ["yes", "no"]

if state in actions.keys():
    return actions[state]
else:
    return []

''' Return the state resulting from playing an action in a state '''
def get_transition(self, state, action):
    transitions = dict()
    if state == 1:
        transitions["2-0"] = 2
        transitions["1-1"] = 3
        transitions["0-2"] = 4
        return transitions[action]
    elif state > 1 and state <= 4:
        transitions["no"] = state * 2 + 1
        transitions["yes"] = state * 2 + 2
        return transitions[action]
    else:
        transitions[action] = []
    return transitions[action]

''' Return the reward for a state, return as a dictionary mapping players to
→ rewards '''
def get_reward(self, state):
    rewards = dict()
    if state > 4:
        rewards[5] = {BROTHER:0, SISTER:0}
        rewards[6] = {BROTHER:2, SISTER:0}
        rewards[7] = {BROTHER:0, SISTER:0}
        rewards[8] = {BROTHER:1, SISTER:1}
        rewards[9] = {BROTHER:0, SISTER:0}
        rewards[10] = {BROTHER:0, SISTER:2}
        return rewards[state]
    else:
        return {BROTHER:0, SISTER:0}

''' Return true if and only if state is a terminal state of this game '''
def is_terminal(self, state):
    return state > 4

''' Return the player who selects the action at this state (whose turn it is) '''
def get_player_turn(self, state):
    if state == 1:
        return BROTHER
    else:
        return SISTER

''' Return the initial state of this game '''
def get_initial_state(self):
    return 1

```

(continues on next page)

(continued from previous page)

```
def to_string(self, state):
    return str(state)
```

We can visualise extensive form games as **game trees**, where the edges are actions, and the nodes are states. The root node is the initial state. Consider the following extensive form game tree for the sharing game. The labels next to nodes indicate whose turn it is (“B” for brother and “S” for sister), and the tuples in nodes are the payoffs at that node (blank if there are no payoffs):

```
from sharing_game import SharingGame
sharing = SharingGame()
from graph_visualisation import GraphVisualisation

gv = GraphVisualisation(max_level = 5)
graph = gv.node_to_graph(sharing, sharing.game_tree())
graph
```

```
<graphviz.graphs.Graph at 0x7fbd3d47c610>
```

6.3 Solutions for extensive form games

Like normal form games, extensive form games have strategies, however, the strategies must tell each agent what to do every time it is their turn to choose the action.

Definition – Pure strategy

A **pure strategy** for an extensive form game G , the pure strategies for a player i is the Cartesian product $\prod_{s \in S, P(s)=i} A(s)$.

Therefore, a pure strategy for player i tells them what move to take in each state where it is their turn.

An optimal solution for an extensive form game is called the **subgame-perfect equilibrium** for that game. Before we define what subgame-perfect equilibria are, we first need to define what sub-games are..

Definition – Sub-game

Given an extensive form game G , the **sub-game** of G rooted at the node $s_g \in S$ is the game $G_{s_g} = (N, S, s_g, A, T, r)$; that is, the part of the game tree in which s_g is the root node and its descendants are the same as its descendants in G .

For any state $s_g \in S$ that occurs in the game tree, we can define a sub-game by taking the descendants of that tree. Therefore, we can define a game as simply the root node s_0 , with the actions in $A(s_0)$ lead to nodes that are the root nodes of its sub-game.

Definition – Subgame-perfect equilibria

The **subgame-perfect equilibria** (SPE) of a game G consists of all strategy profiles for the N agents such that for any subgame G_{s_g} of G , the strategy for player $P(s_g)$ is the best response for that player at s_g .

Therefore, a sub-game perfect equilibria is the best response for every agent in the game when it is their turn.

As an example, consider the sharing game. In this case, the same game on the left and middle have two equilibria because the payoffs for each of the moves are zero for the sister. In such a case, ties can be broken randomly or we can take all

best responses. We can represent the subgame-perfect equilibria for the sharing game as follows, where the thicker arcs specify the move that the player should make (where the ties are broken randomly):

```
from sharing_game import SharingGame
from backward_induction import BackwardInduction
sharing = SharingGame()
backward_induction = BackwardInduction(sharing)
solution = backward_induction.backward_induction(sharing.get_initial_state())

from graph_visualisation import GraphVisualisation
gv = GraphVisualisation()
graph = gv.node_to_graph(sharing, solution)
graph
```

```
<graphviz.graphs.Graph at 0x7fbd3d5b84f0>
```

Note then that breaking ties randomly leads to a strange outcome where the brother plays 2 — 0 (gives nothing to his sister), where he could possibly receive a payoff of 0 if his sister actually chooses *no*. This is the disadvantage of breaking ties randomly.

6.4 Backward induction

Learning outcomes

The learning outcomes of this chapter are:

1. Manually apply backward induction to solve small-scale extensive form games.
 2. Design and implement a backward induction algorithm to solve medium-scale extensive form games automatically.
-

6.4.1 Overview

Backward induction is a model-based technique for solving extensive form games. It solves this by recursively calculating the sub-game equilibrium for each sub-game and then using this to solve the parent node of each subgame. Because it solves subgames first, it is effectively solving the game backwards.

The intuition is as follows: starting at the terminal nodes of the tree (those where $A(s)$ is empty), for the parent, calculate the best move for the agent whose turn it is. This gives us the sub-game equilibrium for the smallest sub-games in the game. As the solutions are reward tuples themselves, we can solve the parent of the parents by using the parent solution as reward for the sub-game, which gives us the sub-game equilibrium for the game that starts at the parents of the parents of the terminal nodes. We progressively induct these values backward up the tree until we reach the start node.

In the pure backward induction that we cover here, the assumption is that only terminal nodes have rewards. If we want to model any situations in which non-terminal nodes have rewards, we simply sum all rewards along the path to the terminal node. Therefore, this solution generalises to the definition of extensive form games in the previous section.

6.4.2 Algorithm

In the following algorithm, *best_child* is an N-tuple that is used to find the best child value for whose turn it is; while *best_child*(*P*(*s*)) and *child_reward*(*P*(*s*)) return the value from the *best_child* and *child_reward* tuple for the player who is choosing the action from state *s*.

Algorithm 17 (Backward induction)

Input : Extensive form game $G = (N, \text{Agt}, S, s_0, A, T, r)$

Output : Sub-game equilibrium for each state $s \in S$

return *BackwardInduction*(s_0)

function *BackwardInduction*($s \in S$)

if $A(s) = \emptyset$ **then**

return $r(s)$

$\text{best_child} \leftarrow (-\infty, \dots, -\infty)$

for each $a \in A(s)$

$s' \leftarrow T(s, a)$

$\text{child_reward} \leftarrow \text{BackwardInduction}(s')$

if $\text{child_reward}(P(s)) > \text{best_child}(P(s))$ **then**

$\text{best_child} \leftarrow \text{child_reward}$

return best_child

So, the solution above is a recursive algorithm that returns the reward tuple for a terminal node, and otherwise finds the best reward tuple for the children of the node. However, “best” is relative to the player whose turn it is. A rational player will choose the outcome that is best for them when it is their turn. The final output of the algorithm is the reward at the terminal state for whose turn it is, which is the sub-game perfect equilibrium for the entire game.

The algorithm can be modified in a straightforward manner to return the paths and strategies for each player by collecting this information at each point.

6.4.3 Implementation

The implementation for this is straightforward from the algorithm above:

```
from extensive_form_game import GameNode

class BackwardInduction:
    def __init__(self, game, do_cache = False):
        self.game = game
        self.do_cache = do_cache
        self.cache = dict()

    def backward_induction(self, state):

        if self.game.is_terminal(state):
            node = GameNode(state, None, self.game.get_reward(state))
            return node

        best_child = None
        best_action = None
        player = self.game.get_player_turn(state)
        children = dict()
        for action in self.game.get_actions(state):
```

(continues on next page)

(continued from previous page)

```

        next_state = self.game.get_transition(state, action)
        child = self.backward_induction(next_state)
        if best_child is None or child.value[player] > best_child.value[player]:
            if best_child is not None:
                best_child.is_best_action = False
            child.is_best_action = True
            best_child = child
        children[action] = child
        node = GameNode(state, player, best_child.value, children = children)
        return node

    def backward_induction_with_cache(self, state):

        state_key = self.game.to_string(state)
        if self.do_cache and state_key in self.cache.keys():
            return self.cache[state_key]

        if self.game.is_terminal(state):
            node = GameNode(state, None, self.game.get_reward(state))
            if self.do_cache:
                self.cache[state_key] = node
            return node

        best_child = None
        best_action = None
        player = self.game.get_player_turn(state)
        children = dict()
        for action in self.game.get_actions(state):
            next_state = self.game.get_transition(state, action)
            child = self.backward_induction(next_state)
            if best_child is None or child.value[player] > best_child.value[player]:
                if best_child is not None:
                    best_child.is_best_action = False
                child.is_best_action = True
                best_child = child
            children[action] = child
        node = GameNode(state, player, best_child.value, children = children)
        if self.do_cache:
            self.cache[state_key] = node
        return node

```

Instead of simply returning the best value from the game, we construct an entire strategy profile for all players using the GameNode objects. The result is a game tree with nodes annotated by their value that is induced up the tree.

Consider the following example, which is just an abstract game with two players:

```

from extensive_form_game import ExtensiveFormGame

ONE = "1"
TWO = "2"

class AbstractExtensiveFormGame(ExtensiveFormGame):

    ''' Get the list of players for this game as a list [1, ..., N] '''
    def get_players(self):
        return [ONE, TWO]

```

(continues on next page)

(continued from previous page)

```

''' Get the valid actions at a state '''
def get_actions(self, state):
    actions = dict()
    actions[1] = ["A", "B"]
    actions[2] = ["C", "D"]
    actions[3] = ["E", "F"]
    actions[7] = ["G", "H"]

    if state in actions.keys():
        return actions[state]
    else:
        return []

''' Return the state resulting from playing an action in a state '''
def get_transition(self, state, action):
    transitions = dict()
    if state == 1:
        transitions["A"] = 2
        transitions["B"] = 3
    elif state == 2:
        transitions["C"] = 4
        transitions["D"] = 5
    elif state == 3:
        transitions["E"] = 6
        transitions["F"] = 7
    elif state == 7:
        transitions["G"] = 8
        transitions["H"] = 9
    else:
        transitions[action] = []
    return transitions[action]

''' Return the reward for a state, return as a dictionary mapping players to
rewards '''
def get_reward(self, state):
    rewards = dict()
    if state in [4,5,6,8,9]:
        rewards[4] = {ONE:3, TWO: 8}
        rewards[5] = {ONE:8, TWO: 3}
        rewards[6] = {ONE:5, TWO: 5}
        rewards[8] = {ONE:2, TWO: 10}
        rewards[9] = {ONE:1, TWO: 0}
        return rewards[state]
    else:
        return {ONE:0, TWO:0}

''' Return true if and only if state is a terminal state of this game '''
def is_terminal(self, state):
    return state in [4,5,6,8,9]

''' Return the player who selects the action at this state (whose turn it is) '''
def get_player_turn(self, state):
    if state in [1,7]:
        return ONE
    else:

```

(continues on next page)

(continued from previous page)

```

        return TWO

    ''' Return the initial state of this game '''
    def get_initial_state(self):
        return 1

    def to_string(self, state):
        return str(state)

```

We can solve this with the following code:

```

from extensive_form_game import ExtensiveFormGame
from backward_induction import BackwardInduction
from graph_visualisation import GraphVisualisation

game = AbstractExtensiveFormGame()
backward_induction = BackwardInduction(game)
solution = backward_induction.backward_induction(game.get_initial_state())

gv = GraphVisualisation(max_level = 5)
graph = gv.node_to_graph(game, game.game_tree(), print_value = False)
graph

```

```
<graphviz.graphs.Graph at 0x7f37b29c09d0>
```

We can see the subgame perfect-equilibria in the bottom-left subgame is (3,8) because player 2 will choose C rather than D, preferring a payoff of 8 more than 3. This value is propagated to the parent node. Subsequently, this becomes the value of the entire game as player 1 will choose A over B, preferring a payoff of 3 rather than 2 in the other sub-game.

6.4.4 Tictactoe

For a slightly larger game (which is still small by standards of games), let's look at Tictactoe. The game itself is too large to display as a game tree, but let's just look at two parts. First, we show just the root node and its children. The equilibria of the sub-games show that even going first, there is no way to ensure you win:

```

from tictactoe import TicTacToe

tictactoe = TicTacToe()
backward_induction = BackwardInduction(tictactoe)
solution = backward_induction.backward_induction(tictactoe.get_initial_state())
gv = GraphVisualisation(max_level = 1)
tictactoe_subgraph = gv.node_to_graph(tictactoe, solution, print_state = True, print_
    value = True)
tictactoe_subgraph

```

```
<graphviz.graphs.Graph at 0x7f37b29c07c0>
```

Next, we show that from the state where the top row of the game is x-o-o, the second is e-e-x (where e is 'empty'), and the third row is empty, playing in the middle cell will guarantee a win for 'x' to win regardless what player 'o' does:

```

from tictactoe import TicTacToe

```

(continues on next page)

(continued from previous page)

```

tictactoe = TicTacToe()
backward_induction = BackwardInduction(tictactoe)
state = [['x', 'o', 'o'],
         [' ', ' ', 'x'],
         [' ', ' ', ' ']]

next_state = tictactoe.get_transition(state, (1, 1))
solution = backward_induction.backward_induction(next_state)
gv = GraphVisualisation(max_level = 100)
tictactoe_subgraph = gv.node_to_graph(tictactoe, solution, print_state = True, print_
↪value = True)
tictactoe_subgraph

```

```
<graphviz.graphs.Graph at 0x7f37b2946dd0>
```

As a result, the equilibrium of this sub-game is (1, -1). No matter which move player ‘o’ takes, they cannot draw or win if player ‘x’ follows the strategy highlighted.

6.5 Multi-agent reinforcement learning

The field of multi-agent reinforcement learning has become quite vast, and there are several algorithms for solving them. We are just going to look at how we can extend the lessons learnt in the first part of these notes to work for **stochastic games**, which are generalisations of extensive form games.

Learning outcomes

The learning outcomes for this chapter are:

1. Define ‘stochastic game’.
2. Explain the difference between single-agent reinforcement learning and multi-agent reinforcement learning.

6.5.1 Overview

Recall the idea of *ExpectiMax trees* which we first encountered in the section on *Monte-Carlo tree search*. ExpectiMax trees are representations of MDPs. Recall that the white nodes are states and the black nodes are what we consider choices by the environment:

An extensive form game tree can be thought of as a slight modification to an ExpectiMax tree, except the choice at the black node is no longer left up to the ‘environment’, but is made by another agent instead:

From this visualisation, we can see that an extensive form game can be solved with model-free reinforcement learning techniques and Monte-Carlo tree search techniques: we can just treat our opponent as the environment; albeit one that has a particular goal itself.

In this section, we just give the intuition of how reinforcement learning techniques can be applied in multi-agent settings like this. Extensions that are specific to multi-agent techniques also exist.

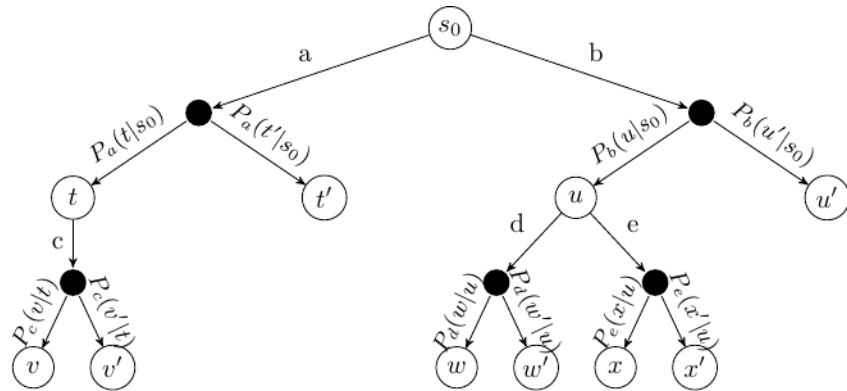


Fig. 6.1: Abstract example of an ExpectiMax Tree

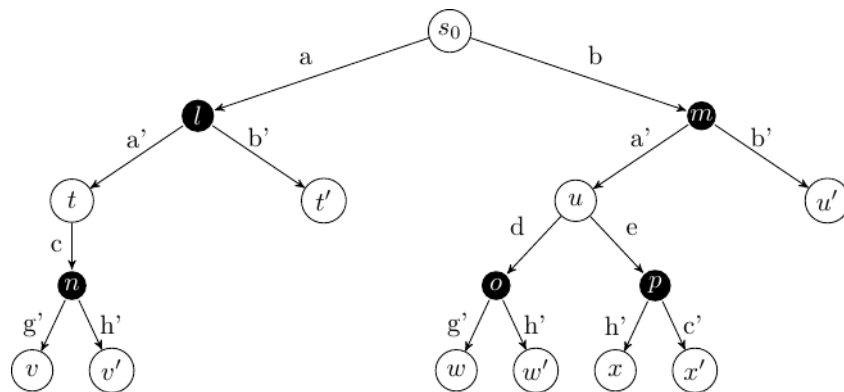


Fig. 6.2: Extensive form game tree as a reinforcement learning problem

6.5.2 Stochastic games

Definition – Stochastic game

A **stochastic game** is a tuple $G = (S, s_0, A^1, \dots, A^n, r^1, \dots, r^n, \text{Agt}, P, \gamma)$ where:

- S is a set of states
- s_0 is the initial state
- A^j is the set of actions for agent j ,
- $P : S \times A^1 \times \dots \times A^n \rightarrow \Omega(S)$ is a transition function that defines the probability of arriving in a state given a starting state and the actions chosen by all players
- $r^j : S \times A^1 \times \dots \times A^n \times S \rightarrow \mathbb{R}$ is the reward function for agent j
- γ is the discount factor

From this definition we can see two things:

1. A stochastic game is simply a version of an MDP in which the actions space and reward space are tuples: one action and one reward for each agent. It is, in fact, a generalisation of MDPs because we can just set $n = 1$ and we have a standard MDP.
2. A stochastic game is a generalisation of an extensive form game. That is, we can express any extensive form game as a stochastic game; but not vice-versa because, for example, actions in extensive form games are deterministic and multiple agents cannot execute actions simultaneously. However, we can simulate an extensive form game by restricting all but one player to playing just a “null” action that has no effect.

6.5.3 Solutions for stochastic games

In a stochastic game, the solution is a set of policies: one policy π^j for each agent j . The **joint policy** of all agents is simply

$$\pi = [\pi^1, \dots, \pi^n]$$

The objective for an agent is to maximise its own expected discounted accumulated reward:

$$V^{\pi^j}(s) = E_{\pi^j} \left[\sum_i \gamma^i r^j(s_i, a, s_{i+1}) \mid s_0 = s, a = \pi(s_i) \right]$$

Note that $a = \pi(s_i)$ is the joint action of all agents. So, each agent’s objective is to maximise its own expected reward considering the possible actions of all other agents.

6.5.4 Multi-agent Q-learning

Given the similarities between MDPs and stochastic games, we can re-cast our standard Q-learning algorithm as a multi-agent version.

Algorithm 18 (Multi-agent Q-learning)

Input : Stochastic game $G = (S, s_0, A^1, \dots, A^n, r^1, \dots, r^n, Agt, P, \gamma)$

Output : Q-function Q^j where j is the *self* agent

Initialise Q^j arbitrarily; e.g., $Q^j(s, a) = 0$ for all states s and joint actions a

repeat

$s \leftarrow$ the first state in episode e

repeat (for each step in episode e)

 Select action a^j to apply in s ;

 e.g. using Q^j and a multi-armed bandit algorithm such as ϵ - greedy

 Execute action a^j in states

 Observe reward r^j and new state s'

$Q^j(s, a) \leftarrow Q^j(s, a) + \alpha \cdot [r^j + \gamma \cdot \max_{a'} Q^j(s', a') - Q^j(s, a)]$

$s \leftarrow s'$

until the end of episode e (a terminal state)

until Q converges

So, this is just the same as standard Q-learning. This will learn well because the model of MDPs is general enough that the opponents' actions can just be approximated as the environment.

However, if we want to solve for just an extensive form game, where players take turns, we can modify the above algorithm slightly to make the opponents' moves more explicit by modifying the execution and update parts of the algorithm:

 Execute action a^j in state s

 Wait for other agents' actions (via simulation or via play)

 Observe rewards $r_t^j, \dots, r_{t+n}^j$ and new state s_{t+n}

$Q^j(s, a) \leftarrow Q^j(s, a) + \alpha \cdot [r_t^j + \dots + r_{t+n}^j + \gamma \cdot \max_{a'} Q^j(s_{t+n}, a') - Q^j(s, a)]$

This algorithm differs from the first in two main ways:

1. After execution our action, we do not observe the next state immediately, but instead we wait until it is our turn again, and observe state s_{t+n} , where $n - 1$ moves have been made by other agents.
2. We observe **all rewards** that we have received over that sequence, including from other agents.

These two algorithms are equivalent for an extensive form game, however, rather than simulating “null” actions for an agent when it is not their turn, it is often simpler and less computationally expensive to instead use the latter.

These algorithms work well as the learning is independent of all of the other agents. This is known as **independent learning**. However, it assumes that all the other agents' policies are **stationary** — that is, it assumes that the other agents' policies do not change over time. This is a limitation, but there are several techniques that do not make this assumption, which are out of scope for these notes. However, we can discuss ways to model our opponent for improving multi-agent Q-learning.

Of course, it is not just Q-learning that can be extended to the multi-agent case. Algorithms like value iteration, SARSA and policy-based methods can similarly be extended. In the simplest case, they require small changes as we have done above; however, more sophisticated versions that are tailored specifically to handle multiple agents will give better results.

Opponent moves

The main question we need to answer is: how can we “decide” our opponents moves if we are learning offline?

In a truly model-free problem, we cannot simulate our opponents, however, our opponent will be executing their own moves, so we don't need to.

However, if we are learning in a simulated environment without real opponents, we will need to simulate their moves ourselves, rather than just “waiting” for their actions. How should we choose actions for opponents' moves? There are a few ways to do this:

1. **Random selection:** Select a random action. This is easy, but it means that we may end up exploring a lot of actions that will never be taken by a good opponent and therefore we will learn poor Q-values for our actions.
2. **Using a fixed policy:** We can use an existing **stochastic** policy that gives reasonable behaviour of the opponent. This could be hand-coded or learnt from a similar game.
3. **Self play:** We can simultaneously learn a policy for both ourselves and our opponents, and we choose actions for our opponents based on the learnt policies. If our actions spaces are the same, such as in games like Chess and Tictactoe, we can learn a single policy and have both ourselves and our opponents use it. This is the technique used by AlphaZero.

6.5.5 Multi-agent Monte-Carlo tree search

Just as we extend Q-learning to a multi-agent case, we can extend *Monte-Carlo tree search* to the multi-agent case with just a small change in our algorithms.

Multi-agent MCTS is similar to single-agent MCTS. We simply modify the basic MCTS algorithm as follows:

1. **Selection:** For ‘our’ moves, we run selection as before, however, we also need to select models for our opponents. In multi-agent MCTS, an easy way to do this is via self-play. Each node has a player whose turn it is, and we use a multi-armed bandit algorithm to choose an action for that player.
2. **Expansion:** Instead of expanding a child node based on $P_a(s' | s)$, we expand one of our actions, which leads to our opponents’ node. In the next iteration, that node becomes a node of expansion.
3. **Simulate:** We then simulate as before, and we learn the rewards when we receive them. Recall that the rewards are a vector of rewards: one for each player.
4. **Backpropagate:** The backpropagation step is the same as before, except that we need to keep the value of the node for every player, not just ourselves.

6.5.6 Takeaways

Takeaways

- For solving extensive form games in a model-free or simulated environment, we can extend techniques like Q-learning, SARSA, and MCTS from single-agent to multi-agent environments in a straightforward way.
 - Other more sophisticated techniques exist for true stochastic games (with simultaneous moves), such as **mean field Q-learning**, which reduces the number of possible interactions between actions by approximating joint actions between all pairs of actions, rather than all global interactions among the agents.
-

MODELLING AND ABSTRACTION FOR MULTI-AGENT GAMES

Learning outcomes

The learning outcomes of this chapter are:

1. Describe modelling and abstraction strategies to scale algorithms for multi-agent games.
 2. Apply modelling and abstraction strategies to non-trivial multi-agent games.
-

7.1 Overview

Just as we can use *modelling and abstraction to help us scale algorithms for MDP problems*, we can use it for multi-agent games. All of the strategies described in the *MDP modelling and abstraction section* can also be applied to multi-agent games. However, when we consider opponents, there are a few other modelling and abstraction strategies we can use.

7.2 Multi-agent to single-agent

A simple and sometimes effective way is to simply **ignore our opponent**, and only search or simulate over our own actions. This may sound like a bad strategy, and in games like Chess and Go where players actively take each others' pieces or surround each others' pieces, this will not be a good abstraction. However, in other games, it can be very useful. Given that we cannot control our opponents' actions directly, in many multi-agent games, it is more important to focus on what we can control: our own actions.

Consider multi-player board games like Cluedo or Scrabble. While our opponents' actions actively influence what we can do and the rewards we receive, quite often most of our strategy is formed by just focusing on our own pieces and what we want to do, because we have very little ability to influence each others' strategy. For example, in Scrabble, we do not spend much time considering what possible tiles our opponents could put down, even though it is a competitive, multi-agent game, and what they do can influence our future actions.

By ignoring our opponents actions and simply solving the problem as a single-player game, we can search/simulate much deeper and broader to gain information about our own actions.

7.3 Limited reasoning of opponent actions

Similar to ignoring the opponent is to limit our own reasoning about our opponents actions.

Focus on known feasible opponent actions. Quite often in multi-agent games, we may have very limited knowledge about what our opponent can do; for example, they have some private information such as cards. So, it is often infeasible to consider all of their possible actions. Instead, we may be better focusing only on the moves that we know our opponent can make, as these are more likely to be made because we know they can be made. Therefore, it is more valuable to spend time analysing these.

Focus on opponents' most promising actions only. Another way is to use *heuristic techniques* over our opponents' moves. Sometimes we may not need to consider ALL possible actions our opponents can make in a state, but just to consider a small handful to get an idea of the value of the state. Of most use will be those that the opponents are more likely to make. We could calculate how likely they are using an heuristic or even a hand-coded opponent. In the former, this is the same as *state/action pruning for single-agent problems*, except that we are pruning our opponents moves. Often, we can prune more aggressively for our opponents because we cannot control their actions. For the latter, we can prune all but the most promising actions. If we have a reasonable hand-coded agent but we want to learn to play better, we can use the hand-coded opponent to give us its best action for our opponent, rather than exploring multiple options. These techniques again allow us to search deeper and broader to gain information about our own actions.

7.4 Takeaways

Takeaways

- The addition of other agents often create further **scalability** issues in stochastic games.
 - Using modelling tricks, we can find problems that are easier to solve, and apply them back to the original problem.
 - Ignoring or approximating opponents can be good in some stochastic games, but not always.
-

Part III

Appendix

INTRODUCTION TO BASIC PROBABILITY THEORY

This chapter covers basic probability theory, with enough understanding for the main chapters of this book.

8.1 Overview

Probability theory is concerned with *chance*. Whenever there is an event or an activity where the outcome is uncertain, then probabilities are involved. We can think of any activity or event with an uncertain outcome as an *experiment* whose outcome we observe. Probabilities are then measures of the likelihood of any of the possible outcomes of an event.

An *experiment* represents an activity whose output is subject to chance (or variation). The output of the experiment is referred to as the *outcome* of the experiment. The set of all possible outcomes is called the *sample space*.

8.2 Some prerequisite set theory

To understand probability theory, we first want an understanding of the three main operation in set theory.

Definition — Set

A **set** is a collection of elements, such as numbers, points in space, shapes, variables, or other sets. A set does not contain duplicate elements.

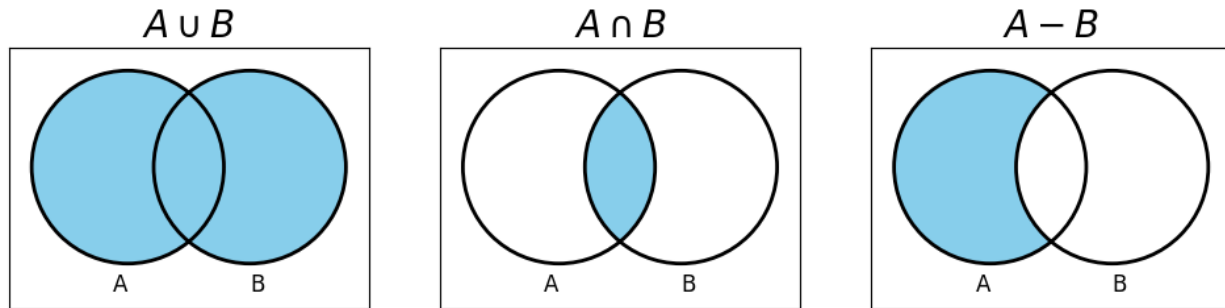
The set with no elements is a special set called the **empty set**, denoted $\emptyset$.

Definition — Set union, intersection, and subtraction

The **set union** $A \cup B$ of two sets A and B is the set of all elements in that are in either set A or in set B . Formally: $A \cup B = \{a \mid a \in A \vee a \in B\}$.

The **set intersection** $A \cap B$ of two sets A and B is the set of all elements in that are in both set A and set B . Formally: $A \cap B = \{a \mid a \in A \wedge a \in B\}$.

The **set subtraction** $A - B$ of two sets A and B is the set of all elements in that are in set A but not in set B . Formally: $A - B = \{a \mid a \in A \wedge a \notin B\}$.



Like any set, $A \cup B$, $A \cap B$, and $A - B$ do not contain duplicate elements. If an element a is in A and in B , it is represented just once in $A \cup B$, in $A \cap B$, and in $A - B$.

8.3 Sample spaces and outcomes

Definition – Sample space and outcomes

The *sample space* Ω is the set of all possible outcomes that might be observed for an experiment.

A set of outcomes $A \subseteq \Omega$ is called an *event*. An event is said to have *occurred* if any one of its elements is the outcome observed in an experiment.

Examples

- An experiment involving the flipping of a coin has two possible outcomes: heads or tails. The sample space is thus $S = \{heads, tails\}$.
 - An experiment involving testing a software system and counting the number of failures experienced after $T = 1$ hour has many possible outcomes: we may experience no failures, 1 failure, 2 failures, 3 failures, The sample space is $S = \{0, 1, 2, 3, \dots\}$ (the set of positive integers).
 - An experiment involving connecting to a web server may have several outcomes, such as successfully connecting, receiving a 404 error, etc. The sample space is the set of possible return values from the web sever.
-

Definition – Independence

Two events are said to be **independent** if the occurrence of one event does not depend on the other and vice versa. For example, if we have two dice, the probability of one falling on the outcome 6 is independent of the outcome of the other dice. However, if we throw both dice, but one falls on the floor out of sight, while the other shows a 3, then the probability of the sum of the two dice equaling 7 is dependent on the probability of the second dice.

8.4 Probability

Probabilities are usually assigned to events.

Definition – Probability

The probability $P(A)$ of an event $A \subseteq \Omega$ is a non-negative real number that relates to the number of times we observe an outcome in A . This can be defined as the fraction of times that we observe an outcome in A over the total number of possible outcomes:

$$P\{A\} = \lim \frac{m}{n}$$

where m is the number of outcomes in A and n is the total number of possible outcomes, that is, the number of elements in S .

We say that $P(A)$ is the *probability measure*, and this measures how likely it is that the actual outcome of the experiment will be a member of the set A .

The three axioms of probability theory

Probabilities must satisfy certain *axioms* to be meaningful measures of likelihood. The following probability laws hold for any event A and any state space Ω :

1. $0 \leq P(A) \leq 1$;
2. $P(\Omega) = 1$; and
3. $P(A \cup B) = P(A) + P(B)$ for disjoint events A and B (that is, $A \cap B = \emptyset$).

That is: (1) the probability of an event occurring is between 0 and 1 inclusive; (2) the probability of the event being part of the sample space is 1, so no events outside the sample space are possible; and (3) the probability of the event $A \cup B$ is the sum of the probabilities of A and of B , assuming that A and B are disjoint.

There are some important consequences of these three axioms:

1. $P(A) = 1 - P(\Omega - A)$.
 2. $P(\emptyset) = 0$, where $\emptyset$ is the empty set.
 3. If $A \subseteq B$ then $P(A) \leq P(B)$.
 4. $P(A \cup B) = P(A) + P(B) - P(A \cap B)$, even if A and B are not disjoint.
-

Definition – Conditional probability

Given two events A and B , if $P(B)$ then the **conditional probability** of A given B is defined as:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

This says that the probability of A occurring, given that we have observed B , is the probability of observing both A and B , divided by the probability of observing event B .

Conditional probability provides us with the tools to reason about partial information, so that we can estimate the probability of an event A that depends on the outcome of event B , even if we do not know the outcome of event B yet.

We say that $P(A)$ is the *prior probability* of A and that $P(A | B)$ is the *posterior probability of A given B* .

Definition – The Product rule

Given two events A and B , if $P(B)$ then the probability of both events A and B occurring is defined by the **product rule**:

$$P(A \cap B) = P(A)P(B | A)$$

The product rule can be useful, however, it is defined in terms of conditional probability, which itself uses the product rule. We can resolve this using Bayes' theorem.

Definition – Bayes' theorem

Given two events A and B , the conditional probability of A given B can be calculated using:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Bayes' theorem allows us to translate causal knowledge about events into diagnostic knowledge. For example, if event A represents a particular disease in a plant, and event B is the event describing a positive outcome of a test that can diagnose that disease, then $P(B | A)$ define the causal relationship: if the plant has disease A , then the probability of observing a positive diagnosis from test B is $P(B | A)$. Once we have the test result, we can determine the probability of A (the plant having the disease) provided we can estimate the prior probabilities $P(A)$ and $P(B)$.

Bayes' theorem is named after [Thomas Bayes](#), the English statistician and philosopher who first defined it.

PROOF INDEX

algorithm:backward-induction

algorithm:backward-induction (multi-agent/backward-induction), 181

algorithm:best-response

algorithm:best-response (multi-agent/normal-form), 168

algorithm:mcts:backpropagation

algorithm:mcts:backpropagation (single-agent/mcts), 93

algorithm:mcts:expand

algorithm:mcts:expand (single-agent/mcts), 93

algorithm:mcts:select

algorithm:mcts:select (single-agent/mcts), 92

algorithm:monte-carlo-reinforcement-learning

algorithm:monte-carlo-reinforcement-learning (single-agent/temporal-difference-learning), 57

algorithm:multi-agent-q-learning

algorithm:multi-agent-q-learning (multi-agent/multi-agent-rl), 187

algorithm:multi-armed-bandit

algorithm:multi-armed-bandit (single-agent/multi-armed-bandits), 43

algorithm:n-step-SARSA

algorithm:n-step-SARSA (single-agent/n-step), 83

algorithm:nash-equilibria

algorithm:nash-equilibria (multi-agent/normal-form), 169

algorithm:policy-iteration

algorithm:policy-iteration (single-agent/policy-iteration), ??

algorithm:q-actor-critic

algorithm:q-actor-critic (single-agent/policy-gradients), ??

algorithm:q-learning

algorithm:q-learning (single-agent/temporal-difference-learning), 60

algorithm:reinforce

algorithm:reinforce (single-agent/policy-gradients), ??

algorithm:sarsa

algorithm:sarsa (single-agent/temporal-difference-learning), 65

algorithm:single-agent-mcts

algorithm:single-agent-mcts (single-agent/mcts), 92

algorithm:value-iteration

algorithm:value-iteration (single-agent/value-iteration), 22